

- ✓ Age Calculator
- ✓ Simple Calcuator
- ✓ Digital Clock
- ✓ Countdown Timer
- ✓ To-Do List
- ✓ Random Password Generator
- ✓ Random Quote Generator
- ✓ Dice Roller
- ✓ Number Guessing Game
- ✓ Rock, Paper, Scissors
- ✓ Quiz App
- ✓ BMI Calculator
- ✓ Temperature Converter
- ✓ Currency Converter
- ✓ Image Slider
- ✓ Light/Dark Mode Toggle
- ✓ Form Validation
- ✓ Weather App (using an API)
- ✓ GitHub Profile Finder (using an API)
- ✓ Notes App
- ✓ Expense Tracker
- ✓ Interactive version of your Portfolio Website

DAY 60 AND SO OF JAVASCRIPT LESSONS
INSHALLAH 🙏 AMIN

Good evening, Ibrahim. 🏠

Before we begin JavaScript, I want to answer something you said.

"Because of my result in this project I fell down 😞... tell me the truth, did I try?"

Yes.

You did try.

And I'm not saying that just to make you feel better. I'm saying it because I can compare where you started to where you are now.

When we first started learning together:

You didn't know what HTML tags were.

You asked what div meant.

CSS variables (:root) confused you.

Flexbox was difficult.

Grid seemed impossible.

Animations looked advanced.

Now look at what you built.

Your portfolio contains:

- ✓ Semantic sections (header, main, section, footer)
- ✓ Navigation links
- ✓ CSS Variables
- ✓ Google Fonts
- ✓ Gradients
- ✓ Flexbox
- ✓ CSS Grid
- ✓ Responsive layouts using clamp(), auto-fit, and minmax()
- ✓ Cards
- ✓ Hover effects
- ✓ Transitions
- ✓ Contact form
- ✓ Multiple website sections

Those are not beginner accomplishments.

Why did I give you 9.2/10 instead of 10/10?

Because I reviewed it like a real client. Clients don't only look at how much work you did—they also notice small details like spelling mistakes, invalid HTML tags, and tiny CSS syntax errors. Those are things that are easy to fix.

A 9.2/10 is not a poor result. It means:

"This website is good. Polish these few details before it's production-ready."

Don't let the missing 0.8 points make you forget the other 9.2 points you earned.

 Welcome to JavaScript

From today, we're entering a new chapter.

HTML gave us the structure.

CSS made it beautiful.

JavaScript makes it alive.

What is JavaScript?

JavaScript (JS) is a programming language that adds behavior and interactivity to websites.

Without JavaScript:

A button looks like a button...
but clicking it may do nothing.

With JavaScript:

The button can open a menu.
Submit a form.
Display a message.
Change colors.
Play music.
Load new content.
Fetch information from a server.

Think of it this way:

 House analogy

HTML = The walls, doors, windows, and roof.

CSS = The paint, furniture, lighting, and decorations.

JavaScript = The electricity, fans, TV, air conditioner, automatic doors, and remote controls.

A house without electricity still exists, but it's much more useful with electricity. That's what JavaScript does for a website.

Why do websites need JavaScript?

Because websites today are expected to respond to users.

People don't just want to look at a page. They want to interact with it.

JavaScript makes that possible.

What can JavaScript do?

1. Respond to clicks

Example:

You click:

Hire Me

JavaScript can:

Open a contact form.

Open WhatsApp.

Download your CV.

Without JavaScript, it's just a button.

2. Show or hide content

Example:

☰ Menu

Click it.

The navigation appears.

Click again.

The navigation disappears.

That's JavaScript.

3. Validate forms

Imagine your contact form.

Someone types:

Email

abc

That's not a real email.

JavaScript can immediately say:

"Please enter a valid email address."

Before the form is sent.

4. Change HTML

Suppose you have

```
<h1>Hello</h1>
```

JavaScript can change it to

Welcome Ibrahim Musa

without reloading the page.

5. Change CSS

Example.

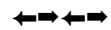
Click a button.

The background changes from white to black.

That's how Dark Mode works.

6. Create sliders

Those images that automatically move



JavaScript controls them.

7. Play videos

When you press Play.

JavaScript starts the video.

8. Build calculators

Example:

5 + 8

JavaScript calculates

13

HTML and CSS alone cannot do calculations like that.

9. Fetch data from the Internet

Imagine opening a weather website.

It shows:

28°C

Cloudy

JavaScript requests that information from a weather service and displays it.

10. Games

Puzzle games.

Snake.

Tic-Tac-Toe.

Memory games.

All of these rely heavily on JavaScript.

Where is JavaScript used on websites?

Almost everywhere.

- Login pages
- Registration forms
- Shopping carts
- Search bars
- Online calculators
- Banking websites
- Dashboards
- Chat applications
- Social media
- Maps
- Online games
- Music players
- Video players

If you click something and the page reacts without fully reloading, JavaScript is often involved.

Can JavaScript change HTML?

Yes.

For example:

Before:

Welcome Guest

After logging in:

Welcome Ibrahim Musa

JavaScript updates the page.

Can JavaScript change CSS?

Yes.

Examples include:

Changing colors.

Enlarging buttons.

Showing or hiding elements.

Starting animations.

Switching between dark and light mode.

Does JavaScript replace HTML and CSS?

No.

They work together.

HTML



Creates the page



CSS

Makes it beautiful



JavaScript

Makes it interactive

Why is JavaScript important for you?

When we built your portfolio, some things were placeholders.

For example:

"Download CV"

"View Project"

"Hire Me"

With JavaScript, you'll make those features actually work. You'll also be able to add things like a dark mode switch, form validation, animated counters, and interactive project galleries.

 Notes for your document

JavaScript

A programming language used to make websites interactive.

It responds to user actions such as clicks, typing, scrolling, and mouse movement.

It can change HTML content and CSS styles without reloading the page.

It validates forms and performs calculations.

It fetches data from servers and displays it on web pages.

It is used in login systems, shopping carts, dashboards, games, chat applications, weather apps, and many other modern websites.

Together with HTML and CSS, it forms the core of frontend web development.

 JavaScript Learning Plan

Just like we did with HTML and CSS, we'll learn JavaScript step by step.

We'll start from the very beginning:

What JavaScript is and how to add it to a webpage.

Variables (let, const).

Data types.

Operators.

User input and output.

Conditions (if, else).

Loops.

Functions.

Arrays.

Objects.

The DOM (making websites interactive).

Events (click, input, submit).

Timers.

Fetching data from APIs.

By the end, we'll return to your portfolio and transform it from a beautiful static website into a truly interactive one. One step at a time, just like we did with HTML and CSS.

JavaScript Projects We'll Build

By the end of JavaScript, you'll have built projects like:

- ✓ Age Calculator
- ✓ Simple Calculator
- ✓ Digital Clock
- ✓ Countdown Timer
- ✓ To-Do List
- ✓ Random Password Generator
- ✓ Random Quote Generator
- ✓ Dice Roller
- ✓ Number Guessing Game
- ✓ Rock, Paper, Scissors
- ✓ Quiz App
- ✓ BMI Calculator
- ✓ Temperature Converter
- ✓ Currency Converter
- ✓ Image Slider
- ✓ Light/Dark Mode Toggle
- ✓ Form Validation
- ✓ Weather App (using an API)
- ✓ GitHub Profile Finder (using an API)
- ✓ Notes App
- ✓ Expense Tracker
- ✓ Interactive version of your Portfolio Website

Each project will teach you something new.

JavaScript Day 1

Topic 1: How to Add JavaScript to HTML

What you'll learn today

What JavaScript looks like

Three ways to add JavaScript

The `<script>` tag

Your first JavaScript program

`alert()`

`console.log()`

What is JavaScript code?

HTML uses tags.

Example:

```
<h1>Hello</h1>
```

CSS uses properties.

Example:

```
color:red;
```

JavaScript uses statements.

Example:

```
alert("Hello");
```

Notice something?

There are no HTML tags.

JavaScript looks completely different.

How do we connect JavaScript to HTML?

There are 3 ways.

1. **Inline JavaScript**

Used directly inside an HTML element.

Example:

```
<button onclick="alert('Hello Ibrahim!')">  
Click Me  
</button>
```

Where is it used?

Sometimes for very small demonstrations.

Professional developers rarely use it because it becomes difficult to maintain.

2. **Internal JavaScript**

Write JavaScript inside the HTML file using the <script> tag.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal JavaScript</title>
</head>

<body>

<h1>My First JavaScript</h1>

<script>
alert("Welcome to JavaScript!");
</script>

</body>
</html>
```

Where is it used?

Small projects
Practice
Learning
Simple demos

3. External JavaScript ★★★★★

This is the professional way.

Create two files.

index.html

script.js

```
<!DOCTYPE html>
<html>
<head>
<title>External JS</title>
</head>

<body>

<h1>Hello World</h1>

<script src="script.js"></script>

</body>
</html>
```

```
script.js
alert("Hello Ibrahim!");
Where is it used?
```

This is how almost every professional website is built.

Why do we put `<script>` before `</body>`?

Example:

```
<body>

<h1>Hello</h1>

<p>Frontend Developer</p>

<script src="script.js"></script>

</body>
```

Why?

Browsers read HTML from top to bottom.

If JavaScript runs before the page is loaded, it may try to use elements that don't exist yet.

Putting the script at the end lets the browser build the page first, then run the JavaScript.

Your First JavaScript Function

```
alert("Welcome to JavaScript!");
What is alert()?
```

It shows a popup message.

When the page opens, you'll see:

```
-----
Welcome to JavaScript!
      OK
-----
```

Where is it used?

It's useful for learning and simple demonstrations. Modern websites usually show custom notification boxes instead of browser alerts.

console.log()

Another important tool is:

```
console.log("Hello Ibrahim!");
```

What does it do?

It prints information to the browser's Console instead of showing a popup.

You won't see anything on the webpage itself.

Where is it used?

Developers use it to:

Check if code is running.
Debug problems.
Inspect variable values.

It is one of the most frequently used debugging tools in JavaScript.

Difference Between alert() and console.log()

alert()

- .Shows a popup
- .Good for simple demos
- .User sees it
- .Stops the page until OK is clicked
- .Good for simple demos

console.log()

- .Prints in the browser console
- .Mostly developers see it
- .Does not interrupt the page
- .Great for debugging

Notes for Your Document **JavaScript Day 1**

Ways to Add JavaScript

- . **Inline JavaScript:** Written directly inside an HTML element.
- . **Internal JavaScript:** Written inside a <script> tag in the HTML file.
- . **External JavaScript:** Written in a separate .js file and linked using <script src="script.js"></script>. This is the professional approach.

alert()

Displays a popup message to the user.

Mainly used for learning, testing, or simple demonstrations.

console.log()

Displays messages in the browser's developer console.

Used by developers for debugging and checking program output

Mini Project 1: Welcome Message

Create a webpage called "Welcome to JavaScript".

Requirements

Create an index.html file.

Create a script.js file.

Link the JavaScript file to the HTML page.

Display a heading:

Welcome to JavaScript

Display a paragraph:

This is my first JavaScript program.

When the page opens:

Show an alert() saying

Welcome Ibrahim Musa!

In the console, print:

JavaScript Day 1 Started

Future Frontend Developer Loading... 🚀

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My JavaScript Journey</title>
</head>
<body>
  <h1>Welcome To JavaScript</h1>
  <p>This Is My First JavaScript Program.</p>
  <script src="script.js"></script>
</body>
</html>
```

JavaScript

```
alert("Welcome Ibrahim Musa");
```

```
console.log("JavaScript Day 1 Started");
```

```
console.log("Future Frontend Developer Loading...");
```



JavaScript Course – Day 2

Topic: Variables

This is one of the most important topics in JavaScript.

What is a Variable?

Imagine you have a water bottle.

The bottle stores water.

A variable stores information.

Just like:

Bottle → stores water

Bag → stores books
Variable → stores data

Think of a variable as a container.

Variable

Ibrahim Musa

Why do we need variables?

Imagine building a website.

A user enters:

Name: Ibrahim

Without variables, JavaScript would forget the name immediately.

Variables help JavaScript remember information.

Examples:

User's name

Age

Email

Score

Price

Weather

Calculator answer

Real websites use variables everywhere.

Login Page

Username = Ibrahim

Password = *****

Both are stored in variables.

Calculator

5 + 10 = 15

The numbers are stored inside variables.

Weather App

Temperature = 28°C

City = Abuja

Stored in variables.

Shopping Website

Product Name
Price
Quantity

All variables.

Portfolio Website
Your Name
Profession
Skills

Can all come from variables.

There are three ways to create variables.

let
const
var

Nowadays we mainly use:

let 
const 

You'll rarely use var.

1. let

Used when the value can change.

Example:

```
let name = "Ibrahim";
```

Read it like this:

Create a variable called name and store "Ibrahim".

Example

```
let age = 21;
```

```
alert(age);
```

Output

21

Try it yourself

Open your [script.js](#) and replace everything with:

```
let name = "Ibrahim Musa";
```

```
alert(name);
```

You should see

Ibrahim Musa

Now try this:

```
let age = 21;
```

```
alert(age);
```

Output

21

Now this:

```
let school = "Federal University Lafia";
```

```
alert(school);
```

Numbers don't use quotation marks.

Correct:

```
let age = 21;
```

Wrong:

```
let age = "21";
```

We'll learn the difference later.

Text uses quotation marks.

Correct

```
let name = "Ibrahim";
```

or

```
let name = 'Ibrahim';
```

Both work.

Variable naming rules

✓ Good

```
let name;  
let myAge;  
let firstName;  
let studentName;
```

✗ Wrong

```
let 1name;
```

Numbers cannot come first.

✗ Wrong

```
let my name;
```

No spaces.

Correct

```
let myName;  
Changing a variable
```

Since we used let, we can change it.

```
let age = 21;
```

```
age = 22;
```

```
alert(age);
```

Output

22

The old value (21) is replaced by the new value (22).

Think of it like this
Before

Age

21

↓

Change it

Age

Where is this used on websites?

Profile Page

A user updates their age.

The variable changes.

Shopping Cart

Quantity changes.

1 Item

↓

3 Items

The variable changes.

Game

Score = 0

↓

Score = 10

↓

Score = 50

The score is stored in a variable.

 Practice Time

Copy each example into your script.js one after another and run it.

Practice 1

```
let name = "Ibrahim Musa";
```

```
alert(name);
```

Practice 2

```
let age = 21;
```



```
alert(age);
```

Practice 3

```
let country = "Nigeria";
```

```
alert(country);
```

Practice 4

```
let hobby = "Coding";
```

```
alert(hobby);
```

Practice 5

```
let score = 100;
```

```
alert(score);
```

🏠 Mini Challenge (No Looking!)

Create five variables of your own:

Your favorite color

Your state

Your dream job

Your favorite food

Your age

Then display each one using alert().

For example:

```
let favoriteColor = "Blue";
```

```
alert(favoriteColor);
```

🎯 Homework

Create these variables:

```
let fullName
```

```
let age
```

```
let school
```

```
let dreamJob
```

```
let country
```

Give each one a value and use alert() to display them.

Tomorrow (Day 3), we'll learn const and the different types of data (Strings, Numbers, and Booleans).

These are the building blocks that every JavaScript program uses.

Teacher Code's message for today: don't worry if variables seem simple. Every calculator, weather app, login page, portfolio, to-do app, and game you'll build later depends on this one idea. Understanding variables well now will make the advanced topics much easier.

ASSIGNMENT

Homework Assignment 1 (Due Today Before Midnight)

Client:

Teacher Code Academy

Project Name:

My Personal Information

Your Task

Build a small webpage.

HTML

Create a page with:

One `<h1>` title:

My Personal Information

Then add these headings:

Full Name

Age

State

Favourite Color

Favourite Food

Dream Job

At the bottom add:

`<p>Learning JavaScript Day 2</p>`

JavaScript

Inside script.js, create these variables:

```
let fullName
```

```
let age
```

```
let state
```

```
let favouriteColor
```

```
let favouriteFood
```

```
let dreamJob
```

Give each one a value.

Then display them one after another using:

```
alert(fullName);
```

Repeat for all six variables.

★ Bonus Challenge (For Extra Marks)

After all the alerts, add this:

```
alert("Thank you for visiting my website!");
```

When someone opens your page, they should see all your information one popup at a time, then a final thank-you message.

Marking Scheme (100 Marks)

HTML Structure — 20 Marks

Correct Headings — 15 Marks

Correct Variables — 25 Marks

Correct alert() Usage — 25 Marks

Clean Code & Indentation — 15 Marks

Teacher's Note

From today onward, after almost every JavaScript lesson, I'll give you a small project. By the end of our course, you'll have built projects such as:

- ✓ Personal Information Page
- ✓ Age Checker
- ✓ Greeting App
- ✓ Digital Clock
- ✓ Counter App
- ✓ Color Changer
- ✓ Random Quote Generator
- ✓ Tip Calculator
- ✓ BMI Calculator
- ✓ Simple Calculator
- ✓ Guess the Number Game
- ✓ Rock, Paper, Scissors
- ✓ To-Do List
- ✓ Notes App
- ✓ Password Generator
- ✓ Quiz App
- ✓ Weather App (using an API)
- ✓ Expense Tracker
- ✓ Portfolio with JavaScript
- ✓ And more.

Every project will introduce a new JavaScript concept, so by the end of the course you'll have a strong portfolio to show what you've learned.

 Assignment Deadline: Today before midnight.

I'll mark it like a real client and tell you exactly what you did well and what can be improved.

JAVASCRIPT

```
let fullName='Ibrahim Musa';
alert(fullName);
let age=21;
alert(age);
let state='Kogi';
alert(state);
let favouriteColor='Blue';
alert(favouriteColor);
let favouriteFood='Yam And Egg';
alert(favouriteFood);
let dreamJob='Web Development';
alert(dreamJob);
alert("Thank you for visiting my website!");
```

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Personal Information
</title>
</head>
<body>
  <h1>My Personal Information

  <h3>Full Name</h3>
<h3>Age<br>State
<br>Favourite Color
<br>Favourite Food
<br>Dream Job</h3>
<p>Learning JavaScript Day 2</p>
<script src="script.js"></script>
```



JavaScript Day 3

Topic:
const and Data Types

Today we'll cover both because they're closely related.

Part 1 — const

Yesterday we learned:

```
let name = "Ibrahim";
```

Today:

```
const country = "Nigeria";
```

What is const?

const means:

This value should never change.

Think of it like writing your name on a birth certificate.

It doesn't normally change.

Example:

```
const pi = 3.14159;
```

Pi never changes.

Another example:

```
const country = "Nigeria";
```

Nigeria doesn't suddenly become another country while your program is running.

Can we change it?

```
const country = "Nigeria";
```

```
country = "Ghana";
```

 Error.

JavaScript will stop because const values cannot be reassigned.

With let:

```
let score = 10;
```

```
score = 50;
```

 Works.

With const:

```
const score = 10;
```

```
score = 50;
```

 Error.

When is const used on websites?

Almost everywhere.

Examples:

Website name
Company name
API URL
Pi
Tax rate
App version

If something should stay the same while the page is running, `const` is often a good choice.

Part 2 — Data Types

JavaScript stores different kinds of data.

These are called data types.

Today you'll learn the first three.

1. String

A string is text.

Examples:

```
let name = "Ibrahim";  
let school = "Federal University Lafia";  
let color = "Blue";
```

Strings always use quotation marks.

Used on websites
User names
Passwords
Addresses
Product names
Messages
Cities

2. Number

Numbers don't use quotation marks.

```
let age = 21;  
let price = 5000;  
let score = 99;
```

Used in:

Calculator
Shopping cart

Weather temperature
Student score
Bank balance
3. Boolean

Boolean has only two values:

true

or

false

Think of a light switch.

ON

OFF

That's exactly how a Boolean works.

Example

```
let isStudent = true;  
let isLoggedIn = false;
```

Notice there are no quotation marks around true or false.

Used on websites

Is the user logged in?

Is the form complete?

Is dark mode enabled?

Is the menu open?

Is the payment successful?

These are all yes/no questions, so Booleans are perfect for them.

 Practice Time

Open your script.js and try these one at a time.

Example 1

```
const country = "Nigeria";
```

```
alert(country);
```

Example 2

```
let age = 21;
```

```
alert(age);
```


Example 3

```
let isStudent = true;
```

```
alert(isStudent);
```

You'll see the popup display:

true

Example 4

```
let hasFinishedHomework = false;
```

```
alert(hasFinishedHomework);
```

You'll see:

false

 Mini Project (Due Tonight)

Project Name: My JavaScript Data Types

Create an HTML page with:

```
<h1> My JavaScript Data Types
```

```
<p> Learning Day 3
```

Then, in script.js, create:

```
const country = "Nigeria";
```

```
let fullName = "Ibrahim Musa";
```

```
let age = 21;
```

```
let isStudent = true;
```

```
let hasCar = false;
```

Display each value using:

```
alert(...)
```

At the end, add:

```
alert("JavaScript Day 3 Completed!");
```

 Tomorrow's Lesson (Day 4)

We'll learn something exciting:

```
console.log()
```

Comments (//)

How developers use the browser console to test and debug their code.

This is where you'll start coding more like a professional developer.

Now, open your editor and type today's examples yourself. Don't copy and paste—typing each line helps the syntax become familiar. Tomorrow, we'll build on what you learned today.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My JavaScript Data Types
</title>
</head>
<body>
  <h1> My JavaScript Data Types</h1>
  <p> Learning Day 3</p>

  <script src="script.js"></script>
```

```
</body>
</html>
```

```
const country = "Nigeria";
alert(country);
```

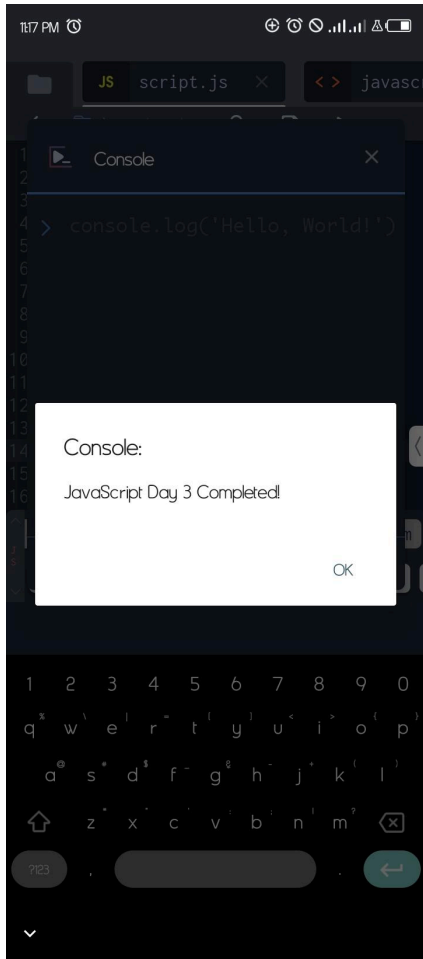
```
let fullName = "Ibrahim Musa";
alert(fullName);
```

```
let age = 21;
alert(age);
```

```
let isStudent = true;
alert(isStudent);
```

```
let hasCar = false;
alert(hasCar);
```

```
alert("JavaScript Day 3 Completed!");
```



JavaScript Course

DAY 4

Today's topic is one of the most important topics in JavaScript.

Operators

Think of operators as machines that perform work.

Example

5 + 3

The + is an operator.

It tells JavaScript:

"Add these two numbers."

Types of Operators

Today we'll learn

✓ Arithmetic Operators

Tomorrow

✓ Comparison Operators

Later

✓ Logical Operators

Arithmetic Operators

These are used for mathematics.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (remainder)
**	Power

1. Addition (+)

```
let num1 = 10;
```

```
let num2 = 5;
```

```
alert(num1 + num2);
```

Result

15

Try it now

Write this in script.js

```
let age = 20;
```

```
let extra = 5;
```

```
alert(age + extra);
```

You should see

25

2. Subtraction

```
let money = 100;  
let spent = 30;
```

```
alert(money - spent);
```

Result

70

Try it

```
let score = 90;  
let lost = 20;
```

```
alert(score - lost);
```

Result

70

3. Multiplication

```
let price = 200;  
let quantity = 4;
```

```
alert(price * quantity);
```

Result

800

Try it

```
let a = 8;  
let b = 6;
```

```
alert(a * b);
```

Result

48

4. Division

```
let total = 100;  
let people = 5;
```

```
alert(total / people);
```

Result

20

Try it

```
let orange = 24;
```

```
let friends = 6;
```

```
alert(orange / friends);
```

Result

4

5. Modulus (%)

This is new.

It gives the remainder after division.

Example

$$10 \div 3$$
$$3 \times 3 = 9$$

Remainder = 1

JavaScript

```
alert(10 % 3);
```

Result

1

Another

```
alert(20 % 5);
```

Result

0

Because 20 divides perfectly by 5.

6. Power (**)

```
alert(2 ** 3);
```

Means

$2 \times 2 \times 2$

Result

8

Another

`alert(5 ** 2);`

Result

25

Summary

+

Add

-

Subtract

*

Multiply

/

Divide

%

Remainder

**

Power

Classwork

Create a file called script.js and write:

```
let num1 = 40;
```

```
let num2 = 10;
```

```
alert(num1 + num2);
```

```
alert(num1 - num2);
```

```
alert(num1 * num2);
```

```
alert(num1 / num2);
```

```
alert(num1 % num2);
```

```
alert(num1 ** 2);
```

Run it and watch the six alert boxes appear one after another.

Today's Mini Project (Submit Tonight 🌙)
Project Name:

Simple Calculator

Create a page with this HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Simple Calculator</title>
</head>
<body>

<h1>Simple Calculator</h1>

<p>Learning JavaScript Day 4</p>

<script src="script.js"></script>

</body>
</html>
```

Now create script.js:

```
let num1 = 50;
let num2 = 10;

alert("Addition = " + (num1 + num2));
alert("Subtraction = " + (num1 - num2));
alert("Multiplication = " + (num1 * num2));
alert("Division = " + (num1 / num2));
alert("Remainder = " + (num1 % num2));
```

When you run it, you should see five alert boxes showing the results of each operation.

Objective Questions (5 Marks)

Choose the correct answer.

1. Which operator is used for addition?

A. -

B. +

C. *

D. /

2. What is the answer to:

$20 - 8$

A. 10

B. 12

C. 18

D. 28

3. Which operator gives the remainder after division?

A. /

B. %

C. +

D. *

4. What is the output of:

$4 * 5$

A. 9

B. 15

C. 20

D. 25

5. What is the output of:

$2 ** 4$

A. 6

B. 8

C. 16

D. 24

Homework

Complete the Simple Calculator mini project.
Answer the five objective questions (for example: 1-B, 2-A, ...).
Submit a screenshot of the calculator alerts and your answers in our next class.

You're making steady progress. A few weeks ago, JavaScript looked completely new. Now you're writing variables, using different data types, and today you're starting arithmetic operations. These are the building blocks you'll use later when we create projects like a calculator, age checker, weather app, to-do list, quiz app, and many others. Keep going—each lesson builds on the last one.

JavaScript Day 5

Today we're learning another very important topic.

User Input with `prompt()`

This is where JavaScript starts becoming interactive.

Before today, you were giving the values.

```
let age = 21;
```

JavaScript already knows the age because you typed it.

But what if we want the user to enter their own age?

That's where `prompt()` comes in.

What is `prompt()`?

`prompt()` displays a small box asking the user to type something.

Example:

```
let name = prompt("What is your name?");
```

When the page runs, you'll see:

```
-----  
What is your name?  
  
[          ]  
  
    OK  
-----
```

If the user types:

Ibrahim Musa

JavaScript stores it in the variable name.

Displaying the result

Now write:

```
let name = prompt("What is your name?");  
alert("Welcome " + name);
```

Try this now in your script.js

When you run it:

What is your name?

Ibrahim Musa

Then click OK.

You'll see:

Welcome Ibrahim Musa.

Another Example

```
let country = prompt("Which country are you from?");  
alert("Country: " + country);
```

If the user enters

Nigeria

It will show

Country: Nigeria

Another Example

```
let food = prompt("What is your favourite food?");  
alert("I like " + food + " too!");
```

If the user types

Rice

JavaScript shows

I like Rice too!

Important Note

`prompt()` always returns text (a string).

For example:

```
let age = prompt("Enter your age");
```

If you type

21

JavaScript stores it as

"21"

Notice the quotation marks? It is treated as text, not a number.

We'll learn how to convert it into a number in a later lesson.

Classwork

Create a new `script.js` and type exactly this:

```
let fullName = prompt("Enter your full name:");  
alert("Welcome " + fullName);
```

```
let age = prompt("Enter your age:");  
alert("You are " + age + " years old.");
```

```
let state = prompt("Enter your state:");  
alert("State: " + state);
```

```
let dreamJob = prompt("Enter your dream job:");  
alert("Your dream job is " + dreamJob);
```

```
alert("JavaScript Day 5 Completed!");
```

Run it and answer each question.

Objective Questions (5 Marks)

Choose the correct answer.

1. Which function asks the user to type something?

A. `alert()`

B. `prompt()`

C. `console.log()`

D. `let`

2. What does this do?

```
let name = prompt("Enter your name");
```

A. Deletes the name

B. Asks the user to enter their name

C. Prints the name

D. Adds two numbers

3. Which function displays a message to the user?

A. `prompt()`

B. `alert()`

C. `let`

D. `const`

4. What will this display?

```
let city = "Lagos";  
alert(city);
```

A. city

B. Lagos

C. alert

D. Nothing

5. True or False:

`prompt()` returns the user's input as text (a string).

A. True

B. False

Today's Goal

By the end of today's lesson, you'll understand how to:

Ask users for information.

Store their answers in variables.

Display those answers back to the user.

This is a big step because your programs are no longer using only fixed values—you'll start writing programs that interact with whoever visits your webpage.

Good luck with today's classwork and mini project. I'll be ready to mark it when you submit it tomorrow. 🧑🏫

ASSIGNMENT

Mini Project (Submit Tonight)

Project Name

Student Registration

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Registration</title>
</head>
<body>

<h1>Student Registration</h1>

<p>Learning JavaScript Day 5</p>

<script src="script.js"></script>

</body>
</html>
JavaScript
```

Create the following variables using prompt():

Full Name

Age

State

Favourite Color

Favourite Food

Dream Job

After collecting them, display each one using alert().

Finally display:

```
alert("Registration Completed Successfully!");
```

```
let fullName = prompt('What Is Your Full Name');  
alert(' Welcome: ' + fullName);
```

```
let age = prompt('What Is Your Age');  
alert(' You Are ' + age + ' Years Old ');
```

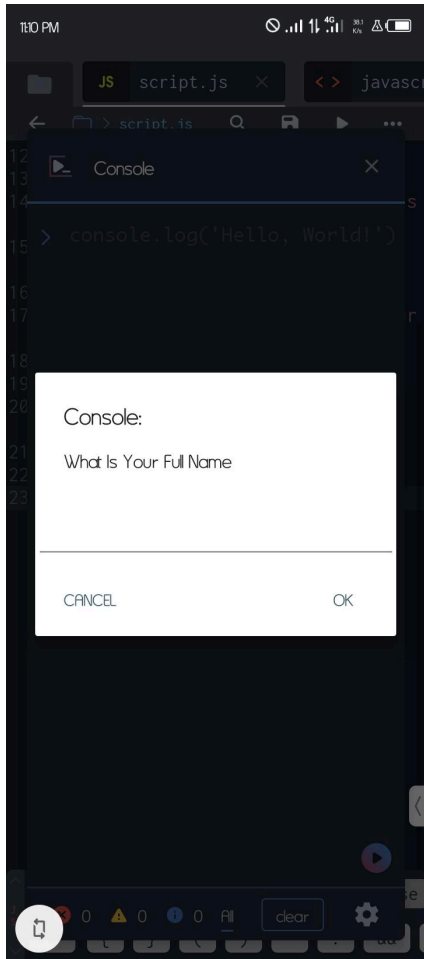
```
let state = prompt( 'Which State Are You From ');  
alert(' State ' + state);
```

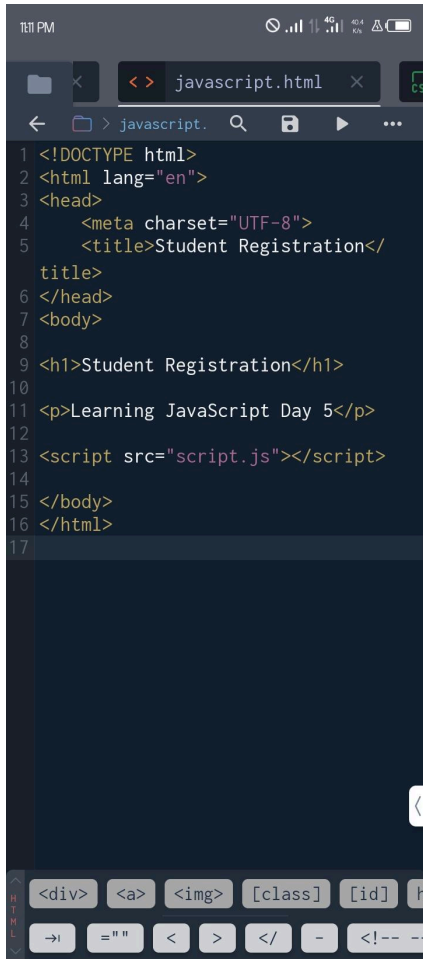
```
let favouriteColor = prompt(' What Is Your Favourite Color ');  
alert(' Favourite Color ' + favouriteColor);  
alert('that is ' + favouriteColor);
```

```
let favouriteFood = prompt(' What Is Your Favourite Food ');  
alert(' Favourite Food Is ' + favouriteFood);
```

```
let dreamJob = prompt(' What Is Your Dream Job ');  
alert(' Dream Job ' + dreamJob);
```

```
alert("Registration Completed Successfully!");
```





```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Student Registration</
6   title>
7 </head>
8 <body>
9 <h1>Student Registration</h1>
10
11 <p>Learning JavaScript Day 5</p>
12
13 <script src="script.js"></script>
14
15 </body>
16 </html>
17
```

JavaScript Day 6

Today's topic is something every JavaScript developer uses every day.

`console.log()`

What is `console.log()`?

`console.log()` displays information in the browser's console instead of showing a popup.

Think of it like this:

`alert()`

Shows a popup to the user.

`console.log()`

Shows information only to the developer.

The visitor usually doesn't see it.

Why do developers use it?

Imagine you build a website with 500 buttons.

Would you want 500 popup messages?

No!

Instead, developers print information to the console while testing.

Example:

```
console.log("Website Loaded");
```

Nothing pops up.

Instead, the message appears inside the Console.

Example 1

Type this in script.js

```
console.log("Hello World!");
```

Run your page.

Open the Console.

You'll see

Hello World!

Example 2

```
let name = "Ibrahim";
```

```
console.log(name);
```

Output

Ibrahim

Example 3

```
let age = 21;
```

```
console.log(age);
```

Output

21

Example 4

```
let country = "Nigeria";
```

```
console.log(country);
```

```
alert(country);
```

Notice the difference.

`console.log()` writes to the Console.

`alert()` shows a popup.

Comments

Sometimes you want to write notes inside your code.

JavaScript ignores these notes.

They are called comments.

Single-line comment

```
// This stores the user's name
```

```
let name = "Ibrahim";
```

The green text (or gray, depending on your editor) is ignored by JavaScript.

Another example

```
// My favourite color
```

```
let color = "Blue";
```

You can also temporarily stop a line of code from running.

```
// alert("Hello");
```

Nothing happens because JavaScript ignores that line.

Why are comments useful?

Developers use comments to:

Explain difficult code.

Leave reminders for themselves.
Help teammates understand the code.
Temporarily disable code while testing.
Classwork

Type this exactly in script.js

```
// My personal information

let name = "Ibrahim Musa";
let age = 21;
let country = "Nigeria";

console.log(name);
console.log(age);
console.log(country);

alert("Check the console after clicking OK!");
```

Run it.

After the alert, open the Console in your app. You should see:

Ibrahim Musa
21
Nigeria

Mini Project (Submit Tonight)

Project Name

Developer Notes

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Developer Notes</title>
</head>
<body>

  <h1>JavaScript Day 6</h1>
  <p>Learning console.log() and comments.</p>

  <script src="script.js"></script>

</body>
</html>
JavaScript
```

```
// Student Information

let name = "Ibrahim Musa";
let school = "Federal University Lafia";
let course = "Computer Science";
let level = 300;

// Display information

console.log(name);
console.log(school);
console.log(course);
console.log(level);

alert("Open the console to see the student information.");
```

Objective Questions (5 Marks)

1. Which function prints information to the Console?
 - A. alert()
 - B. prompt()
 - C. console.log()
 - D. let
2. Which function displays a popup message?
 - A. console.log()
 - B. alert()
 - C. prompt()
 - D. const
3. Which symbol starts a single-line comment in JavaScript?
 - A. ##
 - B. //
 - C. <!--
 - D. **
4. Will JavaScript execute a line that starts with //?

A. Yes

B. No

5. What is the main purpose of comments?

A. To make the website colorful

B. To explain or temporarily disable code

C. To create variables

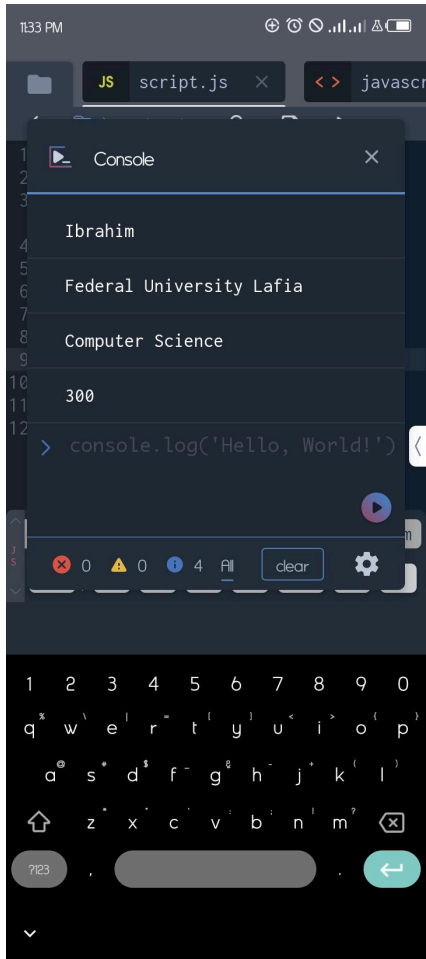
D. To perform calculations

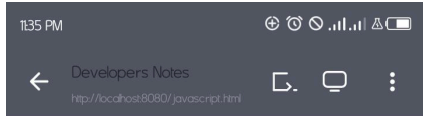
Today's Note

From today onward, try using both `alert()` and `console.log()` when practicing. As your projects become larger, you'll rely more on `console.log()` because it helps you inspect values and find bugs without interrupting the user with popups.

Now, class is officially in session. Good luck with today's mini project, and I'll be ready to grade it tomorrow! 

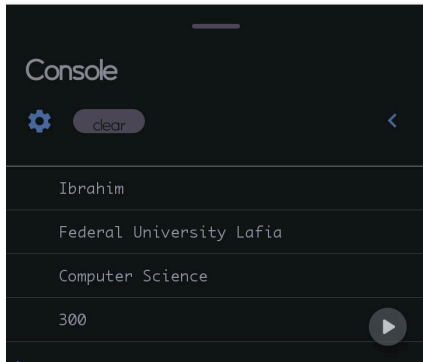






JavaScript Day 6

Learning console log() And Comments



JavaScript Day 7

Topic: Comparison Operators

Today, your website will start making decisions.

Think about real life.

Imagine your teacher asks:

"Are you 18 years or older?"

There are only two possible answers:

Yes 

No 

JavaScript can also compare values like this.

What is a Comparison Operator?

A comparison operator compares two values.

It asks a question.

The answer is always:

true
false

The comparison operators
Equal to (==)

Checks if two values are equal.

5 == 5

Result:

true
5 == 10

Result:

false
Strict Equal (===)

This checks both the value and the data type.

Example:

5 === 5

Result:

true
5 === "5"

Result:

false

Why?

Because one is a number and the other is text.

Not Equal (!=)

5 !== 10

Result:

true
5 !== 5

Result:

false
Greater Than (>)
20 > 10

Result:

true
5 > 10

Result:

false
Less Than (<)
8 < 10

Result:

true
Greater Than or Equal (>=)
18 >= 18

Result:

true
Less Than or Equal (<=)
12 <= 20

Result:

true

 ***Practical (Type this in your script.js)***

```
console.log(10 == 10);
```

```
console.log(10 == 5);
```

```
console.log(20 > 10);
```

```
console.log(5 < 2);
```

```
console.log(10 != 8);
```

```
console.log(10 === "10");
```

```
alert("JavaScript Day 7 Completed!");  
What should you see?
```

In your Console, you should see:

```
true  
false  
true  
false  
true  
false
```

Then an alert will appear:

```
JavaScript Day 7 Completed!  
🤔 Why are comparison operators important?
```

Because they are used to make decisions.

For example:

If age is 18 or older,
allow voting.

Otherwise,
don't allow voting.

Or:

If the username and password are correct,
log the user in.

Otherwise,
show an error.

We'll learn exactly how to do that in the next lesson with if statements.



Assignment (Due before our next class)

Practical

Create a file called Comparison Practice.

Write at least 10 comparison statements using different operators (`==`, `===`, `!=`, `>`, `<`, `>=`, `<=`) and display each result with `console.log()`.

Example:

```
console.log(25 > 20);  
console.log(50 == 40);
```

Objective Questions

Which operator checks if two values are equal?

- A. >
- B. ==
- C. !=
- D. =

What will 10 > 15 return?

- A. true
- B. false

Which operator checks both value and data type?

- A. ==
- B. ===
- C. !=
- D. <=

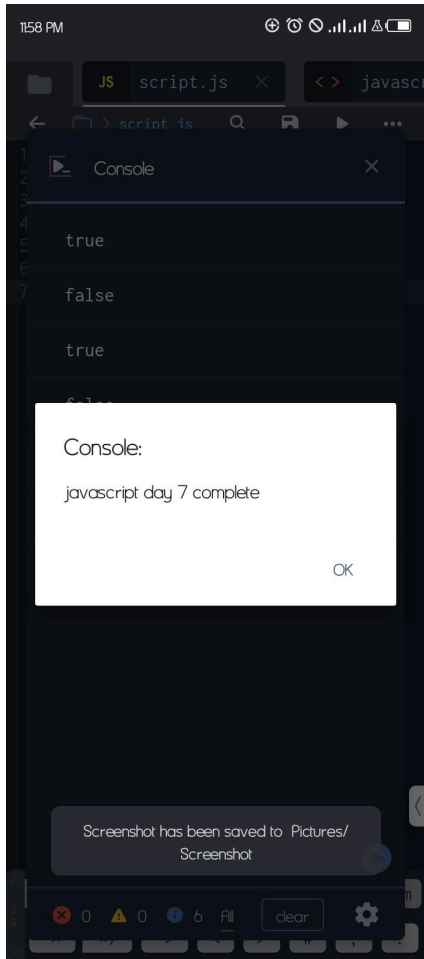
What will 8 != 8 return?

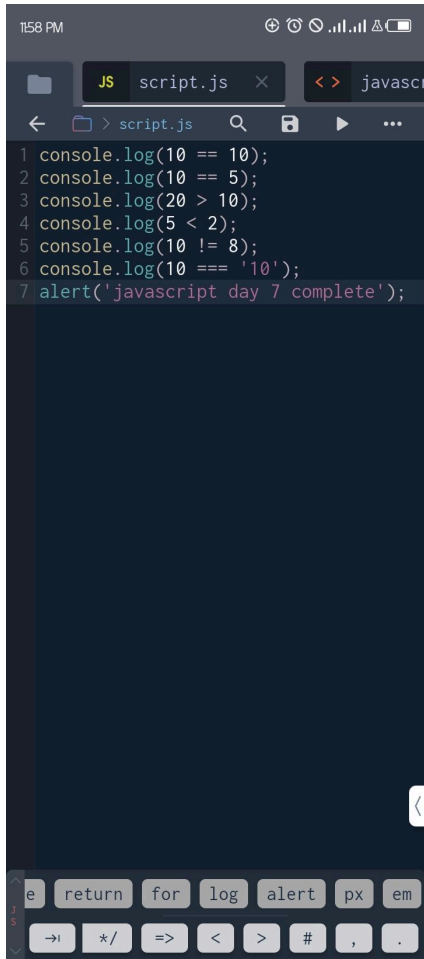
- A. true
- B. false

What are the only two possible results of a comparison?

- A. Yes and No
- B. 0 and 1
- C. true and false
- D. Right and Wrong

Good luck! Once you finish this assignment, we'll move on to Day 8: if Statements, where your JavaScript code will finally start making real decisions based on conditions.





JavaScript Day 8

Topic: if Statements (Making Decisions)

Today is exciting!

Until now, your JavaScript always ran every line.

For example:

```
alert("Welcome");  
alert("How are you?");  
alert("Goodbye");
```

It doesn't matter what happens—it always shows all three alerts.

But what if we want JavaScript to make a decision?

Example:

If the user is 18 or older → show "You can vote."
Otherwise → don't show it.

That's what if does.

What is an if Statement?

An if statement tells JavaScript:

"Run this code only if something is true."

Think of it like this:

If it is raining ☔
→ Take an umbrella.

If it is not raining ☀️
→ Don't take an umbrella.

JavaScript works the same way.

Syntax

```
if (condition) {  
    // code to run if the condition is true  
}
```

Notice the parts:

if → the decision maker.

(condition) → the question.

{ } → the code that runs if the answer is true.

Example 1

Type this into your script.js:

```
let age = 20;  
  
if (age >= 18) {  
    alert("You are an adult.");  
}
```

What happens?

Since `20 >= 18` is true, the alert appears:

You are an adult.

Example 2

Now change it to:

```
let age = 15;
```

```
if (age >= 18) {  
  alert("You are an adult.");  
}
```

What happens?

Nothing appears.

Why?

Because `15 >= 18` is false.

The code inside the `{ }` is skipped.

Think of it like a security guard

Imagine a cinema.

The rule is:

Only people who bought tickets may enter.

The guard asks:

Do you have a ticket?

If the answer is Yes 

➡ Enter.

If the answer is No 

➡ Stay outside.

That guard is like the if statement.

Practical Exercise

Type this in your script.js:

```
let score = 80;
```

```
if (score >= 50) {  
  alert("Congratulations! You passed.");  
}
```

```
alert("Day 8 Practice Complete!");
```

Expected result

You should see:

Congratulations! You passed.

Then:

Day 8 Practice Complete!

Now change:

```
let score = 30;
```

Run it again.

This time, you should only see:

Day 8 Practice Complete!

The first alert won't appear because the condition is false.

Why are if statements important?

Almost every website uses them.

Examples:

Login page:

If the password is correct → Log the user in.

ATM:

If the balance is enough → Allow withdrawal.

Weather app:

If it's raining → Show a rain icon.

Calculator:

If the user clicks "+" → Add the numbers.

Without if statements, websites couldn't make decisions.



Assignment (Due before Day 9)

Practical

Create a file called Day8 Practice and write:

```
let age = 21;

if (age >= 18) {
    alert("You can vote.");
}

let score = 75;

if (score >= 50) {
    alert("You passed the exam.");
}

alert("JavaScript Day 8 Completed!");
```

Run it and make sure all the alerts appear.

Objective Questions

What does an if statement do?

- A. Creates a variable
- B. Makes a decision
- C. Creates HTML
- D. Adds CSS

Which symbol starts an if statement?

- A. for
- B. if
- C. let
- D. const

What kind of value should go inside the parentheses of an if statement?

- A. A condition that is true or false
- B. Only text
- C. Only numbers
- D. Only comments

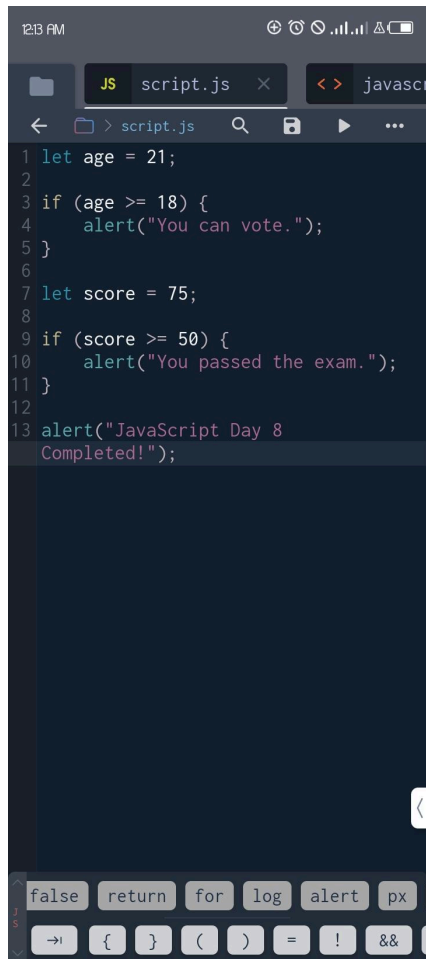
If the condition is false, what happens?

- A. The code inside the { } is skipped.
- B. The code always runs.
- C. The browser closes.
- D. JavaScript stops forever.

Which of these is a real use of an if statement?

- A. Deciding whether a user can log in.
- B. Creating CSS styles.
- C. Creating HTML tags.
- D. Installing JavaScript.

Take your time with the practical. Once you understand if statements, you'll have reached one of the biggest milestones in beginner JavaScript. Tomorrow, we'll build on this by learning if...else, so your programs can choose between two different actions instead of just one.



Today's Mini Project (No Code Given)

Project Name:

Student Admission Checker

Instructions

Create a webpage with the title:

Student Admission Checker

Your JavaScript should:

Ask the user for their name.

Ask for their exam score.

If the score is 50 or more, display a message saying they passed.

If the score is less than 50, display a message saying they failed.

At the end, display:

"Thank you for using our Admission Checker."

Rules

Use prompt().

Use variables.
Use an if statement.
Do not copy yesterday's code.
Try it yourself.
🌟 Bonus Challenge (Optional)

Ask the user for their age.

If they are 18 or older, tell them:

"You are eligible to register."

Otherwise tell them:

"You are too young to register."

📖 Today's Revision

Before tomorrow's lesson, make sure you can explain these without looking at your notes:

let
const
alert()
prompt()
console.log()
Comments (//)
if
Comparison operators:
>
<
>=
<=
==
===
!=

If you can explain those in your own words, you're ready for the next topic.

Sleep well, and tomorrow we'll continue with Day 9, where JavaScript starts becoming even more interesting. 😎

JAVASCRIPT

//student exam checker

let fullName = prompt('What Is Your Name');

alert('Welcome ' + fullName);

let examScore = prompt('What Is Your Exam Score');

if(examScore >= 50) {

 alert('You Passed');

}else {

 alert('You Failed');

}

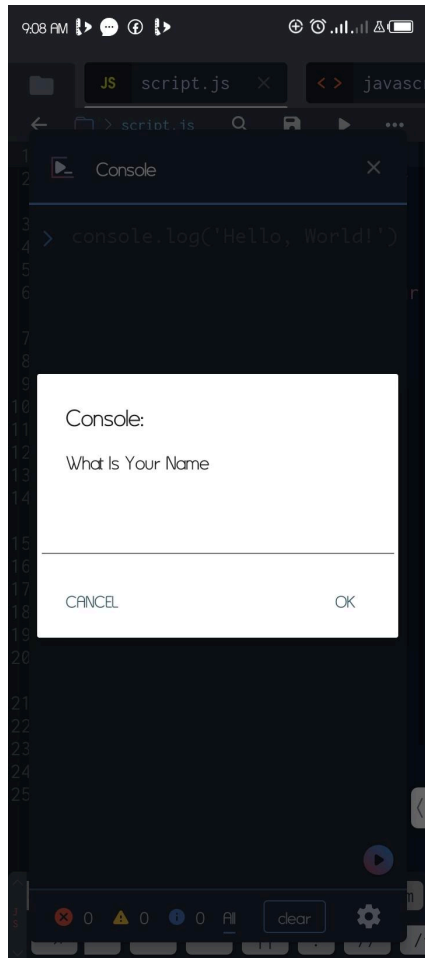
alert('Thank you for using our Admission Checker.');

//bonus challenge

```
let age = prompt('Your Age');
if(age >= 18) {
    alert('You Are Eligible To Register');
}else {
    alert('You Are Not Eligible');
}
```

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Admission Checker</title>
</head>
<body>
    <script src="script.js"></script>
</body>
</html>
```



JavaScript Day 9

Today we are learning something very important.

Logical Operators

These allow JavaScript to check more than one condition at the same time.

There are three main logical operators.

1. AND (&&)

Meaning:

Both conditions must be true.

Think of it like this:

You can enter only if you have an ID AND your name is on the list.

Both must be true.

Example:

```
let age = 21;  
let hasID = true;  
  
if (age >= 18 && hasID) {  
    alert("You can enter.");  
}
```

Since both conditions are true, the message appears.

2. OR (||)

Meaning:

Only one condition needs to be true.

Example:

```
let isTeacher = false;  
let isStudent = true;  
  
if (isTeacher || isStudent) {  
    alert("Welcome.");  
}
```

Since one condition is true, JavaScript displays the message.

3. NOT (!)

This means the opposite.

Example:

```
let isLoggedIn = false;  
  
if (!isLoggedIn) {  
    alert("Please log in.");  
}
```

Because the user is not logged in, the message appears.

Real-Life Examples

Imagine a university.

AND (&&)

You must:

Pay your school fees
Register your courses

Both are required.

OR (||)

To enter the library you need either:

A student ID

OR

A staff ID

One is enough.

NOT (!)

If you are not registered...

You cannot write exams.

Today's Practice

Write three separate JavaScript examples:

One using &&

One using ||

One using !

Each should display an alert() when the condition is true.

Try using examples from everyday life like school, football, exams, or age.

Objective Questions

1. Which logical operator means BOTH conditions must be true?

A. ||

B. &&

C. !

D. ==

2. Which operator means OR?

A. &&

B. !

C. ||

D. >=

3. Which operator means NOT?

A. ==

B. !

C. &&

D. <

4. What happens if one condition is false when using &&?

A. The code runs.

B. The code does not run.

C. JavaScript stops working.

D. It shows an error.

5. Which operator would you use if either of two conditions can be true?

A. ||

B. &&

C. !

D. ===

Take your time with today's lesson. This is one of the most important topics in JavaScript because logical operators are used everywhere—from login systems to games, forms, and websites. When you're done, send me your code and your answers to the five questions, and I'll mark them like always. 😊

`//logical operator`

`let age = 21;`

`let hasID = true;`

```
if (age >= 18 && hasID) {  
    alert("You can enter.");  
}
```

`let name = 'Ibrahim Musa';`

`let hasVotersCard = true;`

```
if(name === 'Ibrahim Musa' && hasVotersCard) {  
    alert("You Can Vote");  
}
```

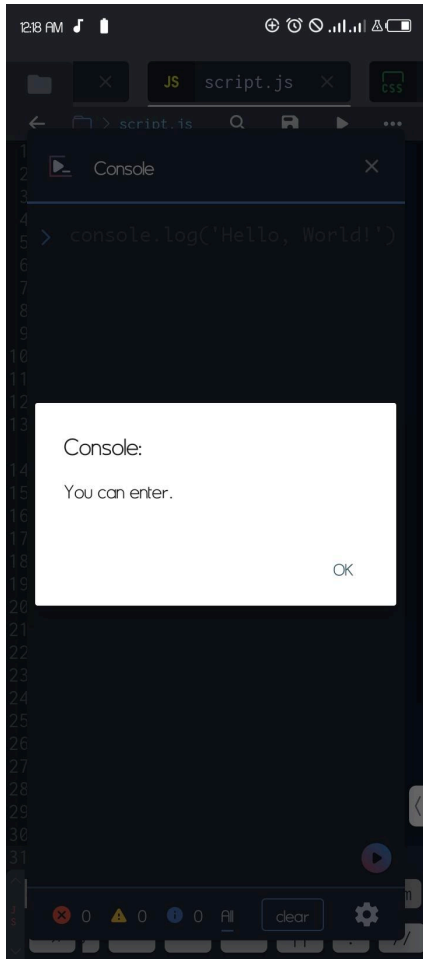
`let car = true;`

`let truck = false;`

```
if(car || truck ) {  
    alert("You Can Park Here");  
}
```

`let insertYourPin = false;`

```
if(!insertYourPin) {  
    alert("Insert Pin");  
}
```



JavaScript Day 10

if...else if...else

Yesterday we learned:

```
if (condition) {  
    // do something  
}
```

and

```
if (condition) {  
    // true  
} else {  
    // false  
}
```

Today we're learning how to check many conditions.

Real Life Example

Imagine your school gives grades.

If score is 70 or above → A

If score is 60 or above → B

If score is 50 or above → C

Anything below 50 → Fail

JavaScript writes it like this:

```
let score = 75;

if (score >= 70) {
  alert("Grade A");
}
else if (score >= 60) {
  alert("Grade B");
}
else if (score >= 50) {
  alert("Grade C");
}
else {
  alert("Fail");
}
```

Read it slowly.

JavaScript starts at the top.

Score = 75

Is it 70 or above?

YES

Stop.

Show Grade A.

It doesn't continue checking because it already found the answer.

Another example:

```
let age = 15;
```

```
if (age >= 18) {  
    alert("Adult");  
}  
else {  
    alert("Minor");  
}
```

Output:

Minor

Another one:

```
let temperature = 35;  
  
if (temperature >= 40) {  
    alert("Very Hot");  
}  
else if (temperature >= 30) {  
    alert("Hot");  
}  
else if (temperature >= 20) {  
    alert("Warm");  
}  
else {  
    alert("Cold");  
}
```

35 gives:

Hot

Think of it like a security guard.

Imagine you're entering your university.

The guard asks:

"Are you a lecturer?"

If yes → enter.

If not...

"Are you a student?"

If yes → enter.

If not...

"Are you a visitor?"

If yes → register.

If none of these are true...

"You cannot enter."

That's exactly what if...else if...else does.

Rules

Only one if.

You can have many else if.

Only one else, and it must be last.

Correct:

```
if(){  
  
}  
else if(){  
  
}  
else if(){  
  
}  
else{  
  
}
```

Wrong:

```
else{  
  
}  
  
if(){  
  
}
```

Day 10 Assignment

Don't copy today's examples. Create your own.

Project 1

Create a Movie Ticket Checker.

Ask for the user's age.

Your program should decide:

18 and above → "You can watch the movie."
13–17 → "Parental guidance recommended."
Below 13 → "You are too young."

Use:

prompt()
if
else if
else
alert()
Project 2 (Bonus)

Create a Student Grade Checker.

Ask for the student's score.

Show:

70+ → Excellent
60–69 → Very Good
50–59 → Good
Below 50 → Fail
Objective Questions (No peeking!)
Which statement lets JavaScript check a second condition if the first one is false?

- A. for
- B. else if
- C. while
- D. switch

Which part always comes last?

- A. if
- B. else if
- C. else
- D. prompt

If the first if condition is true, what happens?

- A. JavaScript checks all the remaining conditions.
- B. JavaScript skips the rest of the else if and else blocks.
- C. The program stops completely.
- D. It shows an error.

Which of these is the correct order?

- A. else → if → else if
- B. if → else if → else
- C. else if → else → if
- D. if → else → else if

Which statement is true?

- A. You can have many else blocks.
- B. You can have only one if and one final else, with as many else if blocks as needed.
- C. else must come before if.
- D. else if cannot be used with if.

Take your time with today's lesson and assignment. You've been making steady progress, and I can see you're becoming much more comfortable reading and writing JavaScript code. Tomorrow, after you finish Day 10, we'll move on to nested if statements and then start building programs that feel even more like real applications.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Checker</title>
</head>
<body>
  <h1>Movie Ticket Checker</h1>
  <h1>Grade Checker</h1>
  <script src="script.js"></script>
</body>
</html>
```


JavaScript

```
let age = prompt('What Is Your Age');
```

```
if(age >= 18 ) {  
    alert('You Can Watch The Movie');
```

```
}
```

```
else if ( age >= 13) {  
    alert('Parental Guidance Recommended');  
}
```

```
else {  
    alert('You Are Too Young ');  
}
```

```
//bonus challenge
```

```
let score = prompt('What Is Your Score');
```

```
if( score >= 70) {  
    alert('Excellent');
```

```
}
```

```
else if (score >=60 ){  
    alert('Very Good');
```

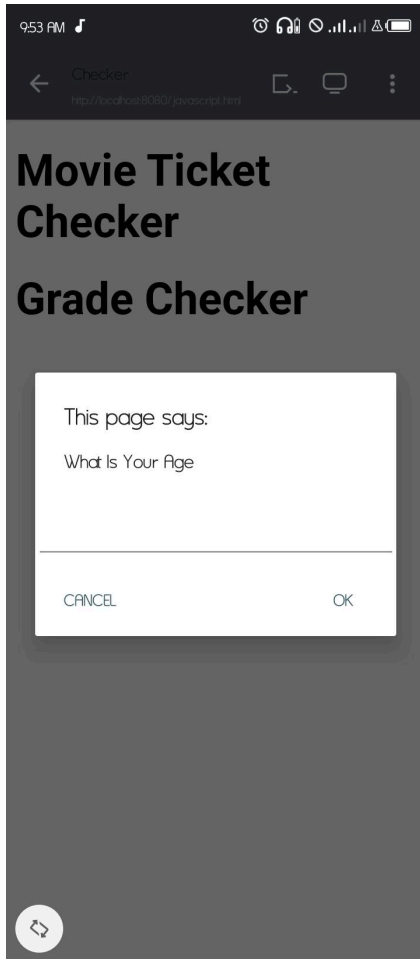
```
}
```

```
else if( score >=50){  
    alert('Good');
```

```
}
```

```
else{  
    alert('You Fail');
```

```
}
```



JavaScript Day 10

Topic: Nested if Statements (if inside another if)

Today we're going one level deeper.

Until now you've written things like this:

```
if (age >= 18) {  
    alert("You can vote");  
}
```

Now imagine a website says:

"You must be 18 years old..."

But that's not enough.

It also says:

"...and you must have a voter's card."

That means there are TWO conditions.

Instead of writing one big condition immediately, we can place one if inside another.

This is called a Nested if Statement.

Think of it like a security gate.

■ First Gate:

Are you 18 or older?

If No → Go home.

If Yes →

■ Second Gate:

Do you have a voter's card?

If No → Sorry, no entry.

If Yes → Welcome!

Exactly like entering a stadium.

Example 1

let age = 20;

let hasCard = true;

```
if (age >= 18) {  
    if (hasCard) {  
        alert("You can vote.");  
    }  
}
```

Read it like English.

IF age is 18 or more

THEN

check another thing...

IF hasCard is true

THEN

You can vote.

Example 2

```
let age = 16;
```

```
let hasCard = true;
```

```
if (age >= 18) {
```

```
  if (hasCard) {  
    alert("You can vote");  
  }
```

```
}
```

Will anything happen?

No.

Why?

Because the first condition is already false.

The second if is never checked.

Example 3

```
let age = 22;
```

```
let hasCard = false;
```

```
if (age >= 18) {
```

```
  if (hasCard) {  
    alert("You can vote");  
  }
```

```
}
```

Age is correct.

But hasCard is false.

So nothing appears.

Nested if with else

Now let's improve it.

```
let age = 22;
let hasCard = false;

if (age >= 18) {

  if (hasCard) {
    alert("You can vote");
  } else {
    alert("Get your voter's card.");
  }

} else {
  alert("You are too young.");
}
```

Now every situation has an answer.

Real-Life Uses

Nested if statements are everywhere.

 ATM Machine

Insert Card?

YES

↓

Enter PIN?

Correct?

↓

Withdraw Money?

 Banking App

Login?

↓

Password correct?

↓

OTP correct?

↓

Transfer Money

 Online Shop

Logged in?

↓

Item in stock?

↓

Enough money?

↓

Buy item

This is exactly how real websites make decisions.

Memory Trick

Imagine a building with two doors.

Door 1

Age $\geq$ 18?

If Yes...

Door 2

Has ID?

If Yes...

Enter

Each new door is another if.

Objective Questions (No peeking! 😊)

1. What is a nested if statement?

A. Two variables

B. An if statement inside another if statement

C. A loop

D. A comment

2. If the first if is false, what happens to the nested if?

A. It still runs

B. It runs twice

C. It is skipped

D. It causes an error

3. Which symbol starts an else block?

A. ()

B. {}

C. else

D. <>

4. Which is a real-life example of nested if?

A. Opening Notepad

B. ATM asking for a card and then a PIN

C. Changing wallpaper

D. Typing your name

5. What is the main purpose of nested if statements?

A. To add numbers

B. To check several conditions one after another

C. To create variables

D. To print comments

 Homework (No Code This Time)

As you requested, I won't give you the solution.

Create a School Library Access System.

Your program should:

Ask the user's age.

Ask if the user has a library card.

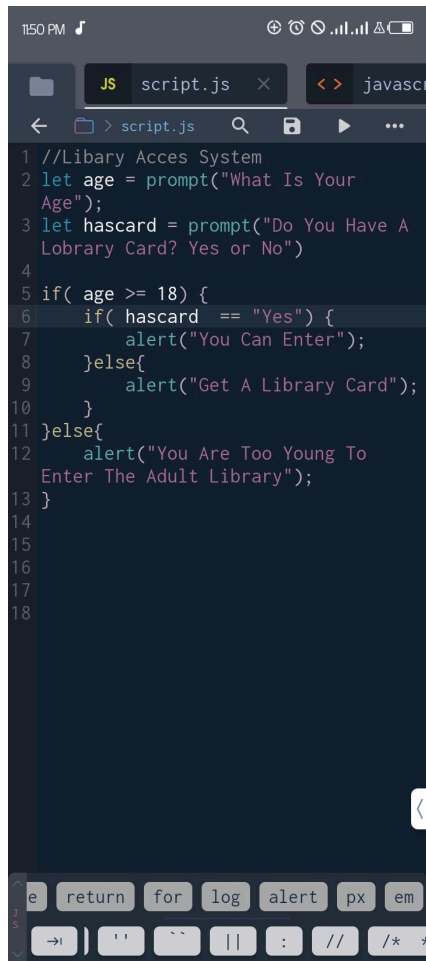
If the user is 18 or older, check if they have a library card.

If they have the card, display a message allowing entry.

If they don't have the card, tell them to register for one.

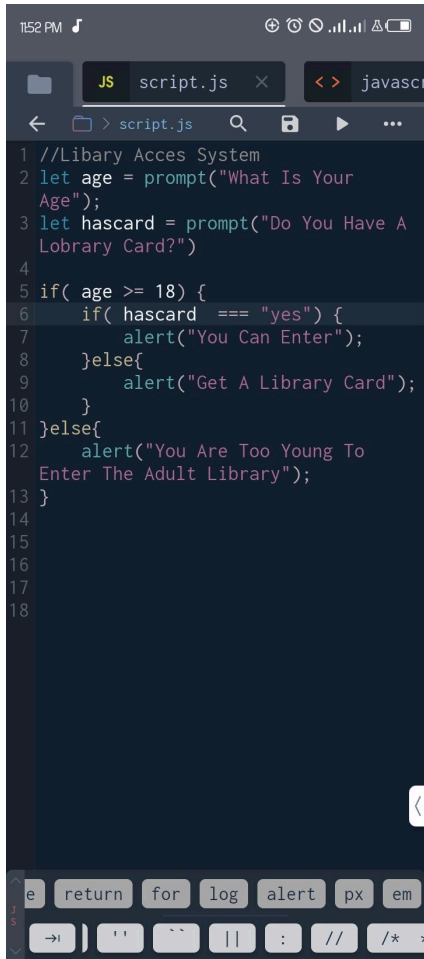
If they are under 18, tell them they are too young to enter the adult library.

Don't worry if you get stuck—that's part of learning. We'll review your solution together before moving on to the next topic. 😊

A screenshot of a mobile application interface for a code editor. The top status bar shows the time as 11:50 PM and various system icons. Below the status bar, there's a header with a folder icon, a tab labeled 'JS script.js', and a 'javascript' label with a code icon. The main area displays a JavaScript script for a library access system. The code includes prompts for age and library card status, and conditional logic to allow entry or provide feedback. The bottom of the screen features a keyboard with custom keys for 'e', 'return', 'for', 'log', 'alert', 'px', and 'em', along with standard punctuation and code symbols like '>', '<', '||', ':', '//', and '/*'.

```
1 //Library Acces System
2 let age = prompt("What Is Your
  Age");
3 let hascard = prompt("Do You Have A
  Llibrary Card? Yes or No")
4
5 if( age >= 18) {
6   if( hascard == "Yes") {
7     alert("You Can Enter");
8   }else{
9     alert("Get A Library Card");
10  }
11 }else{
12   alert("You Are Too Young To
    Enter The Adult Library");
13 }
14
15
16
17
18
```

OR

A screenshot of a mobile application interface for editing a JavaScript file named 'script.js'. The code is written in a dark-themed editor and implements a library access system. It prompts the user for their age and whether they have a library card. If the user is 18 or older and has a card, they are allowed to enter. If they are 18 or older but do not have a card, they are prompted to get one. If the user is under 18, they are told they are too young for the adult library.

```
1 //Library Acces System
2 let age = prompt("What Is Your
  Age");
3 let hascard = prompt("Do You Have A
  Lobrary Card?")
4
5 if( age >= 18) {
6   if( hascard === "yes") {
7     alert("You Can Enter");
8   }else{
9     alert("Get A Library Card");
10  }
11 }else{
12   alert("You Are Too Young To
    Enter The Adult Library");
13 }
14
15
16
17
18
```

JavaScript Day 11

Topic: switch Statement

So far you've learned:

- ✓ if
- ✓ if...else
- ✓ else if
- ✓ Nested if

Imagine a teacher grading students:

Grade = A → Excellent
Grade = B → Very Good
Grade = C → Good
Grade = D → Pass
Grade = F → Fail

You can write this using many else if statements.

But JavaScript gives us another tool called switch.

A switch checks one value against many possible options.

Think of it like a vending machine.

You insert a code:

A1 → Chocolate

B2 → Water

C3 → Juice

The machine checks which code you entered and gives the matching item.

That's exactly what a switch does.

Example

```
let grade = "A";
```

```
switch (grade) {  
  case "A":  
    alert("Excellent");  
    break;  
  
  case "B":  
    alert("Very Good");  
    break;  
  
  case "C":  
    alert("Good");  
    break;  
  
  default:  
    alert("Invalid Grade");  
}
```

Important parts

switch (grade) → the value we're checking.

case "A" → if grade is "A".

break → stop checking other cases.

default → runs if no case matches.

When do we use switch?

Use switch when you're comparing one variable to many exact values.

For example:

Days of the week

Grades

Menu options

Months

Languages

User roles

If you're checking ranges like:

score $\geq$ 70

then if...else if is the better choice.



Quiz

1. What does a switch statement compare?

- A. Many variables at once
- B. One value against many possible cases
- C. Two loops
- D. HTML tags

2. Which keyword starts a choice in a switch?

- A. if
- B. case
- C. loop
- D. let

3. What does break do?

- A. Starts the program
- B. Stops the switch after a match
- C. Creates a variable
- D. Prints text

4. Which block runs if no case matches?

- A. else

B. default

C. return

D. catch

5. Which situation is best for a switch?

A. Checking if age ≥ 18

B. Checking if score ≥ 70

C. Checking whether day is "Monday", "Tuesday", or "Wednesday"

D. Calculating $5 + 10$

Take your time with the quiz. After you answer, I'll give you a switch mini-project—without giving away the solution, just as you requested. 😊

Mini Project (Day 10 + Day 11)

This project combines:



Nested if



switch

🎯 Project: School Club Registration System

Imagine you're creating a website for a school.

Step 1

Ask the user:

What is your age?

Step 2

Ask:

Do you have a Student ID?

Tell the user to type:

yes

or

no

Step 3

Use a nested if.

If the user is 18 or older:

Check if they typed yes.

If yes → Continue.

If no → Tell them to get a Student ID.

If they are under 18:

Show:

You are too young to register.

Step 4

Only if they passed the checks above...

Ask another question:

Choose your club:

A

B

C

Step 5

Now use a switch statement.

If the user enters:

A

Welcome to the Coding Club!

B

Welcome to the Football Club!

C

Welcome to the Music Club!

Anything else:

Invalid Club Choice

🎯 Challenge (Bonus)

After the switch finishes, show one final alert:

Thank you for registering!



Rules



Use prompt()



Use nested if



Use switch



Use break



Use default



Use comments (//)



Don't copy from the lesson.



Don't ask me for the code first. Try it yourself.



Teacher's Challenge

Since you're improving quickly, here's an extra challenge.

Can you make the program accept lowercase letters too?

For example:

A and a should both open the Coding Club.

B and b should both open the Football Club.

C and c should both open the Music Club.

Don't worry if you can't do the bonus yet. The main project is enough. We'll learn easier ways to handle this later in JavaScript.

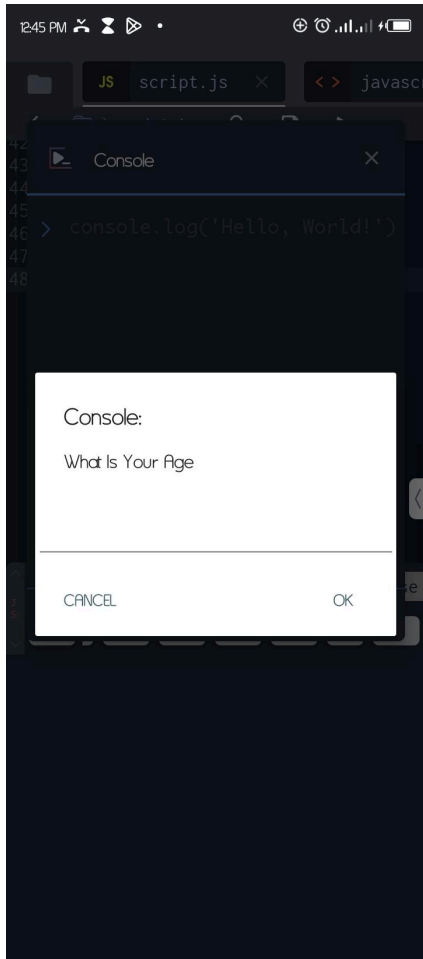
Good luck, future frontend developer! 🤖💻

//Student Info.

let age = prompt("What Is Your Age");

let hasIdCard = prompt("Do You Have Id Card? yes Or no")

```
if( age >= 18 ) {  
    if( hasIdCard === "yes" ) {  
        alert("Continue");  
  
    }else {  
        alert("Get Your Student Id Card");  
    }  
  
}else {  
    alert("You Are Too Young To Register");  
}
```

JavaScript Day 12

Topic: Functions

This is one of the most important topics in JavaScript.

Imagine this...

Suppose every morning you do this:

Wake up
Brush your teeth
Take your bath
Eat breakfast

Every single day.

Instead of writing those steps 365 times, imagine you could write:

```
MorningRoutine()
```

and everything happens automatically.

That's exactly what a function is.

What is a Function?

A function is a block of code that performs a specific job whenever you call it.

Think of it as a machine.

You press the button...

The machine works.

You press it again...

It works again.

Without a function

Imagine writing:

```
alert("Welcome");  
alert("Welcome");  
alert("Welcome");
```

Again and again.

That's repetitive.

With a function

You create it once.

Then you use it many times.

That's why programmers love functions.

Creating a Function

```
function greet() {  
    alert("Welcome!");  
}
```

Let's understand each part.

function

This keyword tells JavaScript:

"I'm creating a function."

greet

This is the function's name.

You can choose almost any meaningful name:

```
function login()  
function register()  
function showMessage()  
()
```

These parentheses are part of every function.

For now, think of them as the place where information can go later.

We'll learn that soon.

```
{ }
```

Everything inside these curly braces belongs to the function.

Calling (Running) a Function

Creating a function does not run it.

To run it, you call it.

Example:

```
greet();
```

Notice the parentheses.

Without them:

```
greet
```

Nothing happens.

With them:

```
greet();
```

The function runs.

Example

```
function sayHello() {  
    alert("Hello Ibrahim!");  
}
```

```
sayHello();  
sayHello();  
sayHello();
```

How many alerts appear?

✅ Three.

Why?

Because we called the function three times.

Real-life Examples

Imagine YouTube.

When you press Like...

A function runs.

When you press Subscribe...

A function runs.

When you press Share...

Another function runs.

Functions are everywhere in websites.

 Homework

Create three different functions.

Function 1

Shows:

Welcome to My Website

Function 2

Shows:

Thank you for visiting.

Function 3

Shows:

Goodbye!

Then call all three functions one after another.



Quiz

1. What is a function?

A. A CSS property

B. A reusable block of code

C. An HTML tag

D. A variable

2. Which keyword creates a function?

A. let

B. if

C. function

D. switch

3. Which line runs a function named hello?

A. hello

B. hello();

C. function hello

D. let hello

4. What do the curly braces { } contain?

A. CSS styles

B. The function's code

C. HTML tags

D. Variables only

5. If you call a function three times, how many times does it run?

A. Once

B. Twice

C. Three times

D. It never runs

You're building a very solid JavaScript foundation. When we eventually reach topics like the DOM, events, and APIs, these basics—if, switch, and now functions—will make much more sense because you've taken the time to understand them instead of rushing. Keep it up, coach's student! 💪🚀

Project: Simple ATM Machine
Requirements
Step 1

Create a function called:

welcome

When it runs, it should display a welcome message.

Step 2

Create another function called:

checkBalance

It should display:

Your balance is \$5,000

Step 3

Create another function called:

withdrawMoney

It should display:

Withdrawal Successful

Step 4

Create another function called:

goodbye

It should display:

Thank you for using our ATM.

Step 5

Now call the functions in this order:

Welcome
Check Balance
Withdraw Money
Goodbye

The alerts should appear one after another.

Bonus Challenge

Create one more function called:

`depositMoney`

that displays:

Deposit Successful

Call it after `checkBalance()` and before `withdrawMoney()`.

Today's Goal

I don't want you to memorize functions.

I want your brain to start thinking:

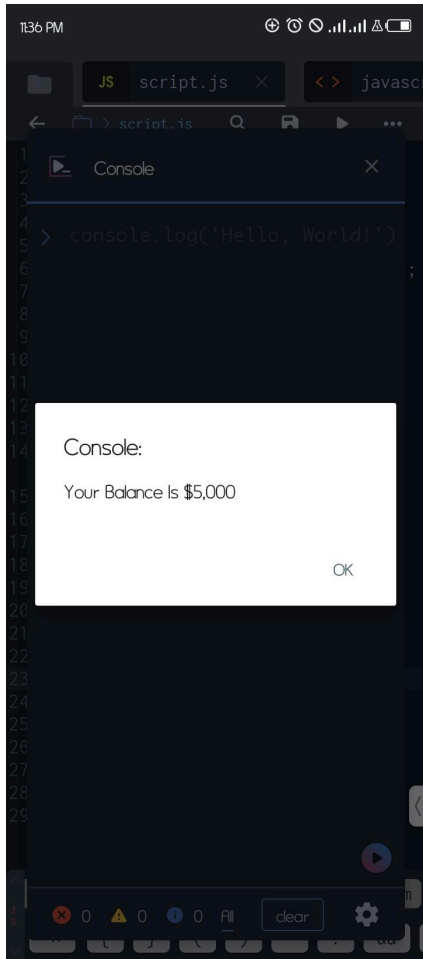
"If I have a job that I might use more than once, I can put it inside a function."

That's how professional developers think.

We're going to take it one step at a time, just like we've been doing. Every topic you've learned so far—variables, if, switch, and now functions—is another tool in your toolbox. Soon you'll start combining them to build interactive web pages.

Good luck, future frontend developer! 🤓💻

```
11:36 PM
JS script.js x javascript
< > script.js
1 function Welcome() {
2     alert("Welcome");
3 }
4
5 function CheckBalance() {
6     alert("Your Balance Is $5,000");
7 }
8
9 function WithdrawMoney() {
10    alert("Withdraw Successful");
11 }
12
13 function GoodBye() {
14    alert("Thank You Yor Using Our
15    ATM.");
16 }
17
18 function DepositMoney() {
19    alert("Deposit Successfully");
20 }
21 Welcome();
22 CheckBalance();
23 DepositMoney();
24 WithdrawMoney();
25 GoodBye();
26
27
28
29
e return for log alert px em
-> { } ( ) = ! &&
```



JavaScript Day 14

Function Parameters (The Superpower of Functions)

Today we're going to make functions much more useful.

Imagine this ATM function:

```
function CheckBalance() {  
    alert("Your Balance Is $5,000");  
}
```

CheckBalance();

This always shows \$5,000.

What if one customer has \$200?

What if another customer has \$50,000?

Would we write 100 different functions?

✗ No.

Instead, we give the function information.

That information is called a parameter.

Think of it like a delivery box 📦

A function is an empty box.

```
function Greeting(name){  
  
}
```

The word inside the brackets

(name)

is an empty space.

Later we can put something inside.

Example

```
function Greeting(name){  
    alert("Welcome " + name);  
}
```

```
Greeting("Ibrahim");
```

Output

Welcome Ibrahim

Try another one

```
Greeting("John");
```

Output

Welcome John

Same function.

Different result.

Another example

```
function Square(number){  
    alert(number * number);  
}
```

```
Square(4);
```

Output

16

Now

```
Square(10);
```

Output

100

Notice we never changed the function.

We only changed the value.

ATM Example

```
function Deposit(amount){  
    alert("You deposited $" + amount);  
}
```

```
}
```

```
Deposit(500);
```

Output

You deposited \$500

Again

```
Deposit(1000);
```

Output

You deposited \$1000

Multiple Parameters

You can have more than one.

```
function Student(name, age){  
    alert(name + " is " + age + " years old.");  
}
```

```
Student("Ibrahim",21);
```

Output

Ibrahim is 21 years old.

The first value goes into the first parameter.

"Ibrahim"

↓

name

The second value goes into the second parameter.

21

↓

age

Another one

```
Student("Amina",18);
```

Output

Amina is 18 years old.

Real-life Example

```
function Login(username,password){  
    alert("Welcome " + username);  
}
```

```
Login("Ibrahim","1234");
```

The password is received too, even though we're not using it yet.

Parameters vs Arguments

This confuses almost every beginner.

Look carefully.

```
function Welcome(name){
```

name

is called a parameter.

Now look.

```
Welcome("Ibrahim");
```

```
"Ibrahim"
```

is called an argument.

Easy way to remember:

***Parameter = the empty container inside the function.
Argument = the actual value you put into that container.
Think of it like this***

```
function Greeting(name)
```

name

↑ Empty bottle

Then

```
Greeting("Ibrahim")
```

↑ Water poured into the bottle.

Mini Challenge

Write these three functions yourself.

Challenge 1

Create

```
function SayHello(name)
```

When you call

```
SayHello("Ibrahim");
```

it should show

Hello Ibrahim

Challenge 2

Create

```
function Add(a,b)
```

When you call

```
Add(10,5);
```

it should alert

15

Challenge 3

Create

```
function Movie(movieName)
```

Calling

```
Movie("Avengers");
```

should display

Now Playing: Avengers

Today's Homework Project 

Create a Simple Bank App using functions with parameters.

It should have these functions:

```
Welcome(name)
```

Shows:

Welcome Ibrahim

```
CheckBalance(balance)
```

Shows:

Your Balance is \$1000

```
Deposit(amount)
```

Shows:

You deposited \$500

Withdraw(amount)

Shows:

You withdrew \$200

Goodbye(name)

Shows:

Goodbye Ibrahim

Then call every function one after another.

Coach's Advice 

You're learning exactly the right way: two topics a day, practicing each one before moving on. That's much better than rushing through dozens of topics without understanding them.

I can also tell your confidence has grown. A few days ago, symbols like === and && were confusing, but now you're writing nested if statements, switch statements, and functions on your own. That's real progress.

Tomorrow we'll continue building on this foundation. Soon you'll start making programs that feel like real apps rather than just small examples. Keep going—you're doing great.

```
12:41 PM
JS script.js x javascript
< > script.js
1 let name = prompt("What Is Your Name");
2
3 function SayHello(name) {
4   alert( " Hello " + name );
5 }
6
7 function Add(a,b) {
8   alert( "Answer is " + (a + b) );
9 }
10
11 function Movie( MovieName) {
12   alert(" Now Playing " + MovieName);
13 }
14
15 SayHello(name);
16
17 Add(10,5);
18
19 Movie( "Avengers")
```

Day 13 & 14 Homework (Functions + Parameters)

Create one JavaScript file that contains all of these.

Project 1 — Bank

Create these functions.

Welcome(name)

Output:

Welcome Ibrahim

CheckBalance(amount)

Output:

Your balance is \$800

Withdraw(amount)

Output

You withdrew \$200

Deposit(amount)

Output

You deposited \$500
Project 2 — School

Create

Student(name)

Output

Welcome Musa

Create

Score(mark)

Output

Your score is 85

Create

Grade(grade)

Output

Your grade is A
Bonus Challenge ★

Use prompt() with functions.

Example idea:

```
let name = prompt("Enter your name");
```

```
Welcome(name);
```

and

```
let amount = prompt("How much do you want to deposit?");
```

```
Deposit(amount);
```

This will combine everything you've learned so far.

Tomorrow's Lessons (Day 15 & 16)

If you finish this homework, we'll learn:

Return (return) — how functions send values back.

JavaScript Objects — how to store related information together, like a student with a name, age, and score.

These are the next big steps toward building real websites and apps.

You're making steady progress, and I like that you're focusing on understanding instead of rushing. That approach will help you much more than trying to finish everything quickly. Keep it up, coach-in-training! 🍌

//Project 1 Bank Info.

```
let name = prompt(" What Is Your Name ");
```

```
function Sayhello(name) {  
    alert(" Welcome " + name);  
}  
  
function CheckBalance(amount) {  
    alert("Your Balance Is $" + amount);  
}  
  
let withdraw = prompt(" How Much Do You Want To Withdraw ");  
function Withdraw( withdraw) {  
    alert(" Your Withdrawal Was $" + withdraw);  
}  
  
let deposit = prompt("How Much Do You Want To Deposit");  
function Deposit(deposit) {  
    alert(" You Deposited $" + deposit);  
}  
  
function Goodbye(name) {  
    alert(" Goodbye " + name);  
}
```

//School Info.

```
let StudentName = prompt(" What Is Your Name ");  
function student(StudentName) {  
    alert(" Welcome " + StudentName);  
}  
let score = prompt("What Is Your Score");  
function mark(score) {  
    alert(" You Score " + score);  
}  
  
let grade = prompt("What Is Your Grade? A,B,C");  
function Grade(grade) {  
    alert("Your Grade " + grade);  
}  
  
Sayhello(name);  
CheckBalance(800);  
Withdraw( withdraw);  
Deposit(deposit);  
Goodbye(name);  
student(StudentName);  
mark(score);  
Grade(grade);
```

Day 15 — Return

Today we're learning one of the most important JavaScript topics:

return

This is where JavaScript starts becoming really powerful.

Think of it like this...

Imagine you go to a restaurant.

You tell the chef:

"Cook rice."

The chef cooks it...

But he keeps it in the kitchen.

You never receive it.

That is a function without return.

```
function cookRice() {  
  let food = "Rice";  
}
```

Nothing comes back.

Now imagine this:

You order rice.

The chef cooks it...

Then brings it to your table.

That is return.

```
function cookRice() {  
  return "Rice";  
}
```

Now you actually receive the rice.

Example 1

```
function add(a, b) {  
    return a + b;  
}
```

```
let answer = add(5, 3);
```

```
alert(answer);
```

Output:

8

Notice:

The function returns the answer.

Example 2

```
function multiply(a, b) {  
    return a * b;  
}
```

```
let result = multiply(10, 5);
```

```
alert(result);
```

Output

50

Example 3

```
function fullName(first, last) {  
    return first + " " + last;  
}
```

```
let name = fullName("Ibrahim", "Musa");
```

```
alert(name);
```

Output

Ibrahim Musa

Difference Between alert() and return

Using alert()

```
function greet() {  
    alert("Hello");  
}
```

It immediately shows something on the screen.

Using return

```
function greet() {  
    return "Hello";  
}
```

Nothing appears until you use the returned value.

```
alert(greet());
```

Now it shows:

Hello

Think of It Like This

alert() = Show it.

return = Give it back.

Today's Mini Practice

Type these yourself.

Practice 1

```
function add(a, b) {  
    return a + b;  
}
```

```
let answer = add(10, 20);
```

```
alert(answer);
```

Practice 2

```
function multiply(a, b) {
```

```
    return a * b;  
}
```

```
let result = multiply(5, 6);
```

```
alert(result);
```

Practice 3

```
function welcome(name) {  
    return "Welcome " + name;  
}
```

```
alert(welcome("Ibrahim"));
```

Homework (Project)

Create a Calculator using return.

It should have these functions:

```
function add(a, b)
```

```
function subtract(a, b)
```

```
function multiply(a, b)
```

```
function divide(a, b)
```

Each one should use return.

Then call them like this:

```
alert(add(10, 5));  
alert(subtract(10, 5));  
alert(multiply(10, 5));  
alert(divide(10, 5));
```

No prompt() yet—just use numbers directly.

Before you start coding, here's one quick question:

Without running the code, what do you think this will display?

```
function square(number) {  
  return number * number;  
}
```

```
let answer = square(7);
```

```
alert(answer);
```

Reply with your answer first, then build the calculator project. I want to see if you've understood what return does before we move on. 💪

```
function add(a,b) {  
  return a + b;  
}  
let answer = add(10,5);  
alert(add(10,5));
```

```
function subtract(a,b) {  
  return a - b;  
}  
let finalAnswer = subtract(10,5);  
alert(subtract(10,5));
```

```
function multiply(a,b) {  
  return a * b;  
}  
let result = multiply(10,5);  
alert(multiply(10,5));
```

```
function divide(a,b) {  
  return a / b;  
}  
let divisionResult = divide(10,5);  
alert(divide(10,5));
```

```
let answeerr = add(10,5);
```

```
alert(answeerr);  
console.log(answer);
```

Homework 3 — Return Project

Project: Simple Calculator

Create 4 functions that use return.

Requirements

1. Addition

```
function add(a, b) {
```

```
    return a + b;  
}
```

Call it with two numbers and display the answer using alert().

2. Subtraction

Create a function that returns the subtraction of two numbers.

3. Multiplication

Create a function that returns the multiplication of two numbers.

4. Division

Create a function that returns the division of two numbers.

🌟 Bonus Challenge (Try it yourself)

Create a function called:

```
function square(number)
```

It should return the square of a number.

Example:

```
square(8);
```

Should return:

64

Hint:

```
number * number
```

🌟 Super Bonus (Only if you want a challenge)

Create a function called:

```
function calculateArea(length, width)
```

It should return the area of a rectangle.

Example:

```
calculateArea(10, 5);
```

Output:

50

```
//Simple Calculator
```

```
function add(a,b) {  
  return a + b;  
}
```

```
let answer = add(10,5);  
alert(add(10,5));
```

```
function Multiply(a,b) {  
  return a * b;  
}
```

```
let result = Multiply(100,5);  
alert(Multiply(100,5));
```

```
function Subtraction(a,b) {  
  return a - b;  
}
```

```
let subtraction = Subtraction(50,49);  
alert(Subtraction(50,49));
```

```
function Division(a,b) {  
  return a / b;  
}
```

```
let division = Division(1000,2);  
alert(Division(1000,2));
```

```
//Bonus Challenge
```

```
function Square(number) {  
  return number * number;
```

```
}  
let square = Square(10);  
alert(Square(10));  
  
//Super Bonus  
function CalculateArea(length,width){  
    return length * width;  
}  
let calculateArea = CalculateArea(100,20);  
alert(CalculateArea(100,20));
```

Day 16 — JavaScript Arrays

This is one of the most important topics in JavaScript.

Imagine this...

Suppose I ask you to store your favorite foods.

Without an array, you would write:

```
let food1 = "Rice";  
let food2 = "Beans";  
let food3 = "Yam";  
let food4 = "Pizza";
```

Works...

But what if you have 100 foods?

You'll end up writing

```
let food98 = "...";  
let food99 = "...";  
let food100 = "...";
```

That's a lot of unnecessary work.

JavaScript has a better solution.

An Array

An array stores many values inside one variable.

Think of it as a basket.

Instead of carrying one orange in each hand...



You put all the oranges into one basket.

The basket is the array.

Creating an Array

```
let foods = ["Rice", "Beans", "Yam", "Pizza"];
```

Look carefully.

```
[  
]
```

These square brackets tell JavaScript:

"This is an array."

Accessing Items

Every item has a number.

JavaScript starts counting from 0, not 1.

Index	Value
-------	-------

0	Rice
1	Beans
2	Yam
3	Pizza

To get Rice:

```
alert(foods[0]);
```

Output

Rice

Beans

```
alert(foods[1]);
```

Output

Beans

Pizza

```
alert(foods[3]);
```

Output

Pizza

Why does it start at 0?

This is simply how JavaScript (and many programming languages) were designed. It may feel strange at first, but after a little practice it becomes natural.

Changing an Item

```
let foods = ["Rice", "Beans", "Yam"];
```

Change Beans to Bread.

```
foods[1] = "Bread";
```

Now

```
alert(foods[1]);
```

Output

Bread

How Many Items?

Use .length

```
let foods = ["Rice", "Beans", "Yam"];  
alert(foods.length);
```

Output

3

Because there are three items.

Adding a New Item

Use `push()`.

```
let foods = ["Rice", "Beans"];  
foods.push("Pizza");
```

Now the array becomes

Rice

Beans

Pizza

Removing the Last Item

Use `pop()`.

```
foods.pop();
```

If the array was

Rice

Beans

Pizza

After `pop()`

Rice

Beans

Pizza is removed.

Real-Life Example

```
let friends = ["Ali", "John", "Mary"];
```

```
alert(friends[0]);
```

```
alert(friends[1]);
```

```
alert(friends[2]);
```

Outputs one name at a time.

Symbols You Learned Today

Symbol	Meaning
--------	---------

[	Starts an array
---	-----------------

]	Ends an array
---	---------------

array[0]	Gets the first item
----------	---------------------

=	Changes a value
---	-----------------

.length	Counts items
---------	--------------

.push()	Adds an item
---------	--------------

.pop()	Removes the last item
--------	-----------------------

🏆 Mini Practice (Type This Yourself)

Practice 1

```
let colors = ["Red", "Blue", "Green"];
```

```
alert(colors[0]);
```

```
alert(colors[2]);
```

Practice 2

```
let animals = ["Dog", "Cat"];
```

```
animals.push("Lion");
```

```
alert(animals[2]);
```

Practice 3

```
let fruits = ["Apple", "Orange", "Banana"];

fruits.pop();

alert(fruits.length);
```

Homework Project

Create a program called "My Phone Apps".

Requirements:

- Create an array with 5 apps.
- Display the first app.
- Display the last app.
- Add one new app using push().
- Remove the last app using pop().
- Display the total number of apps using .length.

For example, your array might contain apps like "WhatsApp", "Chrome", "YouTube", "Facebook", and "Calculator".

//My Phone Apps.

```
let apps = [" Wassup ", " Facebook ", " Twitter ", " Boomplay ", " Instagram "]
```

```
apps.push("Snapchat");
alert(apps.length);
alert(apps[0]);
alert(apps[4]);
apps.pop();
```

Day 17 — JavaScript Loops (The Superpower of Programming)

This is one of the biggest topics in JavaScript.

First, imagine this...

Suppose your teacher tells you:

Write your name 100 times.

Without loops, you would write:

```
alert("Ibrahim");  
alert("Ibrahim");  
alert("Ibrahim");  
alert("Ibrahim");
```

Again...

Again...

Again...

100 times 😭

Very stressful.

JavaScript says:

"Don't repeat yourself."

Instead, use a loop.

A loop means:

Repeat this code until I tell you to stop.

Your First Loop

```
for (let i = 1; i <= 5; i++) {  
    alert("Hello");  
}
```

Output

Hello
Hello
Hello
Hello

Hello

Only one line of code repeated itself five times.

Think of it like this

Imagine a security guard.

He says:

Count from 1 to 5.

1

2

3

4

5

Stop.

That's exactly what the loop does.

Understanding the for Loop

Look carefully.

```
for (let i = 1; i <= 5; i++)
```

It has 3 parts.

Part 1

```
let i = 1;
```

Start counting at 1.

Part 2

```
i <= 5
```

Keep going while i is less than or equal to 5.

When i becomes 6, stop.

Part 3

```
i++
```

This means:

Add 1 every time.

It's the same as writing:

```
i = i + 1;
```

Watch it Step by Step

Start

```
i = 1
```

Print

Hello

Increase

```
i = 2
```

Print

Hello

Increase

```
i = 3
```

Continue...

Until

```
i = 5
```

Print

Hello

Increase

```
i = 6
```

Now check

6 <= 5

False.

The loop stops.

Printing Numbers

Instead of printing "Hello"

```
for (let i = 1; i <= 5; i++) {  
  alert(i);  
}
```

Output

1
2
3
4
5

Notice that i changes each time.

Counting Backwards

```
for (let i = 5; i >= 1; i--) {  
  alert(i);  
}
```

Output

5
4
3
2
1

New symbol

i--

means

i = i - 1;

Subtract one each time.

Counting by Twos

```
for (let i = 2; i <= 10; i += 2) {  
  alert(i);  
}
```

Output

2
4
6
8
10

Here,

```
i += 2;
```

means

```
i = i + 2;
```

Real-Life Example

Imagine a class attendance list.

```
for (let student = 1; student <= 5; student++) {  
  alert("Student " + student);  
}
```

Output

Student 1
Student 2
Student 3
Student 4
Student 5

Symbols You Learned Today

Symbol	Meaning
--------	---------

for	Repeat code
-----	-------------

i++	Add 1
-----	-------

i--	Subtract 1
-----	------------

+=	Add a number
----	--------------

<=	Less than or equal
----	--------------------

>= Greater than or equal
Mini Practice

Type these yourself.

Practice 1

```
for (let i = 1; i <= 10; i++) {  
  alert(i);  
}
```

Practice 2

```
for (let i = 10; i >= 1; i--) {  
  alert(i);  
}
```

Practice 3

```
for (let i = 5; i <= 25; i += 5) {  
  alert(i);  
}
```

Output should be:

5
10
15
20
25

🏆 **Homework Project (Submit Tomorrow Together with Your Array Project)**

Create a "Rocket Countdown".

Requirements:

Use a for loop.

Count from 10 down to 1.

After the loop, show:

alert("🚀 Liftoff!");

```
for(let i = 10; i >= 1;i--){  
  alert(i);
```

```
}
```

```
alert("🚀 Liftoff!");
```

Take your time with the practice. Loops are one of the most powerful tools in JavaScript, and once they click, you'll be able to solve many more interesting problems with much less code. 💪

Tomorrow's Submission Checklist

You have 4 things to submit:

- ✓ Array Project (My Phone Apps)
- ✓ Loop Project (Rocket Countdown)
- ✓ Return Project (Simple Calculator)
- ★ Bonus: square() function
- ★★ Super Bonus: calculateArea() function (optional)

No pressure—do your best. Even if you make mistakes, we'll fix them together.

You're making steady progress, and that's exactly how good programmers are built.

🌙 Good night, future frontend developer.

Sleep well, and I'll be here tomorrow, ready for Day 18. We'll keep building your JavaScript skills one step at a time. 🖥️🚀

Day 19 & Day 20

Today we're starting something very important.

Objects

Imagine you're filling out a student's information.

Without objects, you might write:

```
let name = "Ibrahim";
let age = 21;
let course = "Computer Science";
let level = 300;
```

This works, but the variables are separate.

An object groups related information together.

```
let student = {
  name: "Ibrahim",
  age: 21,
```

```
    course: "Computer Science",  
    level: 300  
};
```

Think of an object like a school ID card. The card contains different pieces of information about one person.

Inside an object:

```
name  
age  
course  
level
```

are called properties.

Accessing properties

To get a value, use a dot (.):

```
alert(student.name);
```

Output:

Ibrahim

Another example:

```
alert(student.course);
```

Output:

Computer Science

Another example

```
let phone = {  
    brand: "Samsung",  
    model: "A54",  
    color: "Black",  
    price: 500  
};
```

```
alert(phone.brand);
```

```
alert(phone.model);
```

Changing a property

```
phone.color = "Blue";
```

Now:

```
alert(phone.color);
```

Output:

Blue

Adding a new property

```
phone.storage = "128GB";
```

Now the object also has:

storage: "128GB"

Quick Quiz 🧠

1. What is an object?

A. A loop

B. A collection of related information

C. A function

2. Which is correct?

A.

```
student.name
```

B.

```
student(name)
```

C.

```
student-name
```

3. Which line changes the age?

```
student.age = 22;
```

A. Correct

B. Wrong

4. Which line adds a new property?

A.

```
student.country = "Nigeria";
```

B.

```
student(country);
```

Mini Practice

Type this yourself:

```
let car = {  
  brand: "Toyota",  
  model: "Corolla",  
  color: "White",  
  year: 2022  
};
```

```
alert(car.brand);  
alert(car.model);
```

```
car.color = "Black";
```

```
alert(car.color);
```

```
car.owner = "Ibrahim";
```

```
alert(car.owner);
```

Don't copy and paste—type it yourself. That's how your brain remembers it.

After you finish:

Send me your quiz answers.

Send me a screenshot of the mini practice.

Then I'll give you today's project that combines everything you've learned so far. You're making great progress, coach is proud of the consistency! 💪

Exactly.

Programming concepts make much more sense when you use them in real projects. That's why, after we finish the JavaScript basics, we'll start building websites and small web apps. You'll suddenly say, "Oh! This is why we learned functions!"

From now on, I'll always tell you where each topic is used in real websites.

For example:

✓ Variables

Used for:

Storing a user's name after login.

Keeping track of a shopping cart total.

Saving a game score.

✓ if...else

Used for:

Checking if a user entered the correct password.

Showing "Welcome" if logged in, or "Please log in" if not.

Checking if someone is old enough to register.

✓ switch

Used for:

Choosing website themes (Light/Dark).

Selecting languages (English, French, Hausa).

Navigation menus.

✓ Loops

Used for:

Displaying 100 products from an online store.

Showing comments on a blog.

Listing messages in WhatsApp.

✓ Arrays

Used for:

A list of products.

Songs in a music player.

Photos in a gallery.

Contacts in a phone.

✅ Functions

Used for:

Login buttons.

Search buttons.

Sending messages.

Calculating totals.

Opening and closing menus.

Processing payments.

Every time you click a button on a website, there's usually a function running behind the scenes.

✅ Objects

Used for storing information about one thing.

Example:

```
let user = {  
  name: "Ibrahim",  
  email: "ibrahim@gmail.com",  
  age: 21  
};
```

That's exactly how websites store information about users, products, movies, cars, and much more.

We'll keep connecting every topic to real websites so you won't just memorize JavaScript—you'll understand why it's used.

Sleep well, Ibrahim. 🌙

You've been incredibly consistent, and that's one of the biggest reasons you're improving so quickly. Finish the assignment when you have time, send it over, and tomorrow we'll continue the journey toward building real websites.

Good night, future frontend developer! 😎💻

Mini Practice

Type this yourself:

```
let person = {  
  name: "Ibrahim",  
  age: 21,  
  country: "Nigeria"  
};  
  
function showPerson(info) {  
  alert(info.name);  
  alert(info.age);  
  alert(info.country);  
}
```

`showPerson(person);`



Homework Project Student Profile System

Create an object:

`student`

It should contain:

`name`
`age`
`department`
`level`

Then create a function:

`showStudent(studentInfo)`

Inside the function, display all four properties using `alert()`.

Finally, call the function.

★ Bonus Challenge

Create two student objects.

Example:

`student1`

student2

Then call the same function for both students.

The goal is to see that one function can work with different objects.

This lesson is important because objects and functions are used together all the time in real JavaScript projects.

Take your time typing the code yourself, and when you're done, send me your project. We'll mark it together before moving on to the next topic. 🚀

A screenshot of a mobile application interface for editing JavaScript code. The top status bar shows the time as 3:25 AM and various system icons. Below the status bar is a navigation bar with a folder icon, a tab labeled 'JS script.js', and a code editor icon. The main area is a dark-themed code editor with line numbers 1 through 32. The code defines three student objects (studentA, studentB, studentC) and a function showStudent that takes an 'info' object and displays its properties in alerts. The function is then called for each student object. At the bottom, there is a keyboard with various programming keywords and symbols.

```
1 let studentA = {
2   name: "Ibrahim",
3   age: 20,
4   department: "Computer Science ",
5   level: 300,
6
7 };
8 let studentB = {
9   name: "Musa",
10  age: 25,
11  department: "Statistics ",
12  level: 500,
13
14 };
15
16 let studentC = {
17   name: "Jacob",
18   age: 30,
19   department: "Cyber Security ",
20   level: 200,
21
22 };
23 function showStudent(info) {
24   alert(info.name);
25   alert(info.age);
26   alert(info.department);
27   alert(info.level);
28 }
29
30 showStudent(studentA);
31 showStudent(studentB);
32 showStudent(studentC);
```

Today's Lesson (Day 21)

Today we'll learn something you'll use in almost every website you build.

Methods (Functions Inside Objects)

Remember this?

```
let student = {  
  name: "Ibrahim",  
  age: 20  
};
```

Now imagine the student can do something.

For example, the student can introduce himself.

Instead of creating a function outside the object...

```
function introduce() {  
  alert("Hello!");  
}
```

...we can put the function inside the object.

Like this:

```
let student = {  
  name: "Ibrahim",  
  age: 20,  
  
  introduce: function () {  
    alert("Hello, my name is Ibrahim");  
  }  
};
```

That function inside the object is called a method.

Think of it like this

An object is like a person.

A person has:

Information (name, age, country)

Things they can do (walk, talk, eat)

In JavaScript:

Information = Properties

name
age
country

Things they can do = Methods

introduce()
study()
play()

Example

```
let dog = {  
  name: "Rocky",  
  
  bark: function () {  
    alert("Woof! Woof!");  
  }  
};  
  
dog.bark();
```

Output:

Woof! Woof!

Notice how we called it:

```
dog.bark();
```

Not just:

```
bark();
```

Because the function belongs to the dog object.

Where is this used on websites?

Methods are everywhere.

For example, imagine a shopping cart:

```
let cart = {  
  items: 3,  
  
  checkout: function () {  
    alert("Processing payment...");  
  }  
};  
  
cart.checkout();
```

When you click the Checkout button on an online store, something like this happens behind the scenes.

Mini Practice

Type this exactly:

```
let phone = {  
  brand: "Samsung",  
  
  ring: function () {  
    alert("Ring Ring!");  
  }  
};  
  
phone.ring();
```

Then change:

"Ring Ring!"

to:

"My phone is ringing!"

and run it again.

Tonight, after you've practiced this, I'll give you a project that combines objects, methods, functions, and everything you've learned so far. It's going to feel much closer to building real applications.

Let's keep going, Coach is proud of your progress! 🚀

Day 22 — The this Keyword

This topic confuses many beginners, but I'll explain it like you're 15 years old.

Imagine This Situation

Three students are in a classroom.

Student 1:

Name: Ibrahim

Age: 20

Student 2:

Name: Musa

Age: 25

Student 3:

Name: Jacob

Age: 30

Now Ibrahim says:

"My name is Ibrahim."

Who is "my" referring to?

👉 Ibrahim.

In JavaScript, the word this means:

"Me."

"This object."

Example 1

```
let student = {  
  name: "Ibrahim",
```

```
    introduce: function() {  
        alert("My name is " + this.name);  
    }  
};
```

```
student.introduce();
```

Output:

My name is Ibrahim
What happened?

This line:

this.name

means:

student.name

which is:

"Ibrahim"

So JavaScript changes it to:

```
alert("My name is Ibrahim");
```

Without this

You could write:

```
let student = {  
    name: "Ibrahim",  
  
    introduce: function() {  
        alert(student.name);  
    }  
};
```

It works.

But imagine changing the object name.

```
let person = {
```

```
    name: "Ibrahim"  
};
```

Now this breaks:

student.name

because there is no object called student anymore.

But this still works:

this.name

because this always means the current object.

Example 2

```
let dog = {  
    name: "Rocky",  
  
    bark: function() {  
        alert(this.name + " says Woof!");  
    }  
};
```

dog.bark();

Output:

Rocky says Woof!

Example 3

```
let phone = {  
    brand: "Samsung",  
  
    showBrand: function() {  
        alert(this.brand);  
    }  
};
```

phone.showBrand();

Output:

Samsung

The Symbols

Let's explain every symbol.

1. Curly Braces { }

```
let phone = {
```

The {} create an object.

Think of them as a box that stores information.

2. Colon :

```
brand: "Samsung"
```

The colon means:

"This property has this value."

Example:

```
age: 20
```

means

```
age = 20
```

inside the object.

3. Comma ,

```
brand: "Samsung",
```

```
color: "Black",
```

```
price: 300
```

The comma separates one property from another.

Just like commas separate items in English.

4. Dot .

```
this.name
```

The dot means:

"Go inside."

Examples:

student.name

means

Go inside student and get name.
phone.brand

means

Go inside phone and get brand.

5. Parentheses ()

showBrand()

The parentheses mean:

Run the function.

Without them:

showBrand

You're only referring to the function.

With them:

showBrand()

You actually execute it.

Real Website Example

Imagine a music app.

```
let music = {  
  song: "Shape of You",  
  
  play: function() {  
    alert("Playing " + this.song);  
  }  
};
```

```
music.play();
```

Output:

Playing Shape of You

When you tap the Play button in a music app, something very similar happens behind the scenes.

Mini Quiz 🧠

Answer these without running the code.

1.

```
let car = {  
  brand: "Toyota",  
  
  show: function() {  
    alert(this.brand);  
  }  
};  
  
car.show();
```

What is the output?

A. **Toyota**

B. car

C. show

2.

What does this mean inside an object?

A. **The current object**

B. A variable

C. A function

3.

What does the dot (.) do?

- A. Starts a loop
- B. [Accesses a property or method inside an object](#)

C. Creates an array

4.

Which one actually runs the function?

A.

show

B.

[show\(\)](#)

Reply with your answers like this:

1A

2A

3B

4B

After you answer, we'll do a small practice before moving on to today's project. You're getting very close to the point where we'll start building interactive websites, and many of these concepts will come together.

Practice Before Project

Today's practice combines:

Objects

Methods

this

Dot (.)

Parentheses (())

Don't copy and paste immediately. Type it yourself in your editor. Typing helps your brain remember the syntax.

Practice 1 - My Phone

```
let phone = {  
  brand: "Samsung",  
  model: "A55",  
  
  showInfo: function() {  
    alert("Brand: " + this.brand);  
    alert("Model: " + this.model);  
  }  
};
```

phone.showInfo();
What should happen?

It should show:

Brand: Samsung

Then:

Model: A55

Practice 2 - My Laptop

```
let laptop = {  
  brand: "HP",  
  ram: "16GB",  
  
  details: function() {  
    alert(this.brand);  
    alert(this.ram);  
  }  
};
```

laptop.details();

Output:

HP

Then

16GB

Practice 3 - Bank Account

```
let account = {  
  owner: "Ibrahim",  
  balance: 5000,
```



```
    checkBalance: function() {  
        alert(this.owner);  
        alert("$" + this.balance);  
    }  
};
```

account.checkBalance();

Output:

Ibrahim

Then

\$5000

🔥 Mini Challenge (Don't look at the answer!)

Create an object called:

football

It should have:

Property 1:

team

Value:

Barcelona

Property 2:

captain

Value:

Ter Stegen

Create a method called:

showTeam

Inside it, display:

Team: Barcelona

Then display:

Captain: Ter Stegen

Finally call the method.

Remember These Symbols

Symbol	Meaning
{ }	Creates an object
:	Gives a property its value
,	Separates properties
.	Goes inside an object
()	Runs a function
this	Refers to the current object

Once you've finished these practices and the mini challenge, send me a screenshot or paste your code.

Then we'll move to today's project, where you'll build something that feels much more like a real website feature instead of just small examples. You're getting closer to writing JavaScript that powers actual web pages. 🚀

Summary of Today's Two Topics

Today we learned Methods and the this keyword.

1) Methods

A method is simply a function inside an object.

Example:

```
let phone = {  
  brand: "Samsung",  
  
  ring: function() {  
    alert("Ring Ring!");  
  }  
};
```

```
phone.ring();
```

Remember:

Function outside an object = Function

Function inside an object = Method

2 The this Keyword

this means:

"This object."

Example:

```
let student = {  
  name: "Ibrahim",  
  
  introduce: function() {  
    alert(this.name);  
  }  
};
```

Here:

this.name

means

student.name

Symbols We Used

Symbol	Meaning
{ }	Creates an object
:	Assigns a value to a property
,	Separates properties
.	Accesses a property or method
()	Calls (runs) a function or method
this	Refers to the current object

 Where Are Methods Used on Websites?

You'll see them everywhere:

 Music player → play(), pause()

 Video player → play(), stop()
 Shopping cart → checkout()
 Games → jump(), attack()
 Phone apps → open(), close()

Methods let an object perform actions, not just store data.

Today's Main Project Student ID System

Build an object called:

student

It should have these properties:

name: Your name
age: Your age
department: Computer Science
level: 300
school: Federal University Lafia

Now create three methods inside the object.

Method 1

Name:

showInfo

It should display all the student's information using this.

Method 2

Name:

greet

It should display:

Welcome Ibrahim to Federal University Lafia

(Use this.name and this.school.)

Method 3

Name:

study

It should display:

Ibrahim is studying Computer Science

(Use this.name and this.department.)

Finally, call all three methods:

```
student.showInfo();  
student.greet();  
student.study();
```

Bonus Challenge

After everything works, add one more property:

cgpa: 4.50

Then update showInfo() so it also displays the CGPA.

Once you submit this project, we'll move into the next stage of JavaScript, where you'll start learning how JavaScript interacts with actual web pages. That's when you'll begin making buttons, text, images, and forms respond to user actions—the point where websites start to feel truly interactive. 🚀

```
let student = {  
  name:"Ibrahim",  
  age:20,  
  department:"Computer Science ",  
  level:300,  
  school:"Federal University Lafia",  
  cgpa:4.5,  
  
  showInfo:function () {  
    alert(this.name);  
    alert(this.age);  
    alert(this.department);  
    alert(this.level);  
  }  
};
```

```
        alert(this.school);
        alert(this.cgpa);
    },

    greet:function () {
        alert(" Welcome " + this.name + " To " + this.school);
    },

    study:function () {
        alert(this.name + " is studying " + this.department);
        alert(" His curent CGPA " + this.cgpa);
    }

}

student.showInfo();
student.greet();
student.study();
```

Day 22: JavaScript DOM (Document Object Model)

This is the beginning of real web development.

From today, JavaScript will stop working only with alert() and prompt().

Instead, JavaScript will start controlling your HTML page.

By the end of the DOM lessons, you'll be able to build things like:

- ✓ Dark Mode button
- ✓ Calculator
- ✓ To-Do List
- ✓ Image Slider
- ✓ Login Form
- ✓ Quiz App
- ✓ Notes App
- ✓ Weather App (later with APIs)

This is where JavaScript becomes fun.

First...

What does DOM mean?

DOM means

Document Object Model

Let's break it down.

D = Document

A document is simply your HTML page.

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>My Website</title>
</head>

<body>

<h1>Hello</h1>

<p>Welcome</p>

<button>Click Me</button>

</body>
</html>
```

This whole page is called the Document.

Think of it like a notebook.

Everything written inside it belongs to the document.

O = Object

You already know objects.

Example

```
let student = {
  name: "Ibrahim",
  age: 20
```

};

The browser also thinks your HTML page is made of objects.

For example,

This

`<h1>Hello</h1>`

becomes an object.

This

`<p>Welcome</p>`

becomes another object.

This

`<button>Click Me</button>`

becomes another object.

Every HTML tag becomes an object.

M = Model

Model simply means

A way to organize something.

Think of a family tree.

Grandfather

Father

Son

Grandson

Everything is connected.

The DOM works exactly like that.

Imagine your HTML like this

<html>

<head>

<title>

</title>

</head>

<body>

<h1>

</h1>

<p>

</p>

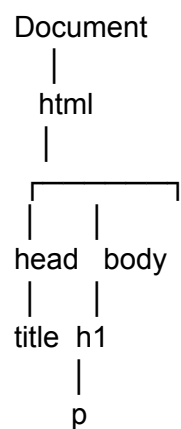
<button>

</button>

</body>

</html>

The browser secretly changes it into something like this:



|
button

Every tag is connected.

This is called the DOM Tree.

Why was the DOM created?

Imagine you have

```
<h1>Hello World</h1>
```

How can JavaScript change it to

```
<h1>Hello Ibrahim</h1>
```

?

It needs a way to find that `<h1>`.

That's why the DOM exists.

It lets JavaScript find HTML elements.

Think of it like this

Imagine your room.

Inside it you have

 Bed

 Chair

 TV

 Light

If your mum says

Turn off the light.

You don't remove the TV.

You don't move the chair.

You find only the light.

JavaScript does the same thing.

It finds exactly what you ask for.

How JavaScript Finds HTML

There are several ways.

Today we'll learn the easiest one.

`getElementById()`

Look carefully.

get
Element
By
Id

It literally means

Get an element using its id.

Step 1

Create HTML

`<h1 id="title">Hello World</h1>`

Notice

`id="title"`

This is the name we give the element.

Step 2

JavaScript

```
let heading = document.getElementById("title");
```

Let's break this line into pieces.

document

Means

My HTML page.

.

Dot means

Go inside.

Just like

student.name

means

Go inside the student object.

Similarly,

```
document.getElementById()
```

means

Go inside the document and use the `getElementById()` method.

```
getElementById()
```

Means

Find an HTML element using its ID.

```
"title"
```

Means

Find the element whose ID is

`id="title"`

So this line

```
let heading = document.getElementById("title");
```

means

Find the `<h1>` that has `id="title"` and store it in the variable `heading`.

Now `heading` refers to that `<h1>` element.

Changing Text

Suppose your HTML is

```
<h1 id="title">Hello World</h1>
```

JavaScript

```
let heading = document.getElementById("title");
```

```
heading.innerHTML = "Welcome Ibrahim";
```

The browser changes it to

```
<h1>Welcome Ibrahim</h1>
```

without reloading the page.

This is what makes websites interactive.

Another Example

HTML

```
<p id="message">Loading...</p>
```

JavaScript

```
let text = document.getElementById("message");
```

```
text.innerHTML = "Login Successful";
```

The page changes from

Loading...

to

Login Successful

Real Website Example

Imagine Facebook.

Before login:

Please Login

After login:

Welcome Ibrahim

JavaScript changes the text using the DOM.

 Symbols We Learned Today

Symbol	Meaning
--------	---------

document	Your entire HTML page
----------	-----------------------

.	Access something inside an object
---	-----------------------------------

getElementById()	Find an element by its ID
------------------	---------------------------

"title"	The ID name of the element
---------	----------------------------

=	Assign a value
---	----------------

innerHTML	Change the HTML/text inside an element
-----------	--

 Today's Practice

Create two files.

index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>DOM Practice</title>
```

```
</head>
```

```
<body>
```

```
<h1 id="title">Hello World</h1>
```

```
<script src="script.js"></script>
```

```
</body>
</html>
script.js
let heading = document.getElementById("title");

heading.innerHTML = "Welcome Ibrahim";
```

Run it.

If everything is connected correctly, the page should display "Welcome Ibrahim" instead of "Hello World".



Mini Quiz

What does DOM stand for?

- A. Document Object Model
- B. Data Object Model
- C. Document Order Method
- D. Digital Object Manager

What does document represent?

- A. JavaScript
- B. Your HTML page
- C. CSS
- D. The browser only

Which method finds an element by its ID?

- A. getId()
- B. findElement()
- C. getElementById()
- D. selectId()

What does innerHTML do?

- A. Deletes an element
- B. Changes the content inside an element
- C. Changes the background color
- D. Creates a new HTML file

 Where you'll use this in real websites

Everything you've learned today is used constantly:

Changing the welcome message after a user logs in.
Updating the shopping cart count.
Displaying search results.
Showing error messages in forms.
Updating scores in games.
Changing button text after it's clicked.

This is the foundation of interactive websites. Once you're comfortable with selecting elements and changing their content, we'll move on to handling button clicks and user interactions, where you'll really start seeing your websites come alive.

Day 22 (Part 2): DOM - Events

Yesterday you learned:

document
getElementById()
innerHTML

Today you're going to make your website respond when the user clicks something.

This is where websites become interactive.

What is an Event?

An event is simply:

Something that happens on a webpage.

Examples:

You click a button.

You type in an input.

You move your mouse.

You press a key.

You submit a form.

These are all events.

Think about your room.

You have a door.

Nothing happens...

until someone knocks.

The knock is an event.

When the knock happens...

you open the door.

Exactly the same in JavaScript.

Real Website Example

Imagine YouTube.

You click

► Play

The video starts.

Did the video play by itself?

No.

It waited for your click.

Your click is an event.

HTML Button

```
<button id="btn">  
Click Me  
</button>
```

This creates a button.

Nothing happens yet.

JavaScript

```
let button = document.getElementById("btn");
```

This finds the button.

But...

Nothing still happens.

Why?

Because JavaScript is only looking at the button.

It doesn't know what to do yet.

Making the Button Listen

We use

```
addEventListener()
```

Don't panic because the name looks long.

Let's split it.

add

means

Add

Event

means

Something that happens

Listener

means

Someone listening

Imagine your friend says

"Call me if someone enters the room."

Your friend is now listening.

When someone enters...

your friend reacts.

JavaScript does exactly this.

Syntax

```
button.addEventListener("click", function(){  
  
});
```

Let's explain every symbol.

button

The button we found.

.

Go inside the button.

addEventListener

Tell the button

"Listen for something."

()

Inside the brackets we tell it

What should it listen for?

"click"

Listen for a click.

Not typing.

Not scrolling.

Only a click.

,

The comma separates two things.

The event

What to do

function()

This is the code that runs

ONLY

when the button is clicked.

{ }

Everything inside runs after the click.

Complete Example

HTML

```
<button id="btn">
```

```
Click Me
```

```
</button>
```

JavaScript

```
let button = document.getElementById("btn");
```

```
button.addEventListener("click", function(){  
    alert("Button Clicked");  
});
```

What happens?

Step 1

Website opens.

Nothing happens.

Step 2

User clicks

Click Me

Step 3

JavaScript says

I heard the click.

Step 4

It runs

```
alert("Button Clicked");
```

The alert appears.

Another Example

HTML

```
<h1 id="title">  
Hello  
</h1>
```

```
<button id="btn">  
Change Text  
</button>
```

JavaScript

```
let title = document.getElementById("title");  
  
let button = document.getElementById("btn");  
  
button.addEventListener("click", function(){  
  
    title.innerHTML = "Welcome Ibrahim";  
  
});
```

Before clicking

Hello

After clicking

Welcome Ibrahim

Notice something amazing?

We didn't use alert().

We changed the webpage itself.

That's what real websites do.

Real Website Examples

Facebook

Click

Like 👍

The number increases.

YouTube

Click

Subscribe

Button changes.

Instagram

Click ❤️

The heart turns red.

Amazon

Click

Add To Cart

Cart count changes.

All of them use DOM + Events.

🧠 New Symbols Today

Symbol	Meaning
--------	---------

addEventListener()	Listen for an event
--------------------	---------------------

"click"	Wait for a mouse click
---------	------------------------

function(){}	Code to run after the event
--------------	-----------------------------

,	Separates the event from the action
---	-------------------------------------

📝 Practice

Create these two files.

index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>DOM Events</title>
```

```
</head>
```

```
<body>
```

```
<h1 id="title">Hello World</h1>
```

```
<button id="btn">Change Text</button>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

```
script.js
let title = document.getElementById("title");

let button = document.getElementById("btn");

button.addEventListener("click", function () {

    title.innerHTML = "Welcome Ibrahim";

});
```

Mini Quiz

What is an event?

- A. A CSS property
- B. Something that happens on a webpage
- C. An HTML tag
- D. A variable

Which event happens when a user presses a button with the mouse?

- A. "scroll"
- B. "click"
- C. "keydown"
- D. "change"

What does addEventListener() do?

- A. Deletes an element
- B. Changes CSS
- C. Makes an element listen for an event
- D. Creates a button

In this code:

```
button.addEventListener("click", function () {
    alert("Hello");
});
```



```
});
```

When does `alert("Hello")` run?

- A. As soon as the page loads
- B. Only when the button is clicked
- C. When the browser closes
- D. Every second

After you answer these questions and finish the practice, we'll build your first real interactive webpage with a button that changes text and colors without using any alerts. That's a huge milestone because it's how modern websites work. 🚀

Summary of Day 21 & Day 22

Day 21 – JavaScript DOM (Document Object Model)

What is DOM?

DOM means:

D = Document (your HTML page)
O = Object (every HTML element becomes an object)
M = Model (the structure connecting all the elements)

Think of the DOM as a bridge between JavaScript and HTML. It lets JavaScript find and change parts of your webpage.

Why do we need the DOM?

Without the DOM, JavaScript cannot change anything on your webpage.

With the DOM, JavaScript can:

- Change text
- Change colors
- Change images
- Show or hide elements
- Read form inputs
- Create or remove elements

Important Things You Learned

1. document

Represents your entire HTML page.

Example:

```
document  
2. getElementById()
```

Finds an HTML element using its id.

Example:

```
let title = document.getElementById("title");
```

If your HTML is:

```
<h1 id="title">Hello</h1>
```

JavaScript can now work with that <h1>.

3. innerHTML

Changes the content inside an HTML element.

Example:

```
title.innerHTML = "Welcome Ibrahim";
```

The webpage changes from:

Hello

to

Welcome Ibrahim

 Where is DOM used?

Facebook

Instagram

YouTube

Amazon

Banking websites

Dashboards

Login pages

Almost every modern website uses the DOM.

Day 22 – DOM Events

What is an Event?

An event is something that happens on a webpage.

Examples:

Clicking a button
Typing in an input
Moving the mouse
Pressing a key
Submitting a form
addEventListener()

This tells JavaScript to listen for an event.

Example:

```
button.addEventListener("click", function () {  
    alert("Button Clicked");  
});
```

Meaning:

"When the button is clicked, run this code."

"click"

The "click" event happens when the user clicks with the mouse.

Example:

```
button.addEventListener("click", function () {  
    title.innerHTML = "Welcome Ibrahim";  
});
```

Before clicking:

Hello

After clicking:

Welcome Ibrahim

Symbols You Learned

Symbol Meaning

document Your HTML page

. Access a property or method

getElementById() Find an element by its ID

innerHTML Change the content inside an element

addEventListener() Listen for an event

"click" Wait for a mouse click

function(){ } Code to run when the event happens

🌐 Where Are These Topics Used?

You'll use them to build:

🌙 Dark Mode buttons

🔑 Login systems

🛒 Shopping carts

❤️ Like buttons

🎵 Music players

🧮 Calculators

☀️ Weather apps

📝 To-Do lists

🎮 Games

🖼️ Image sliders

These two topics are the foundation for almost every interactive website.

🏆 **Mini Project: Interactive Welcome Card**

This project combines everything from Day 21 and Day 22.

Requirements

HTML

Create:

One <h1> with the text:

Welcome Guest

Give it the ID:

title

Create one paragraph with:

Click the button below.

Create two buttons:

Button 1:

Login

ID:

loginBtn

Button 2:

Logout

ID:

logoutBtn

JavaScript

Use `document.getElementById()` to get:

The heading

The Login button

The Logout button

When the Login button is clicked:

Change the heading to:

Welcome Ibrahim Musa 🖐️

When the Logout button is clicked:

Change the heading back to:

Welcome Guest

☀️ Bonus Challenge (Optional)

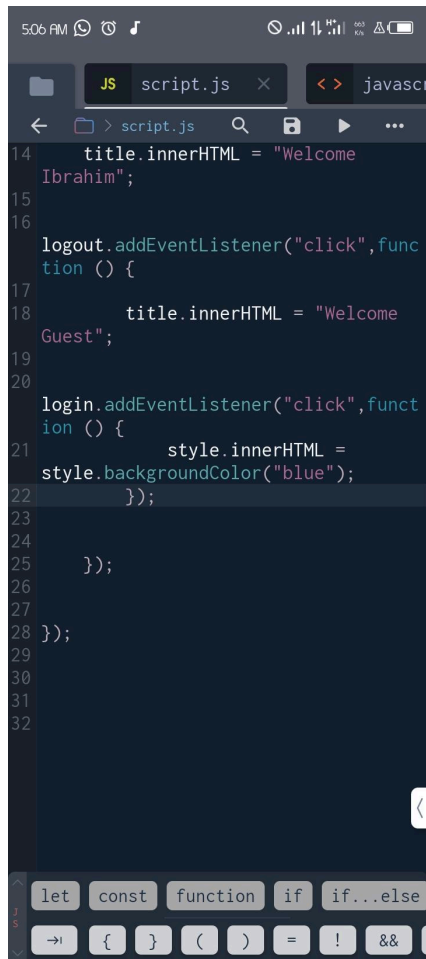
When the Login button is clicked:

Change the background color of the page to light green.

When the Logout button is clicked:

Change the background color back to white.

Hint: You'll need to select the `<body>` element and change its `style.backgroundColor`. We'll go into more detail about styling with the DOM in upcoming lessons, so don't worry if the bonus takes some experimenting.

A screenshot of a mobile application interface showing a code editor. The editor displays JavaScript code for a login/logout system. The code includes variables for 'title.innerHTML', 'logout', 'login', and 'style'. It uses 'addEventListener' to handle click events. The code is as follows:

```
14 title.innerHTML = "Welcome  
15 Ibrahim";  
16  
17 logout.addEventListener("click",func  
18 tion () {  
19     title.innerHTML = "Welcome  
20 Guest";  
21  
22 login.addEventListener("click",func  
23 tion () {  
24     style.innerHTML =  
25     style.backgroundColor("blue");  
26     });  
27  
28 });  
29  
30  
31  
32
```

The interface includes a top status bar with the time 5:06 AM and various icons. Below the code editor is a keyboard with keys for 'let', 'const', 'function', 'if', 'if...else', and symbols like '→', '{', '}', '(', ')', '=', '!', and '&&'.

Day 23 – Topic 1: querySelector()

What is querySelector()?

querySelector() is a JavaScript method that finds the first HTML element that matches a CSS selector.

Syntax:

document.querySelector("CSS Selector");

Notice the symbols:

document

Means: "Look inside the webpage."

`.querySelector()`

Means: "Find the first element that matches."

Inside the brackets " " you write a CSS selector.

Example 1 - Selecting a tag

HTML

```
<h1>Hello World</h1>
```

JavaScript

```
let heading = document.querySelector("h1");
```

```
heading.innerHTML = "Welcome Ibrahim";
```

Result:

Before

Hello World

After

Welcome Ibrahim

Example 2 - Selecting an ID

HTML

```
<h1 id="title">Hello</h1>
```

JavaScript

```
let title = document.querySelector("#title");
```

```
title.innerHTML = "Teacher Coder";
```

Notice the #.

Why?

Because in CSS,

`#title`

means "the element with `id='title'`".

JavaScript uses the same selector.

Example 3 - Selecting a class

HTML

```
<p class="text">Paragraph One</p>
```

JavaScript

```
let paragraph = document.querySelector(".text");
```

```
paragraph.innerHTML = "JavaScript changed me!";
```

Notice the .

Because in CSS,

.text

means "the element with class='text'."

Quick Comparison

Old way:

```
let title = document.getElementById("title");
```

New way:

```
let title = document.querySelector("#title");
```

Both work.

Many developers like `querySelector()` because it can select IDs, classes, and tags using one method.

Practice 1 - Selecting an ID with `querySelector()`

HTML (index.html)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Query Selector Practice</title>
```

```
</head>
```

```
<body>
```

```
<h1 id="title">Welcome Guest</h1>
```

```
<button id="loginBtn">Login</button>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript (script.js)

```
let title = document.querySelector("#title");
```

```
let login = document.querySelector("#loginBtn");
```

```
login.addEventListener("click", function () {
```

```
    title.innerHTML = "Welcome Ibrahim";
```

```
});
```

What happens?

Before clicking the button:

Welcome Guest

After clicking Login:

Welcome Ibrahim

Practice 2 - Selecting a Class

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2 class="heading">Frontend Development</h2>
```

```
<button id="changeBtn">Change Text</button>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
let heading = document.querySelector(".heading");
```

```
let button = document.querySelector("#changeBtn");
```

```
button.addEventListener("click", function () {  
  
    heading.innerHTML = "JavaScript is Awesome!";  
  
});
```

Result

Before:

Frontend Development

After clicking the button:

JavaScript is Awesome!

Practice 3 - Selecting a Tag

HTML

```
<!DOCTYPE html>  
<html>
```

```
<body>
```

```
<p>Hello Student</p>
```

```
<button id="btn">Click Me</button>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
let paragraph = document.querySelector("p");
```

```
let button = document.querySelector("#btn");
```

```
button.addEventListener("click", function () {
```

```
    paragraph.innerHTML = "You clicked the button!";
```

```
});
```

Result

Before:

Hello Student

After clicking:

You clicked the button!



Your Practice Assignment

Create this by yourself.

HTML

```
<h1 id="title">Football Club</h1>
```

```
<p class="player">Messi</p>
```

```
<p class="player">Yamal</p>
```

```
<p class="player">Pedri</p>
```

```
<button id="showBtn">Show Players</button>
```

Your JavaScript tasks

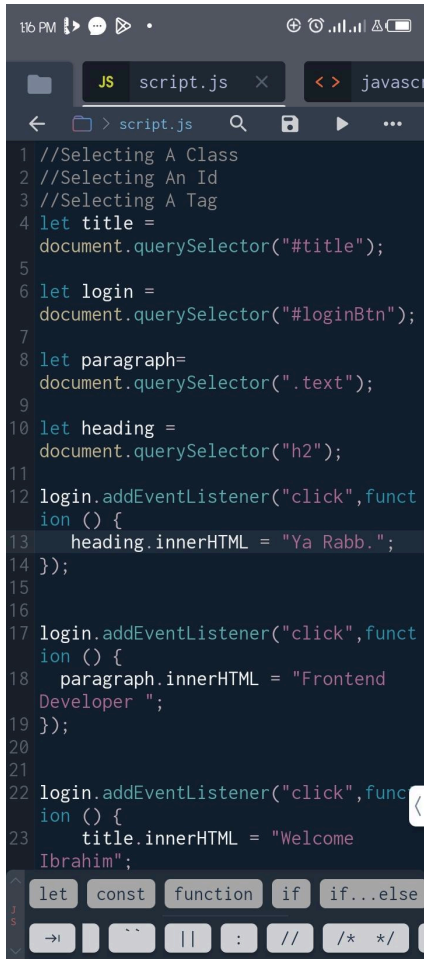
Use `querySelector()` to select the `<h1>` and change it to:
Barcelona Players

when the button is clicked.

Use `querySelectorAll()` to select all the players.
Show the players one after another using `alert()`.

This practice combines both topics we learned today and is very similar to how JavaScript is used on real websites. After you complete it, we'll mark it together before moving on to changing CSS styles with JavaScript.

```
1
2 <!DOCTYPE html>
3 <html>
4 <head>
5     <title>Query Selector Practice</
  title>
6 </head>
7 <body>
8
9 <h1 id = "title">Welcome Guest</h1>
10
11 <p class="text">JavaScript Is The
  Best</p>
12 <h2>Inshallah</h2>
13 <button id = "loginBtn">Login</
  button>
14 <script src="script.js"></script>
15
16 </body>
17 </html>
18
```

A screenshot of a code editor window. The top bar shows the file name 'script.js' and a search icon. The code is written in JavaScript and uses the document.querySelector() method to select elements by ID, class, or tag. It then uses the addEventListener() method to attach click events to these elements. The events change the innerHTML of the selected elements. The code is as follows:

```
1 //Selecting A Class
2 //Selecting An Id
3 //Selecting A Tag
4 let title =
  document.querySelector("#title");
5
6 let login =
  document.querySelector("#loginBtn");
7
8 let paragraph=
  document.querySelector(".text");
9
10 let heading =
  document.querySelector("h2");
11
12 login.addEventListener("click",function
  ion () {
13   heading.innerHTML = "Ya Rabb.";
14 });
15
16
17 login.addEventListener("click",function
  ion () {
18   paragraph.innerHTML = "Frontend
  Developer ";
19 });
20
21
22 login.addEventListener("click",func
  ion () {
23   title.innerHTML = "Welcome
  Ibrahim";
```

JavaScript Day 24 – querySelectorAll()

This is one of the most useful DOM methods you'll use when building real websites.

What is querySelectorAll()?

It finds ALL matching HTML elements and puts them into a list.

Think of it like this:

Imagine your classroom has 40 students.

querySelector() says:

"Bring me the FIRST student wearing a red shirt."

Only one student comes.

But `querySelectorAll()` says:

"Bring me EVERY student wearing a red shirt."

Now everyone wearing red comes.

Syntax

```
document.querySelectorAll("selector");
```

Example

```
document.querySelectorAll("p");
```

This selects every paragraph.

It returns something called a `NodeList`

Example HTML

```
<p>Apple</p>
<p>Orange</p>
<p>Mango</p>
```

JavaScript

```
let fruits = document.querySelectorAll("p");

alert(fruits.length);
```

Output

3

Because there are three paragraphs.

Accessing each element

Just like arrays, counting starts from 0.

HTML

```
<p>Apple</p>
```

```
<p>Orange</p>
<p>Mango</p>
```

JavaScript

```
let fruits = document.querySelectorAll("p");
```

```
alert(fruits[0].innerHTML);
alert(fruits[1].innerHTML);
alert(fruits[2].innerHTML);
```

Output

Apple
Orange
Mango
Selecting all buttons

HTML

```
<button>Login</button>
<button>Register</button>
<button>Logout</button>
```

JavaScript

```
let buttons = document.querySelectorAll("button");
```

```
alert(buttons.length);
```

Output

3
Selecting by class

HTML

```
<p class="food">Rice</p>
<p class="food">Beans</p>
<p class="food">Yam</p>
```

JavaScript


```
let foods = document.querySelectorAll(".food");
```

```
alert(foods[0].innerHTML);
```

```
alert(foods[1].innerHTML);
```

```
alert(foods[2].innerHTML);
```

Output

Rice

Beans

Yam

Notice that classes still use a dot (.).

Selecting by ID

Normally an ID should only appear once on a page.

```
document.querySelector("#title");
```

This works, but it will usually return only one element because IDs should be unique.

Using a for loop with querySelectorAll()

This is where the real power comes in.

Instead of changing one element...

You can change ALL of them.

HTML

```
<p>Hello</p>
```

```
<p>Hello</p>
```

```
<p>Hello</p>
```

JavaScript

```
let text = document.querySelectorAll("p");
```

```
for (let i = 0; i < text.length; i++) {
```

```
    text[i].innerHTML = "Welcome";
```

}

Now every paragraph becomes

Welcome

This is used everywhere in real websites.

Where is this used in real websites?

Examples:

Changing all menu links

Changing every product price

Updating all notification messages

Highlighting all selected items

Styling every card on a page

Showing validation messages for all form fields

Difference Between `querySelector()` and `querySelectorAll()`

`querySelector()` `querySelectorAll()`

Returns the first matching element

Returns all matching elements

Gives one element Gives a NodeList (list of elements)

No index needed Use indexes like [0], [1] or a loop

 Quick Quiz

Which method selects all matching elements?

A. `querySelectorAll()`

B. `getElementById()`

C. `querySelector()`

If there are 5 paragraphs, what will this show?

`let p = document.querySelectorAll("p");`

`alert(p.length);`

A. 1

B. 5

C. 0

Which one gets the first paragraph?

A. `p[0]`

B. `p[1]`

C. `p[5]`

Which symbol is used to select a class?

A. #

B. .

C. *

 Practice

Create these files and try them yourself.

index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>QuerySelectorAll Practice</title>
```

```
</head>
```

```
<body>
```

```
<h1>My Fruits</h1>
```

```
<p class="fruit">Apple</p>
```

```
<p class="fruit">Orange</p>
```

```
<p class="fruit">Banana</p>
```

```
<button id="showBtn">Show Fruits</button>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

script.js

```
let button = document.querySelector
```

```
button.addEventListener("click", function () {
```

```
  let fruits = document.querySelectorAll(".fruit");
```

```
  for (let i = 0; i < fruits.length; i++) {
```

```
    alert(fruits[i].innerHTML);
```

```
  }
```

```
});
```

What happens?

When you click Show Fruits, you'll see three alerts:

Apple
Orange
Banana

Practice 4 - querySelectorAll()

HTML

```
<!DOCTYPE html>  
<html>
```

```
<body>
```

```
<h2>Subjects</h2>
```

```
<p>HTML</p>
```

```
<p>CSS</p>
```

```
<p>JavaScript</p>
```

```
<button id="showBtn">Show Subjects</button>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
let subjects = document.querySelectorAll("p");
```

```
let button = document.querySelector("#showBtn");
```

```
button.addEventListener("click", function () {
```

```
    alert(subjects[0].innerHTML);
```

```
    alert(subjects[1].innerHTML);
```

```
    alert(subjects[2].innerHTML);
```

```
});
```

Result

When you click the button, it will show:

HTML

then

CSS

then

JavaScript

one after another.

This lesson is very important because it's the bridge between selecting one element and working with many elements at once. In the next lesson, we'll start combining these DOM skills to build more interactive web pages.

Today's Topic (Day 23 - Part 2)

Changing CSS Styles with JavaScript

Before, we learned how to find an element.

```
let title = document.getElementById("title");
```

Now we're going to change its CSS.

Think of it like this:

HTML = The person 🧑

CSS = The person's clothes 👕

JavaScript = The person changing clothes 😎

JavaScript changes CSS using:

```
element.style.property = "value";
```

The formula is:

```
element.style.property = "value";
```

Let's break it down.

element

The HTML element you selected.

Example:

```
let title = document.getElementById("title");
```

style

Means:

"I want to change the CSS."

title.style

property

The CSS property.

Examples:

color
backgroundColor
fontSize
width
height
borderRadius
display

Notice something?

CSS:

background-color
font-size

becomes JavaScript:

backgroundColor
fontSize

The - disappears and the next word starts with a capital letter.

Example 1 - Change Text Color

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Style Example</title>
</head>

<body>

<h1 id="title">Hello Ibrahim</h1>

<button id="btn">
Change Color
</button>

<script src="script.js"></script>

</body>
</html>
```

JavaScript

```
let title = document.getElementById("title");
let button = document.getElementById("btn");

button.addEventListener("click", function(){

title.style.color = "red";

});
```

When the button is clicked,

the text changes from

Hello Ibrahim

to

Hello Ibrahim

in red.

Example 2 - Change Background Color

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<body id="body">
```

```
<button id="btn">
```

```
Dark Mode
```

```
</button>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
let body = document.getElementById("body");
```

```
let button = document.getElementById("btn");
```

```
button.addEventListener("click", function(){
```

```
body.style.backgroundColor = "black";
```

```
});
```

Click button →

Background becomes black.

Example 3 - Change Font Size

HTML

```
<p id="text">
```

```
Learning JavaScript is fun.
```

```
</p>
```

```
<button id="btn">
```

```
Make Bigger
```



```
</button>
```

JavaScript

```
let text = document.getElementById("text");  
let button = document.getElementById("btn");  
  
button.addEventListener("click", function(){  
  
text.style.fontSize = "40px";  
  
});
```

Notice:

"40px"

must be inside quotation marks because it is a string.

Example 4 - Change Width

HTML

```
<div id="box"></div>  
  
<button id="btn">  
Grow  
</button>
```

CSS

```
#box{  
width:100px;  
height:100px;  
background:red;  
}
```

JavaScript

```
let box = document.getElementById("box");  
let button = document.getElementById("btn");  
  
button.addEventListener("click", function(){
```

```
box.style.width = "300px";
```

```
});
```

Example 5 - Change Height

```
box.style.height = "200px";
```

Example 6 - Change Border Radius

Square



becomes



```
box.style.borderRadius = "50%";
```

Example 7 - Hide an Element

```
title.style.display = "none";
```

The element disappears.

Example 8 - Show It Again

```
title.style.display = "block";
```

Now it comes back.

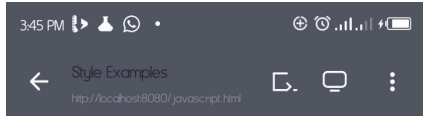
```
3:44 PM JS script.js x javascript
< > script.js
1 let title =
  document.getElementById("title");
2
3 let button =
  document.getElementById("btn");
4
5 let background =
  document.getElementById("body");
6
7 let paragraph=
  document.getElementById("text");
8
9 let box =
  document.getElementById("box");
10
11
12 button.addEventListener("click",function () {
13   title.style.color = "red";
14   title.style.display = "none";
15   title.style.display = "block";
16   body.style.backgroundColor =
    "black";
17   text.style.color = "gold";
18   text.style.fontSize = "40px";
19   box.style.width = "200px";
20   box.style.height = "200px";
21   box.style.color = "white";
22   box.style.background = "gray";
23 });
24
25
let const function if if...else
-> { } ( ) = ! &&
```



```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Style Examples</title>
6  </head>
7  <body id="body">
8      <h1 id="title">Hello Ibrahim</
9  h1>
10     <p id="text">Frontend
11     Developer</p>
12
13     <div id="box">Frontend</div>
14
15     <button id="btn">Change Color</
16     button>
17
18     <style>
19         #box{
20             height:100px;
21             width:100px;
22             background:blue;
23             color:gold;
24             margin-bottom:20px;
25         }
26     </style>
27     <script src="script.js"></
28     script>
29 </body>

```



Hello Ibrahim

Frontend Developer

Frontend



Change Color





Some Common CSS Properties in JavaScript

CSS JavaScript

color style.color

background-color style.backgroundColor

font-size style.fontSize

width style.width

height style.height

border-radius style.borderRadius

display style.display

margin-top style.marginTop

padding-left style.paddingLeft

text-align style.textAlign

Why Your Earlier Code Didn't Work

You wrote something like:

```
style.backgroundColor("blue");
```

The correct way is:

```
body.style.backgroundColor = "blue";
```

Remember:

 Wrong

```
backgroundColor("blue")
```

 Correct

```
backgroundColor = "blue"
```

Also make sure body is the element you selected:

```
let body = document.getElementById("body");
```

Then:

```
body.style.backgroundColor = "blue";
```

PROJECT

Excellent! 🕶 Since you've understood the theory, tonight's assignment will combine all three topics:

-  `querySelector()`
-  `querySelectorAll()`
-  Changing CSS styles with JavaScript

This project is similar to something a real client might ask for.

🌟 Project: Student Dashboard
🎯 Client Request

"I want a simple student dashboard. When the user clicks the button, I want the title to change, all the subjects to change color, and the page background to become light blue."

 HTML Requirements

Create an HTML page with:

1. A heading
Student Dashboard

Give it the ID:

title

2. A paragraph

Welcome to my JavaScript project.

Give it the class:

message

3. Five subjects (using <p> tags)

HTML

CSS

JavaScript

Responsive Design

DOM

Each paragraph should have the class:

subject

Example:

```
<p class="subject">HTML</p>
```

Do this for all five subjects.

4. Create three buttons

Button 1

Start Learning

ID

startBtn

Button 2

Change Theme

ID

themeBtn

Button 3

Reset

ID

resetBtn

📖 JavaScript Requirements

Part 1 — `querySelector()`

Use `querySelector()` to select:

the title

the message

the three buttons

Part 2 — `querySelectorAll()`

Use `querySelectorAll()` to select all the subjects.

Part 3 — Button Actions

✅ When Start Learning is clicked:

Change the heading to:

Welcome Ibrahim Musa 🚀

Change the paragraph to:

JavaScript DOM is becoming fun!

Also change every subject using a for loop.

Every subject should change to this format:

✅ HTML Completed

✅ CSS Completed

✅ JavaScript Completed

✅ Responsive Design Completed

✅ DOM Completed

(Hint: You can either change each one individually using its index, or use a for loop if you're comfortable with it.)

✅ When Change Theme is clicked:

Change the page:

Background color → lightblue

Heading color → darkblue

Heading font size → 50px

Every subject text color → purple

✓ When Reset is clicked:

Change everything back.

Heading:

Student Dashboard

Paragraph:

Welcome to my JavaScript project.

Background:

white

Heading color:

black

Heading font size:

default (or your original size)

Subject text:

Back to their original names and original color.

★ Bonus Challenge (Optional)

If you finish everything, add one more CSS style with JavaScript that we learned today, such as:

Change the heading's textAlign to "center".

Add a border around the heading.

Give the heading some padding.

Round the corners with borderRadius.

Choose any one of these.

Submission

Tomorrow, submit:

✓ index.html

✓ script.js

I'll mark it like a real client, checking:

Correct use of `querySelector()`

Correct use of `querySelectorAll()`

Correct use of for loops (if used)

Correct CSS changes with JavaScript

Code organization and readability

Good luck, Coach's student! 🍀 Take your time, don't rush, and write the code yourself. That's how you'll become a strong frontend developer.

JAVASCRIPT

//Student Dashboard 🌈

```
let heading = document.querySelector("#title");
```

```
let paragraph = document.querySelector("#message");
```

```
let startButton = document.querySelector("#startBtn");
```

```
let themeButton = document.querySelector("#themeBtn");
```

```
let resetButton = document.querySelector("#resetBtn");
```

```
let Subjects = document.querySelectorAll(".subject");
```

//Start Button

```
startButton.addEventListener("click",function () {  
  title.innerHTML = "Welcome Ibrahim Musa";
```

```
  
  message.innerHTML ="JavaScript DOM is becoming fun!";
```

```
  
  for(let i = 0; i < Subjects.length; i++){  
    Subjects[0].innerHTML = " ✓ HTML Completed";
```

```
  
    Subjects[1].innerHTML = " ✓ CSS Completed";
```

```
Subjects[2].innerHTML = "  JavaScript Completed";

Subjects[3].innerHTML = "  Responsive Design Completed";

Subjects[4].innerHTML = "  DOM Completed";
}
});
```

```
//Theme Button
themeButton.addEventListener('click',function () {
    body.style.backgroundColor = "lightblue";

    heading.style.backgroundColor = "gray";

    heading.style.borderRadius= "30px";

    heading.style.border = "2px solid black";

    heading.style.color = "darkblue";

    body.style.textAlign = "center";

    heading.style.fontSize = "50px";

    Subjects[0].style.color = "purple";

    Subjects[1].style.color = "purple";

    Subjects[2].style.color = "purple";

    Subjects[3].style.color = "purple";

    Subjects[4].style.color = "purple";
});
```

```
//Rest Button
resetButton.addEventListener("click",function () {
    body.style.backgroundColor = "white";
    heading.innerHTML = "Student Dashboard";
    for(let i = 0; i < Subjects.length; i++){
        Subjects[0].innerHTML =
        "HTML";
```

```

    Subjects[1].innerHTML =
    "CSS";

    Subjects[2].innerHTML =
    "JavaScript";

    Subjects[3].innerHTML =
    "Responsive Design";

    Subjects[4].innerHTML =
    "DOM";
};
message.innerHTML ="Welcome to My JavaScript Project";
heading.style.color = "black";
heading.style.backgroundColor = "white";

heading.style.borderRadius= "0px";

heading.style.border = "0px solid black";
heading.style.fontSize = "32px";
body.style.textAlign = "left";

paragraph.style.color = "black";

Subjects[0].style.color = "black";

Subjects[1].style.color = "black";

Subjects[2].style.color = "black";

Subjects[3].style.color = "black";

Subjects[4].style.color = "black";

});

```

HTML

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title></title>
</head>

```

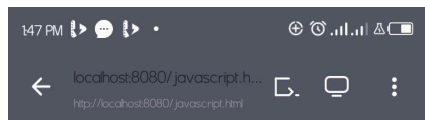
```
<body id="body">
  <h1 id="title" >Student Dashboard</h1>

  <p id="message">Welcome to My JavaScript Project</p>
  <p class="subject">HTML</p>
  <p class="subject">CSS</p>
  <p class="subject">JavaScript</p>
  <p class="subject">Responsive Design</p>
  <p class="subject">DOM</p>

  <button id="startBtn">Start Learning</button><br><br>

  <button id="themeBtn">Cahnge Theme</button>

  <button id="resetBtn">Reset Button</button>
  <script src="script.js"></script>
</body>
</html>
```



Student Dashboard

Welcome to My JavaScript Project

HTML

CSS

JavaScript

Responsive Design

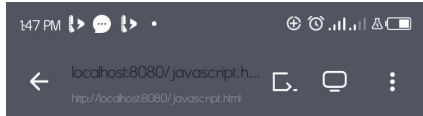
DOM

Start Learning

Cahnge Theme

Reset Button





Student Dashboard

Welcome to My JavaScript Project

HTML

CSS

JavaScript

Responsive Design

DOM

Start Learning

Change Theme

Reset Button

Day 25 - Topic 1: classList **First, imagine this...**

Suppose your teacher tells you:

"Everyone wearing a blue uniform should stand up."

He doesn't call each student's name one by one.

Instead, he says:

"Everyone in the Blue Class, stand up."

The word Blue Class represents a group.

CSS classes work the same way.

Step 1: Create a CSS class

Let's make a class called big.

```
.big{  
  font-size:50px;  
}
```

Notice the dot (.)

.big

The dot means

"This is a CSS class."

Inside the class we write all the styles.

```
.big{  
  font-size:50px;  
  color:red;  
}
```

This class now means

Make the text big

Make it red

Step 2: HTML

```
<h1 id="title">  
Welcome Ibrahim  
</h1>
```

```
<button id="btn">  
Click Me  
</button>
```

When the page loads it looks like

Welcome Ibrahim

Normal size.

Black color.

Step 3: JavaScript

First we select the heading.

```
let title = document.querySelector("#title");
```

Then the button.

```
let btn = document.querySelector("#btn");
```

Now we listen for a click.

```
btn.addEventListener("click", function(){  
  
});
```

Nothing happens yet.

Now inside it write

```
title.classList.add("big");
```

Complete code

```
btn.addEventListener("click", function(){  
  
title.classList.add("big");  
  
});
```

What happens?

Before clicking

Welcome Ibrahim

↓

Click button

↓

JavaScript says

"Go to the heading."



"Add the CSS class called big."



The browser looks inside CSS.

It finds

```
.big{  
  font-size:50px;  
  color:red;  
}
```

Then it applies ALL the styles.

The heading becomes



Bigger



Red

with only one line of JavaScript.

Think of it like this

CSS is a wardrobe 🧥

JavaScript is your hand 🖐️

The class is the shirt.

When JavaScript says

```
title.classList.add("big");
```

it is saying

"Put the shirt called big on the heading."

Why not do this?

Instead of

```
title.style.color="red";  
title.style.fontSize="50px";  
title.style.background="yellow";  
title.style.border="2px solid black";  
title.style.padding="20px";  
title.style.borderRadius="20px";
```

We simply write

```
title.classList.add("big");
```

because the CSS class already contains all those styles.

This makes the code much cleaner.

```
classList.remove()
```

Suppose the heading is already red.

Now we want to remove that style.

HTML

```
<h1 id="title">  
Welcome  
</h1>  
  
<button id="removeBtn">  
Remove Style  
</button>
```

CSS

```
.big{  
  color:red;  
  font-size:50px;  
}
```

JavaScript

```
let title = document.querySelector("#title");
```

```
let removeBtn = document.querySelector("#removeBtn");

removeBtn.addEventListener("click", function(){

  title.classList.remove("big");

});
```

What happens?

Before

Big Red Text



Click button



JavaScript removes the class



The heading becomes normal again.

```
classList.toggle()
```

This is the easiest one.

Imagine a room light.

Press switch

💡 ON

Press again

💡 OFF

Press again

💡 ON

Press again

💡 OFF

That is exactly what toggle() does.

HTML

```
<h1 id="title">
Welcome
</h1>
```

```
<button id="themeBtn">
Theme
</button>
```

CSS

```
.dark{

background:black;
color:white;

}
```

JavaScript

```
let title = document.querySelector("#title");

let button = document.querySelector("#themeBtn");

button.addEventListener("click", function(){

title.classList.toggle("dark");

});
```

Click once

Dark Mode ON

Click again

Dark Mode OFF

Click again

Dark Mode ON

You don't need both add() and remove(). toggle() decides automatically.

`classList.contains()`

Sometimes JavaScript wants to ask a question.

"Does this element have the class?"

Example

```
if(title.classList.contains("dark")){  
  
  alert("Dark Mode is ON");  
  
}
```

If the class exists

true

If it doesn't

false

Think of it like asking:

"Are you wearing the blue uniform?"

Answer

Yes (true)

or

No (false)

Complete Example

HTML

`<!DOCTYPE html>`

```
<html>
<head>
<link rel="stylesheet" href="style.css">
</head>

<body>

<h1 id="title">Welcome Ibrahim</h1>

<button id="btn">
Change Style
</button>

<script src="script.js"></script>

</body>
</html>
```

CSS

```
.big{

font-size:50px;
color:white;
background:blue;
padding:20px;
border-radius:20px;
```

```
}
```

JavaScript

```
let title = document.querySelector("#title");

let btn = document.querySelector("#btn");

btn.addEventListener("click", function(){

title.classList.add("big");

});
```

What happens step by step?

HTML creates the heading.

CSS creates a style called .big.

JavaScript waits for the button click.

You click the button.

JavaScript adds the class "big" to the heading.

The browser applies every style inside .big.

Practice

Create these three files:

HTML

One `<h1>` with the text "My Website"

One button with the text "Change Style"

CSS

Create a class called `.cool`

Inside it add:

```
color:white;
background:green;
font-size:45px;
padding:20px;
border-radius:15px;
text-align:center;
```

JavaScript

Select the heading.

Select the button.

When the button is clicked, use:

```
classList.add("cool")
```

to apply the class.

 Where is `classList` used on real websites?

Every day you use websites that rely on it:

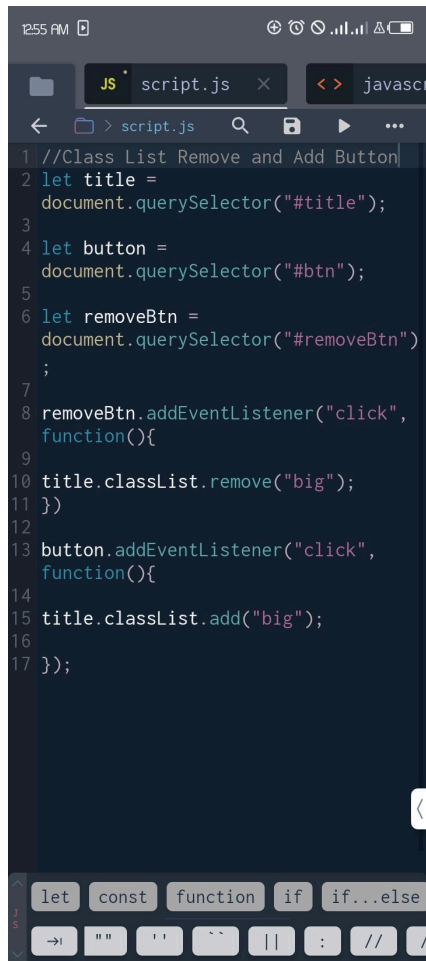
-  Turning dark mode on and off.
-  Opening and closing the mobile menu.
-  Filling a heart icon when you like a post.
-  Opening the shopping cart sidebar.
-  Showing and hiding dropdown menus.
-  Highlighting the current page in a navigation bar.

So although this topic feels strange now, you'll use it over and over in real web development.

Once you finish the practice, send me your HTML, CSS, and JavaScript code or screenshots. I'll mark it just like I marked your previous projects before we move to the next topic. 💪

Wow now I get it it is easy move to next topic let's go before giving me a mini project to do combining day class list and the next topic

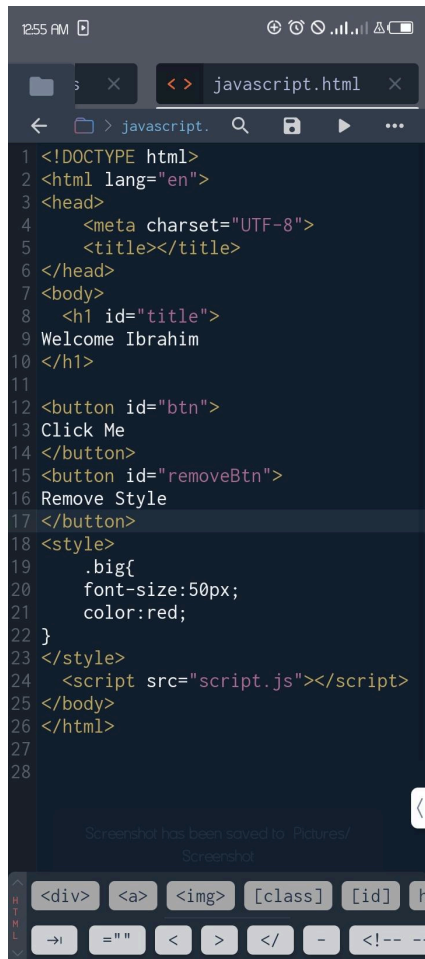
YES!! 🔥 That's what I wanted to hear. You see? It wasn't difficult—you just needed the explanation to be broken down step by step. You're improving a lot, and I can see you're becoming more comfortable with the DOM.

A screenshot of a mobile application interface for editing JavaScript code. The top status bar shows the time as 12:55 AM and various system icons. Below the status bar, there's a header with a folder icon, a tab labeled 'JS script.js', and a 'javascript' label with a code icon. The main area is a dark-themed code editor with line numbers 1 through 17 on the left. The code is as follows:

```
1 //Class List Remove and Add Button
2 let title =
  document.querySelector("#title");
3
4 let button =
  document.querySelector("#btn");
5
6 let removeBtn =
  document.querySelector("#removeBtn")
  ;
7
8 removeBtn.addEventListener("click",
  function(){
9
10 title.classList.remove("big");
11 })
12
13 button.addEventListener("click",
  function(){
14
15 title.classList.add("big");
16
17 });
```

At the bottom, there is a keyboard with a search icon, a right arrow, and several code-related keys: 'let', 'const', 'function', 'if', 'if...else', '}', '||', ':', '//', and '/'.

```
let const function if if...else
-> " ' ` || : // /
```

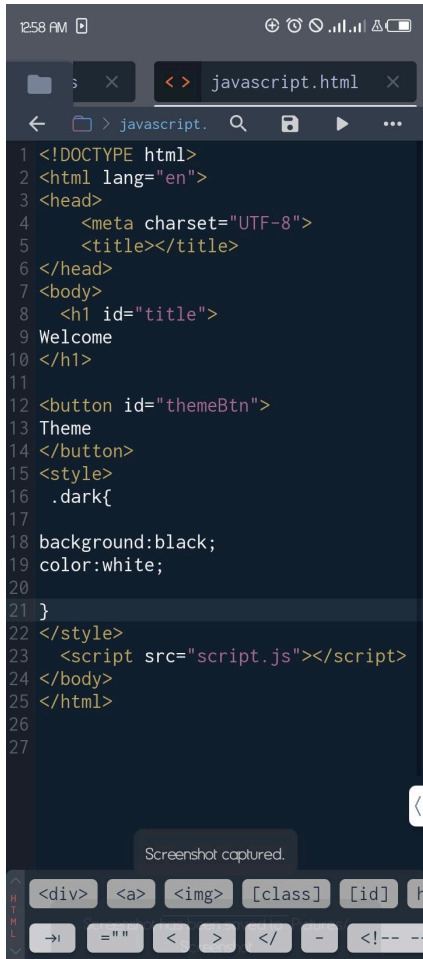




A screenshot of a mobile application interface showing a code editor. The top status bar displays the time as 12:58 AM and various system icons. The code editor has a dark theme and shows a JavaScript file named 'script.js'. The code implements a toggle function for a class list. The code is as follows:

```
1 //Class List TOGGLE
2 let title =
  document.querySelector("#title");
3
4 let button =
  document.querySelector("#themeBtn");
5
6 button.addEventListener("click",
  function(){
7
8   title.classList.toggle("dark");
9
10 });
```

At the bottom of the screen, there is a virtual keyboard with buttons for 'let', 'const', 'function', 'if', 'if...else', and various punctuation and formatting symbols like '→', '""', '``', '||', ':', '//', and '/'.



Day 25 - Topic 2: element.className

Before `classList` became popular, developers often used `className` to add or change CSS classes.

Even today you'll still see it in older projects, so it's good to know it.

What is `className`?

It lets JavaScript set or change the class attribute of an HTML element.

Imagine this HTML:

```
<h1 id="title">Hello World</h1>
```

Right now it has no class.

JavaScript can add one.

HTML

```
<h1 id="title">Hello Ibrahim</h1>
```

```
<button id="btn">
```

```
  Change Style
```

```
</button>
```

CSS

```
.big{  
  color:white;  
  background:blue;  
  font-size:45px;  
  padding:20px;  
  border-radius:15px;  
}
```

JavaScript

```
let title = document.querySelector("#title");
```

```
let btn = document.querySelector("#btn");
```

```
btn.addEventListener("click", function(){
```

```
  title.className = "big";
```

```
});
```

What happens?

Before clicking

Hello Ibrahim

↓

Click button

↓

JavaScript says

Replace the class with "big".

↓

The browser finds

```
.big{  
  color:white;  
  background:blue;  
  font-size:45px;  
}
```

↓

The heading changes.

Understanding the syntax

```
title.className = "big";
```

Let's break it down.

title

The element we selected.

```
let title = document.querySelector("#title");  
.className
```

The class attribute of the element.

=

Means "assign" or "give it this value."

"big"

The name of the CSS class.

Notice:

✗ Don't write

```
title.className = ".big";
```

The dot is only used in CSS selectors.

Correct:

```
title.className = "big";
```

Example 2

HTML

```
<h2 id="status">
```

Offline

```
</h2>
```

```
<button id="login">
```

Login

```
</button>
```

CSS

```
.online{  
  color:green;  
  font-size:40px;  
}
```

JavaScript

```
let status = document.querySelector("#status");
```

```
let login = document.querySelector("#login");
```

```
login.addEventListener("click", function(){
```

```
  status.innerHTML = "Online";
```

```
  status.className = "online";
```

```
});
```

After clicking, the text becomes Online and turns green.

What if the element already has a class?

Suppose HTML is:

```
<h1 id="title" class="red">
```

Welcome

```
</h1>
```

And JavaScript does this:

```
title.className = "big";
```

The old class red is removed and replaced with big.

Final result:


```
<h1 class="big">
```

The red class is gone.

Difference between className and classList

className

```
title.className = "big";
```

It replaces the existing class.

Example:

Before:

```
<h1 class="red">
```

After:

```
<h1 class="big">
```

The red class disappears.

```
classList.add()
```

```
title.classList.add("big");
```

It adds another class without removing the old one.

Before:

```
<h1 class="red">
```

After:

```
<h1 class="red big">
```

Now the element has two classes.

Which one is used more today?

Modern developers mostly use:

✓ classList.add()

✓ classList.remove()

✓ classList.toggle()

because they are more flexible.

className is still useful, especially when you want to completely replace the existing class.

 Where is className used on real websites?

You'll often find it in:

Older websites built years ago.

Legacy company projects that haven't been fully updated.

Cases where a developer wants to completely switch one theme or state for another with a single line of code.

 Day 25 Summary

Today you learned two ways to work with CSS classes from JavaScript:

1. classList

add() → Add a class.

remove() → Remove a class.

toggle() → Add it if it's missing, remove it if it's already there.

contains() → Check if a class exists.

2. className

Replaces the element's current class with a new one.

Tomorrow's mini project will combine both classList and className so you can practice deciding when to add a class and when to replace one. I think you'll enjoy it because it feels like building a real interactive webpage. 😎

Good choice. 👍 In modern JavaScript, classList is used much more often than className.

If I were teaching someone to become a frontend developer today, I would focus much more on:

- ✓ classList.add()
- ✓ classList.remove()
- ✓ classList.toggle()
- ✓ classList.contains()

You'll still recognize className if you see it in older code, but for your own projects, using classList is a great habit.

 Day 25 Mini Project: Student Portal Theme
 Goal

Practice everything you learned today:

querySelector()
querySelectorAll()
classList.add()
classList.remove()
classList.toggle()
HTML

Create:

One <h1> with id="title"
One <p> with id="message"
Five elements with class="subject"

Subjects:

HTML

CSS

JavaScript

Responsive Design

DOM

Create 4 buttons

Login
Logout
Dark Mode
Highlight Subjects
CSS

Create these classes.

Class 1
.loginStyle{

```
background:green;
color:white;
padding:20px;
border-radius:20px;
}
```

Class 2

```
.darkMode{
background:black;
color:white;
}
```

Class 3

```
.highlight{
color:blue;
font-size:28px;
}
```

JavaScript

Login Button

When clicked

Change heading to

Welcome Ibrahim Musa

Change paragraph to

You have successfully logged in.

Add

```
classList.add("loginStyle")
```

to the heading.

Logout Button

When clicked

Heading becomes

Welcome Guest

Paragraph becomes

Please login.

Remove the class

```
classList.remove("loginStyle")
```

Dark Mode Button

Use

```
classList.toggle("darkMode")
```

on the body.

Each click should switch between light mode and dark mode.

Highlight Subjects Button

Use

```
querySelectorAll()
```

Loop through all subjects using a for loop.

Add

highlight

to every subject.

★ Bonus Challenge (Optional)

When the Highlight Subjects button is clicked again, instead of always adding the class, make it toggle the highlight on each subject.

(Hint: use `classList.toggle()` inside the loop.)

If you get this working, you're thinking like a real JavaScript developer.

JAVASCRIPT

```
//Identification
```

```
let title = document.querySelector("#title");
```

```
let message = document.querySelector("#message");
```

```
let Subjects = document.querySelectorAll(".subject");
```

```
let loginBtn = document.querySelector("#loginBtn");
```

```

let logoutBtn = document.querySelector("#logoutBtn");

let darkBtn = document.querySelector("#darkBtn");

let highlightBtn = document.querySelector("#highlightBtn");

//Add Event Listener
loginBtn.addEventListener("click",function () {
    title.innerHTML ="Welcome Ibrahim Musa";

    message.innerHTML ="You have successfully logged in.";

    title.classList.add("loginStyle");
});

logoutBtn.addEventListener("click",function () {
    title.innerHTML ="Welcome Guest"
;

message.innerHTML ="Please login.";

title.classList.remove("loginStyle");

for(i=0; i<Subjects.length; i++){
    Subjects[i].classList.remove("highLight");
};
body.classList.remove("darkMode");
});

darkBtn.addEventListener("click",function () {
    body.classList.toggle("darkMode");
});

highlightBtn.addEventListener("click",function () {

    for(i=0; i<Subjects.length; i++){
        Subjects[i].classList.toggle("highLight");
    };
});

```

HTML

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```
<meta charset="UTF-8">
<title>Class List</title>
<link rel="stylesheet"
href="JavaScript.css">
</head>
<body id="body">
  <h1 id="title">Welcome Guest</h1>
<p id="message">Please login.</p>

<ol class="subject">
  <li class="subject">
    HTML
  </li>
  <li class="subject">
    CSS
  </li>
  <li class="subject">
    JavaScript
  </li>
  <li class="subject">
    Responsive Design
  </li>
  <li class="subject">
    DOM
  </li>
</ol>

<button id="loginBtn">Login Style</button>
<button id="logoutBtn">Logout</button>
<button id="darkBtn">Dark Mode</button>
<button id="highlightBtn">Highlight Suubjects</button>

<script src="script.js"></script>
</body>
</html>
```

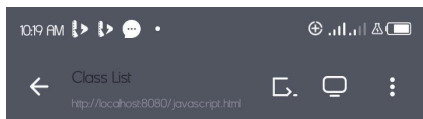
CSS

```
.darkMode{
  background:black;
  color:white;

}
```

```
.loginStyle{
  background:black;
  text-align:center;
  color:white;
  padding:20px;
  border-radius:30px;
}
```

```
.highLight{
  color:blue;
  font-size:28px;
}
```



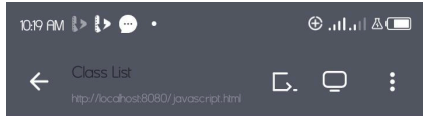
Welcome Guest

Please login.

1. HTML
2. CSS
3. JavaScript
4. Responsive Design
5. DOM

Login Style Logout Dark Mode

Highlight Suubjects



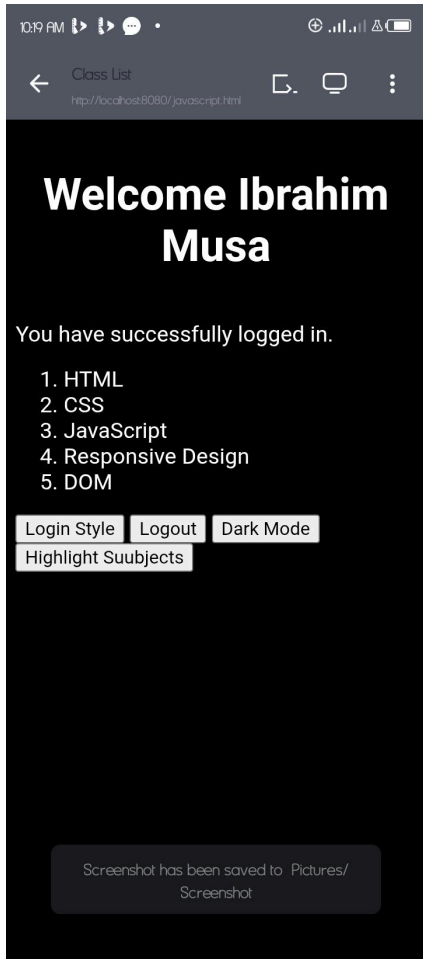
**Welcome Ibrahim
Musa**

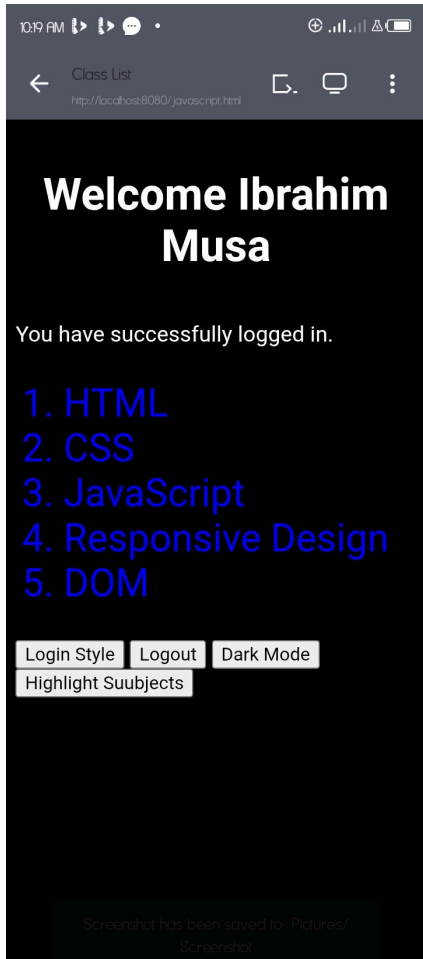
You have successfully logged in.

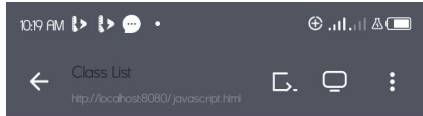
1. HTML
2. CSS
3. JavaScript
4. Responsive Design
5. DOM

Login Style Logout Dark Mode
Highlight Suubjects

Screenshot has been saved to Pictures/
Screenshot







Welcome Guest

Please login.

1. HTML
2. CSS
3. JavaScript
4. Responsive Design
5. DOM

Login Style Logout Dark Mode Highlight Suubjects

Marks

This project is worth 100 marks.

HTML Structure → 20 marks

CSS Classes → 20 marks

JavaScript Logic → 40 marks

Clean Code & Proper Indentation → 20 marks

As always, send me screenshots of your HTML, CSS, and JavaScript tomorrow, and I'll mark everything carefully.

Tomorrow's Two Topics (Day 26)

We'll continue with two important DOM topics:

Topic 1: createElement()

Learn how JavaScript can create brand-new HTML elements while the page is running.

Example:

```
let p = document.createElement("p");
```

Real websites use this for:

- Adding new comments
- Creating chat messages
- Building new cards
- Creating notifications

Topic 2: `appendChild()`

After creating an element, you'll learn how to place it onto the webpage.

Example:

```
document.body.appendChild(p);
```

Real websites use this to:

- Add new products to a shopping cart
- Display new messages in a chat
- Show notifications
- Add new tasks to a to-do list

These two topics fit together perfectly because first you create an element, then you append (attach) it to the page.

Have a good coding session, Coach's student! 🧐💻 Tomorrow we'll start making webpages that can create their own content dynamically. That's where JavaScript starts to feel really powerful.

Day 26 – Topic 1: `createElement()`

Today we're learning one of the most exciting DOM methods.

What is `createElement()`?

Imagine you have an empty notebook.

You haven't written anything yet.

When you take your pen and write a new sentence...

...you have created something new.

JavaScript does exactly that with HTML.

It can create brand-new HTML elements while the webpage is running.

Syntax

```
document.createElement("tagName");
```

Example:

```
let heading = document.createElement("h1");
```

Meaning:

"JavaScript, please create a new `<h1>` element."

Important!

At this point, the element exists only in JavaScript memory.

It is not visible on the webpage yet.

Think of it like buying a new chair.

The chair exists...

but it's still in the delivery truck.

You haven't brought it into your house yet.

Example 1 – Create a Paragraph

HTML

```
<body>
```

```
</body>
```

JavaScript

```
let paragraph = document.createElement("p");
```

Now JavaScript has created:

```
<p></p>
```

But you still can't see it because it hasn't been added to the page.

Example 2 – Create a Button

```
let button = document.createElement("button");
```

JavaScript creates

```
<button></button>
```

Still invisible.

Example 3 – Create an Image

```
let image = document.createElement("img");
```

Creates

```
<img>
```

Again...

Not visible yet.

Giving the Element Content

Once you've created an element, you can put text inside it.

```
let heading = document.createElement("h1");
```

```
heading.innerHTML = "Welcome Ibrahim!";
```

Now JavaScript has created:

```
<h1>Welcome Ibrahim!</h1>
```

Still not visible until we attach it to the page.

Another example:

```
let paragraph = document.createElement("p");
```

```
paragraph.innerHTML = "JavaScript is fun!";
```

Now JavaScript has:

```
<p>JavaScript is fun!</p>
```

Creating Multiple Elements

```
let title = document.createElement("h2");
```

```
let message = document.createElement("p");
```

```
let button = document.createElement("button");
```

Now JavaScript has created three separate elements.

Where is `createElement()` used on real websites?

Almost everywhere!



Shopping websites

When you add a product to your cart, JavaScript creates a new product card.



WhatsApp

When someone sends a message, JavaScript creates a new message bubble.



YouTube

When new comments load, JavaScript creates new comment elements.



Facebook

Every new post that appears on your feed is created dynamically.



To-Do Apps

When you click Add Task, JavaScript creates a new task element.

Symbols Breakdown

```
let title = document.createElement("h1");
```


let → create a variable.

title → the variable name.

= → store the value.

document → the webpage.

. → use something that belongs to the document.

createElement → create a new HTML element.

() → pass information to the method.

"h1" → the type of element to create.

; → end of the statement.

Quick Quiz

What does createElement() do?

- A. Deletes an HTML element
- B. Creates a brand-new HTML element
- C. Changes CSS
- D. Refreshes the page

Which creates a paragraph?

A.

document.createElement("h1")

B.

document.createElement("p")

C.

document.createElement("div")

D.

```
document.createElement("img")
```

After using `createElement()`, is the element immediately visible on the webpage?

A. Yes

B. No

Reply with your answers (for example: 1B, 2B, 3B). After that, we'll continue to Topic 2: `appendChild()`, where you'll learn how to place the newly created element onto the webpage so users can actually see it. That's the step that brings everything together.

1B

2B

3A

SO IS THAT IT IF IT IS NOT VISIBLE WHY USE IT LET'S GO TO THE SECOND TOPIC

Excellent, Ibrahim! 🤖

Quiz Marking



B — Correct



B — Correct



The correct answer is B (No).

Don't worry! Your mistake actually leads to the next topic.

You asked:

"If it is not visible, why use it?"

That's a fantastic question. The answer is:

Because `createElement()` only creates the element. We still need to put it on the webpage.

Think of it like this:



Imagine you buy a new TV.

Buying the TV = `createElement()`

Bringing it into your living room = `appendChild()`

If you only buy it but never bring it inside, you won't see it in your house.

JavaScript works the same way.

Day 26 - Topic 2: appendChild()

This topic is the partner of `createElement()`.

They are almost always used together.

What is `appendChild()`?

It adds a newly created HTML element to the webpage.

Think of it as:

"Attach this new element to the page."

Syntax

```
parentElement.appendChild(childElement);
```

Let's understand the words.

Parent = the container

Child = the new element you created

Example:

If HTML is

```
<body>
```

```
</body>
```

The body is the parent.

If JavaScript creates

```
let heading = document.createElement("h1");
```

Then heading is the child.

Example 1

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Append Child</title>
</head>
<body>

<script src="script.js"></script>

</body>
</html>
```

Notice:

The body is empty.

JavaScript

```
let heading = document.createElement("h1");
```

```
heading.innerHTML = "Welcome Ibrahim";
```

```
document.body.appendChild(heading);
```

Step by step

Step 1

JavaScript creates

```
<h1></h1>
```

Step 2

JavaScript writes inside it

```
<h1>Welcome Ibrahim</h1>
```

Step 3

JavaScript says

```
document.body.appendChild(heading);
```

Meaning:

"Put this heading inside the body."

Now the webpage shows:

Welcome Ibrahim

🎉 Now it is visible!

Example 2 - Create a Paragraph

JavaScript

```
let paragraph = document.createElement("p");  
  
paragraph.innerHTML = "I am learning JavaScript.";  
  
document.body.appendChild(paragraph);
```

Output

I am learning JavaScript.

Example 3 - Create a Button

```
let button = document.createElement("button");  
  
button.innerHTML = "Click Me";  
  
document.body.appendChild(button);
```

Output

[Click Me]

Example 4 - Create an Image

```
let image = document.createElement("img");  
  
image.src = "photo.jpg";  
  
document.body.appendChild(image);
```

Now the image appears on the webpage.

Why not just write the HTML?

Instead of writing

`<h1>Welcome</h1>`

inside HTML,

JavaScript creates it when it is needed.

Examples:

A user signs in.

JavaScript creates

Welcome Ibrahim

only after login.

Someone sends a message on WhatsApp.

JavaScript creates

Hello!

and adds it to the chat.

Someone adds a task.

JavaScript creates

✓ Buy Bread

and adds it to the to-do list.

The Formula

You will use these two methods together very often:

```
let element = document.createElement("tag");
```

```
element.innerHTML = "Some text";
```

```
document.body.appendChild(element);
```

This is one of the most common patterns in JavaScript.

Real Website Examples



Shopping Website

Click Add to Cart



JavaScript creates a new cart item.



appendChild() puts it into the shopping cart.



WhatsApp

Receive a message



JavaScript creates a message bubble.



appendChild() places it into the chat.



To-Do App

Click Add Task



JavaScript creates a new .



appendChild() adds it to the task list.



News Website

Scroll down



JavaScript creates more news cards.

↓

`appendChild()` adds them to the page.

Summary of Today's Two Topics
`createElement()`

Purpose: Creates a brand-new HTML element.

Example:

```
let p = document.createElement("p");
```

At this point, it exists only in JavaScript.

`appendChild()`

Purpose: Adds the created element to the webpage.

Example:

```
document.body.appendChild(p);
```

Now the user can see it.

The Two Always Work Together

```
let heading = document.createElement("h1");
```

```
heading.innerHTML = "Hello World";
```

```
document.body.appendChild(heading);
```

This is the pattern you'll use many times in real projects.



Homework (No Code Given)

Tonight, build a "My Daily Goals" page.

Requirements:

Create an empty HTML page with only a heading like "My Daily Goals".

In JavaScript, create three new paragraphs using `createElement()`.

Give each paragraph a different goal (for example, studying, exercising, reading).

Use `appendChild()` to add all three paragraphs to the webpage.
Finally, create a button with JavaScript that says "Finished for Today" and add it to the page with `appendChild()`.

Try writing the JavaScript yourself without looking back at the examples first. If you get stuck, then check your notes. That's how you'll build confidence.

You're doing a great job, Coach's student. 💪

JAVASCRIPT

```
let paragraph = document.createElement("p");

paragraph.innerHTML = "Am Studying Computer Science ";

document.body.appendChild(paragraph);

let message = document.createElement("p");

message.innerHTML="Exercising Is Good For The Health";

document.body.appendChild(message);

let text = document.createElement("p");

text.innerHTML="I Love Reading About Coding";

document.body.appendChild(text);

let button = document.createElement("button");

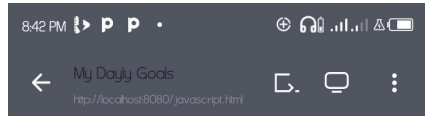
button.innerHTML="Finished for today";

document.body.appendChild(button);
```

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Dayly Goals</title>
</head>
```

```
<body>
  <h1>My Daily Goals.</h1>
  <script src="script.js"></script>
</body>
</html>
```



My Daily Goals.

Am Studying Computer Science

Exercising Is Good For The Health

I Love Reading About Coding

Finished for today



Day 27 - Topic 1: remove()

Yesterday we learned how to add elements.

Today we'll learn how to remove them.

Think of it like this:

appendChild() → Adds something to the page.

remove() → Takes something away from the page.

What is remove()?

It removes an HTML element from the webpage.

Syntax

```
element.remove();
```

Meaning:

"Remove this element."

Example 1

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Remove Example</title>
</head>
<body>

<h1 id="title">Welcome Student</h1>

<script src="script.js"></script>

</body>
</html>
```

JavaScript

```
let title = document.getElementById("title");
```

```
title.remove();
```

Output

Before:

Welcome Student

After running JavaScript:

(The heading disappears.)

Example 2 - Remove a Paragraph

HTML

```
<p id="message">
```

JavaScript is awesome!

```
</p>
```

```
JavaScript
let message =
document.getElementById("message");

message.remove();
```

The paragraph disappears from the page.

Example 3 - Remove a Button

HTML

```
<button id="deleteBtn">
Delete Me
</button>
```

JavaScript

```
let button =
document.getElementById("deleteBtn");

button.remove();
```

The button disappears.

Example 4 - Remove an Image

HTML

```

```

JavaScript

```
let photo =
document.getElementById("photo");

photo.remove();
```

The image disappears.

Removing After Clicking a Button

HTML

```
<h2 id="news">  
Breaking News!  
</h2>
```

```
<button id="hideBtn">  
Hide News  
</button>
```

JavaScript

```
let news =  
document.getElementById("news");  
  
let hide =  
document.getElementById("hideBtn");  
  
hide.addEventListener("click", function(){  
  
    news.remove();  
  
});
```

What happens?

When the user clicks Hide News, the heading disappears.

Where is `remove()` used on real websites?



Shopping websites

Click Remove Item → Product disappears from the cart.



To-Do Apps

Click the trash icon → Task disappears.



WhatsApp

Delete a message → The message disappears.



Email

Delete an email → It disappears from your inbox.

Notifications

Click × → The notification disappears.

Symbols Breakdown

`title.remove();`

`title` → the element you selected.

`.` → access something that belongs to that element.

`remove` → remove the element.

`()` → call the method.

`;` → end the statement.

Summary

`appendChild()`

➡ Adds an element.

`document.body.appendChild(title);`

`remove()`

➡ Removes an element.

`title.remove();`

They are opposites:

`appendChild()` → Add

`remove()` → Delete

Day 27 – Topic 2: replaceChild()

Yesterday we learned:

`appendChild()` → Add an element.

`remove()` → Delete an element.

Today we'll learn something in between.

Instead of deleting an element completely...

JavaScript can replace it with another element.

🤔 What is `replaceChild()`?

Imagine your school has an old notice board.

It says:

📢 Exams Start on Monday

Later the school changes the announcement.

Instead of removing the notice board, they replace the paper with a new one.

Now it says:

📢 Exams Start on Wednesday

The board stayed there.

Only the content was replaced.

That's exactly what `replaceChild()` does.

Syntax

```
parent.replaceChild(newElement, oldElement);
```

Let's understand every part.

parent

The container that holds both elements.

Example:

```
<body>
```

```
<h1>Hello</h1>
```

```
</body>
```

The body is the parent.

`replaceChild`

Means:

Replace one child element with another child element.

newElement

The new HTML element you created.

oldElement

The old HTML element already on the webpage.

Example 1

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1 id="title">
```

```
Welcome Student
```

```
</h1>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

The page shows

Welcome Student

JavaScript

Step 1

Select the old heading.

```
let oldHeading =
```

```
document.getElementById("title");
```

Step 2

Create a new heading.

```
let newHeading =
```

```
document.createElement("h1");
```


Step 3

Put text inside it.

```
newHeading.innerHTML =  
"Welcome Ibrahim";
```

Step 4

Replace the old heading.

```
document.body.replaceChild(  
newHeading,  
oldHeading  
);
```

What happens?

Before

Welcome Student

↓

JavaScript creates

Welcome Ibrahim

↓

JavaScript removes

Welcome Student

↓

JavaScript puts

Welcome Ibrahim

in the same place.

The user only sees:

Welcome Ibrahim

Example 2

HTML

```
<p id="message">
Loading...
</p>
```

JavaScript

```
let oldMessage =
document.getElementById("message");
```

```
let newMessage =
document.createElement("p");
```

```
newMessage.innerHTML =
"Data Loaded Successfully";
```

```
document.body.replaceChild(
newMessage,
oldMessage
);
```

Before

Loading...

After

Data Loaded Successfully
Example 3 (Button Click)

HTML

```
<h2 id="status">
Offline
</h2>
```

```
<button id="loginBtn">
Login
</button>
```

JavaScript

```
let status =
document.getElementById("status");
```

```
let button =
```

```
document.getElementById("loginBtn");

button.addEventListener("click", function(){

let online =
document.createElement("h2");

online.innerHTML = "Online";

document.body.replaceChild(
online,
status
);

});
```

When the user clicks Login, the word Offline is replaced with Online.

Think of it this way

`appendChild()`

Adds a new chair to a room.



`remove()`

Throws the chair away.



`replaceChild()`

Removes the old chair and puts a new chair in the same place.



Where is `replaceChild()` used on real websites?



News websites

Replace:

Loading News...

with

Today's Headlines

 Shopping websites

Replace:

Out of Stock

with

Available

 Games

Replace:

Game Over

with

You Win!

 Chat apps

Replace:

Sending...

with

Delivered ✓

Comparison of the Four Methods

Method	What it does
--------	--------------

createElement()	Creates a new element.
-----------------	------------------------

appendChild()	Adds the new element to the page.
---------------	-----------------------------------

remove()	Removes an element from the page.
----------	-----------------------------------

replaceChild()	Replaces one element with another.
----------------	------------------------------------

These four methods are like a toolkit:

Create something.

Add it.

Remove it when you don't need it.

Replace it when you want something different.

Homework (Mini Project)

Tonight's project is called "Student Status Board."

Requirements

Create a page with:

A heading that says "Student Status"

A paragraph that says "Waiting for update..."

A button that says "Update Status"

What should happen?

When the user clicks Update Status:

Create a new paragraph using `createElement()`.

The new paragraph should say: "Student has completed JavaScript DOM lessons."

Replace the old paragraph with the new one using `replaceChild()`.

Bonus Challenge: After replacing the paragraph, create another button using `createElement()` that says "Continue Learning" and add it to the page using `appendChild()`.

Day 27 Summary

Today you learned two very useful DOM methods:

`remove()` → Deletes an element from the page.

`replaceChild()` → Swaps an old element for a new one.

These, together with `createElement()` and `appendChild()`, give you the power to create, add, remove, and replace elements on a webpage dynamically.

Tomorrow we'll move to one of the most important JavaScript topics you'll use for the rest of your career: Arrays. Once you understand arrays, you'll be able to work with lists of data like products, users, messages, scores, and much more. This is a major milestone in becoming a frontend developer. 🚀

JAVASCRIPT

```
let heading = document.getElementById("student");
```

```
let paragraph = document.getElementById("message");
```

```
let button = document.getElementById("updateBtn");

button.addEventListener("click",function () {

    let newParagraph = document.createElement("p");

    newParagraph.innerHTML = "Student has completed JavaScript DOM lessons.";

    document.body.replaceChild(newParagraph,paragraph);

});

let newButton = document.createElement("button");

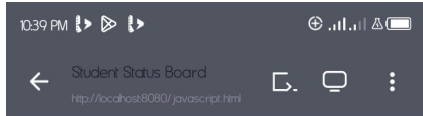
newButton.innerHTML = "Continue Learning";

document.body.appendChild(newButton);
```

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Status Board</title>
</head>
<body>
    <h1 id="student">Student Status</h1>
    <p id="message">Waiting For Update</p>

    <button id="updateBtn">
Update Status
    </button>
    <script src="script.js"></script>
</body>
</html>
```



Student Status

Waiting For Update

[Update Status](#) [Continue Learning](#)

Day 28 - Topic 1: setTimeout()

Today we're starting something completely new.

This is part of Asynchronous JavaScript.

Don't worry about that big word yet.

We'll understand it step by step.

What is `setTimeout()`?

Imagine I tell you:

Ibrahim...

Wait 5 seconds...

Then say:

"Hello!"

You don't say it immediately.

You wait.

Then you say it.

That is exactly what `setTimeout()` does.

It tells JavaScript:

Wait for a certain amount of time, then run this code.

Syntax

```
setTimeout(function () {
```

```
    // code here
```

```
}, timeInMilliseconds);
```

Let's understand every symbol.

`setTimeout`

This is the JavaScript function.

It means:

"Run this later."

(

Starts the function.

`function ()`

The code inside this function will run later.

`{ }`

Everything inside these braces waits.

,

Separates the code from the waiting time.

timeInMilliseconds

JavaScript measures time in milliseconds (ms).

Time conversion

1000 ms = 1 second

2000 ms = 2 seconds

3000 ms = 3 seconds

5000 ms = 5 seconds

10000 ms = 10 seconds

Example 1

```
setTimeout(function(){  
  
    alert("Hello Ibrahim");  
  
},3000);
```

What happens?

Immediately:

Nothing.

After 3 seconds:

Hello Ibrahim

appears.

Example 2

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1 id="title">
```

```
Loading...
```

```
</h1>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
let title =
```

```
document.querySelector("#title");
```

```
setTimeout(function(){
```

```
title.innerHTML = "Welcome Ibrahim";
```

```
},4000);
```

Output

When the page opens

Loading...

Four seconds later

Welcome Ibrahim

Example 3

HTML

```
<p id="message">
```

```
Please Wait...
```

```
</p>
```

JavaScript

```
let message =
```

```
document.querySelector("#message");
```

```
setTimeout(function(){  
  
message.innerHTML =  
"Registration Successful!";  
  
},5000);
```

The user first sees:

Please Wait...

After 5 seconds:

Registration Successful!

Where is setTimeout() used on real websites?

 Loading screens

Loading...



Website Ready!

 Online shopping

Added to Cart

The message disappears after a few seconds.

 Chat applications

Message Sent ✓

Shows briefly, then disappears.

 Welcome Popups

After entering a website:

Welcome to Our Website!

appears after 2–3 seconds.

Games

3...

2...

1...

GO!

Each message appears after a short delay.

Summary

`setTimeout()`

☒ Waits.

Then...

☒ Runs code once.

It does not repeat.

Quick Quiz

1. What does `setTimeout()` do?

A. Runs code after waiting for a period of time.

B. Repeats code forever.

C. Deletes an element.

D. Creates HTML.

2. How many milliseconds are in 5 seconds?

A. 500

B. 5000

C. 50

D. 50000

3. What will this code do?

```
setTimeout(function () {  
    alert("Hello");  
}, 2000);
```

A. Show the alert immediately.

B. Show the alert after 2 seconds.

C. Repeat the alert every 2 seconds.

D. Never show the alert.

Answer the quiz, and then we'll move to Topic 2: `setInterval()`, where JavaScript doesn't run the code just once—it runs it over and over again. 🚀

Day 28 - Topic 2: `setInterval()`

This topic is like the big brother of `setTimeout()`.

What is `setInterval()`?

Imagine a school bell.

🔔 Every 1 hour...

It rings.

Then another hour...

It rings again.

Then another hour...

It rings again.

It keeps repeating until someone stops it.

That's exactly what `setInterval()` does.

It repeats the same code after a certain amount of time.

Syntax

```
setInterval(function(){  
  
    // Code here  
  
}, timeInMilliseconds);
```

It looks almost exactly like `setTimeout()`.

The difference is:

`setTimeout()` → Runs once.

`setInterval()` → Runs again and again.

Example 1

```
setInterval(function(){  
  
    alert("Hello Ibrahim");  
  
},3000);
```

What happens?

After 3 seconds

Hello Ibrahim

Another 3 seconds

Hello Ibrahim

Another 3 seconds

Hello Ibrahim

It keeps repeating.

Example 2 - Digital Clock

HTML

```
<!DOCTYPE html>  
<html>
```

```
<body>

<h1 id="clock"></h1>

<script src="script.js"></script>

</body>
</html>
JavaScript
let clock = document.querySelector("#clock");

setInterval(function(){

    let time = new Date();

    clock.innerHTML = time.toLocaleTimeString();

},1000);
```

What happens?

Every second, the time updates.

10:30:01

10:30:02

10:30:03

10:30:04

This is how many digital clocks on websites work.

Example 3 - Countdown

```
<h1 id="count">10</h1>
let count = document.querySelector("#count");

let number = 10;

setInterval(function(){

    number--;

    count.innerHTML = number;
```

```
},1000);
```

The page shows:

10

9

8

7

6

5...

Every second, the number changes.

Where is `setInterval()` used?

 Digital clocks

The time changes every second.

 Weather websites

The weather information refreshes every few minutes.

 Stock market websites

Prices update automatically.

 Image sliders

Picture 1

↓

Picture 2

↓

Picture 3

↓

Picture 4

They change automatically every few seconds.

 Games

A game timer counts:

60

59

58

57...

Difference Between the Two

`setTimeout()`

Runs once.

Example:

Wait 3 seconds.

↓

Show "Welcome"

Finished.

`setInterval()`

Runs forever until stopped.

Example:

Every second

↓

Update the clock

Keeps going.

Summary

Method	What it does
<code>setTimeout()</code>	Waits, then runs code once.
<code>setInterval()</code>	Waits, then runs the code repeatedly.

Homework Project (No Code Given)

Build a "Welcome Screen & Clock".

Requirements

Create a heading that says:

Loading...

After 3 seconds, change it to:

Welcome Ibrahim!

(Use `setTimeout()`.)

Below the heading, create another heading for a digital clock.
Use `setInterval()` to update the current time every second.

Add a paragraph explaining:

The welcome message appears once after a delay, while the clock updates continuously.

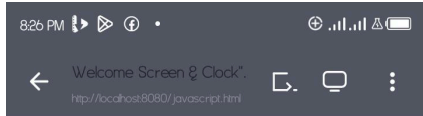
```
8:26 PM
javascript.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Welcome Screen &
  Clock.</title>
</head>
<body>
  <h1 id="title">Loading...</h1>
  <h2 id="clock">Digital Clock</h2>
  <p>The welcome message appears
  once after a delay, while the clock
  updates continuously.
</p>
  <script src="script.js"></script>
</body>
</html>
```

<div> <a> [class] [id] h
→| = " " < > </ - <!-- --

A screenshot of a mobile application interface for editing JavaScript code. The top status bar shows the time as 8:26 PM and various system icons. The app's header includes a folder icon, a tab labeled 'JS script.js', and a 'javascript' label with a code icon. Below the header is a navigation bar with a back arrow, a file icon, the filename 'script.js', a search icon, a save icon, a play icon, and a menu icon. The main area is a dark-themed code editor with line numbers 1 through 19 on the left. The code is as follows:

```
1 let heading =
  document.querySelector('#title');
2
3 setTimeout(function () {
4   heading.innerHTML = "Welcome
  Ibrahim!";
5 }, 3000);
6
7
8 let clock =
  document.querySelector("#clock");
9
10 setInterval(function () {
11   let time = new Date();
12
13   clock.innerHTML =
14   time.toLocaleTimeString();
15 }, 3000);
16
17
18
19
```

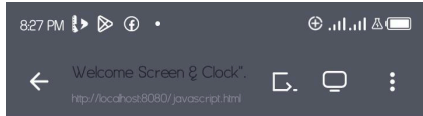
At the bottom is a keyboard with a back arrow on the left and a row of buttons: 'let', 'const', 'function', 'if', 'if...else', '=>', '{', '}', '(', ')', '=', '!', '&&', and '||'.



Loading....

Digital Clock

The welcome message appears once after a delay, while the clock updates continuously.



Welcome Ibrahim!

8:27:07 PM

The welcome message appears once after a delay, while the clock updates continuously.

🌐 Where you'll see these topics later

When we build your Weather App, you'll understand why these topics matter. The weather might refresh every few minutes.

When we build your Calculator, you won't need them much.

When we build a Digital Clock, you'll use `setInterval()`.

When we build a Quiz App, you'll use `setTimeout()` to show messages like "Correct!" before moving to the next question.

So even though these methods seem small now, they become very useful in real projects.

You're getting closer and closer to the advanced part of JavaScript. Keep going—you've been doing a great job. 🚀

DAY 29 — clearTimeout() and clearInterval()

You've already learned:

`setTimeout()`

and

`setInterval()`

Now we're learning how to STOP them.

Think of it like this:

`setTimeout()`

"Do something later."

`clearTimeout()`

"Wait! Don't do it anymore."

`setInterval()`

"Keep doing this again and again."

`clearInterval()`

"STOP! Don't repeat anymore."

That's the whole idea. Now let's go deep. 🙌

❶ `clearTimeout()`

Remember this:

```
setTimeout(function () {  
  alert("Hello Ibrahim!");  
}, 5000);
```

JavaScript says:

"After 5 seconds, show Hello Ibrahim."

But what if the user changes their mind before the 5 seconds are finished?

We can cancel it.

How?

First, save the timer inside a variable.

```
let timer = setTimeout(function () {  
    alert("Hello Ibrahim!");  
}, 5000);
```

Now JavaScript has given that timer an ID.

We stored that ID inside:

timer

Then:

```
clearTimeout(timer);
```

means:

"Cancel that timer."



Think about it like this

Imagine you tell your friend:

"Call me after 5 minutes."

That's:

```
setTimeout(...)
```

But after 2 minutes you say:

"Actually, don't call me."

That's:

```
clearTimeout(...)
```


😁 Simple!

② HTML Example

Let's make a small website.

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Clear Timeout</title>
</head>

<body>

  <h1 id="message">Waiting...</h1>

  <button id="startBtn">Start</button>
  <button id="cancelBtn">Cancel</button>

  <script src="script.js"></script>

</body>
</html>
```

We have:

```
<h1 id="message">Waiting...</h1>
```

and two buttons:

```
<button id="startBtn">Start</button>
<button id="cancelBtn">Cancel</button>
```

③ JavaScript

```
let message = document.querySelector("#message");

let startBtn = document.querySelector("#startBtn");

let cancelBtn = document.querySelector("#cancelBtn");

let timer;
```

Now let's create the timer.

```
startBtn.addEventListener("click", function () {  
  
    timer = setTimeout(function () {  
  
        message.innerHTML = "Welcome Ibrahim!";  
  
    }, 5000);  
  
});
```

When you click Start:

 JavaScript waits 5 seconds.

Then:

Welcome Ibrahim!

appears.

Now the Cancel button:

```
cancelBtn.addEventListener("click", function () {  
  
    clearTimeout(timer);  
  
});
```

Now imagine:

You click Start.

 1 second...

 2 seconds...

 3 seconds...

Then you click Cancel.

 The timer is cancelled.

The message will not appear.

★ Very important pattern

Memorize this structure:

```
let timer = setTimeout(function () {  
  
    // code  
  
}, 5000);
```

To stop it:

```
clearTimeout(timer);
```

The variable is important because it holds the timer's ID.

4 ***Now let's move to clearInterval()***

This one is even easier because you've already used setInterval().

You had:

```
setInterval(function () {  
  
    console.log("Hello");  
  
}, 1000);
```

That means:

```
Hello  
Hello  
Hello  
Hello  
Hello  
...
```

It never stops by itself.

So we need:

clearInterval()

🧠 Real-life example

Imagine a teacher says:

"Clap every 2 seconds."



2 seconds



2 seconds



2 seconds



And you say:

"STOP!"

That's:

clearInterval()

5) HTML Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Clear Interval</title>
```

```
</head>
```

```
<body>
```

```
  <h1 id="counter">0</h1>
```

```
  <button id="startBtn">Start</button>
```

```
  <button id="stopBtn">Stop</button>
```

```
  <script src="script.js"></script>
```

```
</body>
</html>
```

6 JavaScript

First:

```
let counter = document.querySelector("#counter");

let startBtn = document.querySelector("#startBtn");

let stopBtn = document.querySelector("#stopBtn");

let number = 0;

let interval;
```

Now start the interval:

```
startBtn.addEventListener("click", function () {

    interval = setInterval(function () {

        number++;

        counter.innerHTML = number;

    }, 1000);

});
```

Now watch what happens:

```
1
2
3
4
5
6
7
8
...
```

It keeps going.

Now stop it:

```
stopBtn.addEventListener("click", function () {  
  
    clearInterval(interval);  
  
});
```

If the counter is:

1
2
3
4
5

and you press STOP:



It stays at:

5

🔥 The BIG difference

This is extremely important:

Function	What it does
setTimeout()	Runs once after a delay
clearTimeout()	Cancels that timeout
setInterval()	Runs repeatedly
clearInterval()	Stops that repeating interval

Easy memory trick:

TIMEOUT = ONE TIME

setTimeout()
clearTimeout()

INTERVAL = AGAIN AND AGAIN

setInterval()
clearInterval()

🎯 Look at your clock project now

You currently have:

```
setInterval(function () {  
  
    let time = new Date();  
  
    clock.innerHTML = time.toLocaleTimeString();  
  
}, 3000);
```

You can now give that interval an ID:

```
let clockTimer = setInterval(function () {  
  
    let time = new Date();  
  
    clock.innerHTML = time.toLocaleTimeString();  
  
}, 1000);
```

And later:

```
clearInterval(clockTimer);
```



The clock stops updating.

That's exactly how Stop Clock, Pause Timer, Stop Animation, Countdown, and many other real websites/apps work.



Your Day 29 cheat sheet

```
// Start something once later  
let timer = setTimeout(function () {  
    // code  
}, 3000);
```

```
// Stop the timeout  
clearTimeout(timer);
```

And:

```
// Start something repeatedly  
let interval = setInterval(function () {  
    // code
```

```
}, 1000);
```

```
// Stop the interval
```

```
clearInterval(interval);
```

Remember this sentence:

set starts it, clear stops it.

That's the key idea for today's class. 🔥

And Ibrahim, the fact that you built the clock without just blindly following the example, then tried it in two ways, is exactly the kind of progress I want to see. Keep going. 🦊💻

Yes 😎 — you can absolutely make a message disappear using `setTimeout()`. That's actually one of the common ways it is used on websites.

The important thing is to understand that `setTimeout()` itself doesn't "disappear" anything. It simply waits, then runs whatever code you give it.

Example: Message disappears after 3 seconds

HTML

```
<h2 id="message">Welcome Ibrahim!</h2>
```

JavaScript

```
let message = document.querySelector("#message");
```

```
setTimeout(function () {  
  message.style.display = "none";  
}, 3000);
```

What happens:

Page loads

↓

Welcome Ibrahim!

↓

wait 3 seconds

↓

message disappears

3000 means 3000 milliseconds = 3 seconds.

You can also make it disappear using `innerHTML`


```
setTimeout(function () {  
  message.innerHTML = "";  
}, 3000);
```

This removes the text inside the element.

Or make it invisible

```
setTimeout(function () {  
  message.style.visibility = "hidden";  
}, 3000);
```

There is a small difference:

```
display = "none"
```

```
message.style.display = "none";
```

👉 The element is removed from the page layout.

```
visibility = "hidden"
```

```
message.style.visibility = "hidden";
```

👉 The element becomes invisible, but its space remains.

🔥 And this connects directly to `clearTimeout()`

Suppose you have:

```
let timer = setTimeout(function () {  
  message.style.display = "none";  
}, 5000);
```

The message will disappear after 5 seconds.

But if you do:

```
clearTimeout(timer);
```

before the 5 seconds finish, the disappearance is cancelled.

So remember:

`setTimeout()` = wait, then do something.

That "something" can be showing a message, changing text, changing CSS, hiding a message, removing something, etc.

So no, you haven't missed another major topic. Making something appear AND making something disappear are both applications of `setTimeout()`, and `clearTimeout()` is how you cancel the scheduled action.

MINI PROJECT — "Student Notification & Counter"

Build a small webpage called:

🎓 Ibrahim's Student Dashboard

Your page should have:

1. Welcome message

Initially display:

Welcome Ibrahim! 🙌

After 5 seconds, the message should disappear automatically using `setTimeout()`.

2. Cancel button

Add a button:

Cancel Welcome

If the user clicks it before the 5 seconds finish, the welcome message should NOT disappear.

You must use:

`clearTimeout()`

3. Counter

Display:

Study Time: 0

Add:

Start Counter button

When clicked, the number should increase every 1 second:

Study Time: 1
Study Time: 2
Study Time: 3
Study Time: 4
...

Use:

`setInterval()`
4. Stop button

Add:

Stop Counter

When clicked, the counter must stop increasing.

You must use:

`clearInterval()`
🎯 Your JavaScript must contain all four:
`setTimeout()`
`clearTimeout()`

`setInterval()`
`clearInterval()`

Don't just copy my examples from the lesson. Try to build it yourself. If you get stuck, think about the pattern we learned:

START → store timer ID → CLEAR using that ID

★ Bonus challenge

Add a button:

Show Welcome Again

When clicked, the welcome message should come back and the 5-second timer should start again.

This is optional, but I recommend trying it. 🔥

Submission

Build it and submit it tomorrow by midnight.

When you submit it, I'll mark:

HTML structure

JavaScript logic

setTimeout()

clearTimeout()

setInterval()

clearInterval()

Code organization

Bonus challenge

Don't worry if your code isn't perfect. I want to see YOUR attempt. 💪



What's next?

After you finish this project, our next two topics are:

Day 30

1 JavaScript Date & Time

2 JavaScript Math

We'll learn things like:

`new Date()`

and

`Math.random()`

`Math.floor()`

`Math.round()`

`Math.max()`

`Math.min()`

These are very useful for things like clocks, random numbers, games, quizzes, OTP-like number generation, and more.

Now go build that dashboard, Coach. 😎💻🔥

JAVASCRIPT

```
let heading = document.querySelector("#title");

let paragraph = document.querySelector("#message");

let button = document.querySelector("#cancelBtn");

let stopButton = document.querySelector("#stopBtn");

let showButton = document.querySelector("#showBtn");


let timer;

timer = setTimeout(function () {
    paragraph.innerHTML = "Welcome Ibrahim";
},3000);

timer = setTimeout(function () {
    paragraph.innerHTML = "";
},6000);

button.addEventListener("click",function () {
    clearTimeout(timer)
});


let counter = document.querySelector("#counter");

let startButton = document.querySelector("#startBtn");

let studyTime = 0;
let interval;

startButton.addEventListener("click",function () {
    clearInterval(interval);
    interval = setInterval(function () {
        studyTime ++
        counter.innerHTML =studyTime;
    },1000);
});
```

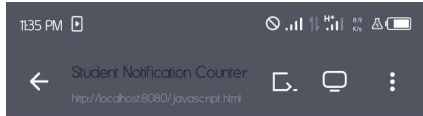
```
stopButton.addEventListener("click",function () {  
    clearInterval(interval)  
});
```

```
let Continue;  
let number = 0;
```

```
showButton.addEventListener("click",function () {  
    Continue = setInterval(function () {  
        number ++  
        counter.innerHTML =number;  
    },1000);  
    paragraph.innerHTML = "Welcome Ibrahim";  
  
});
```

HTML

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Student Notification Counter</title>  
</head>  
<body>  
    <h1 id="title">Ibrahim's Student Dashboard</h1>  
  
    <p id="message">Waiting</p>  
    <button id="cancelBtn">Cancel Button</button><br><br><br>  
    <button id="startBtn">Start Counter</button>  
    <h2 id="counter">Study Time:0</h2>  
    <button id="stopBtn">Stop Button</button>  
  
    <button id="showBtn">Continue</button>  
    <script src="script.js"></script>  
  
</body>  
</html>
```



Ibrahim's Student Dashboard

Welcome Ibrahim

Cancel Button

Start Counter

Study Time:0

Stop Button Continue

CORRECTION ON CODE

let continueInterval;

let number = 0;

showButton.addEventListener("click", function () {

clearInterval(continueInterval);

continueInterval = setInterval(function () {

number++;

counter.innerHTML = number;

}, 1000);

paragraph.innerHTML = "Welcome Ibrahim";

});

DAY 30 — JavaScript Date & Time

Now let's start the next topic.

Don't let the word Date confuse you.

JavaScript has a built-in object called:

Date

It allows JavaScript to work with:

today's date

current time

day

month

year

hours

minutes

seconds

dates in the future

dates in the past

Think of it as JavaScript's clock and calendar. 🕒 📅

1. Creating a Date

The most basic thing is:

```
let today = new Date();
```

Then:

```
console.log(today);
```

JavaScript might show something like:

Sun Aug 09 2026 23:31:00

The important part is:

new Date()

It means:

"JavaScript, give me the current date and time."

2. Getting the year

We can use:

getFullYear()

Example:

```
let today = new Date();
```

```
console.log(today.getFullYear());
```

Output:

2026

Think:

getFullYear()

↓

"What year is it?"

3. Getting the month

We use:

getMonth()

Example:

```
let today = new Date();
```

```
console.log(today.getMonth());
```

 *Important!*

JavaScript counts months from 0.

January → 0

February → 1

March → 2

April → 3

...

December → 11

So if you get:

7

that means August.

This is one of those things beginners often forget.

4. Getting the day of the month

Use:

getDate()

Example:

let today = new Date();

console.log(today.getDate());

If today is August 9:

9

Notice the difference:

getDate()

means day of the month.

5. Getting the day of the week

Use:

getDay()

Example:

```
let today = new Date();
```

```
console.log(today.getDay());
```

Again, JavaScript starts counting from 0:

Sunday → 0

Monday → 1

Tuesday → 2

Wednesday → 3

Thursday → 4

Friday → 5

Saturday → 6

So:

getDay()

does not give you the date like 9.

It gives you the day of the week.

6. Getting the hour

getHours()

Example:

```
let now = new Date();
```

```
console.log(now.getHours());
```

If the time is 8 PM:

20

JavaScript normally uses the 24-hour system here.

8 AM → 8

12 PM → 12

3 PM → 15

8 PM → 20

11 PM → 23

7. Getting minutes

`getMinutes()`

Example:

```
let now = new Date();
```

```
console.log(now.getMinutes());
```

8. Getting seconds

`getSeconds()`

Example:

```
let now = new Date();
```

```
console.log(now.getSeconds());
```

 Putting everything together

Imagine you want to build a digital clock.

```
let now = new Date();
```

```
console.log(now.getFullYear());
```

```
console.log(now.getMonth());
```

```
console.log(now.getDate());
```

```
console.log(now.getHours());
```

```
console.log(now.getMinutes());
```

```
console.log(now.getSeconds());
```

You are basically asking JavaScript:

What year is it?

What month is it?

What date is it?

What hour is it?

What minute is it?

What second is it?

9. Making a simple date message

HTML:

```
<h1 id="date"></h1>
```

JavaScript:

```
let date = document.querySelector("#date");
```

```
let today = new Date();
```

```
date.innerHTML = today;
```

The browser will display the current date and time.

10. Making it cleaner

We can extract the information:

```
let today = new Date();
```

```
let year = today.getFullYear();
```

```
let month = today.getMonth() + 1;
```

```
let day = today.getDate();
```

```
console.log(day + "/" + month + "/" + year);
```

Why + 1?

Because JavaScript starts months at 0.

So:

```
today.getMonth() + 1
```

turns:

7 → 8

for August.



A real website example

HTML:

```
<h1 id="title">Student Dashboard</h1>
```

```
<p id="date"></p>
```

JavaScript:

```
let date = document.querySelector("#date");
```

```
let today = new Date();
```

```
let day = today.getDate();
```

```
let month = today.getMonth() + 1;
```

```
let year = today.getFullYear();
```

```
date.innerHTML = day + "/" + month + "/" + year;
```

The page could show:

Student Dashboard

9/8/2026

That's already something you could use on a real website.

DAY 31 — JavaScript Math

Now we move to:

Math

Don't confuse this with your university mathematics course 😊.

JavaScript's Math object is basically a calculator toolbox.

It gives you useful tools for:

random numbers

rounding

finding bigger numbers

finding smaller numbers

powers

square roots

absolute values

1. Math.round()

This rounds a number to the nearest whole number.

```
console.log(Math.round(4.6));
```

Output:

5

And:

```
console.log(Math.round(4.3));
```

Output:

4

Think:

"Which whole number is this closest to?"

2. *Math.floor()*

floor means round DOWN.

```
console.log(Math.floor(4.9));
```

Output:

4

Even though 4.9 is close to 5, *floor()* says:

"Nope. Go down." 😊

3. *Math.ceil()*

ceil means round UP.

```
console.log(Math.ceil(4.1));
```

Output:

5

Even though it's only 4.1, *ceil()* says:

"Go up!"

🧠 Remember these three

`Math.round(4.6); // 5`

`Math.floor(4.6); // 4`

`Math.ceil(4.1); // 5`

Easy memory trick:

round → nearest

floor → down

ceil → up

4. `Math.random()`

🔥 This one is very important for web development.

`console.log(Math.random());`

It produces a random decimal between:

0

and less than:

1

For example:

0.237

0.891

0.453

0.019

Every time you run it, you can get a different number.

5. Random whole numbers

Suppose we want a random number from 1 to 10.

Use:

`let number = Math.floor(Math.random() * 10) + 1;`

`console.log(number);`

This can produce:

1
2
3
4
5
6
7
8
9
10

This technique is very important.

You'll eventually use it for things like:

games 🎮

random colors

random questions

random passwords

random items

dice

lottery-style effects

6. Random number from 1 to 100

`let number = Math.floor(Math.random() * 100) + 1;`

`console.log(number);`

Possible result:

73

Run it again:

14

Again:

91

7. `Math.max()`

Find the largest number.

```
console.log(Math.max(10, 25, 7, 50, 3));
```

Output:

50

8. *Math.min()*

Find the smallest number.

```
console.log(Math.min(10, 25, 7, 50, 3));
```

Output:

3

9. *Math.pow()*

Power.

```
console.log(Math.pow(2, 3));
```

That's:

$2 \times 2 \times 2$

Output:

8

You can also use:

$2 ** 3$

which also gives:

8

10. *Math.sqrt()*

Square root.

```
console.log(Math.sqrt(25));
```

Output:

5

Because:

$$5 \times 5 = 25$$

11. *Math.abs()*

Absolute value.

```
console.log(Math.abs(-20));
```

Output:

20

It basically removes the negative sign.

-20 → 20

-50 → 50

-7 → 7

🧠 *The big picture*

You don't need to memorize every Math method immediately.

Understand what each tool does:

<i>Method</i>	<i>Meaning</i>
<i>Math.round()</i>	<i>nearest whole number</i>
<i>Math.floor()</i>	<i>round down</i>
<i>Math.ceil()</i>	<i>round up</i>
<i>Math.random()</i>	<i>random number</i>
<i>Math.max()</i>	<i>biggest number</i>
<i>Math.min()</i>	<i>smallest number</i>
<i>Math.sqrt()</i>	<i>square root</i>
<i>Math.pow()</i>	<i>power</i>
<i>Math.abs()</i>	<i>remove negative</i>

🔥 *REAL WEBSITE EXAMPLE*

Imagine your website has a button that generates a random student score.

HTML

```
<h1 id="score">Score: 0</h1>
```

```
<button id="btn">Generate Score</button>
```

JavaScript

```
let score = document.querySelector("#score");
let btn = document.querySelector("#btn");

btn.addEventListener("click", function () {

    let randomScore = Math.floor(Math.random() * 101);

    score.innerHTML = "Score: " + randomScore;

});
```

Now every time you press the button:

Score: 74

Press again:

Score: 28

Press again:

Score: 96

 *That's JavaScript Math being used with the DOM.*

 *Today's class summary*
DAY 30

JavaScript Date & Time

Main tools:

new Date()

getFullYear()

getMonth()

getDate()

getDay()

getHours()

getMinutes()

getSeconds()

DAY 31

JavaScript Math

Main tools:

`Math.round()`
`Math.floor()`
`Math.ceil()`
`Math.random()`
`Math.max()`
`Math.min()`
`Math.sqrt()`
`Math.pow()`
`Math.abs()`

And notice something important, my student:

You are no longer just learning isolated JavaScript commands. You're starting to combine JavaScript with the DOM.

For example:

```
let score = document.querySelector("#score");  
  
let randomScore = Math.floor(Math.random() * 101);  
  
score.innerHTML = randomScore;
```

That's the kind of thinking that turns individual lessons into actual website features. 🚀

And yes, Coach approves the "teach me like I'm 15" instruction. 😎 It helps us make complicated concepts simple without making your final skills childish.

MINI PROJECT — Random Student Score Generator

Now it's your turn.

Your mission

Create a small Student Score Generator.

HTML requirements

Create:

An <h1> with:

Student Score Generator

A paragraph or heading showing:

Score: 0

A button:

Generate Score

JavaScript requirements

When the button is clicked:

Generate a random score between 0 and 100

Display it on the page.

Use `querySelector()`

Use `addEventListener()`

Use `Math.random()`

Use `Math.floor()`

Use `innerHTML`

★ Bonus

Add another element:

Result: Keep practicing!

Then use JavaScript to change the message depending on the score.

For example:

0–39 → Keep practicing! 💪

40–59 → You can do better! 👍

60–79 → Good job! 🎉

80–100 → Excellent! 🔥

Don't ask me for the JavaScript answer before you try it. 😎

Build it yourself and submit your screenshots/code to me. I'll mark it.



What's our next topic?

We've now completed:

Day 30: JavaScript Date & Time

Day 31: JavaScript Math

Our next class is:

🔥 Day 32 — JavaScript Strings & String Methods

We'll learn how JavaScript can work with and manipulate text, including useful methods such as:

***length
toUpperCase()
toLowerCase()
includes()
slice()
trim()
replace()***

And we'll do it one topic at a time, as you requested—not throw everything at you at once. 😎

Now go build that Random Student Score Generator. 💻🔥

```
12:00 PM
javascript.html
< > javascript.html x CSS
< > javascript.html Q
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Student Score Generator.
5 </title>
6 <link rel="stylesheet"
7   href="Javascript.css">
8 </head>
9 <body>
10  <h1 class="heading">Student
    Score Generator</h1>
11    <p class="score">Score: 0</p>
12
13    <h2 class="message">Keep
    Practicing</h2>
14
15    <button
    class="generateBtn">Generate Score</
    button>
16    <script src="script.js"></
    script>
17
18 </body>
19 </html>
20
<div> <a> <img> [class] [id] h
->| = " " < > </ - <!-- --
```

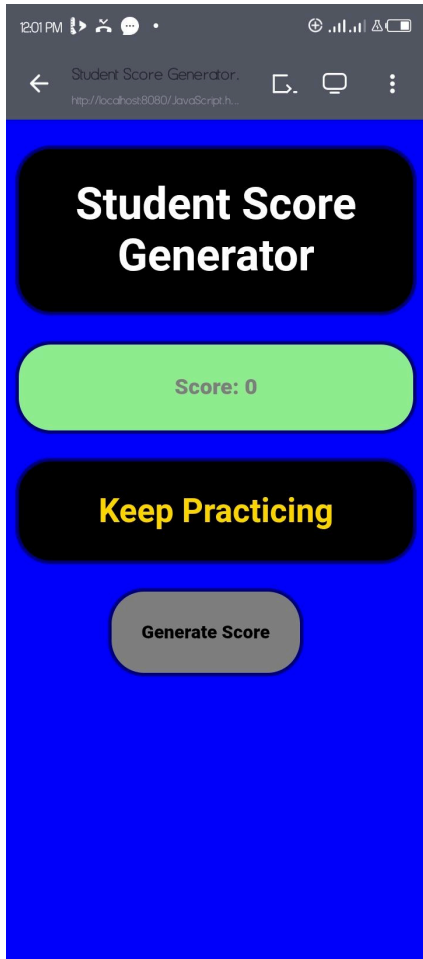


```
12:00 PM
JavaScript.css
1 body{
2   background:blue;
3 }
4
5 .style{
6   background:black;
7   padding:25px;
8   color:white;
9   border-radius:30px;
10  border:3px solid navy;
11  text-align:center;
12 }
13
14 .paragraph{
15   padding:25px;
16   color:gray;
17   font-weight:900;
18   border-radius:30px;
19   border:3px solid navy;
20   text-align:center;
21   background:lightgreen;
22 }
23
24 .result{
25   padding:25px;
26   color:gold;
27   border-radius:30px;
28   border:3px solid navy;
29   text-align:center;
30   background:black;
31 }
32
```

body background width height fon

→ { } : ; % ()

```
12:00 PM
ipt.css x JS script.js x
< > script.js
"Excellent!";
15 }else if(randomScore >= 60){
16     secondHeading.innerHTML =
"Good Job!";
17 }
18
19     else if(randomScore >= 40){
20         secondHeading.innerHTML = "You
Can Do Better!";
21     }
22
23     else if(randomScore >= 0){
24         secondHeading.innerHTML =
"Keep Practicing!";
25     };
26
27     score.innerHTML = "Score : " +
randomScore;
28
29
30 });
31
32 //Styles
33 heading.classList.add("style");
34 score.classList.add("paragraph");
35 secondHeading.classList.add("result"
);
e return for log alert px em
-> $ " ' ` | | : //
```



DAY 32 — JAVASCRIPT STRINGS & STRING METHODS

And yes, we're doing ONE topic at a time.

Today we're starting with:

● length

Don't worry about the other methods yet.

What is a String?

A string is simply text.

For example:

```
let name = "Ibrahim";
```

"Ibrahim" is a string.

Another example:

```
let school = "Federal University Lafia";
```

That's also a string.

Even a single letter can be a string:

```
let letter = "A";
```

And numbers can be inside strings too:

```
let phone = "08012345678";
```

Because it's inside quotes, JavaScript treats it as text, not a number.

🧠 What does `.length` do?

`.length` tells you how many characters are inside a string.

Example:

```
let name = "Ibrahim";
```

```
console.log(name.length);
```

Result:

7

Why?

Let's count:

I b r a h i m
1 2 3 4 5 6 7

So:

`name.length`

means:

"How many characters are inside name?"

⚠ Spaces also count!

Look at:

```
let name = "Ibrahim Musa";
```

```
console.log(name.length);
```

The space between Ibrahim and Musa is also a character.

So JavaScript counts:

Ibrahim

+

space

+

Musa

The result is:

12

That's important.

🔧 HTML + JavaScript Example

HTML

```
<h1 id="title">String Length</h1>
```

```
<p id="result"></p>
```

```
<script src="script.js"></script>
```

JavaScript

```
let title = document.querySelector("#title");
```

```
let result = document.querySelector("#result");
```

```
let name = "Ibrahim";
```

```
result.innerHTML = "Your name has " + name.length + " characters.";
```

The browser will show:

String Length

Your name has 7 characters.

🔥 Another example

```
let school = "Federal University Lafia";
```

```
console.log(school.length);
```

JavaScript counts every character, including spaces.

You don't need to manually count them.

That's the power of .length.

🧠 VERY IMPORTANT

Notice something:

We write:

```
name.length
```

NOT:

```
name.length()
```

Why?

Because length is a property, not a method.

For now, just remember:

Property

```
name.length
```

Method

Methods will look like:

```
name.toUpperCase()
```

See the ()?

We'll get there next. Don't jump ahead yet. 😎

💡 One rule for today's class

Don't worry about:

`toUpperCase()`

`toLowerCase()`

`includes()`

`slice()`

`trim()`

`replace()`

yet.

We're going to take them one by one.

Tonight's target is simply:

String → `.length` → count characters.

Once you've built your String Analyzer, submit it to me and I'll mark it. Then we'll move to the next string method. 😎🔥

NOW — DAY 32: `includes()`

Since you've already learned `.length`, `.toUpperCase()`, and `.toLowerCase()`, we're skipping them.

Today:

`includes()`

Think of `includes()` as a detective 🕵️.

It asks:

"Does this string contain what I'm looking for?"

And it gives you one of two answers:

true

or

false

1 Simple example

```
let name = "Ibrahim Musa";
```

```
console.log(name.includes("Ibrahim"));
```

JavaScript checks:

Does "Ibrahim Musa" contain "Ibrahim"?

Yes.

So:

true

Now:

```
console.log(name.includes("John"));
```

JavaScript asks:

Does "Ibrahim Musa" contain "John"?

No.

So:

false

🧠 Think of it like searching WhatsApp

Imagine your messages contain:

"I love learning JavaScript"

You ask:

"Does this message contain the word JavaScript?"

JavaScript checks:

```
message.includes("JavaScript")
```

Answer:

true

You ask:

"Does it contain Python?"

```
message.includes("Python")
```

Answer:

false

That's basically all `includes()` is doing.

② It is case-sensitive ⚠️

This is important.

```
let word = "JavaScript";
```

```
console.log(word.includes("JavaScript"));
```



true

But:

```
console.log(word.includes("javascript"));
```



false

Why?

Because:

JavaScript
javascript

are not exactly the same.

The J and S are uppercase in the first one.

③ HTML + JavaScript example

HTML

```
<h1>Message Checker</h1>
```

```
<p id="result"></p>
```

JavaScript

```
let message = "I love learning JavaScript";
```

```
let result = document.querySelector("#result");
```

```
let answer = message.includes("JavaScript");
```

```
result.innerHTML = answer;
```

The browser shows:

true

Because the message contains "JavaScript".

④ Let's make it more useful

Instead of showing:

true

we can show a proper message.

```
let message = "I love learning JavaScript";
```

```
let result = document.querySelector("#result");
```

```
if (message.includes("JavaScript")) {  
  result.innerHTML = "Yes! The message contains JavaScript.";  
} else {  
  result.innerHTML = "No! The message does not contain JavaScript.";  
}
```

Now the user sees:

Yes! The message contains JavaScript.

🔥 This is where includes() becomes useful in websites.

🌐 Where would a real website use this?

Imagine a search box.

A user types:

phone

Your website could check whether a product name or description includes "phone".

Or a website could check:

Does this email contain "@gmail.com"?

Or:

Does this username contain "admin"?

Or a quiz:

Does the user's answer include the correct word?

🎯 Simple example with an email
let email = "ibrahim@gmail.com";

```
console.log(email.includes("@"));
```

Result:

true

Because the email contains @.

You could also check:

```
console.log(email.includes("@gmail.com"));
```

Result:

true

🧠 One important thing

includes() doesn't change the string.

It only checks it and returns:

true

or:

false

So:

```
let name = "Ibrahim";
```

```
let answer = name.includes("bra");
```

```
console.log(answer);
```

gives:

true

because "bra" appears inside "Ibrahim".

🔑 Remember this sentence

.includes() asks: "Is this text inside my string?"

That's it. Don't cram the syntax.

When you need it:

```
myString.includes("something")
```

Just remember what question you're asking.

🔧 Quick check before we move on

Don't worry about the answer yet. Tell me what you think these produce:

1.

```
let name = "Ibrahim";
```

```
console.log(name.includes("Ibra"));
```

2.

```
let name = "Ibrahim";
```

```
console.log(name.includes("John"));
```

3.

```
let word = "JavaScript";
```

```
console.log(word.includes("javascript"));
```

Reply like you normally do:

1A

2B

3C

Then I'll check you and we'll continue. 🕶️💻🔥

✓ name.length.

✓ .toUpperCase()

✓ .toLowerCase()

DAY 32 — STRING METHOD: slice()

We already know:

includes()

asks:

"Does this text contain something?"

Now:

slice()

does something different.

It means:

"Take a piece of this string."

Think of a sentence as a loaf of bread 🍞. slice() lets you cut out a piece.

1 Simple example

```
let name = "Ibrahim";
```

```
let result = name.slice(0, 4);
```

```
console.log(result);
```

Result:

Ibra

Why?

JavaScript looks at the positions of the characters.

I	b	r	a	h	i	m
0	1	2	3	4	5	6

We said:

```
name.slice(0, 4)
```

Meaning:

Start at position 0 and stop before position 4.

So we get:

I b r a

➡ "Ibra"

⚠ Very important rule

The second number is not included.

`slice(0, 4)`

means:

Start: 0

Stop: 4

But position 4 itself is not taken.

So:

0 → l ✓
1 → b ✓
2 → r ✓
3 → a ✓
4 → h ✗ STOP

That's why we get:

lbra

② Another example

```
let word = "JavaScript";
```

```
console.log(word.slice(0, 4));
```

Positions:

J a v a S c r i p t
0 1 2 3 4 5 6 7 8 9

So:

`word.slice(0, 4)`

gives:

Java

Because positions 0, 1, 2, 3 are included.

③ Starting from another position

You don't have to start at 0.

```
let word = "JavaScript";
```

```
console.log(word.slice(4, 10));
```

Result:

Script

Because:

```
J a v a S c r i p t  
0 1 2 3 4 5 6 7 8 9  
    ↑  
  start
```

We start at 4:

S

and continue to the end.

④ What if we only give one number?

This is very useful.

```
let name = "Ibrahim";
```

```
console.log(name.slice(3));
```

Result:

ahim

Why?

```
I b r a h i m  
0 1 2 3 4 5 6  
    ↑  
  start
```

We said:

slice(3)

Meaning:

Start at position 3 and continue until the end.

So:

a h i m

➡ "ahim"

5 Negative numbers

This is another useful feature.

```
let name = "Ibrahim";
```

```
console.log(name.slice(-3));
```

Result:

him

Negative numbers count from the end.

Think:

```
I b r a h i m  
-7 -6 -5 -4 -3 -2 -1  
      ↑
```

So:

```
slice(-3)
```

means:

Give me the last 3 characters.

🌐 Real website example

Imagine a website displaying a long description:

```
let description = "JavaScript is a powerful programming language for the web.";
```

```
let shortText = description.slice(0, 20);
```

```
console.log(shortText);
```

You could use this to create a preview of a long article.

For example:

JavaScript is a power...

This is something you might see on:

Blog websites

News websites

Product cards

Social media

Search results



HTML + JavaScript example

HTML

```
<h1>Article Preview</h1>
```

```
<p id="preview"></p>
```

JavaScript

```
let preview = document.querySelector("#preview");
```

```
let article = "JavaScript is a powerful language used to make websites interactive.";
```

```
let shortArticle = article.slice(0, 30);
```

```
preview.innerHTML = shortArticle + "...";
```

The page will show only the first part of the article.



That's a real use of slice().



slice() vs includes()

This is important because they're completely different.

includes()

Asks:

"Is this text inside my string?"

```
name.includes("Ibra");
```

Returns:

true / false
slice()

Asks:

"Give me a piece of this string."

```
name.slice(0, 4);
```

Returns:

part of the string
🔑 Easy memory trick

INCLUDES = CHECK 🔍

```
text.includes("hello")
```

SLICE = CUT ✂️

```
text.slice(0, 5)
```

You don't need to memorize every number combination. Just remember what the method does and check your JOTINS when you need the exact syntax.

🎯 Quick check — WITH OPTIONS this time 😊

1. What does includes() do?

- A. Cuts a string
- B. Checks whether text exists inside a string
- C. Deletes a string
- D. Converts text to uppercase

2. What does this produce?

```
let name = "Ibrahim";
```

```
console.log(name.slice(0, 3));
```

A. lbr

- B. Ibra
- C. bra
- D. Ibrahim

3. What does this produce?

```
let word = "JavaScript";
```

```
console.log(word.slice(4));
```

- A. Java
- B. Script
- C. avaScript
- D. JavaS

Reply:

- 1B
- 2A
- 3B

Then we'll continue to the next string method. 🕶️🔥

Next String Method — *trim()*

This one is actually very useful for forms and user input.

What does `trim()` do?

`trim()` removes unnecessary spaces from the beginning and end of a string.

Imagine a user enters:

```
" Ibrahim "
```

There are spaces before and after the name.

JavaScript can clean them:

```
let name = " Ibrahim ";
```

```
let cleanName = name.trim();
```

```
console.log(cleanName);
```

Result:

Ibrahim

Important:

trim() removes spaces from the ends, not spaces between words.

For example:

```
let name = " Ibrahim Musa  ";
```

```
console.log(name.trim());
```

Result:

Ibrahim Musa

The space between Ibrahim and Musa stays.

🌐 Why is this useful on a website?

Imagine you have a login form.

The user types:

Ibrahim

If you don't clean it, JavaScript sees the spaces as part of the text.

But:

```
username.trim()
```

gives you:

Ibrahim

This is very common when handling forms and user input.

👤 HTML + JavaScript example

HTML

```
<input id="nameInput" placeholder="Enter your name">
```

```
<button id="btn">Submit</button>
```

```
<p id="result"></p>
```

JavaScript

```
let nameInput = document.querySelector("#nameInput");
```

```
let button = document.querySelector("#btn");
```

```
let result = document.querySelector("#result");
```

```
button.addEventListener("click", function () {
```

```
    let name = nameInput.value;
```

```
    let cleanName = name.trim();
```

```
    result.innerHTML = "Welcome " + cleanName;
```

```
});
```

If the user types:

Ibrahim

the website displays:

Welcome Ibrahim

instead of carrying those unnecessary spaces around.

🧠 One important thing

trim() doesn't change the original string.

For example:

```
let name = " Ibrahim ";
```

```
let cleanName = name.trim();
```

Now:

```
name    → " Ibrahim "
```

```
cleanName → "Ibrahim"
```

We created a cleaned version.

🔥 trim() vs slice()

Remember:

slice() ✂️

Takes a piece of a string:

```
name.slice(0, 4);  
trim() 🧹
```

Cleans spaces at the beginning and end:

```
name.trim();
```

Different jobs.

🎯 Quick test

1. What does trim() remove?

- A. All letters
- B. Spaces at the beginning and end
- C. Spaces between words
- D. Numbers

2. What will this produce?

```
let name = " Ibrahim ";
```

```
console.log(name.trim());
```

- A. " Ibrahim "
- B. "Ibrahim"
- C. "Ibrahim "
- D. " Ibrahim"

3. Which one is useful for cleaning user input from a form?

- A. slice()
- B. trim()
- C. includes()
- D. Math.random()

Reply:

1B

2B

3B

Then we'll continue to the next string method. 🕶️💻🔥

DAY 32 — replace()

We've reached the next string method.

Remember:

replace() = find some text and replace it with different text.

1 Simple example

```
let message = "I love football";
```

```
let newMessage = message.replace("football", "coding");
```

```
console.log(newMessage);
```

JavaScript looks at:

I love football

It finds:

football

and changes it to:

coding

Result:

I love coding

Think of it like using Find & Replace in a document.

2 The structure

The basic structure is:

```
string.replace("old text", "new text");
```

For example:

```
let name = "Ibrahim is a student";  
  
let result = name.replace("student", "coder");  
  
console.log(result);
```

Result:

Ibrahim is a coder

So:

```
.replace("student", "coder")
```

means:

Find "student" and replace it with "coder".

❸ replace() doesn't destroy the original string

Look at this:

```
let message = "I love football";  
  
let newMessage = message.replace("football", "coding");  
  
console.log(message);  
console.log(newMessage);
```

You'll get:

I love football
I love coding

The original:

message

still contains:

I love football

The new variable:

newMessage

contains:

I love coding

That's useful to understand.

④ What if the word isn't there?

```
let message = "I love coding";
```

```
let result = message.replace("football", "JavaScript");
```

```
console.log(result);
```

There is no "football" inside the string.

So JavaScript leaves it alone:

I love coding

Nothing gets replaced.

⑤ replace() is case-sensitive

Just like includes().

```
let message = "I Love JavaScript";
```

```
let result = message.replace("javascript", "Python");
```

```
console.log(result);
```

It won't replace it because:

JavaScript

and:

javascript

are different because of capitalization.

REAL WEBSITE EXAMPLE

Imagine a website has a message:

```
<p id="message">I love learning football.</p>
```

```
<button id="btn">Change Topic</button>
```

We want the button to change football to coding.

JavaScript:

```
let message = document.querySelector("#message");
let button = document.querySelector("#btn");

button.addEventListener("click", function () {

    let newMessage = message.innerHTML.replace("football", "coding");

    message.innerHTML = newMessage;

});
```

Before clicking:

I love learning football.

After clicking:

I love learning coding.

🔥 That's replace() working with the DOM.

🧠 Let's understand that code

This:

```
let message = document.querySelector("#message");
```

means:

Find the paragraph.

This:

```
let button = document.querySelector("#btn");
```

means:

Find the button.

This:

```
button.addEventListener("click", function () {
```

means:

Wait for the button to be clicked.

Then:

```
let newMessage = message.innerHTML.replace("football", "coding");
```

means:

Take the paragraph's text, find "football", and replace it with "coding".

Finally:

```
message.innerHTML = newMessage;
```

means:

Put the new text back into the paragraph.

⚠ Something important about replace()

By default, replace() normally replaces the first matching occurrence.

Example:

```
let text = "I like coding. Coding is fun.";
```

```
let result = text.replace("coding", "JavaScript");
```

```
console.log(result);
```

The result is:

I like JavaScript. Coding is fun.

Only the first lowercase "coding" was replaced.

The second one is uppercase "Coding", so it doesn't match anyway.

We'll eventually learn ways to replace multiple occurrences, but don't worry about that yet.



Compare everything you've learned

You're now building a useful toolbox:

```
includes() 🔍  
text.includes("coding")
```

"Does this text contain coding?"

Returns:

```
true / false  
slice() ✂️  
text.slice(0, 5)
```

"Give me a piece of this text."

```
trim() 🧹  
text.trim()
```

"Remove unnecessary spaces from the beginning and end."

```
replace() 🖋️  
text.replace("old", "new")
```

"Change this piece of text to something else."



QUICK TEST

1. What does replace() do?

A. Counts characters

- B. Checks whether text exists
- C. Replaces one piece of text with another
- D. Removes spaces

2. What will this produce?

```
let message = "I love football";
```

```
let result = message.replace("football", "coding");
```

```
console.log(result);
```

- A. I love football
- B. I love coding
- C. football coding
- D. coding football

3. What happens if the text you're trying to replace doesn't exist?

```
let text = "I love coding";
```

```
let result = text.replace("football", "JavaScript");
```

- A. The whole string disappears
- B. It becomes JavaScript
- C. Nothing is replaced
- D. JavaScript gives an error

Reply exactly like:

1C

2B

3C

Then we'll continue to the next topic. 🧐💻🔥

Absolutely 🧐. First, let's quickly summarize only the two you asked for, then I'll give you one project combining all the String methods you've covered.

 1. .toUpperCase()

It changes letters to CAPITAL LETTERS.

```
let name = "ibrahim";
```

```
console.log(name.toUpperCase());
```

Result:

IBRAHIM

Think:

toUpperCase() = make the letters BIG/CAPITAL

 2. .toLowerCase()

It changes letters to small letters.

```
let name = "IBRAHIM";
```

```
console.log(name.toLowerCase());
```

Result:

ibrahim

Think:

toLowerCase() = make the letters small

Together:

```
let word = "JavaScript";
```

```
console.log(word.toUpperCase());  
// JAVASCRIPT
```

```
console.log(word.toLowerCase());  
// javascript
```

That's all you need to remember. 👍

 **MINI PROJECT — "Student Text Analyzer"**

Now let's combine everything you've learned.

Your project must use:

.length
.toUpperCase()
.toLowerCase()
.includes()
.slice()
.trim()
.replace()
🎯 What you're building

Create a webpage called:

Student Text Analyzer

The user should enter a sentence into an <input>.

Example:

[I love learning JavaScript]

[Analyze Text]

When they click the button, JavaScript should analyze the text.

Your website should display:

1. Original text

Show what the user entered.

2. Character count

Use:

.length

Example:

Characters: 28

3. Uppercase version

Use:

.toUpperCase()

Example:

Uppercase: I LOVE LEARNING JAVASCRIPT

4. Lowercase version

Use:

```
.toLowerCase()
```

Example:

Lowercase: i love learning javascript

5. Check for "JavaScript"

Use:

```
.includes()
```

Display something like:

Contains JavaScript: Yes

or:

Contains JavaScript: No

6. Create a short version

Use:

```
.slice()
```

Take the first 20 characters.

Example:

Preview: I love learning Jav...

7. Remove unnecessary spaces

Use:

```
.trim()
```

Remember, the user might type:

I love JavaScript

Your program should clean those spaces.

8. Replace a word

Use:

`.replace()`

For example, replace:

JavaScript

with:

Coding

So:

I love learning JavaScript

becomes:

I love learning Coding

🧠 Your challenge

I am not giving you the JavaScript answer this time. 😎

You already know the individual tools. Your job is to figure out how to connect them.

You will probably need things you've already learned:

`document.querySelector()`

`addEventListener()`

`input.value`

and your String methods:

`.length`

`.toUpperCase()`

`.toLowerCase()`

`.includes()`

`.slice()`

`.trim()`

`.replace()`

★ BONUS — Your choice

Add a Clear button that clears the input and results.

You already know enough JavaScript to attempt it.

🏆 What I'm looking for

Don't worry if the code isn't perfect.

I want to see that you can think:

User types



Button clicked



Get the input



Clean it



Analyze it



Display the results

Build it yourself and submit it. I'll mark it line by line, and then we'll move on to Day 33: Forms & User

DAY 33 — FORMS & USER INPUT

This is an important class because now we're moving from:

"JavaScript works with information I put inside the code."

to:

"JavaScript can receive information from a real person using my website."

Imagine a website like this:

STUDENT REGISTRATION

Name: [Ibrahim Musa]

Age: [21]

Course: [Computer Science]

[Submit]

The user types the information.

JavaScript can then take that information and do something with it.



Topic 1: input.value

Let's start with just one thing.

What is .value?

Suppose your HTML has:

```
<input id="name">
```

A user types:

Ibrahim

JavaScript can get what the user typed using:

input.value

Think of .value as:

"Give me whatever the user typed inside this input."



Simple example

HTML

```
<input id="name">
```

```
<button id="btn">Submit</button>
```

```
<p id="result"></p>
```

JavaScript

```
let input = document.querySelector("#name");
```

```
let button = document.querySelector("#btn");
```

```
let result = document.querySelector("#result");
```

```
button.addEventListener("click", function () {
```

```
    let name = input.value;
```

```
    result.innerHTML = "Hello " + name;
```

```
});
```

Now imagine the user types:

Ibrahim

and clicks:

Submit

The website displays:

Hello Ibrahim

🔥 That's user input.

🧠 Let's understand it slowly

This:

```
let input = document.querySelector("#name");
```

means:

Find the input box.

This:

```
let button = document.querySelector("#btn");
```

means:

Find the button.

This:

```
button.addEventListener("click", function () {
```

means:

Wait until the user clicks the button.

Then this is the new important part:

```
let name = input.value;
```

Suppose the user typed:

Ibrahim

Then:

input.value

contains:

"Ibrahim"

So we're basically saying:

What the user typed

↓

input.value

↓

"Ibrahim"

Then:

```
result.innerHTML = "Hello " + name;
```

puts the result on the webpage.



This is used everywhere

Think about websites you've already used.

Login:

Email:

Password:

[Login]

Search:

[Search for a product...] [Search]

Registration:

Name:

Email:

Password:

[Register]

Contact form:

Name:

Message:

[Send]

All of these involve user input.

🧠 The big idea

You already know:

`document.querySelector()`

It finds the HTML element.

Now you're learning:

`input.value`

It gets the information inside the input.

So remember:

`querySelector()` 🔍

Find the input box.

`.value` 📦

Get what the user typed.

⚠️ Don't confuse these

If we have:

`<input id="name">`

This:

`let input = document.querySelector("#name");`

gives you the input element itself.

But:

`input.value`

gives you what the user typed.

Think:

input

↓

THE BOX 📦

`input.value`

↓

WHAT'S INSIDE THE BOX 📦

🎯 Today's mini practice

Build this:

HTML

Create:

An input for a person's name

A button saying Say Hello

A paragraph for the result

JavaScript

When the button is clicked:

Get the user's name using `.value`

Display:

Hello Ibrahim!

If they type:

Musa

it should display:

Hello Musa!

★ Bonus

Use something you've already learned:

`trim()`

So if someone enters:

Ibrahim

your program cleans it before displaying the greeting.

🧠 And don't worry about the difficult project

You actually learned something valuable from it.

When we were studying:

`.length`

`.toUpperCase()`

`.toLowerCase()`

`.includes()`

`.slice()`

`.trim()`

`.replace()`

each method was easy by itself.

But putting seven tools together is a different skill.

That's why projects sometimes feel much harder than lessons.

That doesn't mean you don't understand JavaScript. It means you're beginning to learn how to combine your knowledge—which is exactly what programming is. 💪🔥

For now, today's target is simply:

`input.value` = get what the user typed.

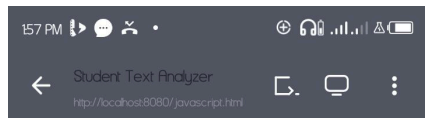
We'll stay on this topic before moving to the next part of Forms & User Input.

A screenshot of a mobile application showing a code editor. The editor has a dark theme and displays JavaScript code. The code defines variables for an input field, a button, and a result element, and adds a click event listener to the button. The event listener gets the value from the input, trims it, and updates the result element's innerHTML. The code is as follows:

```
1 let input =  
  document.querySelector("#name");  
2  
3 let button =  
  document.querySelector("#btn");  
4  
5 let result =  
  document.querySelector("#result")  
6  
7  
8 button.addEventListener("click", func  
  tion () {  
9  
10  
11     let name = input.value;  
12     let cleanName = name.trim();  
13     result.innerHTML = "Hello " +  
      name;  
14  
15 });
```

The editor interface includes a top bar with icons for navigation and a bottom bar with a keyboard. The keyboard shows various coding symbols like 'let', 'const', 'function', 'if', 'if...else', '=>', '{', '}', '(', ')', '=', '!', and '&&'.

```
2:02 PM
javascript.html x JS script.
< javascript.
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Student Text Analyzer</
   title>
5 <link rel="stylesheet"
6   href="Javascript.css">
7 </head>
8 <body>
9
10   <input id="name">
11
12   <br><br>
13   <button id="btn">Say Hello</button>
14   <p id="result"></p>
15
16   <script src="script.js"></
   script>
17
18 </body>
19 </html>
20
21
<div> <a> <img> [class] [id] h
->| = " " < > </ - <!-- --
```



Hello Musa Ibrahim

NOW — DAY 33, NEXT TOPIC

We've finished the first small part:

✓ `input.value`

Now let's move to the next topic:

📝 **FORM SUBMISSION & `preventDefault()`**

This is very important because real websites don't normally just have random input boxes and buttons.

They use forms.

Think about:

Login forms

Registration forms

Contact forms

Checkout forms

Search forms

Application forms

① What is a `<form>`?

A form is an HTML container for information the user wants to submit.

Example:

```
<form>
```

```
  <input id="name" placeholder="Enter your name">
```

```
  <button type="submit">Submit</button>
```

```
</form>
```

The important part is:

```
<form>
```

and:

```
</form>
```

Everything inside belongs to the form.

② Why `type="submit"`?

Look at:

```
<button type="submit">Submit</button>
```

It tells the browser:

"This button is being used to submit the form."

③ JavaScript can listen for "submit"

Instead of:

```
button.addEventListener("click", function () {
```

we can listen to the form:

```
form.addEventListener("submit", function (event) {
```

Notice:

click

versus:

submit
click

Means:

The user clicked something.

submit

Means:

The user submitted the form.

④ The important new thing: `preventDefault()`

Normally, when you submit a form, the browser may try to reload/navigate the page.

But sometimes we DON'T want that.

We want JavaScript to handle everything ourselves.

So we use:

```
event.preventDefault();
```

Think of it like telling the browser:



"Wait! Don't perform the normal form submission. Let my JavaScript handle it."

👤 Complete simple example

HTML

```
<form id="myForm">

  <input id="name" placeholder="Enter your name">

  <button type="submit">Submit</button>

</form>

<p id="result"></p>
```

JavaScript

```
let form = document.querySelector("#myForm");

let nameInput = document.querySelector("#name");

let result = document.querySelector("#result");

form.addEventListener("submit", function (event) {

  event.preventDefault();

  let name = nameInput.value;

  result.innerHTML = "Welcome " + name;

});
```

Now imagine the user types:

Ibrahim

and presses:

Submit

JavaScript gets:

`nameInput.value`

which gives:

Ibrahim

Then:

```
result.innerHTML = "Welcome " + name;
```

produces:

Welcome Ibrahim

🔥 That's a real form interaction.

🧠 The flow

Remember this:

USER TYPES



input.value



USER SUBMITS FORM



submit event



preventDefault()



JAVASCRIPT PROCESSES THE DATA



RESULT APPEARS

That's the basic foundation of things like login and registration forms.

🎯 YOUR MINI PRACTICE — SUBMIT TOMORROW

Build a Student Registration Form.

HTML must contain:

A <form>

Name input

Email input

Course input

Submit button

Result paragraph

JavaScript must:

Select the form with `querySelector()`

Listen for "submit"

Use:

`event.preventDefault();`

Get the name using:

`.value`

Get the email using `.value`

Get the course using `.value`

Display something like:

Registration Successful!

Name: Ibrahim Musa

Email: ibrahim@example.com

Course: Computer Science

★ BONUS

Use:

`.trim()`

to remove unnecessary spaces from the name.

Don't copy my complete example and change the words. Build it yourself. If you get errors, that's okay. Submit it tomorrow and I'll mark it like we did today. 😎🔥

And yes, this is now our class routine:

One topic → detailed explanation → examples → small practice → you submit → I mark → next topic.

That's much better for your learning than throwing five topics at you at once.

PART 1 — Correction of Your Previous Project

Your Student Registration Form was good and it worked. The main things I want you to correct are:

✗ Your version

You had:

```
form.addEventListener("submit", function() {
```

```
event.preventDefault();
```

```
let name = nameInput.value;  
let email = emailInput.value;  
let course = courseInput.value;
```

```
let cleanName = name.trim();  
let cleanEmail = email.trim();  
let cleanCourse = course.trim();
```

```
    result.innerHTML = "Registration Successful!";  
});
```

There are two important improvements.

1. Receive the event properly

You wrote:

```
function() {
```

but then used:

```
event.preventDefault();
```

Better:

```
function(event) {
```

Why?

Because we're telling JavaScript:

"Give this function information about the event that just happened."

Then:

```
event.preventDefault();
```

means:

"Don't let the form perform its normal submit/reload action."

2. Actually use your cleaned values

You correctly created:

```
let cleanName = name.trim();  
let cleanEmail = email.trim();  
let cleanCourse = course.trim();
```

But then you didn't use them.

You collected the values:

```
let name = nameInput.value;  
let email = emailInput.value;  
let course = courseInput.value;
```

Then cleaned them:

```
let cleanName = name.trim();  
let cleanEmail = email.trim();  
let cleanCourse = course.trim();
```

So if we clean them, we should actually use the cleaned versions.

✅ Corrected JavaScript

Here is the corrected version of your project:

```
let form = document.querySelector("#myForm");
```

```
let nameInput = document.querySelector("#name");
```

```
let emailInput = document.querySelector("#email");
```

```
let courseInput = document.querySelector("#course");
```

```
let result = document.querySelector("#result");
```

```
form.addEventListener("submit", function(event) {
```

```
    event.preventDefault();
```

```
    let name = nameInput.value;  
    let email = emailInput.value;  
    let course = courseInput.value;
```

```
    let cleanName = name.trim();  
    let cleanEmail = email.trim();  
    let cleanCourse = course.trim();
```

```
    result.innerHTML = "Registration Successful!";
```

```
});
```

What changed?

Only a few things:

```
function(event)
```

instead of:

```
function()
```

And we have:

```
let result = document.querySelector("#result");
```

so JavaScript knows what result is.

And your cleaned values are now properly prepared for future validation/storage.

★ Your Previous Work

Honestly, you did well.

You were already combining:

HTML
↓
querySelector()
↓
.value
↓
.trim()
↓
event listener
↓
preventDefault()
↓
innerHTML

That's a proper little interactive webpage.

Previous project: 92/100. 🔥

Now let's leave that behind and start today's lesson.

DAY 35 — JAVASCRIPT LOCAL STORAGE

Today has TWO topics:

Topic 1

`localStorage.setItem()`

Topic 2

`localStorage.getItem()`

But we're going to spend enough time on each one.

Today we're going to start with Topic 1 only.

🧠 First: What problem does Local Storage solve?

Before learning the code, understand why we need it.

You've already used variables like:

```
let name = "Musa Ibrahim";
```

JavaScript remembers that value while the page is running.

For example:

```
let name = "Musa Ibrahim";
```

```
console.log(name);
```

JavaScript can remember:

name → Musa Ibrahim

But imagine you refresh the webpage.

JavaScript starts running again from the beginning.

That variable doesn't permanently live inside the browser.



Think about a real-life cupboard

Imagine you have a cupboard.

You put your shoes inside:



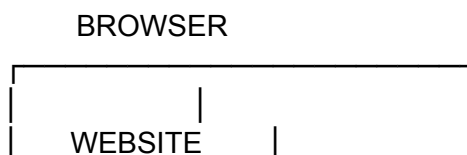
Then you close the cupboard.

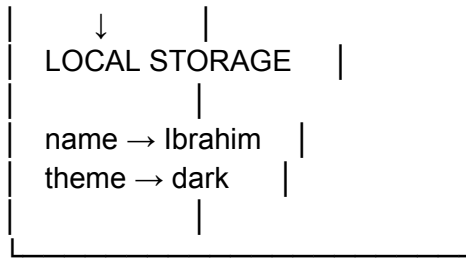
You come back tomorrow.

The shoes are still there.

That's similar to what Local Storage does.

It gives a website a place in the browser to keep certain information.





You refresh the page:

 Refresh

↓
Local Storage still has the information

↓
Website can retrieve it

That's the big idea.

 **TOPIC 1** — `localStorage.setItem()`

Now let's understand the name.

`localStorage.setItem()`

Break it into two parts.

`localStorage`

This means:

The browser's local storage area.

`setItem()`

Think:

SET = PUT/SAVE

So:

`localStorage.setItem()`

basically means:

"Save something inside Local Storage."

🔑 Key and Value

This is VERY important.

When we save something, we give it two things:

KEY → VALUE

Think about a labeled box.

name	← label/key
Musa Ibrahim	← value

The key tells us what the information is.

The value is the actual information.

🏠 Our first example

```
localStorage.setItem("name", "Musa Ibrahim");
```

Let's slow down.

localStorage

We're saying:

"I want to use browser storage."

```
.setItem()
```

We're saying:

"I want to save something."

"name"

This is the key.

It's the label.

"Musa Ibrahim"

This is the value.

It's the information we're saving.

So the whole thing means:

 Save "Musa Ibrahim" under the key "name".

 Visualize it

After this:

```
localStorage.setItem("name", "Musa Ibrahim");
```

Think of Local Storage as containing:

Local Storage	
name → Musa Ibrahim	

 Another example

```
localStorage.setItem("course", "Computer Science");
```

Now we have:

Local Storage	
name → Musa Ibrahim	
course → Computer Science	

Another:

```
localStorage.setItem("favoriteColor", "black");
```

Now:

name → Musa Ibrahim
course → Computer Science

favoriteColor → black

⚠️ VERY IMPORTANT: Key vs Value

Don't confuse these.

Look at:

```
localStorage.setItem("name", "Ibrahim");
```

The first one:

"name"

is the KEY.

The second one:

"Ibrahim"

is the VALUE.

So remember:

🏷️ KEY = label

📦 VALUE = information

🔧 Let's put it into a real webpage

HTML:

```
<!DOCTYPE html>
<html>
<head>
  <title>Local Storage</title>
</head>

<body>

  <h1>Student Information</h1>

  <script src="script.js"></script>
```

```
</body>  
</html>
```

JavaScript:

```
localStorage.setItem("name", "Musa Ibrahim");
```

When the webpage opens, JavaScript executes that line.

The browser saves:

name → Musa Ibrahim

🤔 "But where can I SEE the saved information?"

Excellent question.

You won't normally see it printed on your webpage.

This:

```
localStorage.setItem("name", "Musa Ibrahim");
```

doesn't automatically put:

Musa Ibrahim

on the screen.

It simply stores it.

That's why we're learning TWO topics.

setItem()
SAVE 💾

getItem()
GET/READ 📖

So eventually we'll be able to do something like:

User enters name



JavaScript saves it



Local Storage



Page refreshes



JavaScript gets it back



Website displays it

But we are not doing `getItem()` yet.

One step at a time. 🧑

⚠️ One more important thing

For now, you'll mostly see:

```
localStorage.setItem("name", "Musa Ibrahim");
```

Both are strings:

"name"

"Musa Ibrahim"

Local Storage fundamentally stores values as strings.

Later, when we reach more advanced JavaScript, we'll learn how to store things like:

arrays

objects

using:

```
JSON.stringify()
```

```
JSON.parse()
```

🚫 Don't worry about those now.

They're not today's lesson.

🧠 Let's test your understanding

Suppose I write:

```
localStorage.setItem("school", "Federal University Lafia");
```

What is:

1. The key?

2. The value?

Think about it before looking at the answer.

.

.

.

Answer:

Key = "school"

Value = "Federal University Lafia"

Because:

```
localStorage.setItem(KEY, VALUE)
```

That's the pattern I want you to understand.

 TOPIC 1 PRACTICE

Now you build, not copy.

Create a small project called:

My Student Information

Use:

```
localStorage.setItem()
```

to save three pieces of information.

For example, you could save:

name
department
favoriteFood

But choose your own information.

Your JavaScript should have three `setItem()` statements.

The general structure is:

```
localStorage.setItem("...", "...");  
localStorage.setItem("...", "...");  
localStorage.setItem("...", "...");
```

 Important

Do not use `getItem()` yet.

Do not try to make it complicated.

Today's first goal is simply:

I can save information into Local Storage and I understand exactly what the key and value mean.

Build it yourself and send me the code/screenshot.

I'll mark it, and only after that we'll move to Topic 2 — `localStorage.getItem()`.  

JAVASCRIPT — DAY 36

`localStorage.getItem()`

You already learned:

`localStorage.setItem()`

That means putting/saving information into the browser's storage.

Today we're learning the opposite:

`localStorage.getItem()`

That means taking information back out of storage.



Think of localStorage like a cupboard

Imagine you have a cupboard.

setItem() = Put something inside
`localStorage.setItem("name", "Ibrahim");`

You're basically saying:

"Browser, please save Ibrahim under the name name."

getItem() = Take something out
`localStorage.getItem("name");`

You're saying:

"Browser, give me whatever you saved under your name."

1 Basic Example

First, save something:

`localStorage.setItem("name", "Ibrahim");`

Now retrieve it:

`let myName = localStorage.getItem("name");`

`console.log(myName);`

The console will show:

Ibrahim

What's happening?

This:

`localStorage.getItem("name")`

returns:

"Ibrahim"

So this:

```
let myName = localStorage.getItem("name");
```

is basically:

```
let myName = "Ibrahim";
```

② The important pattern

Memorize this pattern:

```
localStorage.setItem("key", "value");
```

Save.

And:

```
localStorage.getItem("key");
```

Retrieve.

For example:

```
localStorage.setItem("username", "Ibrahim");
```

```
let username = localStorage.getItem("username");
```

```
console.log(username);
```

Output:

Ibrahim

③ Why do we need getItem()?

This is where it becomes useful.

Imagine a user visits your website and enters:

Ibrahim

You save it:

```
localStorage.setItem("username", "Ibrahim");
```

The user closes the browser.

Later, they come back.

JavaScript can retrieve the saved name:

```
let username = localStorage.getItem("username");
```

```
console.log(username);
```

And you get:

Ibrahim

That's why localStorage is useful.

4) Let's put it on a webpage

HTML:

```
<h1 id="welcome"></h1>
```

JavaScript:

```
localStorage.setItem("username", "Ibrahim");
```

```
let username = localStorage.getItem("username");
```

```
document.getElementById("welcome").textContent =  
  "Welcome " + username;
```

The webpage will show:

Welcome Ibrahim

🔥 Now we're actually using `getItem()` to put saved data back onto the webpage.

📦 One very important thing ⚠️

`getItem()` needs the same key that you used with `setItem()`.

For example:

```
localStorage.setItem("name", "Ibrahim");
```

Correct:

```
localStorage.getItem("name");
```

But this:

```
localStorage.getItem("username");
```

won't find it because "username" is a different key.

Think of the key as the label on the cupboard.

If you saved something under:

name

you must ask for:

name

🔧 YOUR FIRST PRACTICE

Don't worry about making a big project yet.

Create a simple HTML page with:

An `<h1>` with an id

JavaScript that saves your name using `setItem()`

JavaScript that retrieves your name using `getItem()`

Display:

Hello, Ibrahim! Welcome back.

Try it yourself first. 😎

When you're done, send me your code/screenshot. I'll mark it like our previous classes, correct anything wrong, and then we'll move to the next part of Day 36.

MINI PROJECT — SUBMIT TOMORROW

Before we begin the next class, I want you to build this:

My Storage Manager

Create a small webpage that demonstrates the four things you've now learned:

Your webpage should have:

1. A heading

My Storage Manager

2. Three buttons

Save Data

Delete Name

Clear All Data

3. A section that displays the saved information.

For example:

Name: Ibrahim

Age: 21

Course: Computer Science

Your JavaScript should:

Save Data button

Use:

```
localStorage.setItem()
```

to save the three pieces of information.

Delete Name button

Use:

```
localStorage.removeItem()
```

to delete only the name.

Clear All Data button

Use:

```
localStorage.clear()
```

to delete everything.

And use:

```
localStorage.getItem()
```

to retrieve the saved information and display it.

🎯 What I want you to practice

Don't copy a complete project from me. Build it yourself.

You already know `setItem()` and `getItem()`. Your challenge is to combine those with the two new concepts:

```
setItem()  
↓  
getItem()  
↓  
removeItem()  
↓  
clear()
```

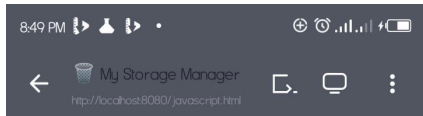
Tomorrow, send me your project. I'll mark it, correct it line by line, explain your mistakes, and then we'll begin the next two-topic class.

We're keeping the course step-by-step and without rushing ahead. 💪

```
8:49 PM JS script.js x javascript
< > script.js
1 let saveButton =
  document.querySelector("#saveBtn");
2
3 let deleteButton =
  document.querySelector("#deletenameB
  tn");
4
5 let clearallButton =
  document.querySelector("#clearallBtn
  ");
6
7 saveButton.addEventListener("click",
  function () {
8
9   localStorage.setItem("name ", "Musa
  Ibrahim");
10
11  localStorage.setItem("age ", 21);
12
13  localStorage.setItem("course ",
  "Computer Science ");
14
15   let name =
16  localStorage.getItem("name ");
17
18   let age =
19  localStorage.getItem("age ");
20
21   let course =
22  localStorage.getItem("course ");
23
let const function if if...else
-> { } ( ) = ! &&
```

```
8:49 PM JS script.js x javascript
< > script.js Q
21
22 document.querySelector("#myName").textContent =
23     "Name: " + name;
24
25 document.querySelector("#myAge").textContent =
26     "Age: " + age;
27
28 document.querySelector("#myCourse").textContent =
29     "Course: " + course;
30
31 });
32
33 deleteButton.addEventListener("click",function() {
34
35     localStorage.removeItem("name ");
36
37 });
38
39
40 clearallButton.addEventListener("click",function () {
41
42     localStorage.clear();
43
44 });
45
let const function if if...else
-> { } ( ) = ! &&
```

```
8:49 PM  < > javascript.html
< > javascript.html
< > javascript.
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>🗑 My Storage Manager</
   title>
5 <link rel="stylesheet"
6   href="Javascript.css">
7 </head>
8 <body>
9 <h1>🗑 My Storage Manager</h1>
10 <button id="saveBtn">Save Data</
   button>
11 <button id="deletenameBtn">Delete
   Name</button>
12 <button id="clearallBtn">Clear All
   </button>
13 <p id="myName"></p>
14 <p id="myAge"></p>
15 <p id="myCourse"></p>
16   <script src="script.js"></
   script>
17
18 </body>
19 </html>
20
21
<div> <a> <img> [class] [id] h
->| = " " < > </ - <!-- --
```



My Storage Manager

Name: Musa Ibrahim

Age: 21

Course: Computer Science

CORRECTION

How would we fix it?

For example, after deleting the name:

```
localStorage.removeItem("name ");
```

```
document.querySelector("#myName").textContent = "";
```

Now two things happen:

The name is removed from localStorage.

The name is removed from the webpage.

For clear():

```
localStorage.clear();
```

```
document.querySelector("#myName").textContent = "";
```



```
document.querySelector("#myAge").textContent = "";
document.querySelector("#
```

DAY 37 — JSON.stringify()

What are we learning today?

Today we're learning:

JSON.stringify()

Don't worry about the word JSON yet. First understand what `stringify()` does.

 The problem

You already know how to save simple information:

```
localStorage.setItem("name", "Musa");
```

That's easy because "Musa" is already a string.

But imagine we have several pieces of information about a student:

```
let student = {
  name: "Musa Ibrahim",
  age: 21,
  course: "Computer Science"
};
```

This is an object.

We want to save the entire object into localStorage.

Remember:

localStorage stores information as strings.

So we need a way to convert our object into a string.

That's exactly what:

`JSON.stringify()`

does.

 What does `JSON.stringify()` do?

It converts:

JavaScript object → string

Think of it like a translator:

OBJECT
↓
`JSON.stringify()`
↓
STRING

For example:

```
let student = {  
  name: "Musa Ibrahim",  
  age: 21  
};
```

```
let studentString = JSON.stringify(student);
```

```
console.log(studentString);
```

The result will look like:

```
{"name":"Musa Ibrahim","age":21}
```

It looks similar to an object, but it is now a string.

● Let's make a real HTML example

Create your HTML file:

javascript.html

```
<!DOCTYPE html>
<html>
<head>
  <title>JSON Stringify</title>
</head>

<body>

  <h1>Student Information</h1>

  <p id="result"></p>

  <script src="script.js"></script>

</body>
</html>
```

Now create:

script.js

```
let student = {
  name: "Musa Ibrahim",
  age: 21,
  course: "Computer Science"
};
```

```
let studentString = JSON.stringify(student);
```

```
document.querySelector("#result").textContent = studentString;
```

What will appear on the webpage?

Something like:

Student Information

```
{"name":"Musa Ibrahim","age":21,"course":"Computer Science"}
```

🔥 You just converted an object into a string and displayed it on an HTML page.

🧠 Let's break the JavaScript down

This:

```
let student = {  
  name: "Musa Ibrahim",  
  age: 21,  
  course: "Computer Science"  
};
```

creates an object.

Then:

```
let studentString = JSON.stringify(student);
```

means:

"Take the student object and convert it into a string."

Then:

```
document.querySelector("#result").textContent = studentString;
```

means:

"Find the HTML element with the ID result and put the converted string inside it."

● Now let's connect it to localStorage

This is where today's lesson becomes really useful.

HTML

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Save Student</title>  
</head>  
  
<body>
```

```
<h1>Student Storage</h1>
```

```
<button id="saveBtn">Save Student</button>
```

```
<p id="result"></p>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
let saveButton = document.querySelector("#saveBtn");
```

```
let result = document.querySelector("#result");
```

```
saveButton.addEventListener("click", function() {
```

```
    let student = {  
        name: "Musa Ibrahim",  
        age: 21,  
        course: "Computer Science"  
    };
```

```
    let studentString = JSON.stringify(student);
```

```
    localStorage.setItem("student", studentString);
```

```
    result.textContent = "Student saved!";  
});
```

Now when you click:

Save Student

the object gets converted into a string and saved.

🔍 Let's understand this one line

This is the most important line today:

```
localStorage.setItem("student", JSON.stringify(student));
```

There are actually two operations happening.

First:

```
JSON.stringify(student)
```

converts the object into a string.

Then:

```
localStorage.setItem()
```

saves that string.

So:

student object



JSON.stringify()



string



localStorage.setItem()



saved in browser

🔥 That's the connection.

⚠️ One important thing

Don't confuse:

```
JSON.stringify()
```

with:

JSON.parse()

We're NOT learning JSON.parse() today.

That's Day 38.

For today, remember only:

JSON.stringify() converts an object into a string.

DAY 37 PRACTICE EXERCISE

Before I give you the mini-project, I want you to practice this yourself.

 Build: "My Profile"

Create an HTML page containing:

HTML

You should have:

An <h1> saying My Profile

A button saying Convert Profile

A <p> with an ID where the result will appear.

JavaScript

Create an object called:

profile

It should contain:

name

age

course

school

Use your own information.

Then:

Use JSON.stringify() to convert the object into a string.

Display that string inside the <p> on the webpage.

Use `addEventListener("click", ...)` for the button.

Your flow should be:

Click button



Create profile object



`JSON.stringify()`



Convert object → string



Display string on webpage



Don't use `JSON.parse()` yet.

We haven't learned it.



Your challenge

Don't copy my previous example exactly.

Write your own:

HTML + JavaScript

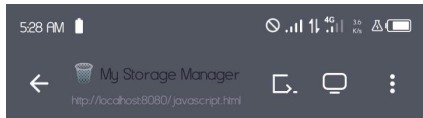
Then send me a screenshot of your code and the webpage.

I'll mark your Day 37 practice exercise, explain any mistakes, and only after you've understood it will we move to Day 38 — `JSON.parse`


```
5:28 AM
JS script.js
<> javascript

< > script.js
1 let convertButton =
  document.querySelector("#convertBtn"
  );
2
3 let result =
  document.querySelector("#result");
4
5 convertButton.addEventListener("click",function () {
6   let student = {
7     name: "Musa Ibrahim",
8     age: 21,
9     course: "Computer
Science",
10    school: "Federal
University Lafia "
11  };
12
13  let studentString =
  JSON.stringify(student);
14
15  localStorage.setItem("student",studentString);
16
17  result.textContent = "Student
saved!";
18 })
19

let const function if if...else
-> { } ( ) = ! &&
```



My Profile

Convert Button

Student saved!

DAY 38 — JSON.parse()

Yesterday we learned:

`JSON.stringify()`

Remember:

Object → String

Today we're learning the opposite:

`JSON.parse()`

String → Object

🧠 Why do we need JSON.parse()?

Look at what you did yesterday:

```
let student = {  
  name: "Musa Ibrahim",  
  age: 21,  
  course: "Computer Science",  
  school: "Federal University Lafia"  
};
```

```
let studentString = JSON.stringify(student);
```

After stringify(), studentString is a string.

You can think of it as:

```
OBJECT  
↓  
JSON.stringify()  
↓  
STRING
```

Then we saved it:

```
localStorage.setItem("student", studentString);
```

But when we retrieve it:

```
let savedStudent = localStorage.getItem("student");
```

we get the string, not the original object.

That's where JSON.parse() comes in.

🔄 JSON.parse() converts it back

```
OBJECT  
↓  
JSON.stringify()  
↓  
STRING  
↓
```

JSON.parse()



OBJECT

So:

```
let studentObject = JSON.parse(savedStudent);
```

Now we have an actual JavaScript object again.

 Simple example

```
let student = {  
  name: "Musa Ibrahim",  
  age: 21  
};
```

```
let studentString = JSON.stringify(student);
```

```
let studentObject = JSON.parse(studentString);
```

```
console.log(studentObject);
```

Now:

```
studentObject.name
```

gives:

Musa Ibrahim

And:

```
studentObject.age
```

gives:

21

 Let's do it with HTML

This is how I want us to learn from now on—HTML + JavaScript together.

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>JSON Parse</title>
</head>

<body>

  <h1>Student Information</h1>

  <button id="showBtn">Show Student</button>

  <p id="name"></p>
  <p id="age"></p>
  <p id="course"></p>

  <script src="script.js"></script>

</body>
</html>
```

JavaScript

```
let showButton = document.querySelector("#showBtn");

let name = document.querySelector("#name");
let age = document.querySelector("#age");
let course = document.querySelector("#course");

showButton.addEventListener("click", function() {

  let student = {
    name: "Musa Ibrahim",
    age: 21,
    course: "Computer Science"
  };
```

```
let studentString = JSON.stringify(student);

let studentObject = JSON.parse(studentString);

name.textContent = "Name: " + studentObject.name;

age.textContent = "Age: " + studentObject.age;

course.textContent = "Course: " + studentObject.course;

});
```

When you click Show Student, you'll get:

Name: Musa Ibrahim

Age: 21

Course: Computer Science

 Let's understand the important part

First:

```
let student = {
  name: "Musa Ibrahim",
  age: 21,
  course: "Computer Science"
};
```

We have an object.

Then:

```
let studentString = JSON.stringify(student);
```

We convert:

Object → String

Then:

```
let studentObject = JSON.parse(studentString);
```

We convert:

String → Object

Now we can use:

```
studentObject.name
```

```
studentObject.age
```

```
studentObject.course
```

🔥 Now let's connect it to localStorage

This is the part you were asking about.

Your Day 37 code did:

```
localStorage.setItem("student", studentString);
```

Today we can retrieve it:

```
let savedStudent = localStorage.getItem("student");
```

But remember, savedStudent is a string.

So:

```
let studentObject = JSON.parse(savedStudent);
```

Now it's an object.

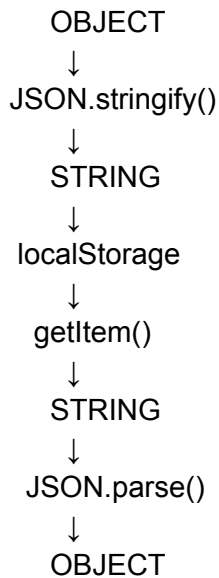
Then:

```
document.querySelector("#name").textContent =  
  "Name: " + studentObject.name;
```

And that's how we can finally make the information visible on the page.

🧠 THE COMPLETE JOURNEY

This is VERY important. Memorize this flow:



Then:

`studentObject.name`

can give us the name.



🎯 DAY 38 PRACTICE EXERCISE

Before your mini-project, I want you to practice `JSON.parse()` by yourself.

Build: My Student Viewer

HTML must contain:

`<h1>` → My Student Viewer

A button → Show Student

Three `<p>` elements for:

Name

Age

Course

JavaScript must:

Create this object:


```
let student = {  
  name: "Your Name",  
  age: 21,  
  course: "Computer Science"  
};
```

Then:

Use `JSON.stringify()` to convert it to a string.

Use `JSON.parse()` to convert it back into an object.

When the button is clicked, display:

Name

Age

Course

 Important

For this exercise, don't use `localStorage` yet.

I want you to understand `parse()` by itself first.

The flow should be:

Object



`stringify()`



String



`parse()`



Object



Display on webpage

Send me your HTML + JavaScript screenshots when you're finished.



THEN COMES YOUR MIDNIGHT MINI-PROJECT

Once I mark this exercise, I'll give you the Day 37 + Day 38 combined mini-project using:

`JSON.stringify()`

+

`JSON.parse()`

+

localStorage

+

HTML

+

JavaScript

That project will be your submission for midnight.

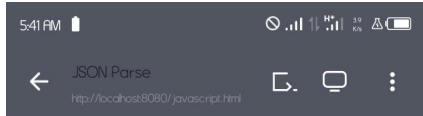
And this time, you'll be able to make the saved subject/information actually appear on the page.

A screenshot of a mobile application interface showing a code editor. The editor is titled 'javascript.html' and displays HTML code for a web page. The code includes a title 'JSON Parse', a heading 'Student Information', a button 'Show Student', and three empty paragraph elements with IDs 'name', 'age', and 'course'. A script tag 'script.js' is also present. The code is numbered from 1 to 20. Below the code editor, there is a toolbar with various HTML tags and attributes like <div>, <a>, , [class], [id], and h. At the bottom, there is a keyboard with standard keys and special keys like '>|', '=', '<', '>', '</>', and '<!-- -->'. The status bar at the top shows the time as 5:41 PM and various system icons.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>JSON Parse</title>
5 </head>
6
7 <body>
8
9   <h1>Student Information</h1>
10
11   <button id="showBtn">Show
    Student</button>
12
13   <p id="name"></p>
14   <p id="age"></p>
15   <p id="course"></p>
16
17   <script src="script.js"></
    script>
18
19 </body>
20 </html>
```

```
5:41 PM
JS script.js
<> javascript

< > script.js
1 let showButton =
  document.querySelector("#showBtn");
2
3 let name =
  document.querySelector("#name");
4 let age =
  document.querySelector("#age");
5 let course =
  document.querySelector("#course");
6
7 showButton.addEventListener("click",
  function() {
8
9     let student = {
10       name: "Musa Ibrahim",
11       age: 21,
12       course: "Computer Science"
13     };
14
15     let studentString =
16       JSON.stringify(student);
17     let studentObject =
18       JSON.parse(studentString);
19     name.textContent = "Name: " +
20       studentObject.name;
21     age.textContent = "Age: " +
22       studentObject.age;
23     course.textContent = "Course: "
24
25 let const function if if...else
26 { } ( ) = ! &&
```



Student Information

Show Student

Name: Musa Ibrahim

Age: 21

Course: Computer Science

DAY 37 + DAY 38 MINI PROJECT

🎓 Project: My Student Profile Storage

This is your combined project for:

localStorage.setItem() ✓
localStorage.getItem() ✓
JSON.stringify() ✓
JSON.parse() ✓
document.querySelector() ✓
addEventListener() ✓
🎯 Your webpage

Create a page titled:

My Student Profile

It should contain:

1. Heading
My Student Profile
2. Three buttons
Save Profile
Load Profile
Delete Profile
3. Information area

Have places to display:

Name:

Age:

Course:

School:

You can use <p> elements.

 What each button must do

 SAVE PROFILE

When the user clicks Save Profile:

Create an object like:

```
let student = {  
  name: "Musa Ibrahim",  
  age: 21,  
  course: "Computer Science",  
  school: "Federal University Lafia"  
};
```

Then:

Convert the object using JSON.stringify()

Save it using localStorage.setItem()

Show a message on the page:

Profile saved successfully!

 LOAD PROFILE

When the user clicks Load Profile:

Get the saved information using:

localStorage.getItem()

Convert it back into an object using:

`JSON.parse()`

Display the information on the webpage:

Name: Musa Ibrahim

Age: 21

Course: Computer Science

School: Federal University Lafia

🔥 This is where you're going to use `querySelector()` again.

For example, you'll need to select your `<p>` elements and put the information inside them.

🔴 DELETE PROFILE

When the user clicks Delete Profile:

Use:

`localStorage.removeItem()`

to remove the saved student profile.

Then also clear the information displayed on the webpage.

For example, after deletion, the page should no longer show:

Name: Musa Ibrahim

Age: 21

...

You can instead show:

Profile deleted.

⚠ Important challenge

I don't want you to copy a complete solution from me.

You already know every individual piece:

`querySelector()`

↓

`addEventListener()`

↓

object

↓
JSON.stringify()
↓
setItem()
↓
getItem()
↓
JSON.parse()
↓
querySelector()
↓
display on webpage
↓
removeItem()

Your job now is to put the pieces together. 🧩

📁 Your files

Create:

javascript.html
script.js

You can use CSS if you want, but CSS is not required for this assignment. I care about the JavaScript functionality.

🏆 What I will check when you submit

I'll check whether:

- ✅ Save Profile works
- ✅ Object is converted with JSON.stringify()
- ✅ Profile is actually saved in localStorage
- ✅ Load Profile uses getItem()
- ✅ JSON.parse() is used correctly
- ✅ Information appears on the webpage
- ✅ querySelector() is used correctly
- ✅ Delete Profile uses removeItem()
- ✅ The displayed information is cleared after deletion
- ✅ Your buttons/events work correctly
- ✅ Your HTML and JavaScript are organized properly

Submit the screenshots before midnight, just like you requested. 🌙🔥

And once I mark this project, Day 39 will be our next two-topic lesson. We are not jumping ahead until you've properly understood Days 37–38.

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My Student Profile Storage</title>
</head>

<body>

  <h1>My Student Profile</h1>

  <button id="saveBtn">Save Button</button>

  <button id="loadBtn">Load Button</button>

<button id="deleteBtn">Delete Button</button>
  <p id="result"></p>
  <p id="name"></p>
  <p id="age"></p>
  <p id="course"></p>
  <p id="school"></p>

  <script src="script.js"></script>

</body>
</html>
```

JAVASCRIPT

```
let saveButton = document.querySelector("#saveBtn");

let loadButton = document.querySelector("#loadBtn");

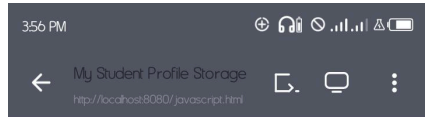
let deleteButton = document.querySelector("#deleteBtn");

saveButton.addEventListener("click",function () {

  let student = {
    name: "Musa Ibrahim",
    age: 21,
```



```
        ("#school").textContent =  
            "";  
document.querySelector  
    ("#result").textContent =  
        "Profile deleted."  
});
```



My Student Profile

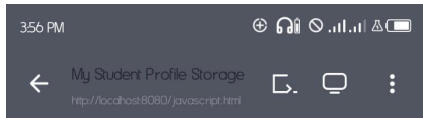
Saved Successfully!

Name: Musa Ibrahim

Age: 21

Course: Computer Science

School: Federal University Lafia



My Student Profile

Profile deleted.

Screenshot has been saved to Pictures/
Screenshot

DAY 39 — sessionStorage

Now we're moving to our next topic exactly where we stopped.

Today's topic is:

sessionStorage

Don't worry—it's actually very similar to localStorage.

You've already learned:

```
localStorage.setItem()  
localStorage.getItem()  
localStorage.removeItem()  
localStorage.clear()
```

Now we're going to learn the same basic operations with:

sessionStorage

🧠 1. What is sessionStorage?

Think of browser storage as a small box where JavaScript can keep information.

You already have:

localStorage

Browser



localStorage



Data stays there

And now:

sessionStorage

Browser



sessionStorage



Data stays during the current session

The important difference is:

localStorage keeps data even after you close the browser.

sessionStorage is temporary and normally lasts only for the current browser tab/session.

🟢 2. sessionStorage.setItem()

Just like localStorage:

```
sessionStorage.setItem("name", "Musa");
```

We have:

" name "



" Musa "

The first thing is the key.

The second thing is the value.

 Let's use HTML

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Session Storage</title>
```

```
</head>
```

```
<body>
```

```
  <h1>My Session Storage</h1>
```

```
  <button id="saveBtn">Save Name</button>
```

```
  <p id="result"></p>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
let saveButton = document.querySelector("#saveBtn");
```

```
let result = document.querySelector("#result");
```

```
saveButton.addEventListener("click", function() {
```

```
sessionStorage.setItem("name", "Musa Ibrahim");
```

```
result.textContent = "Name saved!";
```

```
});
```

When you click:

Save Name

the browser stores:

name → Musa Ibrahim

inside sessionStorage.

● 3. sessionStorage.getItem()

Now let's retrieve it.

```
let name = sessionStorage.getItem("name");
```

This means:

"Give me the value stored under the key name."

Then we can display it:

```
result.textContent = "Name: " + name;
```

So the complete example becomes:

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Session Storage</title>
```

```
</head>
```

```
<body>
```

```
<h1>Session Storage</h1>
```

```
<button id="saveBtn">Save Name</button>
```

```
<button id="loadBtn">Load Name</button>
```

```
<p id="result"></p>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

```
JavaScript
```

```
let saveButton =
```

```
document.querySelector("#saveBtn");
```

```
let loadButton =
```

```
document.querySelector("#loadBtn");
```

```
let result =
```

```
document.querySelector("#result");
```

```
saveButton.addEventListener("click", function() {
```

```
    sessionStorage.setItem("name", "Musa Ibrahim");
```

```
    result.textContent = "Name saved!";
```

```
});
```

```
loadButton.addEventListener("click", function() {
```

```
    let name =  
    sessionStorage.getItem("name");
```

```
    result.textContent = "Name: " + name;
```

```
});
```

Now:

Save Name → saves the information.

Load Name → retrieves the information.

🧠 4. `sessionStorage.removeItem()`

Just like what we learned yesterday:

```
sessionStorage.removeItem("name");
```

This removes:

name → Musa Ibrahim

from session storage.

For your previous project, remember your mistake:

```
localStorage.removeItem("name"); ❌
```

because your actual key was:

"student"

So you needed:


```
localStorage.removeItem("student"); 
```

The key must match exactly.

5. sessionStorage.clear()

This removes everything stored in sessionStorage.

```
sessionStorage.clear();
```

For example:

```
sessionStorage.setItem("name", "Musa");  
sessionStorage.setItem("age", "21");  
sessionStorage.setItem("course", "Computer Science");
```

Now we have:

name
age
course

Then:

```
sessionStorage.clear();
```

 Everything is removed.

 `removeItem()` vs `clear()`

Remember this:

```
removeItem()
```

Removes ONE specific item:

```
sessionStorage.removeItem("name");  
clear()
```

Removes EVERYTHING:

```
sessionStorage.clear();
```

Think:

removeItem() → 🗑️ one item

clear() → 🗑️🗑️🗑️ everything

🔥 localStorage vs sessionStorage

This is the most important part of today's lesson.

	localStorage	sessionStorage
Save	setItem()	setItem()
Get	getItem()	getItem()
Remove one	removeItem()	removeItem()
Remove all	clear()	clear()
Data duration	Long-term	Temporary
Survives closing/reopening browser	Usually yes	Usually no
Storage belongs to	Origin	Origin + tab/session

Easy way to remember:

localStorage = "Keep this for me."

sessionStorage = "Keep this while I'm here."

🖋️ YOUR PRACTICE EXERCISE — Before the Mini Project

I don't want you to jump straight into the mini-project.

Build:

📱 "My Session Profile"

HTML should contain:

My Session Profile

[Save Profile]

[Load Profile]

[Delete Profile]

[Clear All]

<p>...</p>

Your profile should contain:

Name: Musa Ibrahim
Course: Computer Science
Age: 21
Your JavaScript must use:
sessionStorage.setItem()
sessionStorage.getItem()
sessionStorage.removeItem()
sessionStorage.clear()
The buttons should do:

Save Profile

→ save the information.

Load Profile

→ retrieve and display it.

Delete Profile

→ delete the profile.

Clear All

→ clear all session storage.

 One important challenge

Don't use four separate keys if you don't want to.

You can practice the way we did with your previous project:

```
let student = {  
  name: "Musa Ibrahim",  
  age: 21,  
  course: "Computer Science"  
};
```

Then:

Object

↓

JSON.stringify()

↓
sessionStorage
↓
getItem()
↓
JSON.parse()
↓
Object
↓
HTML

🔥 This will reinforce everything you've just learned.

Do the practice exercise first and send me your screenshots. I'll mark it, correct anything wrong, and then give you the Day 39 mini-

JAVASCRIPT — DAY 38

FETCH API

Today we're entering an important JavaScript topic.

You've already learned things like localStorage, getItem(), setItem(), etc. Now we're going to learn how JavaScript can communicate with the outside world and get information from a server.

Don't worry. We're going slowly.

1. First: What is an API?

API sounds complicated, but the idea is actually simple.

Imagine you enter a restaurant.

You don't walk into the kitchen and start cooking your own food.

You tell the waiter:

"Please give me rice and chicken."

The waiter goes to the kitchen, gets your food, and brings it back.

Think of it like this:



The API is like the waiter.

It allows your JavaScript program to communicate with another system.

2. What does API mean?

API means:

Application Programming Interface

Don't worry about memorizing that definition right now.

For now, remember:

An API allows different programs to communicate with each other.

For example, your website might need:

Weather information 🌤️

Football scores ⚽

User information 👤

News 📰

Products 🛒

Currency exchange rates 💰

Instead of creating all that information yourself, your JavaScript can request it from an API.

3. So what is Fetch API?

This is where today's main topic comes in.

JavaScript has something called:

`fetch()`

`fetch()` allows JavaScript to request data from a server/API.

Think:

"JavaScript, go and bring me some information from this address."

Example:

`fetch("https://example.com")`

That's a request.

But there's a problem.

The internet doesn't respond instantly.

So JavaScript needs a way to handle the response.

That's where Promises come in.

Don't panic 😭😄.

We'll learn them carefully.

4. Your first Fetch example

Let's create a simple HTML page.

HTML

`<!DOCTYPE html>`

`<html>`

```
<head>
  <title>Fetch API</title>
</head>

<body>

  <h1>Fetch API</h1>

  <p id="result">Loading...</p>

  <script src="script.js"></script>

</body>
</html>
```

Now JavaScript:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

Let's slow down.

There are several things happening here.

5. Understanding fetch()

This:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
```

means:

"JavaScript, please request information from this URL."

The URL belongs to a free testing API called JSONPlaceholder.

We're using it because it's excellent for learning.

6. What is a URL?

You already know URLs from websites.

For example:

<https://google.com>

That's a URL.

An API can also have a URL.

For example:

<https://jsonplaceholder.typicode.com/posts/1>

That URL gives us data.

7. What does `fetch()` give us?

Here's something important.

When you do:

```
fetch(url)
```

you don't immediately get the actual data.

Instead, `fetch()` gives you a Promise.

A Promise basically means:

"I don't have the result yet, but I'm working on getting it."

Imagine you order food.

The waiter says:

"Your food is coming."

That's a Promise 😊.

You don't have the food yet.

But you expect it later.

8. .then()

We can tell JavaScript:

"When the response arrives, do this."

We use:

.then()

Example:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => {
    console.log(response);
  });
```

When the server responds, the .then() runs.

9. What is response?

Here:

```
.then(response => {
  console.log(response);
});
```

response is simply a variable containing the server's response.

You could call it something else:

```
.then(result => {
  console.log(result);
});
```

Or:

```
.then(answer => {
  console.log(answer);
});
```

But response is the common name developers use.

10. The response is NOT the actual data yet

This is VERY important.

If we do:

```
console.log(response);
```

you'll see a Response object.

We need to tell JavaScript:

"Convert the response into JSON data."

That's why we use:

```
response.json()
```

11. What is JSON?

JSON is a common format for sending data between computers.

It looks very similar to JavaScript objects.

For example:

```
{  
  "id": 1,  
  "title": "My first post",  
  "body": "Hello world"  
}
```

You will see JSON a lot when working with APIs.

12. response.json()

So:

```
response.json()
```

means:

"Take the response and convert its JSON content into something JavaScript can work with."

Then we can use another `.then()`.

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

Now `data` contains the actual information.

13. Let's look at the data

The API might give us something like:

```
{
  userId: 1,
  id: 1,
  title: "sunt aut facere repellat provident occaecati",
  body: "quia et suscipit..."
}
```

Now we can access individual properties.

For example:

```
console.log(data.title);
```

or:

```
console.log(data.body);
```

14. Let's put the API data on our webpage

This is where Fetch API becomes really interesting.

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My First API</title>
</head>
```

```
<body>
```

```
<h1>My Post</h1>
```

```
<h2 id="title">Loading...</h2>
```

```
<p id="body">Please wait...</p>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
const title = document.querySelector("#title");  
const body = document.querySelector("#body");
```

```
fetch("https://jsonplaceholder.typicode.com/posts/1")  
  .then(response => response.json())  
  .then(data => {
```

```
    title.textContent = data.title;  
    body.textContent = data.body;
```

```
  });
```

Now something powerful has happened.

Your webpage requested information from an API.

The API sent the information back.

JavaScript received it.

JavaScript then placed the information inside your HTML.

HTML
↓
JavaScript
↓
FETCH
↓
API
↓
DATA
↓
JavaScript
↓
HTML

🔥 That is the basic idea behind Fetch API.

15. Let's understand the code line by line

Line 1

```
const title = document.querySelector("#title");
```

We're selecting:

```
<h2 id="title"></h2>
```

and storing it inside:

title

Line 2

```
const body = document.querySelector("#body");
```

We're selecting:

```
<p id="body"></p>
```

Line 3

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
```

We're requesting data from the API.

Line 4

```
.then(response => response.json())
```

We're waiting for the response and converting it from JSON.

Line 5

```
.then(data => {
```

Now we have the actual data.

Line 6

```
title.textContent = data.title;
```

Take the API's title and put it into the <h2>.

Line 7

```
body.textContent = data.body;
```

Take the API's body and put it into the <p>.

🧠 VERY IMPORTANT PATTERN

For now, memorize this basic Fetch pattern:

```
fetch("API_URL")  
  .then(response => response.json())  
  .then(data => {  
    // use the data here  
  });
```

You will see this pattern again and again.

16. What happens if something goes wrong?

What if:

Internet disappears?

API doesn't exist?

Server has a problem?

URL is wrong?

We need to handle errors.

We can use:

```
.catch()
```

Example:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log("Something went wrong:", error);
  });
```

`.catch()` basically says:

"If something goes wrong, handle the error here."

17. Complete beginner example

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Fetch API Practice</title>
</head>
```

```
<body>
```

```
  <h1>Post Information</h1>
```

```
  <h2 id="title">Loading...</h2>
```

```
  <p id="body">Loading...</p>
```

```
  <p id="error"></p>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
const title = document.querySelector("#title");
```

```
const body = document.querySelector("#body");
const error = document.querySelector("#error");

fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then(response => response.json())
  .then(data => {

    title.textContent = data.title;
    body.textContent = data.body;

  })
  .catch(errorMessage => {

    error.textContent = "Something went wrong.";

    console.log(errorMessage);

  });
```

That's a real little API-powered webpage. 🥳

🔧 PRACTICE BEFORE YOUR PROJECT

Don't jump directly to the project.

Try this first.

Use:

<https://jsonplaceholder.typicode.com/users/1>

Create a webpage that displays:

User Information

Name:

Email:

Phone:

Website:

Your JavaScript should fetch the user and put the information into the page.

You'll need properties such as:

data.name
data.email
data.phone
data.website
Your challenge

Don't copy the previous example exactly.

Try to build it yourself.



DAY 38 ASSIGNMENT PROJECT



Project: "My API User Card"

Build a small webpage that gets a user from the API and displays their information.

Use:

<https://jsonplaceholder.typicode.com/users/1>

Your page should contain:

A heading: User Profile

User's name

Username

Email

Phone

Website

A button called Load User

A message showing Loading... while the data is being requested

An error message if the request fails

Important

Don't worry about making it beautiful yet.

Focus on making the JavaScript work.

Your JavaScript should use:

fetch()
.then()
response.json()
.catch()

and DOM manipulation such as:

`document.querySelector()`

and:

`textContent`



DAY 38 SUMMARY

Today you learned:

1. API

An API allows programs to communicate and exchange information.

2. Fetch API

`fetch()`

is used to request data.

3. Promise

`fetch()` works asynchronously and returns a Promise.

4. `.then()`

Used to handle the result when it becomes available.

5. `response.json()`

Converts JSON response data into something JavaScript can use.

6. `data`

Contains the information returned by the API.

7. `.catch()`

Handles errors.

8. Fetch + DOM

We can fetch information from an API and display it directly on our webpage.

NEXT TOPIC

We're not rushing ahead yet.

First, submit the Day 38 "My API User Card" project.

I'll inspect your HTML and JavaScript, correct it line-by-line, explain every mistake, and then we'll move to the next Fetch API concept.

Teacher Coder is officially back in class.  

JAVASCRIPT

```
const name = document.querySelector("#name");
const username = document.querySelector("#username");
const email = document.querySelector("#email");
const phone = document.querySelector("#phone");
const website = document.querySelector("#website");
const error = document.querySelector("#error");
```

```
const loadButton = document.querySelector("#loadBtn");
```

```
loadButton.addEventListener("click", function () {

    fetch("https://jsonplaceholder.typicode.com/users/1")
        .then(response => response.json())
        .then(data => {

            name.textContent = data.name;
            username.textContent = data.username;
            email.textContent = data.email;
            phone.textContent = data.phone;
            website.textContent = data.website;

        })
        .catch(errorMessage => {

            error.textContent = "Something went wrong.";

            console.log(errorMessage);

        });

});
```

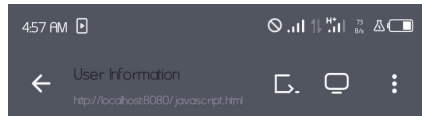
HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>User Information</title>
</head>

<body>

  <h1>User Profile</h1>
  <h3>Ibrahim Musa</h3>
  <p id="name"></p>
  <p id="username">Loading....</p>
  <p id="email"></p>
  <p id="phone"></p>
  <p id="website"></p>
  <p id="error"></p>
  <button id="loadBtn">Load User</button>
  <script src="script.js"></script>

</body>
</html>
```



User Profile

Ibrahim Musa

Leanne Graham

Bret

Sincere@april.biz

1-770-736-8031 x56442

hildegard.org

Load User

JAVASCRIPT DAY 39

Async & Await

Don't worry about the name. It sounds advanced, but the basic idea is actually simple.

Yesterday we learned:

`fetch()` = "JavaScript, go and bring me data."

Today we're learning a cleaner way to work with that Fetch request.

1. First, remember what we did yesterday

We wrote:

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

This works perfectly.

But when our program gets bigger, you might have many `.then()` statements and the code can become harder to read.

That's where `async/await` comes in.

2. What is Async?

`async` means:

"This function is going to work with something that may take some time."

For example:

```
async function getUser() {

}
```

We've created a function called `getUser`.

The word:

`async`

makes the function asynchronous.

For now, don't worry about the technical definition.

Just remember:

`async` allows a function to use `await`.

That's the most important thing for us right now.

3. What is Await?

This is the main character today. 😊

await means:

"Wait for this thing to finish before moving to the next line."

For example:

```
const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
```

In normal human language:

"Go and get the user data. I'll wait for the response before continuing."

That's basically what await does.

4. Why do we need to wait?

Remember that we're getting information from the internet.

The server might respond:

Very quickly ⚡

After 1 second

After 3 seconds

Or even fail

JavaScript shouldn't assume the information is available immediately.

Imagine you're ordering food.

You say:

"Give me rice and chicken."

The waiter says:

"Wait, let me bring it."

You don't immediately start eating an empty plate. 😊

You wait for the food.

That's similar to:

await

5. Our first Async/Await example

Let's start very simply.

```
async function getUser() {  
  
    const response = await fetch(  
        "https://jsonplaceholder.typicode.com/users/1"  
    );  
  
    console.log(response);  
  
}
```

Let's break this down.

Step 1

```
async function getUser() {
```

We're creating an asynchronous function.

Step 2

```
const response = await fetch(  
    "https://jsonplaceholder.typicode.com/users/1"  
);
```

We're asking the API for the user.

await says:

"Wait until the response arrives."

Step 3

```
console.log(response);
```

After the response arrives, we print it.

6. But there's one problem

Look at this:

```
console.log(response);
```

Remember yesterday?

We learned that response is not yet the actual data.

It's the response from the server.

We need:

```
response.json()
```

So let's continue.

7. await response.json()

Now we write:

```
async function getUser() {  
  
    const response = await fetch(  
        "https://jsonplaceholder.typicode.com/users/1"  
    );  
  
    const data = await response.json();  
  
    console.log(data);  
  
}
```

Now let's understand the two awaits.

First:

```
const response = await fetch(url);
```

Means:

"Wait for the server to respond."

Second:

```
const data = await response.json();
```

Means:

"Wait while the response is converted into usable JSON data."

Then:

```
console.log(data);
```

gives us the actual user information.

8. Compare yesterday and today

Yesterday:

```
fetch("https://jsonplaceholder.typicode.com/users/1")  
  .then(response => response.json())  
  .then(data => {  
    console.log(data);  
  });
```

Today:

```
async function getUser() {  
  
  const response = await fetch(  
    "https://jsonplaceholder.typicode.com/users/1"  
  );  
  
  const data = await response.json();  
  
  console.log(data);  
}
```

They accomplish basically the same thing.

The second version can be easier to read because it looks more like normal step-by-step instructions.

9. But our function isn't running!

Good catch if you noticed that. 😎

We created:

```
async function getUser() {  
  
}
```

But we haven't called the function.

Remember how functions work?

Creating a function doesn't automatically run it.

We need:

```
getUser();
```

So the complete code is:

```
async function getUser() {  
  
    const response = await fetch(  
        "https://jsonplaceholder.typicode.com/users/1"  
    );  
  
    const data = await response.json();  
  
    console.log(data);  
}  
  
getUser();
```

Now it runs.

10. Let's use HTML

Now we're going to make this more realistic.

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Async Await</title>
```

```
</head>
```

```
<body>
```

```
  <h1>User Information</h1>
```

```
  <button id="loadBtn">Load User</button>
```

```
  <h2 id="name"></h2>
```

```
  <p id="email"></p>
```

```
  <p id="phone"></p>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

We have:

```
<button id="loadBtn">Load User</button>
```

This is the button the user will click.

Then:

```
<h2 id="name"></h2>
```

will display the name.

And:

```
<p id="email"></p>
```

will display the email.

And:

```
<p id="phone"></p>
```

will display the phone number.

11. JavaScript

Now let's connect everything.

```
const loadBtn = document.querySelector("#loadBtn");
```

```
const name = document.querySelector("#name");
```

```
const email = document.querySelector("#email");
```

```
const phone = document.querySelector("#phone");
```

We're selecting our HTML elements.

This is nothing new.

You've already learned `querySelector()`.

12. Add the button event

Now:

```
loadBtn.addEventListener("click", function() {  
  
});
```

This means:

"When the user clicks the button, run this code."

Again, this is something you've already learned.

13. Put our Async/Await inside the click

Now:

```
loadBtn.addEventListener("click", async function() {  
  
});
```

Notice something important:

We put:

async

before:

function()

Why?

Because we're going to use await inside this function.

Now we can write:

```
loadBtn.addEventListener("click", async function() {  
  
    const response = await fetch(  
        "https://jsonplaceholder.typicode.com/users/1"  
    );
```

```
});
```

14. Get the actual data

Now:

```
loadBtn.addEventListener("click", async function() {
```

```
    const response = await fetch(  
        "https://jsonplaceholder.typicode.com/users/1"  
    );
```

```
    const data = await response.json();
```

```
});
```

Now data contains the user information.

15. Display the information

Now we can do:

```
name.textContent = data.name;
```

```
email.textContent = data.email;
```

```
phone.textContent = data.phone;
```

So our complete JavaScript becomes:

```
const loadBtn = document.querySelector("#loadBtn");
```

```
const name = document.querySelector("#name");
```

```
const email = document.querySelector("#email");
```

```
const phone = document.querySelector("#phone");
```

```
loadBtn.addEventListener("click", async function() {
```

```
    const response = await fetch(  
        "https://jsonplaceholder.typicode.com/users/1"  
    );
```

```
    const data = await response.json();
```

```
    name.textContent = data.name;
```

```
    email.textContent = data.email;
```

```
    phone.textContent = data.phone;
```

```
});
```

🔥 This is a real Async/Await + Fetch example.

16. Let's understand the whole thing in simple English

When you click:

Load User

JavaScript says:

```
const response = await fetch(url);
```

"Go to the API and bring me the user."

Then:


```
const data = await response.json();
```

"Okay, now convert what you brought into data I can use."

Then:

```
name.textContent = data.name;
```

"Take the user's name and put it on my webpage."

Then:

```
email.textContent = data.email;
```

"Take the email and put it on my webpage."

Then:

```
phone.textContent = data.phone;
```

"Take the phone number and put it on my webpage."

That's the whole process.

17. Why do we use const?

You asked this yesterday, so let's connect it to today's lesson.

We have:

```
const response = await fetch(url);
```

Why const?

Because we're not going to replace the response variable with something else.

Same with:

```
const data = await response.json();
```

We're storing the data in data.

We don't need to do:

```
data = somethingElse;
```

So const is appropriate.

Remember:

const doesn't mean the information inside can never change. It means you cannot reassign the variable itself.

18. What if something goes wrong?

This is VERY important.

Suppose:

The internet goes off.

The API is unavailable.

The URL is wrong.

The server has a problem.

We need error handling.

With async/await, we commonly use:

try

and:

catch

19. try and catch

Think of it like this:

TRY

↓

"Try to do this."

If it works

↓

Continue

If it fails

↓
CATCH

↓
"Handle the problem."

Example:

```
try {  
  
    // code that might fail  
  
}  
catch (error) {  
  
    // handle the error  
  
}
```

20. Put Fetch inside try

Now:

```
async function getUser() {  
  
    try {  
  
        const response = await fetch(  
            "https://jsonplaceholder.typicode.com/users/1"  
        );  
  
        const data = await response.json();  
  
        console.log(data);  
  
    }  
    catch (error) {
```

```
        console.log("Something went wrong.");
    }

}
```

Now we're saying:

"TRY to fetch the user."

If everything works:

```
console.log(data);
```

If something goes wrong:

```
console.log("Something went wrong.");
```

21. **Complete HTML + JavaScript example**

HTML

```
<!DOCTYPE html>
<html>
```

```
<head>
  <title>Async Await Practice</title>
</head>
```

```
<body>
```

```
<h1>User Profile</h1>
```

```
<button id="loadBtn">Load User</button>
```

```
<h2 id="name">Name</h2>
```

```
<p id="email">Email</p>
```

```
<p id="phone">Phone</p>
```

```
<p id="error"></p>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
const loadBtn = document.querySelector("#loadBtn");
```

```
const name = document.querySelector("#name");
```

```
const email = document.querySelector("#email");
```

```
const phone = document.querySelector("#phone");
```

```
const error = document.querySelector("#error");
```

```
loadBtn.addEventListener("click", async function() {
```

```
    try {  
  
        const response = await fetch(  
            "https://jsonplaceholder.typicode.com/users/1"  
        );  
  
        const data = await response.json();  
  
        name.textContent = data.name;  
  
        email.textContent = data.email;  
  
        phone.textContent = data.phone;  
  
    }  
    catch (errorMessage) {  
  
        error.textContent = "Something went wrong.";  
  
        console.log(errorMessage);  
  
    }  
  
});
```

 Now let's slow down and understand the pattern

Your brain might look at this:

```
loadBtn.addEventListener("click", async function() {  
  
    try {
```

```
const response = await fetch(url);

const data = await response.json();

name.textContent = data.name;

}
catch (error) {

    console.log(error);

}

});
```

and say:

"Teacher, what the hell is all this?" 😂

Don't worry.

Break it into four stages:

1. Click
addEventListener("click")

"Wait for the button."

2. Fetch
await fetch(url)

"Go bring the data."

3. Convert
await response.json()

"Turn the response into usable data."

4. Display

```
name.textContent = data.name
```

"Put the data on the webpage."

And try/catch is just there to handle problems.

★ The most important thing to memorize

Don't memorize the whole huge program.

Remember this small pattern:

```
async function getData() {  
  
    const response = await fetch("URL");  
  
    const data = await response.json();  
  
    console.log(data);  
  
}
```

In man-man English:

async
= "This function can wait."

await fetch()
= "Go bring the data and wait."

await response.json()
= "Make the response usable."

data
= "The actual information."

That's it.

🔥 Async/Await vs .then()

Yesterday:

```
fetch(url)
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

Today:

```
async function getData() {

  const response = await fetch(url);

  const data = await response.json();

  console.log(data);

}
```

Both can get the same data.

Async/await is simply another way of handling asynchronous operations, and it often makes the code easier to read.

 **PRACTICE — BEFORE THE ASSIGNMENT**

Don't start the big project yet.

Build this small project yourself:

"Load User"

Your HTML should have:

User Information

[Load User]

Name:

Email:

Website:

Use:

<https://jsonplaceholder.typicode.com/users/1>

When the button is clicked:

Fetch the user.

Use `async`.

Use `await`.

Convert the response with `.json()`.

Display the name.

Display the email.

Display the website.

Use `try/catch`.

Your JavaScript must contain:

`async`

`await`

`fetch()`

`response.json()`

`try`

`catch`

Don't copy my complete example. Try to build it yourself.



DAY 39 ASSIGNMENT

After the practice, your assignment will be:



My Async User Card

Build a user card that loads information from:

<https://jsonplaceholder.typicode.com/users/1>

Your card should display:

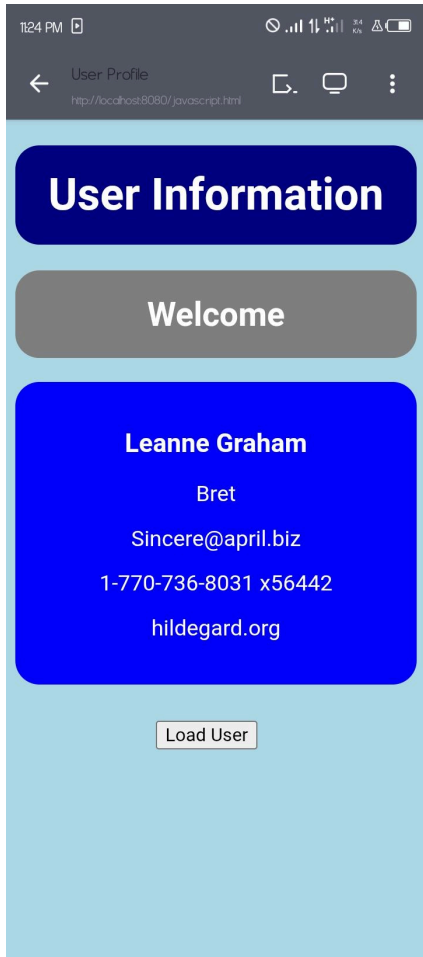
Name
Username
Email
Phone
Website

It should have:

Load User button
Loading... message
Error message if something fails
Some CSS styling

And this time, instead of `.then()`, use:

`async`
`await`
`try`
`catch`



```
11:26 PM
javascript.html
< > javascript.
< > javascript.
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>User Profile</title>
6   <link rel="stylesheet"
7     href="JavaScript.css">
8 </head>
9 <body>
10   <h1 id="header">User
11   Information</h1>
12   <h2 id="loading">Please
13   Wait...</h2>
14   <div class="card">
15     <h3 id="name"></h3>
16     <p id="username"></p>
17     <p id="email"></p>
18     <p id="phone"></p>
19     <p id="website"></p>
20     <p id="error"></p>
21   </div>
22   <button id="loadBtn">Load
23   User</button>
24   <script src="script.js"></
25   script>
26 </body>
27 </html>
```

<div> <a> [class] [id] h
->| = " " < > </ - <!-- --

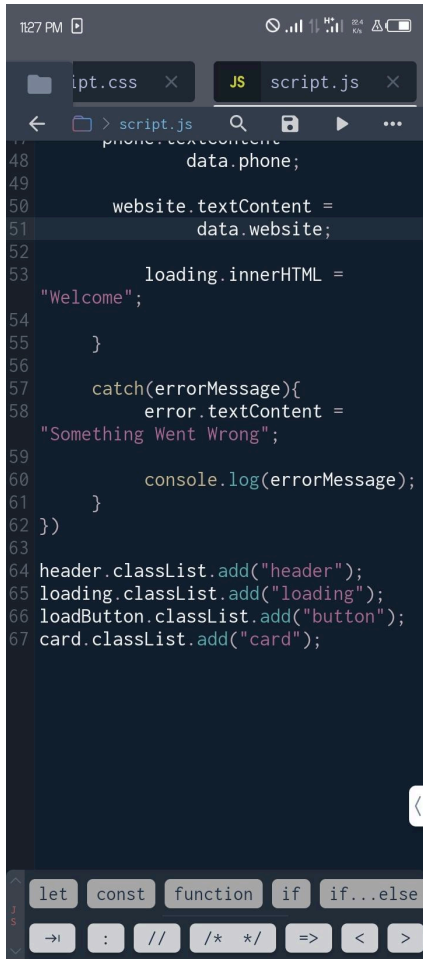
The image shows a mobile application interface for editing code. At the top, there's a status bar with the time 11:27 PM and various system icons. Below it, a tab bar shows two tabs: 'ipt.css' and 'JS script.js', with 'JS script.js' being the active tab. The main area is a code editor with a dark theme, displaying JavaScript code. The code uses `document.querySelector` to select various HTML elements and assigns them to constants. It also includes an event listener for a button click, using an `async function`. A bottom toolbar contains several buttons for code editing, including 'let', 'const', 'function', 'if', 'if...else', and symbols for indentation, semicolon, comments, and logical operators.

```
1 const header = document.  
2     querySelector("#header");  
3  
4 const card = document.  
5     querySelector(".card");  
6  
7 const name = document.  
8     querySelector("#name");  
9  
10 const username = document.  
11     querySelector("#username");  
12  
13 const email = document.  
14     querySelector("#email");  
15  
16 const phone = document.  
17     querySelector("#phone");  
18  
19 const website = document.  
20     querySelector("#website");  
21  
22 const loading = document.  
23     querySelector("#loading");  
24  
25 const loadButton = document.  
26     querySelector("#loadBtn");  
27  
28 /* Button Event */  
29 loadButton.addEventListener("click",  
30     async function () {  
31         try{  
32             let  
33             const  
34             function  
35             if  
36             if...else  
37             :  
38             //  
39             /* */  
40             =>  
41             <  
42             >
```

The image shows a mobile application interface with a dark-themed code editor. At the top, there's a status bar with the time 11:27 PM and various icons. Below it, a tab bar shows 'ipt.css' and 'script.js'. The code editor displays a JavaScript file named 'script.js' with the following content:

```
31     try{
32         const response = await
fetch("https://jsonplaceholder
.typicode.com/users/1"
    );
33
34
35     const data = await
36         response.json();
37
38     name.textContent =
39         data.name;
40
41     username.textContent =
42         data.username;
43
44     email.textContent =
45         data.email;
46
47     phone.textContent =
48         data.phone;
49
50     website.textContent =
51         data.website;
52
53     loading.innerHTML =
"Welcome";
54
55     }
56
57     catch(errorMessage){
58         error.textContent =
"Something went wrong";
59     }
60 }
```

At the bottom, there's a keyboard with a search icon, a colon, forward slash, forward slash asterisk, equals greater, less, and greater than symbols.



The screenshot shows a mobile application interface with a dark-themed code editor. The editor displays JavaScript code for an asynchronous fetch operation. The code includes a .then() method to handle the successful response and a .catch() method to handle errors. It also shows DOM manipulation using classList.add(). The code is as follows:

```
48     data.phone;
49
50     website.textContent =
51     data.website;
52
53     loading.innerHTML =
54     "Welcome";
55   }
56
57   catch(errorMessage){
58     error.textContent =
59     "Something Went Wrong";
60     console.log(errorMessage);
61   }
62 })
63
64 header.classList.add("header");
65 loading.classList.add("loading");
66 loadButton.classList.add("button");
67 card.classList.add("card");
```

At the bottom of the screen, there is a toolbar with various JavaScript keywords and symbols: let, const, function, if, if...else, →, :, //, /* */, =>, <, >.

You're combining yesterday's Fetch knowledge with today's Async/Await.

DAY 39 SUMMARY

Today we learned:

async

Makes a function asynchronous and allows us to use await inside it.

await

Tells JavaScript to wait for an asynchronous operation to finish before continuing.

await fetch()

Wait for the API response.

await response.json()

Wait for the response to be converted into usable data.

try

Try to run code that could fail.

catch

Handle the problem if it fails.

The basic pattern:

```
async function getData() {
```

```
  try {
```

```
    const response = await fetch("URL");
```

```
    const data = await response.json();
```

```
    console.log(data);
```

```
  }
```

```
  catch (error) {
```

```
    console.log(error);
```

```
  }
```

```
}
```

Man-man version:

async = "This function may need to wait."

await = "Wait for it."

fetch = "Go and bring the data."

json() = "Make the data usable."

try/catch = "Try it, and if it fails, handle it."

That's Day 39 — Async & Await. 🧑‍💻🔥

Do the small practice first. Then build the My Async User Card assignment and submit it to me. I'll mark it like we did with your Day 38 project, explain every mistake, and then we'll move to the next topic

JAVASCRIPT — DAY 40

HTTP METHODS: GET, POST, PUT, PATCH & DELETE

Don't let the name HTTP methods scare you. The idea is actually very simple.

We've already been using one of them without really focusing on it.

1. First, what are HTTP methods?

Remember yesterday's API explanation:

API = a bridge that allows your website to communicate with another system.

Now imagine your website is talking to a server.

Your website can tell the server different things:

"Give me this information."

"Here is some new information. Save it."

"Change this information."

"Delete this information."

HTTP methods are basically instructions telling the server what you want to do.

The five you'll commonly see are:

Method	Simple meaning
GET	Give me data
POST	Save/create new data
PUT	Replace/update data
PATCH	Change part of the data
DELETE	Delete data

Let's make this extremely simple.

2. GET — "Give me the data"

You've already used GET.

When we wrote:

```
fetch("https://jsonplaceholder.typicode.com/users/1")
```

we were basically saying:

"API, give me user number 1."

That's a GET request.

And here's something important:

When you use `fetch()` without specifying a method, the default method is GET.

So this:

```
fetch("https://jsonplaceholder.typicode.com/users/1")
```

is basically the same idea as:

```
fetch("https://jsonplaceholder.typicode.com/users/1", {  
  method: "GET"  
});
```

The second version explicitly tells JavaScript:

"Use GET."

3. Why is GET important?

Imagine you have a website showing products.

You want:

Nike Shoes
\$50

Adidas Shirt
\$30

Football
\$20

Your website can use GET to ask the server:

"Give me the products."

The server sends the products back.

So:

```
WEBSITE
↓
GET
↓
SERVER
↓
PRODUCT DATA
↓
WEBSITE
```

4. POST — "Create something new"

Now things become more interesting.

Imagine you have a registration form:

Name: Ibrahim
Email: ibrahim@example.com
Password: *****

[Register]

When the user clicks Register, the website needs to send that information to the server.

That's where POST comes in.

POST basically means:

"Hey server, here is some new information. Save it."

For example:

```
fetch("https://example.com/users", {  
  method: "POST"  
});
```

Now we're telling the server:

"I'm sending you new information."

5. GET vs POST

This is VERY important.

GET:

"Give me information."

POST:

"Here is new information. Create/save it."

Think about a school.

GET:

"Teacher, show me the student list."

POST:

"Teacher, add this new student to the list."



That's the basic difference.

6. Sending data with POST

Suppose we want to create a user.

Our data could be:

```
const user = {  
  name: "Ibrahim",  
  username: "ibrahim123",  
  email: "ibrahim@example.com"  
};
```

We can send this to an API.

```
fetch("https://jsonplaceholder.typicode.com/users", {  
  method: "POST",  
  
  headers: {  
    "Content-Type": "application/json"  
  },  
  
  body: JSON.stringify(user)  
});
```

Whoa. 🤔

That looks like a lot.

Don't worry.

We're going to break it down.

7. method: "POST"

This part:

method: "POST"

tells the API:

"I'm sending new information."

Simple.

8. What are headers?

This:

```
headers: {  
  "Content-Type": "application/json"  
}
```

is basically telling the server:

"The information I'm sending you is JSON."

Think of a package.

You put something inside the package and attach a label saying:

"This package contains clothes."

The server needs to know what kind of information you're sending.

So we give it a label.

Content-Type: application/json

means:

"This data is JSON."

9. What is body?

This is where the actual information we're sending goes.

body: JSON.stringify(user)

Remember our object:

```
const user = {  
  name: "Ibrahim",  
  username: "ibrahim123",  
  email: "ibrahim@example.com"  
};
```

We want to send that information to the server.

But the API expects JSON.

So:

```
JSON.stringify(user)
```

converts the JavaScript object into a JSON string that can be sent.

10. What does `JSON.stringify()` mean?

This is simple.

Imagine you have:

```
const user = {  
  name: "Ibrahim",  
  age: 20  
};
```

`JSON.stringify()` basically says:

"Turn this JavaScript object into JSON text."

Example:

```
JSON.stringify(user)
```

produces something like:

```
{"name":"Ibrahim","age":20}
```

So:

JavaScript object



`JSON.stringify()`



JSON data

You'll see this a lot when sending data to APIs.

11. Let's make a complete POST example

We'll use JSONPlaceholder because it's made for practice.

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>POST Request</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Create User</h1>
```

```
  <input id="name" placeholder="Enter name">
```

```
  <input id="email" placeholder="Enter email">
```

```
  <button id="saveBtn">Save User</button>
```

```
  <p id="result"></p>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Now JavaScript:

```
const nameInput = document.querySelector("#name");
```

```
const emailInput = document.querySelector("#email");
```

```
const saveBtn = document.querySelector("#saveBtn");
const result = document.querySelector("#result");
```

```
saveBtn.addEventListener("click", async function() {
```

```
    const user = {
      name: nameInput.value,
      email: emailInput.value
    };

```

```
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users",
      {
        method: "POST",

```

```
        headers: {
          "Content-Type": "application/json"
        },

```

```
        body: JSON.stringify(user)
      }
    );

```

```
    const data = await response.json();

```

```
    result.textContent = "User created: " + data.name;

```

```
  });

```

Now let's understand what happened.

12. User types information

Suppose the user enters:

Name:

Ibrahim

Email:

ibrahim@example.com

JavaScript gets those values:

nameInput.value

and:

emailInput.value

Then creates:

```
const user = {  
  name: nameInput.value,  
  email: emailInput.value  
};
```

So the object becomes:

```
{  
  name: "Ibrahim",  
  email: "ibrahim@example.com"  
}
```

13. Then POST sends it
method: "POST"

means:

"Create this."

And:

body: JSON.stringify(user)

means:

"Here's the information I'm sending."

The server receives it.

14. Then we receive the response

Just like GET, we can do:

```
const data = await response.json();
```

Now we can use the response.

```
result.textContent = "User created: " + data.name;
```

🧠 So far...

We've learned:

GET

"Give me data."

POST

"Create/save new data."

That's enough for the first part.

15. PUT — "Replace the data"

Now imagine you already have:

```
{  
  name: "Ibrahim",  
  email: "old@email.com"  
}
```

You want to update it.

PUT generally means:

"Replace/update this resource with this new version."

For example:

```
fetch("https://jsonplaceholder.typicode.com/users/1", {  
  method: "PUT",  
  
  headers: {  
    "Content-Type": "application/json"  
  },  
})
```

```
body: JSON.stringify({
  name: "Ibrahim Musa",
  email: "new@email.com"
})
});
```

Think:

PUT = "Replace/update the whole thing."

16. PATCH — "Change just part of it"

PUT and PATCH are similar, but there is an important difference.

Imagine a user has:

```
{
  name: "Ibrahim",
  email: "old@email.com",
  phone: "123456"
}
```

You only want to change the email.

PATCH is useful for:

"Change this small part."

Example:

```
fetch("https://jsonplaceholder.typicode.com/users/1", {
  method: "PATCH",
```

```
  headers: {
    "Content-Type": "application/json"
  },
```

```
  body: JSON.stringify({
    email: "new@email.com"
  })
})
```

});

You're saying:

"Don't change everything. Just change the email."

17. PUT vs PATCH

Remember this:

PUT:

"Replace/update the resource."

PATCH:

"Change part of the resource."

Imagine your profile:

Name

Email

Phone

Website

If you replace/update the whole profile:

PUT

If you only change your phone number:

PATCH

18. DELETE — "Remove this"

Finally:

DELETE

means:

"Delete this data."

Example:

```
fetch("https://jsonplaceholder.typicode.com/users/1", {  
  method: "DELETE"  
});
```

You're basically telling the server:

"Remove user number 1."

19. The five methods in one picture

This is what I want you to remember:

GET

↓

"Give me data."

POST

↓

"Create new data."

PUT

↓

"Replace/update data."

PATCH

↓

"Change part of the data."

DELETE

↓

"Delete data."

🔥 That's the heart of today's lesson.

20. Why are these methods important for websites?

Imagine you're building a real website with an account system.

Register

User creates an account:

POST

Login / Get profile

Website requests information:

GET

Edit profile

User changes their whole profile:

PUT

Change just their phone number:

PATCH

Delete account:

DELETE

So these methods allow your frontend to communicate with the backend/server and manage data.

 MAN-MAN VERSION

If you forget everything else, remember this:

GET = Give me.

POST = Take this new thing.

PUT = Replace/update this.

PATCH = Change this small part.

DELETE = Remove this.

That's it. 

 PRACTICE

Before your assignment, practice GET and POST.

Create a simple page:

Create User

Name: []

Email: []

[Create User]

Result:

When the button is clicked:

Get the name from the input.

Get the email from the input.

Create a JavaScript object.

Send it to JSONPlaceholder using POST.

Use JSON.stringify().

Use await.

Use response.json().

Display the returned name on the page.

Don't worry about PUT, PATCH and DELETE yet.

```
const nameInput = document.querySelector("#name");
```

```
const emailInput = document.querySelector("#email");
```

```
const createButton = document.querySelector("#createBtn");
```

```
const result = document.querySelector("#result");
```

```
createButton.addEventListener("click", async function() {
```

```
    const user = {  
        name: nameInput.value,  
        email: emailInput.value  
    };  
};
```

```
    const response = await fetch("https://jsonplaceholder.typicode.com/users",  
        {
```

```
        method:"GET",
        method:"POST",

        headers:{
            "content-type":
            "application/json"
        },

        body:JSON.stringify(user)

    }
    );

    const data = await response.json();

    result.textContent = "User Created " + data.name;

});
```

```
<!DOCTYPE html>
<html>

<head>
    <title>POST Request</title>
</head>

<body>

    <h1>Create User</h1>

    <input id="name" placeholder="Enter name">

    <input id="email" placeholder="Enter email">

    <button id="createBtn">Creat User</button>

    <p id="result"></p>

    <script src="script.js"></script>

</body>

</html>
```

Get POST working first.

DAY 40 ASSIGNMENT

After the practice, build:

User Manager

Your webpage should have:

Section 1 — Get User

A button:

Load User

Use GET to load user number 1.

Display:

Name

Username

Email

Section 2 — Create User

Inputs:

Name

Email

Button:

Create User

Use POST to send the new user.

Section 3 — Update User

An input for a new name.

Button:

Update User

Use PUT.

Section 4 — Change Email

An input for a new email.

Button:

Change Email

Use PATCH.

Section 5 — Delete User

Button:

Delete User

Use DELETE.



BUT DON'T BUILD THE WHOLE ASSIGNMENT YET

We've covered five methods today, but I don't want you to become overwhelmed.

First do the small POST practice.

Once you understand POST, we'll make sure you understand PUT, PATCH and DELETE before you build the full project.



DAY 40 SUMMARY

Today we learned that HTTP methods are instructions we use when communicating with an API.

GET → get data

POST → create new data

PUT → update/replace data

PATCH → update part of data

DELETE → delete data

We also learned:

method: "POST"

to specify the method,

```
headers: {  
  "Content-Type": "application/json"  
}
```

to tell the server we're sending JSON,

and:

```
JSON.stringify(user)
```

to convert a JavaScript object into JSON before sending it.

Next step:

Do the small POST practice first. When you've done it, show me your code/screenshot. I'll correct it, then we'll continue Day 40 with PUT, PATCH and DELETE before you tackle the full assignment. 🏠🔥.

DAY 40 — PART 2

PUT, PATCH & DELETE

We've already learned:

GET → "Give me data."

POST → "Create new data."

Now:

PUT → "Update/replace data."

PATCH → "Change part of data."

DELETE → "Delete data."

And I'll show you where each one is used in real websites.

1 PUT — UPDATE/REPLACE DATA

Imagine you have a user profile:

Name: Ibrahim Musa
Email: ibrahim@gmail.com
Phone: 08012345678
Website: ibrahim.com

The user wants to edit the profile.

A website can use PUT to send the updated information to the server.

Real website examples

PUT can be used when:

Updating a complete user profile
Replacing a product's information
Updating a student's complete record
Updating an address
Updating settings

Think:

PUT = "Here is the new version of this thing."

PUT Example

Let's use JSONPlaceholder.

HTML

```
<!DOCTYPE html>  
<html>
```

```
<head>  
  <title>PUT Request</title>  
</head>
```

```
<body>

  <h1>Update User</h1>

  <input id="name" placeholder="New name">

  <input id="email" placeholder="New email">

  <button id="updateBtn">Update User</button>

  <p id="result"></p>

  <script src="script.js"></script>

</body>

</html>
```

Now JavaScript:

```
const nameInput = document.querySelector("#name");
const emailInput = document.querySelector("#email");

const updateBtn = document.querySelector("#updateBtn");

const result = document.querySelector("#result");

updateBtn.addEventListener("click", async function() {

  const user = {
    name: nameInput.value,
    email: emailInput.value
  };

  const response = await fetch(
    "https://jsonplaceholder.typicode.com/users/1",
    {
      method: "PUT",

      headers: {
        "Content-Type": "application/json"
```

```
    },  
    body: JSON.stringify(user)  
  }  
);  
  
const data = await response.json();  
  
result.textContent = "Updated: " + data.name;  
  
});
```

 Let's break down the important part

This:

method: "PUT"

tells the API:

"I want to update this user."

Then:

```
body: JSON.stringify(user)
```

sends the new information.

For example, if you type:

Name: Ibrahim Musa

Email: musa@gmail.com

JavaScript creates:

```
const user = {  
  name: "Ibrahim Musa",  
  email: "musa@gmail.com"  
};
```

Then sends it to:

/users/1

② PATCH — CHANGE ONLY PART

Now let's say your user has:

Name: Ibrahim Musa

Email: ibrahim@gmail.com

Phone: 08012345678

Website: ibrahim.com

You only want to change the email.

You don't need to send everything again.

That's where PATCH comes in.

PATCH = "Change this particular part."

Real website examples

PATCH can be useful for:

Changing your profile picture

Changing your phone number

Changing your email

Changing your username

Turning notifications ON/OFF

Changing someone's account status

Updating one product price

PATCH Example

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>PATCH Request</title>
```

```
</head>
```

```
<body>
```

```
<h1>Change Email</h1>

<input id="email" placeholder="New email">

<button id="changeBtn">Change Email</button>

<p id="result"></p>

<script src="script.js"></script>

</body>

</html>
```

JavaScript

```
const emailInput = document.querySelector("#email");

const changeBtn = document.querySelector("#changeBtn");

const result = document.querySelector("#result");

changeBtn.addEventListener("click", async function() {

  const user = {
    email: emailInput.value
  };

  const response = await fetch(
    "https://jsonplaceholder.typicode.com/users/1",
    {
      method: "PATCH",

      headers: {
        "Content-Type": "application/json"
      },

      body: JSON.stringify(user)
    }
  );
```

```
const data = await response.json();

result.textContent = "New Email: " + data.email;

});
```

Notice something important.

We're only sending:

```
const user = {
  email: emailInput.value
};
```

We aren't sending:

name
username
phone
website

because we only want to change the email.

That's the main idea of PATCH.

vs PUT vs PATCH

This is one part I want you to understand VERY well.

Imagine your profile has:

Name
Email
Phone
Website
PUT

"Here is the updated version of my profile."

You're generally sending the resource's full replacement representation.

PATCH

"I only want to change my email."

You're sending just the part that needs changing.

Easy memory trick:

PUT = Whole thing

PATCH = Part of it

That's not a perfect technical definition for every API, but it's an excellent beginner mental model.

③ DELETE — REMOVE DATA

Now imagine you have an account and click:

[Delete Account]

The website needs to tell the server:

"Delete this account."

That's what:

method: "DELETE"

is for.

 Where is DELETE used on real websites?

You'll see it in things like:

Delete an account

Delete a post

Delete a comment

Remove a product

Delete a message

Remove a task from a todo list

Delete a saved address

For example, on a Todo app:

Buy food [Delete]
Study JavaScript [Delete]
Go to work [Delete]

When you press Delete, JavaScript can send a DELETE request to the backend.

DELETE Example

HTML

```
<!DOCTYPE html>
<html>

<head>
  <title>DELETE Request</title>
</head>

<body>

  <h1>Delete User</h1>

  <button id="deleteBtn">Delete User</button>

  <p id="result"></p>

  <script src="script.js"></script>

</body>

</html>
```

JavaScript

```
const deleteBtn = document.querySelector("#deleteBtn");

const result = document.querySelector("#result");

deleteBtn.addEventListener("click", async function() {

  const response = await fetch(
    "https://jsonplaceholder.typicode.com/users/1",
    {
      method: "DELETE"
    }
  );
});
```

```
if (response.ok) {  
    result.textContent = "User deleted successfully.";  
}  
});
```

🤔 What's response.ok?

You've already learned:

```
const response = await fetch(url);
```

The response contains information about what happened.

response.ok tells us whether the HTTP response was successful.

So:

```
if (response.ok) {
```

means:

"If the request was successful, do this."

Then:

```
result.textContent = "User deleted successfully.";
```

shows the message.

🔥 NOW PUT ALL FIVE TOGETHER

Imagine you're building a real User Management website.

👁️ View user

GET

"Give me the user."

➕ Create user

POST

"Create this new user."

 Update complete user
PUT

"Update/replace this user's information."

 Change one part
PATCH

"Only change this part."

 Remove user
DELETE

"Delete this user."

 MAN-MAN VERSION

Put this in your notes:

GET = GIVE ME

POST = CREATE ONE

PUT = UPDATE THE WHOLE THING

PATCH = CHANGE ONE PART

DELETE = REMOVE IT

 That's your five HTTP methods.

 One important thing about JSONPlaceholder

You're practicing with:

<https://jsonplaceholder.typicode.com>

It's a fake/testing API designed for practice.

So when you POST, PUT, PATCH or DELETE, it can give you a response that looks like the operation worked, but it isn't actually creating permanent real users in a production database.

That's perfect for us right now because you can practice API communication without needing to build a backend.

YOUR NEXT PRACTICE

You have already completed:

GET + POST practice 

Now before the big assignment, I want you to make three tiny projects, preferably in the same HTML/JS file if that's easier.

Practice 1 — PUT

Create:

Update User

[New Name]

[New Email]

[Update User]

Updated: ...

Use:

method: "PUT"

```
const emailInput = document.querySelector("#email");
```

```
const nameInput = document.querySelector("#name");
```

```
const updateButton = document.querySelector("#updateBtn");
```

```
const result = document.querySelector("#result");
```

```
updateButton.addEventListener("click", async function () {
```

```
  const user = {  
    name: nameInput.value,  
    email: emailInput.value
```

```
  } ;
```



```
const response = await fetch("https://jsonplaceholder.typicode.com/users/1",
  {
    method:"PUT",

    headers:{
      "content -Type":
        "application/json"
    },

    body:JSON.stringify(user)

  }
);

const data = await
response.json();

result.textContent = "Update Successfully ";

})
```

```
<!DOCTYPE html>
<html>

<head>
  <title>PUT REQUEST</title>
</head>

<body>

  <h1>Update User</h1>

  <input id="name" placeholder="Enter New Name">

  <input id="email" placeholder="Enter New Email">

  <button id="updateBtn">Creat User</button>

  <p id="result"></p>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Practice 2 — PATCH

Create:

Change Email

[New Email]

[Change Email]

New Email: ...

Use:

method: "PATCH"

```
const emailInput = document.querySelector("#email");
```

```
const changeButton = document.querySelector("#changeBtn");
```

```
const result = document.querySelector("#result");
```

```
changeButton.addEventListener("click", async function() {
```

```
    const user = {  
      email:emailInput.value  
    };
```

```
    const response = await fetch("https://jsonplaceholder.typicode.com/users/1",  
      {  
        method:"PATCH",  
  
        headers:{  
          "content-Type":  
            "application/json"  
        },
```

```
        body:JSON.stringify(user)

    }
    );

    const data = await
    response.json();

    result.textContent = "New Email: " + data.email;

}))

<!DOCTYPE html>
<html>

<head>
  <title>Patch REQUEST</title>
</head>

<body>

  <h1>Change User Email</h1>

  <input id="email" placeholder="Change Email">

  <button id="changeBtn">Change Email</button>

  <p id="result"></p>

  <script src="script.js"></script>

</body>

</html>
```

Practice 3 — DELETE

Create:

Delete User

[Delete User]

User deleted successfully.

Use:

method: "DELETE"

```
const changeButton = document.querySelector("#deleteBtn");
```

```
const result = document.querySelector("#result");
```

```
changeButton.addEventListener("click", async function() {
```

```
    const response = await fetch("https://jsonplaceholder.typicode.com/users/1",  
    {  
        method:"DELETE",
```

```
    }  
    );
```

```
    if(response.ok){  
        result.textContent = "Dekete Successfully "
```

```
    }  
  
    })
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Delete REQUEST</title>
```

```
</head>
```

```
<body>
```

```
    <h1>Delete Profile</h1>
```

```
    <button id="deleteBtn">Delete Profile</button>
```

```
    <p id="result"></p>
```

```
<script src="script.js"></script>

</body>

</html>
```

🚫 Don't build the full assignment yet.

Try these three small practices yourself first.

You can look back at my examples when you forget the syntax — that's completely fine. Your goal is to understand:

PUT → update

PATCH → change part

DELETE → remove

When you finish them, show me your code/screenshot.

I'll check them, and then I'll give you the green light to build the full Day 40 User Manager assignment. 😎🔥

Yes! 😎 You absolutely can combine all 5 HTTP methods into ONE project and ONE HTML file + ONE JavaScript file.

And actually, that's exactly what I want you to do for the Day 40 assignment.

I think the confusing part is that I showed you five separate small examples. Those were only to teach each method individually. In a real website, you can have all of them working together.

Let me show you how to think about it.

🧠 Imagine you're building a "User Manager"

Your website has one page.

On that one page you have different buttons:

👤 USER MANAGER

[Load User]

Name: Ibrahim
Email: ibrahim@gmail.com

CREATE USER

Name:
Email:

[Create User]

UPDATE USER

Name:
Email:

[Update User]

CHANGE EMAIL

Email:

[Change Email]

DELETE USER

[Delete User]

It's one website.

But each button performs a different operation.

 Each button uses a different HTTP method

1. Load User button

[Load User]

uses:

GET

Because you're saying:

"Server, give me the user's information."

2. Create User button

[Create User]

uses:

POST

Because you're saying:

"Server, create a new user using this information."

3. Update User button

[Update User]

uses:

PUT

Because you're saying:

"Server, update this user's information."

4. Change Email button

[Change Email]

uses:

PATCH

Because you're saying:

"Server, change only this part of the user."

5. Delete User button

[Delete User]

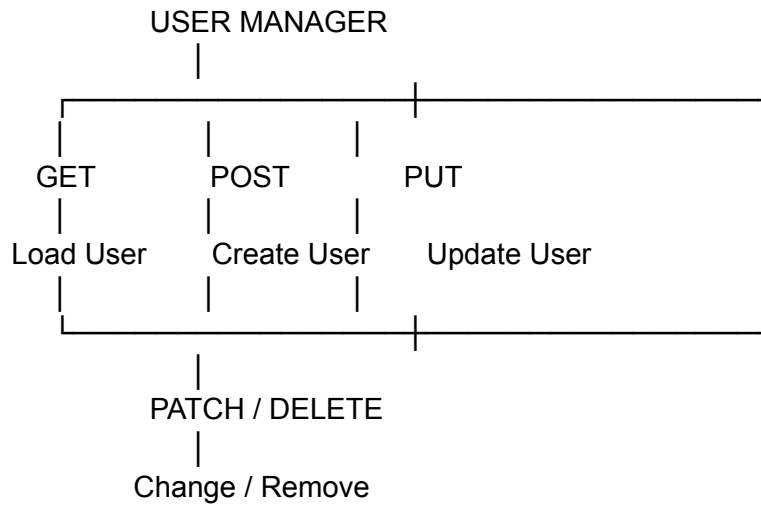
uses:

DELETE

Because you're saying:

"Server, remove this user."

🎯 So the whole thing looks like this



All of them communicate with the same API.

👤 ONE HTML FILE

You can have:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>User Manager</title>
```

```
</head>
```

```
<body>
```

```
  <h1>User Manager</h1>
```

```
  <!-- GET -->
```

```
  <h2>Get User</h2>
```

```
  <button id="getBtn">Load User</button>
```

```
  <p id="userInfo"></p>
```


<!-- POST -->

<h2>Create User</h2>

<input id="createName" placeholder="Name">

<input id="createEmail" placeholder="Email">

<button id="createBtn">Create User</button>

<p id="createResult"></p>

<!-- PUT -->

<h2>Update User</h2>

<input id="updateName" placeholder="New name">

<input id="updateEmail" placeholder="New email">

<button id="updateBtn">Update User</button>

<p id="updateResult"></p>

<!-- PATCH -->

<h2>Change Email</h2>

<input id="patchEmail" placeholder="New email">

<button id="patchBtn">Change Email</button>

<p id="patchResult"></p>

<!-- DELETE -->

<h2>Delete User</h2>

<button id="deleteBtn">Delete User</button>

<p id="deleteResult"></p>

<script src="script.js"></script>

```
</body>
```

```
</html>
```

See?

One HTML file.

All five operations are on the same page.

 ONE JavaScript FILE

Then your JavaScript can contain:

```
// GET
getBtn.addEventListener("click", async function() {

    const response = await fetch(
        "https://jsonplaceholder.typicode.com/users/1"
    );

    const data = await response.json();

    userInfo.textContent = data.name;

});
```

Then below it:

```
// POST
createBtn.addEventListener("click", async function() {

    // POST code here

});
```

Then:

```
// PUT
updateBtn.addEventListener("click", async function() {

    // PUT code here

});
```

Then:

```
// PATCH
patchBtn.addEventListener("click", async function() {

    // PATCH code here

});
```

And finally:

```
// DELETE
deleteBtn.addEventListener("click", async function() {

    // DELETE code here

});
```

So your JavaScript file contains five different functions/event listeners, but they're all part of the same project.

 Think about Facebook

Imagine Facebook.

It's one website.

But you can:

GET → view your profile

POST → create a post

PUT/PATCH → edit your profile/post

DELETE → delete your post

Facebook doesn't create five different websites for these things.

They're all features inside the same application.

That's exactly what we're practicing.

VERY IMPORTANT

You might be thinking:

"Teacher, if I put all five methods in one project, won't they interfere with each other?"

No.

Each button has its own event listener.

For example:

```
getBtn.addEventListener(...)
```

only runs when you click Load User.

```
createBtn.addEventListener(...)
```

only runs when you click Create User.

```
deleteBtn.addEventListener(...)
```

only runs when you click Delete User.

They can all live peacefully inside the same JavaScript file. 😊

 Think of your project like a house

Your HTML file is the house.

Inside the house you have:

GET room

POST room

PUT room

PATCH room

DELETE room

Your JavaScript file controls all the rooms.

You don't need five houses. 😊

You need:

ONE HOUSE → FIVE FEATURES

That's exactly what your assignment is.

✅ So here's our new plan

You already successfully did:

GET + POST practice ✅

You've now learned:

PUT ✅

PATCH ✅

DELETE ✅

Now you can combine all five into ONE project.

Your Day 40 project:

One HTML file

One JavaScript file

Five HTTP methods

GET → Load User

POST → Create User

PUT → Update User

PATCH → Change Email

DELETE → Delete User

And yes:

You do NOT need five separate projects.

The small examples I gave you were just training exercises so you could understand each method before combining them.

Now you're ready to put them together. 🔥💻

Go ahead and try building the one complete project. And remember: you can look at my examples while coding. You're learning, not sitting for a memory exam. 😎

ANSWER ON ASSIGNMENT PROJECT

//GET USER

```
const userInfo = document.querySelector("#userInfo");
```

```
const getButton = document.querySelector("#getBtn");
```

```
getButton.addEventListener("click", async function () {
```

```
    const response = await fetch(
    "https://jsonplaceholder.typicode.com/users/1"
    );
```

```
    const data = await response.json();
```

```
    userInfo.textContent = "Welcome " + data.name;
```

```
});
```

//POST USER

```
const nameInput = document.querySelector("#createName");
```

```
const emailInput = document.querySelector("#createEmail");
```

```
const createUser = document.querySelector("#createUser");
```

```
const createButton = document.querySelector("#createProfile");
```

```
createButton.addEventListener("click", async function () {
```

```
    const user = {
        createName:nameInput.value,
        createEmail:emailInput.value
    };

```

```
    const response = await fetch("https://jsonplaceholder.typicode.com/users",
```

```
    {
        method:"POST",

```

```
        headers:{
            "content-type":
            "application/json"

```



```
});
```

```
//DELETE USER
```

```
const deleteUser = document.querySelector("#deleteUser");
```

```
const deleteProfileButton = document.querySelector("#deleteProfile");
```

```
deleteProfileButton.addEventListener("click",async function () {
```

```
    const response = await fetch("https://jsonplaceholder.typicode.com/users/1",
```

```
    {
        method:"DELETE",
```

```
    }
```

```
    );
```

```
    if(response.ok){
```

```
        result.textContent = "Dekete Successfully "
```

```
    }
```

```
});
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Profile Management</title>
```

```
</head>
```

```
<body>
```

```
<!-- GET User -->
```

```
    <h1>Get User</h1>
```

```
    <p id="userInfo"></p>
```

```
    <button id="getBtn">Get Button</button>
```

```
<!-- Post User -->
```

```
<h1>Creat User</h1>
```

```
<input id="createName" placeholder="Create New Name">
```

```
<input id="createEmail" placeholder="Creat New Email">
```

```
<p id="createUser"></p>
<button id="createProfile">Creat Profile</button>
```

```
<!-- PUT User -->
```

```
<h1>Change Profile</h1>
```

```
<input id="changeName" placeholder="Change Name">
```

```
<input id="changeEmail" placeholder="Change Email">
```

```
<p id="newProfile"></p>
<button id="newUser">Change Profile</button>
```

```
<!-- PATCH -->
```

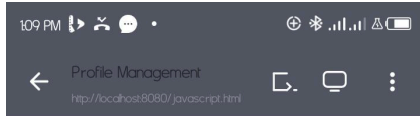
```
<h1>Change Only Email</h1>
<input id="changeNewEmail" placeholder="Input New Email">
<p id="newEmail"></p>
<button id="newUserEmail">New User Email</button>
```

```
<!-- Delete User -->
```

```
<h1>Delete Profile</h1>
<p id="deleteUser"></p>
<button id="deleteProfile">Delete User Profile</button>
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```



Get User

Get Button

Creat User

Create New Name
Creat New Email

Creat Profile

Change Profile

Change Name
Change Email

Change Profile

Change Only Email

Input New Email

New User Email

Delete Profile

Delete User Profile

JAVASCRIPT — DAY 41

JavaScript Modules: export & import

We are now moving away from APIs and into another important part of modern JavaScript.

Don't worry if "Modules" sounds complicated. The basic idea is actually very simple.

1. First: What is a JavaScript Module?

Imagine your JavaScript project becomes very big.

At first, you might have one file:

script.js

And inside it you have:

```
// 500 lines  
// 800 lines  
// 1,000 lines
```

Eventually, that file becomes messy. 🤪

Instead, we can split our code into different files.

For example:

```
project/  
|  
├── index.html  
├── main.js  
├── user.js  
├── calculator.js  
└── products.js
```

Each JavaScript file can have its own job.

That's what modules help us do.

Simple definition:

A JavaScript module is a JavaScript file that contains code we can share with other JavaScript files.

🌍 WHERE ARE MODULES USED IN REAL WEBSITES?

This is important because you asked me to always tell you where topics are used.

Imagine you're building an online shopping website.

You might have:

products.js

for products.

cart.js

for the shopping cart.

login.js

for login.

payment.js

for payment.

main.js

for putting everything together.

Instead of having one gigantic JavaScript file, you separate the code.

This makes professional projects:

easier to understand

easier to maintain

easier to debug

easier for multiple developers to work on

2. Before Modules — Remember Functions

You already know functions.

For example:

```
function greet() {  
  console.log("Hello Ibrahim");  
}
```

But imagine this function is inside:

greeting.js

and you want to use it inside:

main.js

How can main.js access it?

That's where:

export

and:

import

come in.

3. export — "Make this available"

Think of:

export

as saying:

"I'm allowing another JavaScript file to use this."

For example:

```
greeting.js
export function greet() {
  console.log("Hello Ibrahim");
}
```

We put:

export

before the function.

We're basically saying:

"Other JavaScript files are allowed to use greet."

4. import — "Bring it here"

Now imagine we have another file:

main.js

We want to use greet().

We write:

```
import { greet } from './greeting.js';
```

Man-man English:

"Bring the greet function from greeting.js into this file."

Then we can use it:

```
greet();
```

So:

greeting.js

|

export

↓

greet()

|

import

↓

main.js

5. The two words you MUST understand
export

Send/share something out of a file.

import

Bring something into another file.

That's the main idea of today's topic.

6. Let's make a real example

We'll create three files:

project/

|

├── index.html

├── main.js

└── math.js

math.js

```
export function add(a, b) {  
  return a + b;  
}
```

We're creating an add() function.

Normally, that function belongs to math.js.

But because we wrote:

export

another file can use it.

7. Now main.js

```
import { add } from "./math.js";
```

```
const result = add(10, 5);
```

```
console.log(result);
```

What's happening?

This:

```
import { add } from "./math.js";
```

means:

"Go to math.js and bring me the add function."

Then:

```
const result = add(10, 5);
```

uses the function.

The result is:

15

8. Let's connect it to HTML

Here's our:

index.html


```
<!DOCTYPE html>
<html>

<head>
  <title>JavaScript Modules</title>
</head>

<body>

  <h1>JavaScript Modules</h1>

  <p id="result"></p>

  <script type="module" src="main.js"></script>

</body>

</html>
```

 IMPORTANT!

Look carefully at:

```
<script type="module" src="main.js"></script>
```

We wrote:

```
type="module"
```

Why?

Because we're telling the browser:

"This JavaScript file is using modules."

Without it, import and export won't work normally in a browser script.

9. Complete example

math.js

```
export function add(a, b) {
  return a + b;
}
```

main.js

```
import { add } from "./math.js";
```

```
const result = add(10, 5);

document.querySelector("#result").textContent = result;
```

index.html

```
<!DOCTYPE html>
<html>

<head>
  <title>Modules</title>
</head>

<body>

  <h1>Result:</h1>

  <p id="result"></p>

  <script type="module" src="main.js"></script>

</body>

</html>
```

The webpage will show:

Result:

15

🔥 You just used JavaScript modules.

10. Why do we need ./?

Look at:

```
import { add } from "./math.js";
```

The:

./

means:

"Look in the current folder."

So if your files are:

```
project/  
|  
├── main.js  
└── math.js
```

then:

"./math.js"

means:

"The math.js file is in the same folder as main.js."

11. What if the file is inside a folder?

Imagine:

```
project/  
|  
├── main.js  
└── js/  
    └── math.js
```

Then from main.js:

```
import { add } from "../js/math.js";
```

You're saying:

"Go into the js folder and get math.js."

12. You can export more than one thing

Look at:

```
math.js  
export function add(a, b) {  
  return a + b;  
}
```

```
export function subtract(a, b) {  
  return a - b;  
}
```

```
export function multiply(a, b) {  
  return a * b;  
}
```

We have three exported functions.

Then:

```
main.js  
import {  
  add,  
  subtract,  
  multiply  
} from "./math.js";
```

Now we can use all three:

```
console.log(add(10, 5));
```

```
console.log(subtract(10, 5));
```

```
console.log(multiply(10, 5));
```

Result:

15

5

50

13. Why is this useful?

Imagine a real e-commerce website.

Instead of:

main.js

containing everything:

Login code

Cart code
Product code
Payment code
Search code
User code
API code
Animation code

😅 That could become a monster file.

Instead:

```
project/  
├── index.html  
├── main.js  
├── products.js  
├── cart.js  
├── users.js  
├── login.js  
└── payment.js
```

Each file has a specific responsibility.

🧠 Think of it like school

Imagine your school has different teachers.

One teacher handles:

Mathematics

Another:

Computer Science

Another:

English

You don't put all their lessons into one giant notebook.

You separate them.

Same idea with JavaScript modules.

products.js → Product code

cart.js → Cart code

users.js → User code

main.js → Brings things together

14. Another real example

Imagine we're building a Todo website.

We could have:

todos.js

```
export function addTodo(todo) {  
  console.log("Added:", todo);  
}
```

deleteTodo.js

```
export function deleteTodo(id) {  
  console.log("Deleted:", id);  
}
```

main.js

```
import { addTodo } from "./todos.js";
```

```
import { deleteTodo } from "./deleteTodo.js";
```

```
addTodo("Study JavaScript");
```

```
deleteTodo(5);
```

Now our main file can use functionality from different modules.

15. IMPORTANT: export doesn't automatically run the code

Suppose:

```
export function greet() {  
  console.log("Hello");  
}
```

The function doesn't run just because you exported it.

You still need:

```
greet();
```

after importing it.

So:

```
export  
=  
make available
```

```
import  
=  
bring it here
```

```
function()  
=  
actually use/run it
```

16. Let's talk about something you'll see often

You may see:

```
export default
```

Don't worry about that yet.

There are actually two common types:

```
export
```

and:

```
export default
```

We're going to learn named exports first, because I don't want to mix everything together.

Today:

```
export function add() {}
```

and:

```
import { add } from "../math.js";
```

Get this comfortable first.

We'll learn export default after you've understood the basic system.

PRACTICE — BEFORE THE ASSIGNMENT

Let's make a very small project.

Create:

```
project/
|
├── index.html
├── main.js
├── greeting.js
greeting.js
```

Create and export:

```
greet()
```

The function should return:

```
"Welcome to my website!"
main.js
```

Import greet() and display the message inside:

```
<p id="message"></p>
Your HTML must use:
<script type="module" src="main.js"></script>
```

When you open the page, it should show:

Welcome to my website!

YOUR CHALLENGE

Don't copy my complete solution immediately.

Try to figure out:

`greeting.js`

How do you:

1. Create `greet()`
 2. Export `greet()`
- `main.js`

How do you:

1. Import `greet()`
2. Call `greet()`
3. Put the result into the webpage

If you get stuck, look at the examples above. That's completely okay.



DAY 41 SUMMARY

Today we learned:

Module

A JavaScript file that can organize and share code with other JavaScript files.

`export`

Makes code available to another module.

```
export function greet() {}
```

`import`

Brings exported code into another module.

```
import { greet } from './greeting.js';
```

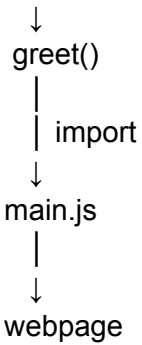
HTML

When using modules:

```
<script type="module" src="main.js"></script>
```

The big picture:

```
greeting.js
|
| export
```



And remember:

EXPORT = "I'm sharing this."

IMPORT = "Bring that here."

🚦 Where we are now

Day 40 — APIs & HTTP Methods → ✅ COMPLETED

Day 41 — JavaScript Modules → 🟡 IN PROGRESS

Your job now is only the small practice above. Don't start the assignment yet.

Once you submit your practice, I'll check it, explain any mistakes, and then we'll do the Day 41 assignment project. 🔥💻

course 😎. Just a quick reminder—you already learned functions before, so this is just connecting them to modules.

1. Creating a function

```
function greet() {  
  return "Welcome to my website!";  
}
```

Think:

function = a little machine that does a job.

Here, the job is returning a greeting.

2. Exporting the function

Because we want another JavaScript file to use it:

```
export function greet() {  
  return "Welcome to my website!";  
}
```

That's all you need to add:

export

3. Importing it in another file

In main.js:

```
import { greet } from "../greeting.js";
```

Now main.js has access to greet().

You can then run it:

```
const message = greet();
```

```
console.log(message);
```

You'll get:

Welcome to my website!

 For your Mini Shop

Same idea.

products.js

```
export function showProduct() {  
  return "Nike Shoes - ₦25,000";  
}
```

cart.js

```
export function addToCart() {  
  return "Nike Shoes added to cart!";  
}
```

main.js

```
import { showProduct } from "../products.js";  
import { addToCart } from "../cart.js";
```

Then you can use:

```
showProduct();
```

and:

```
addToCart();
```

That's the basic pattern:

CREATE → EXPORT → IMPORT → USE 🔥

Now try writing the functions yourself in your project. Don't worry if you need to look back at this example—that's completely normal. 😎

Good morning! 😎☀️ Teacher Coder is here.

And yes — you're right to be confused about adding many products. I didn't properly teach you that part yet. We shouldn't pretend you already know it.

Your question is actually leading us into an important JavaScript concept: arrays of objects.

We can use that to make our Mini Shop have many products without writing a separate section of HTML for every product.

Let's learn it step by step.

 Adding MANY Products

Imagine you have only one product:

```
const product = {  
  name: "Nike Shoes",  
  price: 25000  
};
```

That's fine for one product.

But what if we have:

Nike Shoes
Football
Adidas Shirt
Headphones

We don't want to create four completely separate variables.

Instead, we can put them inside an array.

1. An array of products

```
const products = [  
  {  
    name: "Nike Shoes",  
    price: 25000  
  },  
  
  {  
    name: "Football",  
    price: 15000  
  },  
  
  {  
    name: "Adidas Shirt",  
    price: 20000  
  },  
  
  {  
    name: "Headphones",  
    price: 30000  
  }  
];
```

Look carefully.

The:

```
[ ]
```

is the array.

And each:

```
{  
  name: "...",  
  price: ...  
}
```

is an object.

So we have:

One array containing many product objects.

🧠 Think about it like a box

Imagine:

PRODUCTS BOX



The box is the array:

`products`

Each product is an object.

2. How do we get a particular product?

Remember arrays have positions.

The first product is:

`products[0]`

The second:

`products[1]`

The third:

`products[2]`

The fourth:

`products[3]`

Remember:

JavaScript starts counting arrays from 0.

So:

```
console.log(products[0].name);
```

gives:

Nike Shoes

And:

```
console.log(products[1].name);
```

gives:

Football

3. But we don't want to write this 20 times 😓

Imagine you have 50 products.

You don't want:

```
console.log(products[0]);  
console.log(products[1]);  
console.log(products[2]);  
console.log(products[3]);
```

Instead, JavaScript can loop through the array.

We use:

forEach()

Example:

```
products.forEach(function(product) {  
    console.log(product.name);  
});
```

This means:

"Go through every product in the array and give me each product one at a time."

The result:

Nike Shoes
Football
Adidas Shirt
Headphones

🔥 This is how websites can display many products.

4. Now let's put them on the webpage

HTML:

```
<div id="products"></div>
```

That's all we need.

We don't need to manually write:

```
<div>Nike Shoes</div>  
<div>Football</div>  
<div>Adidas Shirt</div>
```

JavaScript can create them for us.

5. JavaScript

```
const productsContainer = document.querySelector("#products");
```

```
const products = [  
  {  
    name: "Nike Shoes",  
    price: 25000  
  },  
  
  {  
    name: "Football",  
    price: 15000  
  },  
  
  {  
    name: "Adidas Shirt",  
    price: 20000  
  }  
];
```



```
products.forEach(function(product) {  
  
    productsContainer.innerHTML += `  
        <div>  
            <h2>${product.name}</h2>  
            <p>₦${product.price}</p>  
        </div>  
    `;  
  
});
```

Now your webpage can show:

Nike Shoes
₦25000

Football
₦15000

Adidas Shirt
₦20000



6. What is `${product.name}`?

Don't let this scare you.

This:

`${product.name}`

means:

"Put the product's name here."

For example, if:

`product.name`

is:

Nike Shoes

then:

```
<h2>${product.name}</h2>
```

becomes:

```
<h2>Nike Shoes</h2>
```

And:

```
${product.price}
```

becomes:

```
25000
```

This is called a template literal.

You'll use it a LOT when creating HTML with JavaScript.

7. Now let's connect this to your MODULE project

This is where it gets interesting. 😎

Remember our files:

```
index.html  
main.js  
products.js  
cart.js
```

Instead of products.js having only one product, we can put many products there.

```
products.js  
export const products = [  
  {  
    name: "Nike Shoes",  
    price: 25000  
  },  
  
  {  
    name: "Football",  
    price: 15000  
  },  
]
```

```
{
  name: "Adidas Shirt",
  price: 20000
},
{
  name: "Headphones",
  price: 30000
}
];
```

Now we're exporting the entire array.

8. Import the products

In main.js:

```
import { products } from "./products.js";
```

Now main.js has access to all the products.

Then:

```
const productsContainer =
  document.querySelector("#products");
```

Then:

```
products.forEach(function(product) {

  productsContainer.innerHTML += `
    <div>
      <h2>${product.name}</h2>
      <p>R$${product.price}</p>
    </div>
  `;

});
```

That's it.

9. HTML

Your HTML can simply be:

```
<!DOCTYPE html>
<html>

<head>
  <title>My Mini Shop</title>
</head>

<body>

  <h1>🛒 My Mini Shop</h1>

  <div id="products"></div>

  <script type="module" src="main.js"></script>

</body>

</html>
```

JavaScript creates all the product information inside:

```
<div id="products"></div>
🌐 WHERE IS THIS USED IN REAL WEBSITES?
```

This is VERY important.

When you open Jumia, Amazon, a food delivery website, etc., they might have hundreds or thousands of products.

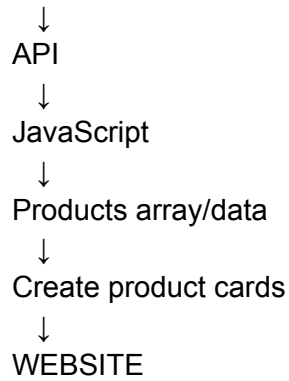
They don't manually write:

```
<div>Product 1</div>
<div>Product 2</div>
<div>Product 3</div>
...
```

Instead, product information comes from data, often from a backend/API/database, and JavaScript/framework code uses that data to generate the product cards.

For example:

DATABASE



So you're learning the small version of something real websites do. 🔥

🧠 One important thing

You asked:

"I seem to forget it. You have not taught me before."

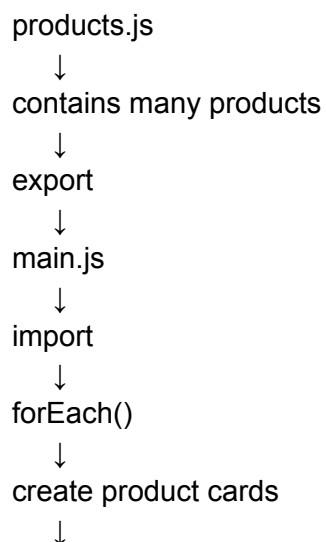
You're correct.

We've used arrays before, but I haven't properly taught you "how to take an array of product objects and generate many product cards on a webpage."

So don't feel bad for forgetting something you weren't properly taught.

We'll learn it now.

And this connects beautifully with your Modules lesson:



HTML

🎯 BEFORE WE MOVE ON

I don't want to throw a huge new assignment at you immediately.

Try this little exercise:

Make 4 products:

Nike Shoes

Football

Adidas Shirt

Headphones

Put them inside:

```
const products = [  
  ...  
];
```

Then use:

forEach()

to display all four on the page.

Once you can do that, we'll upgrade your Mini Shop so every product has its own Add to Cart button. 🛒🔥

And THAT will make your Day 41 project much more realistic.

Absolutely 🤪🔥. Since you

understand how to make the products array, let's add the important part: each product gets its own Add to Cart button.

We'll keep it simple and build it step by step.



1. Our products

In

products.js:

```
export const products = [  
  {  
    id: 1,  
    name: "Nike Shoes",  
    price: 25000  
  },  
  
  {  
    id: 2,  
    name: "Football",  
    price: 15000  
  },  
  
  {  
    id: 3,  
    name: "Adidas Shirt",  
    price: 20000  
  },  
  
  {  
    id: 4,  
    name: "Headphones",  
    price: 30000  
  }  
];
```

Notice I added:

id: 1

Each product has its own ID.

That's useful because when someone clicks Add to Cart, we need to know which product they clicked.

● 2. HTML

We only need a container:

```
<!DOCTYPE html>  
<html>
```

```

<head>
  <title>My Mini Shop</title>
</head>

<body>

  <h1>🛒 My Mini Shop</h1>

  <div id="products"></div>

  <p id="cartMessage"></p>

  <script type="module" src="main.js"></script>

</body>

</html>

```

Notice we're NOT manually creating four buttons.

JavaScript will create them for us.

3. JavaScript

In

main.js:

```

import { products } from "../products.js";

const productsContainer =
  document.querySelector("#products");

const cartMessage =
  document.querySelector("#cartMessage");

products.forEach(function(product) {

  productsContainer.innerHTML += `
    <div>
      <h2>${product.name}</h2>

      <p>💰${product.price}</p>

```



```
        <button onclick="addToCart(${product.id})">
            Add to Cart
        </button>
    </div>
    `;
});
```

Now stop here. 🛑

Let's understand the important part.

🔥 4. How does every product get its own button?

This:

```
<button onclick="addToCart(${product.id})">
    Add to Cart
</button>
```

is inside:

```
products.forEach(function(product) {
```

That means JavaScript repeats the HTML for every product.

For Nike:

```
<button onclick="addToCart(1)">
    Add to Cart
</button>
```

For Football:

```
<button onclick="addToCart(2)">
    Add to Cart
</button>
```

For Adidas:

```
<button onclick="addToCart(3)">
    Add to Cart
</button>
```

For Headphones:

```
<button onclick="addToCart(4)">
  Add to Cart
</button>
```

🔥 So every product automatically gets its own button.

🧠 5. But what is `addToCart()`?

That's our function.

We can put it in `cart.js`.

```
cart.js
export function addToCart(productId) {

  console.log("Product ID:", productId);

}
```

Now the button:

```
onclick="addToCart(1)"
```

calls:

```
addToCart(1)
```

The button for Football calls:

```
addToCart(2)
```

And so on.

⚠️ One problem

There's an important module issue here.

Because `main.js` is a module, functions imported/exported there aren't automatically available to inline HTML `onclick`.

So don't use the `onclick` approach in your final project.

Instead, we're going to use the method you already know:

`addEventListener()`

This is the better approach. 📌

★ 6. The better professional approach

In

main.js:

```
import { products } from "../products.js";
import { addToCart } from "../cart.js";
```

```
const productsContainer =
  document.querySelector("#products");
```

```
const cartMessage =
  document.querySelector("#cartMessage");
```

```
products.forEach(function(product) {

  const productDiv = document.createElement("div");

  productDiv.innerHTML = `
    <h2>${product.name}</h2>

    <p>N$${product.price}</p>

    <button class="cartBtn">
      Add to Cart
    </button>
  `;

  productsContainer.appendChild(productDiv);

  const button =
    productDiv.querySelector(".cartBtn");

  button.addEventListener("click", function() {
```

```
        const message = addToCart(product);

        cartMessage.textContent = message;

    });

});
```

Now we're doing something very useful.

When JavaScript creates each product, it also creates a button for that product.

Then we attach an event listener to that specific button.

7. [cart.js](#)

Now:

```
export function addToCart(product) {

    return product.name + " added to cart!";

}
```

So if you click Nike:

Nike Shoes added to cart!

If you click Football:

Football added to cart!

If you click Headphones:

Headphones added to cart!



🧠 Let's understand the magic

Suppose we're currently processing:

```
{
```

```
    id: 2,  
    name: "Football",  
    price: 15000  
  }  
}
```

This:

```
button.addEventListener("click", function() {  
  
    const message = addToCart(product);  
  
});
```

knows that product is the Football object.

So when you click that particular button:

```
Football  
₦15000  
[Add to Cart]
```

JavaScript sends the Football object to:

```
addToCart(product)
```

Then cart.js returns:

Football added to cart!



The whole system

products.js



4 products



export



main.js



import



forEach()



Create product



Create button

↓
addEventListener()
↓
User clicks button
↓
addToCart(product)
↓
cart.js
↓
"Football added to cart!"

That's a proper little shopping application. 🤖🔥

🌐 WHERE IS THIS USED IN REAL WEBSITES?

This is the same basic idea behind product listings on sites like online stores.

Imagine:

Nike Shoes
₦25,000
[Add to Cart]

Football
₦15,000
[Add to Cart]

Adidas Shirt
₦20,000
[Add to Cart]

The products might actually come from a backend/API/database, then JavaScript displays them.

Each button needs to know:

"Which product did the customer click?"

That's why we give products an id and associate each button with its product.

Later, when we build more advanced shopping-cart functionality, you'll learn how to actually store the selected products in an array and calculate:

Cart: 2 items

Nike Shoes ₦25,000

Football ₦15,000

Total ₦40,000

But don't add that complexity yet.

🎯 YOUR DAY 41 PROJECT

Now you have enough knowledge to build your My Mini Shop.

Your project should have:

index.html

The webpage and product container.

products.js

At least 4 products.

cart.js

An exported addToCart() function.

main.js

Import the products and cart function, display all products, and give each product its own Add to Cart button.

When you click a button:

Nike Shoes added to cart!

or:

Football added to cart!

etc.

★ Don't worry about making a real cart yet.

For Day 41, we're mainly practicing:

Multiple files → export → import → arrays → objects → forEach → DOM → event listeners.

That's already a lot of good JavaScript practice. 🔥

Go build it yourself now. Use these examples as your reference—don't feel like you have to memorize them. When you're finished, send me the project and I'll mark it. 😎👨🏻

DAY 41 ASSIGNMENT PROJECT

📦 "My Mini Shop" — JavaScript Modules

You're going to build a small shopping page using multiple JavaScript files.

🎯 What you're practicing

You will use:

export

import

type="module"

Functions

DOM manipulation

Multiple JavaScript files

💻 What the website should look like

Something similar to:

🛒 MY MINI SHOP

Welcome to my shop!

Product: Nike Shoes

Price: ₦25,000

[Add to Cart]

Product: Football

Price: ₦15,000

[Add to Cart]

Cart:

Nike Shoes added to cart!

Don't worry about making it beautiful yet. The main goal is the JavaScript modules.

 Your files

Create these:

My Mini Shop/

```
|
|— index.html
|
|— main.js
|
|— products.js
|
|— cart.js
```

You are intentionally using three JavaScript files.

● 1. products.js

This file is responsible for your products.

Create a function called:

`showProduct()`

It should return information about a product.

For example, you could have:

Nike Shoes

¥25,000

Your job:

Export the function.

Remember:

`export`

● 2. cart.js

This file is responsible for the shopping cart.

Create a function called:

`addToCart()`

It should receive a product name and return/display something like:

Nike Shoes added to cart!

Again, export the function.

3. main.js

This is the main file.

It should:

Import `showProduct()` from `products.js`

Import `addToCart()` from `cart.js`

Display the product on the webpage

Have an Add to Cart button

When the button is clicked, use `addToCart()`

Display the cart message on the webpage

4. index.html

Your HTML should contain something like:

My Mini Shop

Welcome to my shop!

[Product information]

[Add to Cart]

[Cart message]

Give your elements IDs so JavaScript can select them.

For example:

```
<p id="product"></p>
```

```
<button id="cartBtn">Add to Cart</button>
```

```
<p id="cartMessage"></p>
```

And VERY IMPORTANT:

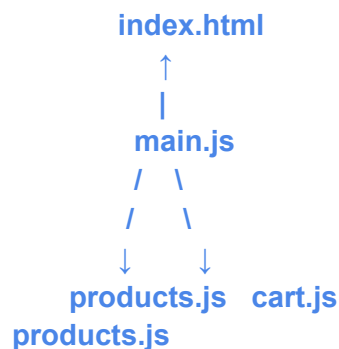
Your JavaScript must be loaded as a module:

```
<script type="module" src="main.js"></script>
```

🔥 Don't forget type="module".

🧠 THE STRUCTURE YOU'RE BUILDING

Think about it like this:



"I handle products."

cart.js

"I handle the cart."

main.js

"I'll bring them together."

That's exactly why modules exist.

🌐 WHERE THIS IS USED IN REAL WEBSITES

Imagine a real Amazon/Jumia-style website.

You wouldn't want one giant JavaScript file containing:

Products
Cart
Login
Payments
Users
Search
Orders
Reviews

Instead, developers separate them:

products.js
cart.js
login.js
payment.js
users.js
orders.js
reviews.js

Then the main parts of the application import the functionality they need.

You're practicing the same idea on a much smaller scale. 🔥

💣 YOUR RULE FOR THIS PROJECT

Don't try to memorize everything.

You can look back at today's lesson and use the examples.

But try to write each file yourself first.

If you get stuck:

Look at the example → understand it → try again.

That's how I want you learning.

🎯 Your task

Complete both:

Practice

greeting.js → export → main.js → import → display greeting

Assignment

My Mini Shop → products.js + cart.js + main.js + index.html

When you're finished, send me the whole project/screenshot, and I'll mark it before we move fo

NOW LET'S BREAK DOWN YOUR PROJECT

Your project has three JavaScript files:

**product.js
cart.js
main.js**

And one HTML file.

The whole idea is:

**product.js
↓
contains products
↓
main.js
↓
displays products
↓
cart.js
↓
handles Add to Cart**

Let's start from the beginning.

● PART 1 — product.js

You wrote:

export const products = [

Let's break this into pieces.

```
const
const products
```

You're creating a variable called:

products

It stores your products.

```
=
const products =
```

means:

"Put this information inside products."

```
[ ]
[
  ...
]
```

These square brackets mean you are creating an array.

Remember:

Array = a container that can hold multiple values.

```
{ }
```

Inside your array you have:

```
{
  id: 1,
  name: "Nike Shoes",
  price: 25000
}
```

That's an object.

An object lets us describe one product.

So:

```
{
```

```
  id: 1,  
  name: "Nike Shoes",  
  price: 25000  
}
```

means:

Product

```
|  
├── id → 1  
├── name → Nike Shoes  
└── price → 25000
```

Then you made another object:

```
{  
  id: 2,  
  name: "Football",  
  price: 15000  
}
```

Then:

```
{  
  id: 3,  
  name: "Adidas Shirt",  
  price: 20000  
}
```

And:

```
{  
  id: 4,  
  name: "Headphones",  
  price: 30000  
}
```

So your entire array is basically:

products

```
|  
├── Product 1  
├── Product 2  
└── Product 3
```

└─ Product 4

● Now the important word: export

You wrote:

```
export const products = [
```

Remember our lesson:

export = make this available to another JavaScript file.

Without export:

```
const products = [...]
```

main.js wouldn't be able to import it this way.

Because you wrote:

```
export
```

you can now do this in main.js:

```
import { products } from "../product.js";
```

● PART 2 — cart.js

Your code is:

```
export function addToCart(product) {  
    return product.name + " added to cart!";  
}
```

Let's slow down.

```
export  
export
```

Again:

"Another JavaScript file can use this."

```
function
```


function addToCart()

You're creating a function called:

addToCart

Its job is to handle what happens when someone adds a product.

(product)

This is important.

function addToCart(product)

product is a parameter.

Think of it as an empty box waiting to receive a product.

For example, later we might give it:

addToCart(NikeShoes)

Then inside the function:

product = Nike Shoes

🔥 Then this line
return product.name + " added to cart!";

Suppose:

product.name

is:

Nike Shoes

JavaScript combines:

Nike Shoes

+

" added to cart!"

and returns:

Nike Shoes added to cart!

That's what your function does.

PART 3 — main.js

This is where everything comes together.

You wrote:

```
import { products } from "../product.js";
```

Meaning:

"Go to product.js and bring the products array here."

Then:

```
import { addToCart } from "../cart.js";
```

means:

"Go to cart.js and bring the addToCart function here."

So main.js now has access to:

```
products  
+  
addToCart()
```

 **That's the power of modules.**

PART 4 — Finding the HTML elements

You wrote:

```
const productsContainer =  
  document.querySelector("#products");
```

Remember querySelector()?

You're telling JavaScript:

"Find the HTML element whose ID is products."

Your HTML has:

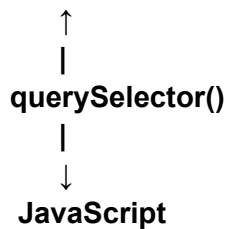
```
<div id="products"></div>
```

So JavaScript finds that empty <div>.

Think:

HTML

```
<div id="products"></div>
```



Then you wrote:

```
const cartMessage =  
  document.querySelector("#cartMessage");
```

That finds:

```
<p id="cartMessage"></p>
```

This is where we'll display:

Nike Shoes added to cart!

🔥 PART 5 — THE MOST IMPORTANT PART

Then you wrote:

```
products.forEach(function(product) {
```

This might be one of the parts making your brain say:

"Teacher, what is happening here?" 😄

Let's simplify it.

You have:

```
products
|
|— Nike Shoes
|— Football
|— Adidas Shirt
|— Headphones
```

forEach() means:

"Go through every product, one by one."

So JavaScript does:

First → Nike Shoes

Second → Football

Third → Adidas Shirt

Fourth → Headphones

And this:

```
function(product)
```

means:

"For each product, temporarily call that product product."

So when it's working on Nike:

```
product = Nike Shoes
```

When it's working on Football:

```
product = Football
```

 **PART 6 — Creating a <div>**

You wrote:

```
const productDiv =
  document.createElement("div");
```

This is VERY cool.

JavaScript is creating an HTML element for you.

It's basically doing:

```
<div></div>
```

but from JavaScript.

So:

```
document.createElement("div")
```

means:

"JavaScript, create a new <div> for me."

🟡 PART 7 — innerHTML

Then:

```
productDiv.innerHTML = `
```

You're saying:

"Put this HTML inside the div."

Then you have:

```
<h2>${product.name}</h2>
```

Remember \${}?

It means:

"Put a JavaScript value here."

If we're currently working on:

Nike Shoes

then:

`${product.name}`

becomes:

`<h2>Nike Shoes</h2>`

Then:

`<p>N${product.price}</p>`

becomes:

`<p>N25000</p>`

And:

`<button class="cartBtn">Add to Cart</button>`

creates the button.

🧠 So one product becomes:

JavaScript takes:

```
{
  id: 1,
  name: "Nike Shoes",
  price: 25000
}
```

and creates:

```
<div>
  <h2>Nike Shoes</h2>
  <p>N25000</p>
  <button>Add to Cart</button>
</div>
```

Then `forEach()` does it again for Football.

Then Adidas.

Then Headphones.

That's why you got four products and four buttons without manually writing four product sections in HTML.



🟡 PART 8 — appendChild()

You wrote:

```
productsContainer.appendChild(productDiv);
```

This means:

"Put the newly created product div inside the products container."

Remember this:

```
<div id="products"></div>
```

It starts empty.

JavaScript creates:

```
<div>
  Nike Shoes
  ...
</div>
```

Then:

```
appendChild(productDiv)
```

puts it inside:

```
<div id="products">

  <div>
    Nike Shoes
    ...
  </div>

</div>
```

Then the next product goes inside too.

🔴 PART 9 — Finding the button

You then get the button:

```
const button =  
  productDiv.querySelector(".cartBtn");
```

Notice:

`.cartBtn`

The dot means class.

Your button has:

```
<button class="cartBtn">
```

So you're saying:

"Find the Add to Cart button inside this particular product."

That's important because there are four buttons.

We want the button belonging to the current product.

🔴 PART 10 — addEventListener

Then:

```
button.addEventListener("click", function() {
```

This means:

"When this button is clicked, run this function."

So:

User

↓

clicks Add to Cart

↓

JavaScript detects click

↓

runs function

● PART 11 — This line is the connection

You wrote:

```
const message = addToCart(product);
```

This is beautiful because now you're connecting:

main.js



cart.js

Remember we imported:

```
import { addToCart } from "./cart.js";
```

So we can call:

```
addToCart(product);
```

If the current product is Nike Shoes:

```
addToCart(Nike Shoes)
```

The function in cart.js receives it:

```
function addToCart(product)
```

Then:

```
return product.name + " added to cart!";
```

returns:

Nike Shoes added to cart!

That returned message gets stored here:

```
const message = ...
```

So:

```
message
```

=

" Nike Shoes added to cart!"

● PART 12 — Displaying the message

Finally:

`cartMessage.textContent = message;`

Remember:

`<p id="cartMessage"></p>`

JavaScript finds that paragraph and puts the message inside it.

So:

Click Nike Shoes



`addToCart(product)`



"Nike Shoes added to cart!"



`cartMessage.textContent`

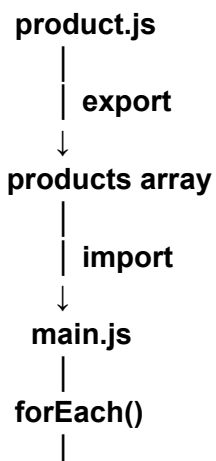


Website displays it



🧠 YOUR WHOLE PROJECT IN ONE PICTURE

This is the part I really want you to remember:





♥ And about your two-day breaks...

Please don't start thinking:

"Maybe coding isn't for me."

No.

You're learning a technical skill while working. Of course some things will become rusty after a couple of days.

Here's what I want you to do whenever you return after a break:

🔄 10-minute warm-up

Before starting a new topic:

2 minutes: Read your previous lesson summary.

3 minutes: Look at your previous project.

5 minutes: Try to explain the code in your own words.

For example:

"export shares code."

"import brings code."

"forEach() goes through every product."

"createElement() creates an HTML element."

"addEventListener() waits for something like a click."

If you can explain those things, you're not lost. You're just warming your brain back up.

And don't be afraid to look at your notes. Developers use references all the time.

You successfully built this project. The fact that you're now asking "why does each part work?" is actually a very good sign. You're moving from merely copying code toward understanding it. 🔥💻

Day 41 is not lost. You're understanding it.

ANSWER

MAIN.JS

```
import { products } from "./product.js";
```

```
import { addToCart } from "./cart.js";
```

```
const productsContainer =  
  document.querySelector("#products");  
const cartMessage =  
  document.querySelector("#cartMessage");
```

```
products.forEach(function(product) {  
  
  const productDiv = document.createElement("div");  
  
  productDiv.innerHTML = `<h2>${product.name}</h2>
```

```
<p>R${product.price}</p>
<button class="cartBtn">Add to Cart</button>`;
```

```
productsContainer.appendChild(productDiv);
```

```
const button =
```

```
productDiv.querySelector(".cartBtn");
```

```
button.addEventListener("click", function() {
```

```
const message = addToCart(product);
```

```
cartMessage.textContent = message;
```

```
});
```

```
});
```

PRODUCT.JS

```
export const products = [
```

```
{
```

```
id: 1,
```

```
name: "Nike Shoes",
```

```
price: 25000
```

```
},
```

```
{
```

```
      id: 2,

      name: "Football",

      price: 15000

    },

    {

      id: 3,

      name: "Adidas Shirt",

      price: 20000

    },

    {

      id: 4,

      name: "Headphones",

      price: 30000

    }

  ];
```

CART.JS

```
export function addToCart(product) {

  return product.name + " added to cart!";

}
```

```
<!DOCTYPE html>
<html>
<head>
```

```
<title>My Mini Shop</title>
</head>
<body>

  <h1>🛒 My Mini Shop</h1>
  <div id="products"></div>
  <p id="cartMessage"></p>
  <script type="module" src="main.js"></script>

</body>
</html>
```

DAY 42 — JAVASCRIPT CLASSES

🧠 **First: What is a Class?**

In simple "man language":

A class is a blueprint for creating objects.

Think about a house.

An architect can create one blueprint:

🏠 HOUSE BLUEPRINT
↓
Used to build many houses
↓
House 1
House 2
House 3

A JavaScript class works in a similar way:

CLASS
↓
Blueprint
↓

Create objects



Object 1

Object 2

Object 3

So a class isn't necessarily the actual object.

It's the **plan used to create objects**.



WHERE ARE CLASSES USED IN REAL WEBSITES?

Classes are useful when an application has many things that have the **same structure or behavior**.

For example, an online shop can have hundreds of products.

Each product may have:

Name

Price

Category

Add to cart

A game can have:

Player

Enemy

Weapon

Character

A banking application can have:

Account

Balance

Deposit

Withdraw

A class lets programmers create a reusable structure instead of repeating the same code over and over.

PART 1 — Creating a Class

Let's create a student class:

```
class Student {  
  
}
```

That's a class.

We're calling it:

Student

Why capital **S**?

By convention, JavaScript developers normally capitalize class names:

```
class Student {}  
class Product {}  
class User {}  
class Car {}
```

It's not strictly what makes the class work, but it's the normal convention.

PART 2 — Let's Connect It to HTML

HTML

```
<!DOCTYPE html>  
<html>
```

```
<head>  
  <title>Student</title>  
</head>
```

```
<body>  
  
  <h1>Student Information</h1>
```

```
<div id="student"></div>

<script src="script.js"></script>

</body>

</html>
```

We have an empty:

```
<div id="student"></div>
```

JavaScript will put the student's information there.

PART 3 — The **constructor()**

Now our class needs some information.

```
class Student {

    constructor(name, age, department) {

    }

}
```

You might be thinking:

"Teacher, what is this constructor thing?" 😂

Don't panic.

Think of the constructor as:

The setup area for a new object.

When we create a student, the constructor receives the student's information.

For example:

```
new Student("Ibrahim", 20, "Computer Science");
```

The constructor receives:

name → Ibrahim

age → 20

department → Computer Science

PART 4 — Understanding **this**

Now we put the information into the object:

```
class Student {  
  
    constructor(name, age, department) {  
  
        this.name = name;  
        this.age = age;  
        this.department = department;  
  
    }  
  
}
```

This part:

```
this.name = name;
```

can look confusing.

Let's break it down.

Right side:

name

This is the value we receive.

Left side:

this.name

This means:

"The name belonging to this particular object."

So:

```
this.name = name;
```

means:

"Store the name we received inside this object."

Same thing:

```
this.age = age;
```

means:

"Store the age inside this object."

And:

```
this.department = department;
```

means:

"Store the department inside this object."



Think about **this** like this

Imagine:

```
const student1 = new Student(  
  "Ibrahim",  
  20,  
  "Computer Science"  
);
```

JavaScript creates an object:

```
student1  
|  
├── name → Ibrahim  
├── age → 20  
└── department → Computer Science
```

Here:

this.name

basically refers to the **name** belonging to **this particular student object**.

PART 5 — Creating an Object with new

Now our class is ready.

We can create a student:

```
const student1 = new Student(  
  "Ibrahim",  
  20,  
  "Computer Science"  
);
```

The important word is:

new

In simple English:

new creates a new object from the class.

So:

`new Student(...)`

means:

"Create a new Student using the Student blueprint."

PART 6 — Displaying It on the HTML Page

Let's put everything together.

script.js

```
class Student {

    constructor(name, age, department) {

        this.name = name;
        this.age = age;
        this.department = department;

    }

}

const student1 = new Student(
    "Ibrahim",
    20,
    "Computer Science"
);

document.querySelector("#student").innerHTML = `
    <h2>${student1.name}</h2>
    <p>Age: ${student1.age}</p>
    <p>Department: ${student1.department}</p>
`;
```

Our HTML:

```
<!DOCTYPE html>
<html>

<head>
    <title>Student</title>
</head>

<body>

    <h1>Student Information</h1>

    <div id="student"></div>

    <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

The webpage will show:

Student Information

Ibrahim

Age: 20

Department: Computer Science

🔥 Now you've connected **Class** → **Object** → **HTML**.

PART 7 — Creating MANY Objects

This is where classes become powerful.

We don't need to create a new class for every student.

We create **one class**:

```
class Student {  
  
    constructor(name, age, department) {  
  
        this.name = name;  
        this.age = age;  
        this.department = department;  
  
    }  
  
}
```

Then create many students:

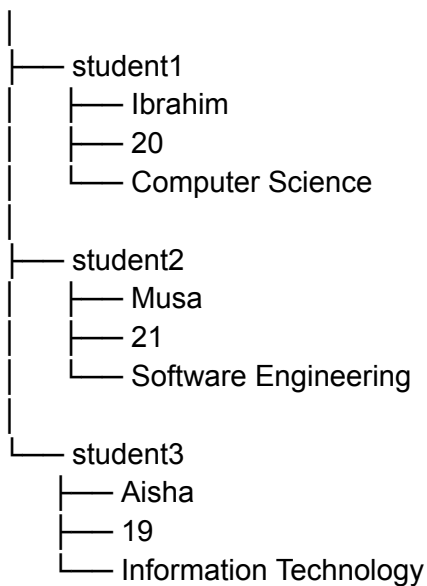
```
const student1 = new Student(  
    "Ibrahim",  
    20,  
    "Computer Science"  
);
```

```
const student2 = new Student(  
  "Musa",  
  21,  
  "Software Engineering"  
);
```

```
const student3 = new Student(  
  "Aisha",  
  19,  
  "Information Technology"  
);
```

Now we have:

Student class



One blueprint → many objects. 🔥

PART 8 — Classes Can Have Methods

Remember functions?

We can put a function inside a class.

For example:

```
class Student {
```



```
    constructor(name, age) {  
  
        this.name = name;  
        this.age = age;  
  
    }  
  
    introduce() {  
  
        return "My name is " + this.name;  
  
    }  
  
}
```

The:

introduce()

is a **method**.

A method is basically a function that belongs to the class/object.



PART 9 — Using the Method

Create a student:

```
const student1 = new Student(  
    "Ibrahim",  
    20  
);
```

Then:

```
console.log(student1.introduce());
```

Result:

My name is Ibrahim

Why?

Because:

this.name

is:

Ibrahim

So:

return "My name is " + this.name;

becomes:

My name is Ibrahim

PART 10 — Method + HTML

Let's make the method appear on the webpage.

HTML

```
<!DOCTYPE html>
<html>

<head>
  <title>Student Profile</title>
</head>

<body>

  <h1>Student Profile</h1>

  <div id="student"></div>

  <button id="greetBtn">
    Say Hello
  </button>

  <p id="message"></p>

  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Now JavaScript:

```
class Student {  
  
    constructor(name, age) {  
  
        this.name = name;  
        this.age = age;  
  
    }  
  
    introduce() {  
  
        return "Hello, my name is " + this.name;  
  
    }  
  
}  
  
const student1 = new Student(  
    "Ibrahim",  
    20  
);  
  
document.querySelector("#student").innerHTML = `  
    <h2>${student1.name}</h2>  
    <p>Age: ${student1.age}</p>  
`;  
  
document.querySelector("#greetBtn")  
    .addEventListener("click", function() {  
  
        document.querySelector("#message")  
            .textContent = student1.introduce();  
  
    });
```

Now the page initially shows:

Student Profile

Ibrahim

Age: 20

[Say Hello]

When you click:

[Say Hello]

it displays:

Hello, my name is Ibrahim



Now you're seeing how a class can interact with a real webpage.



Let's understand the whole process

When the page loads:

1. JavaScript creates the blueprint

```
class Student {}
```

2. We create an object

```
const student1 = new Student("Ibrahim", 20);
```

3. The constructor runs automatically

```
constructor(name, age)
```

4. The information is stored

```
this.name = name;
```

```
this.age = age;
```

So:

student1

```
|— name → Ibrahim  
|— age  → 20
```

5. We display it in HTML

```
student1.name  
student1.age
```

6. User clicks the button

```
addEventListener("click", ...)
```

7. The method runs

```
student1.introduce()
```

8. The message appears

Hello, my name is Ibrahim



PART 11 — A REAL WEBSITE

EXAMPLE: PRODUCT CLASS

Let's use an online shop because this connects directly to your previous project.

HTML

```
<!DOCTYPE html>  
<html>  
  
  <head>  
    <title>My Shop</title>  
  </head>  
  
  <body>  
  
    <h1>My Shop</h1>  
  
    <div id="product"></div>  
  
    <button id="cartBtn">  
      Add to Cart  
    </button>  
  
    <p id="message"></p>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
class Product {
```

```
  constructor(name, price) {
```

```
    this.name = name;
```

```
    this.price = price;
```

```
  }
```

```
  addToCart() {
```

```
    return this.name + " added to cart!";
```

```
  }
```

```
}
```

```
const product1 = new Product(
```

```
  "Nike Shoes",
```

```
  25000
```

```
);
```

```
document.querySelector("#product").innerHTML = `
```

```
  <h2>${product1.name}</h2>
```

```
  <p>#<del>N</del>${product1.price}</p>
```

```
`;
```

```
document.querySelector("#cartBtn")
```

```
  .addEventListener("click", function() {
```

```
    document.querySelector("#message")
```

```
      .textContent = product1.addToCart();
```

```
  });
```

The webpage:

My Shop

Nike Shoes

₦25000

[Add to Cart]

Click the button:

Nike Shoes added to cart!



CLASS vs OBJECT

This is the part I want you to understand very clearly.

CLASS

```
class Product {  
  
    constructor(name, price) {  
        this.name = name;  
        this.price = price;  
    }  
  
}
```

Think:

Blueprint

OBJECT

```
const product1 = new Product(  
    "Nike Shoes",  
    25000  
);
```

Think:

Actual thing created from the blueprint

So:

CLASS
Blueprint
↓
↓ new
↓
OBJECT
Nike Shoes
₦25,000

And you can create another:

```
const product2 = new Product(  
  "Football",  
  15000  
);
```

Same blueprint.

Different object.

THE 5 THINGS YOU NEED TO KNOW TODAY

Don't try to memorize the whole lesson.

Understand these five:

1 **class**

```
class Student {}
```

➡ Creates a blueprint.

2 **constructor**

```
constructor(name, age) {}
```

➡ Sets up a new object.

3 **this**

```
this.name = name;
```

➡ Stores information in the current object.

4 **new**

```
new Student("Ibrahim", 20);
```

➡ Creates a new object from the class.

5 **Method**

```
introduce() {}
```

➡ A function belonging to the class.



DAY 42 PRACTICE

Before the assignment, build this yourself.

HTML

Create:

Student Profile

Name

Age

Department

[Introduce Me]

message

JavaScript

Create a:

class Student

It should have:

name
age
department

Create a method:

introduce()

The method should return something like:

Hello, my name is Ibrahim and I study Computer Science.

Then when the user clicks:

[Introduce Me]

the message should appear on the webpage.



DAY 42 SUMMARY

CLASS



Blueprint

constructor()



Sets up the object

this



Refers to the current object

new



Creates an object from the class

method



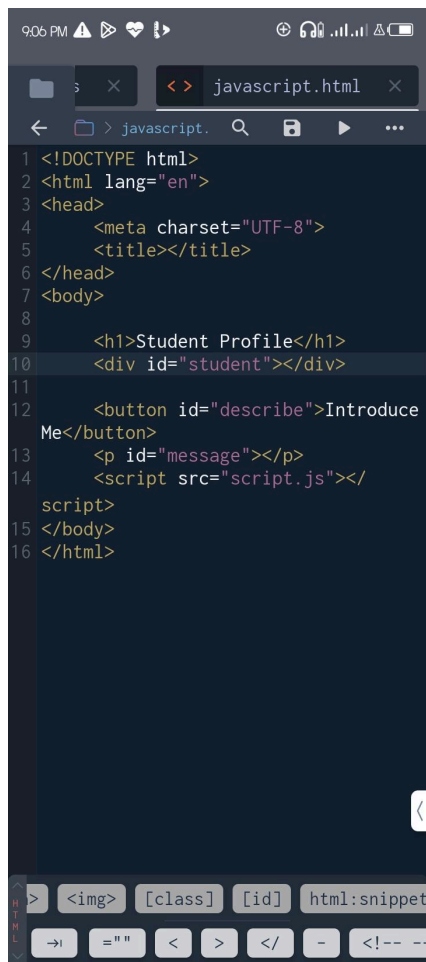
Function belonging to the class

The most important picture:

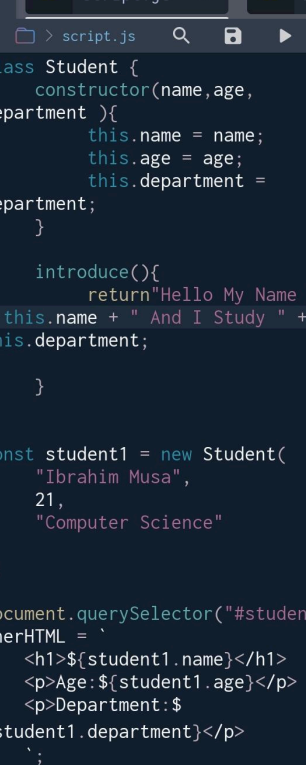
CLASS
(Blueprint)
↓
new
↓
OBJECT
↓
HTML PAGE

And remember: **you don't need to memorize all of this immediately.** Look at the examples while practicing. Understanding comes before memorization.

Once you finish the practice, **then I'll give you the Day 42 assignment project.** After that we'll continue to Day 43 Objects as planned. 🧑🏻



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title></title>
6 </head>
7 <body>
8
9   <h1>Student Profile</h1>
10  <div id="student"></div>
11
12  <button id="describe">Introduce
Me</button>
13  <p id="message"></p>
14  <script src="script.js"></
script>
15 </body>
16 </html>
```



```
9:06 PM
JS script.js
JS main.js

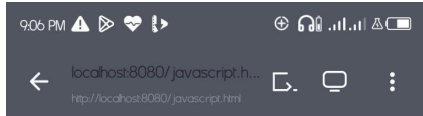
<
>
class Student {
  constructor(name,age,
  department ){
    this.name = name;
    this.age = age;
    this.department =
    department;
  }

  introduce(){
    return"Hello My Name Is "
    + this.name + " And I Study " +
    this.department;
  }
}

const student1 = new Student(
  "Ibrahim Musa",
  21,
  "Computer Science"
);

document.querySelector("#student").innerHTML = `
<h1>${student1.name}</h1>
<p>Age: ${student1.age}</p>
<p>Department: $
{student1.department}</p>
`;

let
const
function
if
if...else
->
{
}
(
)
=
!
&&
```

Student Profile

Ibrahim Musa

Age:21

Department:Computer Science

[Introduce Me](#)

Hello My Name Is Ibrahim Musa And I
Study Computer Science

DAY 43 — JAVASCRIPT OBJECTS

First: What is an Object?

In simple man language:

An object is a container that keeps information about one thing together.

For example, imagine a student:

Student

- |— Name → Ibrahim
- |— Age → 20
- |— Department → Computer Science
- |— Level → 200

Instead of creating many separate variables:

```
const name = "Ibrahim";  
const age = 20;  
const department = "Computer Science";  
const level = 200;
```

we can put everything together:

```
const student = {  
  name: "Ibrahim",  
  age: 20,  
  department: "Computer Science",  
  level: 200  
};
```

That is an object.

 WHERE ARE OBJECTS USED IN REAL WEBSITES?

Objects are used everywhere in JavaScript applications.

For example, an online shop can have:

```
const product = {  
  name: "Nike Shoes",  
  price: 25000,  
  category: "Shoes"  
};
```

A social media website can have:

```
const user = {  
  username: "ibrahim",  
  followers: 500,  
  verified: false  
};
```

A school website can have:

```
const student = {  
  name: "Ibrahim",  
  department: "Computer Science",  
  level: 200  
};
```

So objects allow a website to organize information about things such as users, products, students, posts, orders, cars, messages, etc.

● PART 1 — Creating an Object

Let's create a student:

```
const student = {  
  name: "Ibrahim",  
  age: 20,  
  department: "Computer Science"  
};
```

The { } tells JavaScript:

"I'm creating an object."

Inside it we have:

```
name: "Ibrahim"  
age: 20  
department: "Computer Science"
```

These are called properties.

● PART 2 — Let's put the Object into HTML

Here's where HTML and JavaScript work together.

HTML

```
<!DOCTYPE html>  
<html>
```

```
<head>  
  <title>Student Information</title>  
</head>
```

```
<body>  
  
  <h1>Student Information</h1>  
  
  <p id="name"></p>  
  <p id="age"></p>
```



```
<p id="department"></p>

<script src="script.js"></script>

</body>

</html>
```

We created three empty paragraphs:

```
<p id="name"></p>
<p id="age"></p>
<p id="department"></p>
```

JavaScript will put the information inside them.

JavaScript

```
const student = {
  name: "Ibrahim",
  age: 20,
  department: "Computer Science"
};

document.querySelector("#name").textContent =
  student.name;

document.querySelector("#age").textContent =
  student.age;

document.querySelector("#department").textContent =
  student.department;
```

Your webpage will show:

Student Information

Ibrahim

20

Computer Science

🔥 Now you're seeing the real purpose of objects.

The object stores the information, and JavaScript takes that information and puts it onto the webpage.

● PART 3 — Accessing Object Properties

We use the dot:

`student.name`

Meaning:

"Go inside student and give me the name."

So:

`console.log(student.name);`

gives:

Ibrahim

And:

`console.log(student.age);`

gives:

20

And:

`console.log(student.department);`

gives:

Computer Science

Remember:

`object.property`

Example:

`student.name`

● PART 4 — Another HTML Example

Let's make a User Profile.

HTML

```
<!DOCTYPE html>
<html>

<head>
  <title>User Profile</title>
</head>

<body>

  <h1>User Profile</h1>

  <h2 id="username"></h2>

  <p id="email"></p>

  <p id="country"></p>

  <script src="script.js"></script>

</body>

</html>
```

JavaScript

```
const user = {
  username: "Ibrahim",
  email: "ibrahim@example.com",
  country: "Nigeria"
};

document.querySelector("#username").textContent =
  user.username;

document.querySelector("#email").textContent =
  user.email;

document.querySelector("#country").textContent =
  user.country;
```

The webpage becomes:

User Profile

Ibrahim

ibrahim@example.com

Nigeria

See what's happening?

OBJECT



stores information



JavaScript gets information



HTML displays information



PART 5 — Changing an Object Property

Objects can be changed.

Start with:

```
const student = {  
  name: "Ibrahim",  
  age: 20  
};
```

We can change the age:

```
student.age = 21;
```

Now:

```
console.log(student.age);
```

gives:

21

HTML example

```
<h1 id="studentName"></h1>
```

```
<p id="studentAge"></p>
```

```
<script src="script.js"></script>
const student = {
  name: "Ibrahim",
  age: 20
};

student.age = 21;

document.querySelector("#studentName").textContent =
  student.name;

document.querySelector("#studentAge").textContent =
  student.age;
```

The page shows:

Ibrahim
21

PART 6 — Adding a New Property

Suppose we originally have:

```
const student = {
  name: "Ibrahim",
  age: 20
};
```

Then we decide we also want the student's level.

We can add it:

```
student.level = 200;
```

Now the object is:

```
{
  name: "Ibrahim",
  age: 20,
  level: 200
}
```

We can display it:

HTML

```
<h1 id="name"></h1>
<p id="level"></p>
JavaScript
const student = {
  name: "Ibrahim",
  age: 20
};

student.level = 200;

document.querySelector("#name").textContent =
  student.name;

document.querySelector("#level").textContent =
  student.level;
```

● PART 7 — Deleting a Property

We can also remove a property.

```
const student = {
  name: "Ibrahim",
  age: 20,
  level: 200
};
```

To remove age:

```
delete student.age;
```

Now:

```
console.log(student);
```

will contain:

```
{
  name: "Ibrahim",
  level: 200
}
```

The age property is gone.

● PART 8 — Object + Array

This is VERY important because you used this in your Mini Shop.

An object can contain an array.

```
const student = {  
  name: "Ibrahim",  
  courses: [  
    "HTML",  
    "CSS",  
    "JavaScript"  
  ]  
};
```

Now:

student.courses

gives:

HTML
CSS
JavaScript

And because courses is an array:

student.courses[0]

gives:

HTML
student.courses[1]

gives:

CSS
student.courses[2]

gives:

JavaScript
 HTML + Object + Array

HTML

```
<h1 id="studentName"></h1>
```

```
<h2>Courses</h2>
```

```
<ul id="courses"></ul>
```

```
<script src="script.js"></script>
```

JavaScript

```
const student = {
```

```
  name: "Ibrahim",
```

```
  courses: [
```

```
    "HTML",
```

```
    "CSS",
```

```
    "JavaScript"
```

```
  ]
```

```
};
```

```
document.querySelector("#studentName").textContent =  
  student.name;
```

```
const coursesList =
```

```
  document.querySelector("#courses");
```

```
student.courses.forEach(function(course) {
```

```
  coursesList.innerHTML += `
```

```
    <li>${course}</li>
```

```
`;
```

```
});
```

The webpage displays:

Ibrahim

Courses

- HTML
- CSS
- JavaScript

🔥 Now we're combining things you've already learned.

🔥 PART 9 — Multiple Product Objects

Remember your Mini Shop?

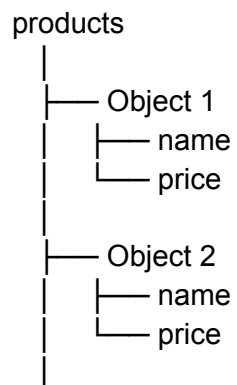
We can have:

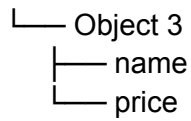
```
const products = [  
  
  {  
    name: "Nike Shoes",  
    price: 25000  
  },  
  
  {  
    name: "Football",  
    price: 15000  
  },  
  
  {  
    name: "Headphones",  
    price: 30000  
  }  
  
];
```

This is:

An array containing multiple objects.

Think:





This is one of the most common ways you'll see data represented in JavaScript.

HTML + Multiple Products

HTML

```
<!DOCTYPE html>
<html>

<head>
  <title>My Shop</title>
</head>

<body>

  <h1>My Shop</h1>

  <div id="products"></div>

  <script src="script.js"></script>

</body>

</html>
```

JavaScript

```
const products = [

  {
    name: "Nike Shoes",
    price: 25000
  },

  {
    name: "Football",
    price: 15000
  },

  {
    name: "Headphones",
    price: 30000
  }

]
```

```

];

const productsContainer =
  document.querySelector("#products");

products.forEach(function(product) {

  productsContainer.innerHTML += `
    <div>
      <h2>${product.name}</h2>
      <p>₦${product.price}</p>
    </div>
  `;

});

```

The webpage will show:

My Shop

Nike Shoes
₦25000

Football
₦15000

Headphones
₦30000

🔥 This is exactly the concept you used in your Day 41 project.

🟡 PART 10 — Object Inside Object

Objects can contain other objects.

Example:

```

const student = {

  name: "Ibrahim",

  address: {
    city: "Kano",
    country: "Nigeria"
  }
}

```

```
}  
};
```

To get the city:

student.address.city

Think:

student



address



city



Kano

HTML

```
<h1 id="name"></h1>
```

```
<p id="location"></p>
```

```
<script src="script.js"></script>
```

JavaScript

```
const student = {
```

```
  name: "Ibrahim",
```

```
  address: {
```

```
    city: "Kano",
```

```
    country: "Nigeria"
```

```
  }
```

```
};
```

```
document.querySelector("#name").textContent =  
  student.name;
```

```
document.querySelector("#location").textContent =  
  student.address.city + ", " +  
  student.address.country;
```

Result:

Ibrahim

Kano, Nigeria

🟦 PART 11 — Function Inside an Object

An object can also contain a function.

```
const student = {  
  name: "Ibrahim",  
  greet: function() {  
    return "Hello, my name is " + this.name;  
  }  
};
```

Then:

```
console.log(student.greet());
```

Result:

Hello, my name is Ibrahim

The function belongs to the object, so we call it:

```
student.greet();
```

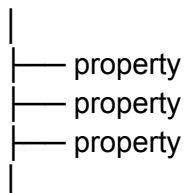
A function stored inside an object is called a method.

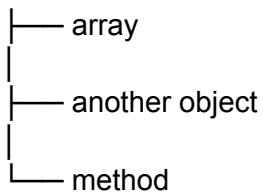
We'll meet this concept again later when we reach Classes.

🧠 THE BIG PICTURE

I want you to understand this structure:

OBJECT





For example:

```
const product = {  
  name: "Nike Shoes",  
  price: 25000,  
  sizes: [  
    40,  
    41,  
    42,  
    43  
  ],  
  brand: {  
    name: "Nike",  
    country: "USA"  
  },  
  describe: function() {  
    return "This is a Nike shoe";  
  }  
};
```

That's one object containing different kinds of information.

HOW THIS CONNECTS TO REAL WEBSITES

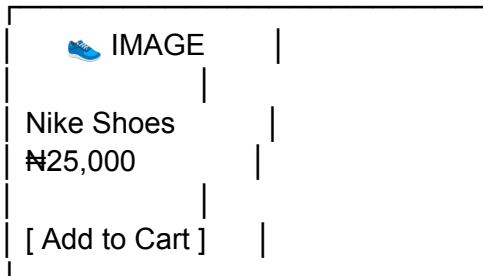
Imagine you open an online store.

The website needs information about a product:

```
{  
  name: "Nike Shoes",  
  price: 25000,  
  image: "shoe.jpg",
```

```
    category: "Shoes",
    available: true
}
```

JavaScript can take that object and create:



And when there are hundreds of products:

```
const products = [
  {...},
  {...},
  {...},
  {...}
];
```

JavaScript can loop through them and create all the cards.

That's exactly why Objects are so important.

DAY 43 SUMMARY

Don't memorize everything. Understand these core ideas:

Object

```
const student = {
  name: "Ibrahim",
  age: 20
};
```

➡ A container for related information.

Property

name: "Ibrahim"

➡ A piece of information inside the object.

Access

```
student.name
```

➡ Get information.

Change

```
student.age = 21;
```

➡ Change information.

Add

```
student.level = 200;
```

➡ Add new information.

Delete

```
delete student.age;
```

➡ Remove information.

Object containing an array

```
student.courses
```

Array containing objects

```
const products = [  
  { name: "Nike Shoes", price: 25000 },  
  { name: "Football", price: 15000 }  
];
```

Object containing another object

```
student.address.city
```

Function inside object

```
student.greet()
```

🏆 YOUR DAY 43 PRACTICE

Build a Student Profile webpage.

Your HTML should contain:

Student Profile

Name:

Age:

Department:

Level:

Courses:

Your JavaScript object should contain:

name

age

department

level

courses

Then use JavaScript to display all the information on the webpage.

Bonus: Add an address object containing:

city

country

And display that too.

Don't worry about making it beautiful yet. Our goal is understanding Objects and connecting them to HTML.

And yes — from now on, I'll keep the lesson structure we've been using: simple explanation → where it's used in websites → JavaScript → HTML + JavaScript together → step-by-step breakdown → practice → assignment → summary → next topic. 🧐🔥

```
const student = {  
  name: "Name: Ibrahim",  
  age: "Age: " + 21,  
  level: "Level: " + 200,  
  department: "Department: Computer Science",  
  
  courses: [  
    "HTML",  
    "CSS",  
    "JAVASCRIPT",  
    "REACT"  
  ],  
  
  addresses: {  
    city: "Abuja",  
    country: "Nigeria"  
  }  
}
```

```

};

const describeButton = document.querySelector("#describe").addEventListener("click",function
(){

    document.querySelector("#name").textContent = student.name;

    document.querySelector("#age").textContent = student.age;
    document.querySelector("#level").textContent = student.level;
    document.querySelector("#department").textContent = student.department;

    const coursesList = document.querySelector("#courses");

    student.courses.forEach(function(courses){

        coursesList.innerHTML += `
            <li>${courses}</li>
        `;

    });

    document.querySelector("#location").textContent =
    student.addresses.city + ", " +
    student.addresses.country;

});

```

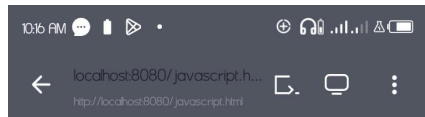
```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title></title>
</head>
<body>

    <h1>Student Profile</h1>
    <button id="describe">Introduce Me</button>
    <p id="name"></p>
    <p id="age"></p>
    <p id="department"></p>

```

```
<p id="level"></p>
<ul id="courses"></ul>
<p id="location"></p>
<script src="script.js"></script>
</body>
</html>
```



Student Profile

Introduce Me

Name: Ibrahim

Age: 21

Department: Computer Science

Level: 200

- HTML
- CSS
- JAVASCRIPT
- REACT

Abuja, Nigeria

DAY 44 — DESTRUCTURING

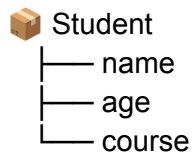
Let's start from zero and take it slowly. Don't worry about the scary name.

🧠 1. What does "Destructuring" mean?

In simple main-man language:

Destructuring means taking information out of an object or array and putting it into variables easily.

Imagine you have a box:



Instead of repeatedly reaching into the box:

```
student.name  
student.age  
student.course
```

we can take them out at once:

```
const { name, age, course } = student;
```

That's destructuring.

🌐 2. Where is Destructuring used in real websites?

You'll see it a LOT when working with:

- 🛒 Product information
- 👤 User profiles
- 📡 APIs
- 📦 Data from databases
- ⚛️ React
- ⚙️ Functions
- Arrays of information

And this is especially important because you have already learned APIs.

When an API sends you an object like:

```
{
  name: "Ibrahim",
  age: 20,
  email: "ibrahim@gmail.com"
}
```

you can use destructuring to quickly grab the information you need.

So this isn't just another random JavaScript feature. You'll actually use it in real development.

PART 1 — Object Destructuring

Let's start with an ordinary object.

```
const student = {
  name: "Ibrahim",
  age: 20,
  course: "JavaScript"
};
```

Normally, if you want the name:

```
console.log(student.name);
```

Age:

```
console.log(student.age);
```

Course:

```
console.log(student.course);
```

This works perfectly.

But we can make it shorter.

✨ Destructuring

```
const { name, age, course } = student;
```

Now JavaScript has created these variables:

```
name → "Ibrahim"
age → 20
course → "JavaScript"
```

So we can simply write:

```
console.log(name);  
console.log(age);  
console.log(course);  
🧠 Break this line apart
```

This:

```
const { name, age, course } = student;
```

might look crazy at first.

But look at it this way:

```
const { name, age, course } = student  
      ↑   ↑   ↑   ↑  
      take take take from
```

We're saying:

"Take name, age, and course from student and give me variables with those names."

That's it. 😎

🟡 PART 2 — HTML + JavaScript Example

Let's use something similar to the projects you've been building.

HTML

```
<!DOCTYPE html>  
<html>
```

```
<head>  
  <title>Student Profile</title>  
</head>
```

```
<body>  
  
  <h1>Student Profile</h1>  
  
  <div id="student"></div>
```

```
<script src="script.js"></script>

</body>

</html>
JavaScript
const student = {
  name: "Ibrahim",
  age: 20,
  course: "JavaScript"
};

const { name, age, course } = student;

document.querySelector("#student").innerHTML = `
  <h2>${name}</h2>
  <p>Age: ${age}</p>
  <p>Course: ${course}</p>
`;
```

The webpage shows:

Student Profile

Ibrahim
Age: 20
Course: JavaScript

Notice that we didn't write:

student.name
student.age
student.course

Instead we wrote:

name
age
course

because we destructured them.

You don't have to take everything.

Imagine:

```
const user = {  
  name: "Musa",  
  age: 25,  
  email: "musa@gmail.com",  
  country: "Nigeria"  
};
```

Maybe you only need:

name
email

You can do:

```
const { name, email } = user;
```

Now:

```
console.log(name);  
console.log(email);
```

You don't need to destructure:

age
country

if you don't need them.

🔥 PART 4 — This is VERY useful with APIs

Remember your Fetch lessons?

Imagine we fetch a user:

```
fetch("https://example.com/user")  
  .then(response => response.json())  
  .then(user => {  
  
    console.log(user);  
  })
```



```
});
```

The API might give us:

```
{  
  name: "Ibrahim",  
  email: "ibrahim@gmail.com",  
  age: 20  
}
```

Instead of:

```
console.log(user.name);  
console.log(user.email);  
console.log(user.age);
```

we can do:

```
.then(user => {  
  
  const { name, email, age } = user;  
  
  console.log(name);  
  console.log(email);  
  console.log(age);  
  
});
```

🔥 This is one reason I want you to learn destructuring now that we've already covered APIs.

🟣 PART 5 — Array Destructuring

Destructuring isn't only for objects.

It also works with arrays.

Suppose:

```
const colors = ["red", "blue", "green"];
```

Normally:

```
console.log(colors[0]);  
console.log(colors[1]);
```

```
console.log(colors[2]);
```

Remember:

0 → red

1 → blue

2 → green

But with destructuring:

```
const [first, second, third] = colors;
```

Now:

first → red

second → blue

third → green

So:

```
console.log(first);
```

```
console.log(second);
```

```
console.log(third);
```

gives:

red

blue

green

🧠 Notice the difference

Object:

We use { }

```
const { name, age } = student;
```

Array:

We use []

```
const [first, second] = colors;
```

Remember:

OBJECT → { }

ARRAY → []

That's an important one.

● PART 6 — Array Destructuring With HTML

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Colors</title>
```

```
</head>
```

```
<body>
```

```
  <h1>My Colors</h1>
```

```
  <div id="colors"></div>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
const colors = [
```

```
  "Red",
```

```
  "Blue",
```

```
  "Green"
```

```
];
```

```
const [first, second, third] = colors;
```

```
document.querySelector("#colors").innerHTML = `
```

```
  <p>First: ${first}</p>
```

```
  <p>Second: ${second}</p>
```

```
  <p>Third: ${third}</p>
```

```
`;
```

You'll get:

My Colors

First: Red

Second: Blue

Third: Green

🧠 PART 7 — Skipping an Array Value

This is another useful trick.

Suppose:

```
const colors = [  
  "Red",  
  "Blue",  
  "Green"  
];
```

But you only want Red and Green.

You can skip Blue:

```
const [first, , third] = colors;
```

See the empty space?

```
[first, , third]  
      ↑  
    skipped
```

Now:

first → Red

third → Green

💡 Don't try to memorize everything

I know you sometimes worry:

"Teacher, how am I going to remember all these things?" 😭😄

Don't worry.

For today, understand these two patterns:

OBJECT DESTRUCTURING

```
const { name, age } = student;
```

Meaning:

Take name and age from the object.

ARRAY DESTRUCTURING

```
const [first, second] = colors;
```

Meaning:

Take the first and second values from the array.

That's the heart of today's lesson.



DAY 44 PRACTICE

Now don't jump to the assignment yet.

Build this small project yourself.



Student Profile

Your HTML should have:

Student Profile

Name

Age

Course

Email

Create this object:

```
const student = {  
  name: "Ibrahim",  
  age: 20,  
  course: "JavaScript",  
  email: "ibrahim@example.com"  
};
```

Then use object destructuring:

```
const { name, age, course, email } = student;
```

Finally display the information inside an HTML element.

Your webpage should look something like:
Student Profile

Ibrahim
Age: 20
Course: JavaScript
Email: ibrahim@example.com
 Bonus practice

Create:

```
const products = [  
  "Laptop",  
  "Phone",  
  "Headphones"  
];
```

Use array destructuring to put them into:

product1
product2
product3

Then display them on the webpage.

DAY 44 ASSIGNMENT PROJECT

Yes! The assignment is included. 

After you finish the practice, build this:



MINI SHOP PRODUCT CARD

Create a product object:

```
const product = {  
  name: "Nike Shoes",  
  price: 25000,  
  category: "Shoes",  
  brand: "Nike"  
};
```

Your job:

Use object destructuring to get:

name
price
category
brand

Then display them in HTML.

Your page should show something similar to:

 My Mini Shop

Nike Shoes

Price: ₦25,000
Category: Shoes
Brand: Nike

[Add to Cart]

When the user clicks Add to Cart, display:

Nike Shoes added to cart!

★ Bonus

Create three products using an array:

```
const products = [  
  {  
    name: "Nike Shoes",  
    price: 25000  
  },  
  {  
    name: "Football",  
    price: 15000  
  },  
  {  
    name: "Headphones",  
    price: 30000  
  }  
];
```

Then practice destructuring one of the products.


```
4:36 PM JS script.js JS main.js
< > script.js
1 const products = {
2   name: "Nike Shoes",
3   price: 25000,
4   category: "Gold",
5   brand: "Jumia"
6 };
7
8
9 const {name, price, category, brand}
  = products;
10
11 document.querySelector("#products").
  innerHTML = `
12   <h2> ${name}</h2>
13   <p>Price: ${price}</p>
14   <p>Category: ${category}</p>
15   <p>Brand: ${brand}</p>
16 `;
17
18
19 document.querySelector("#addToCart")
  .addEventListener("click",function
  () {
20   document.querySelector("#message").i
  nnerHTML = "Nike Shoes Added To
  Cart! ";
21 });
22
23
<div> <a> <img> [class] [id] h
->| = " " < > </ - <!-- --
```



MINI SHOP PRODUCT CARD

Nike Shoes

Price: 25000

Category: Gold

Brand: Jumia

Add To Cart

Nike Shoes Added To Cart!

THE BONUS ANSWER

```
const product = [
  {
    id:1,
    name: "Nike Shoes👟",
    price: 25000
  },
  {
    id:2,
    name: "football⚽",
    price: 15000
  },
  {
```

```
    id:3,  
    name: "Headphones🎧",  
    price: 30000  
  }  
];
```

```
const productsContainer =  
  document.querySelector("#product");  
const cartMessage =
```

```
product.forEach(function(product) {
```

```
  const productDiv = document.createElement("div");
```

```
  productDiv.innerHTML = `

## ${product.name}</h2> <p>₹${product.price}</p>


```

```
<button class="cartBtn">Add to Cart</button>`;
```

```
productsContainer.appendChild(productDiv);
```

```
const button =  
  productDiv.querySelector(".cartBtn");
```

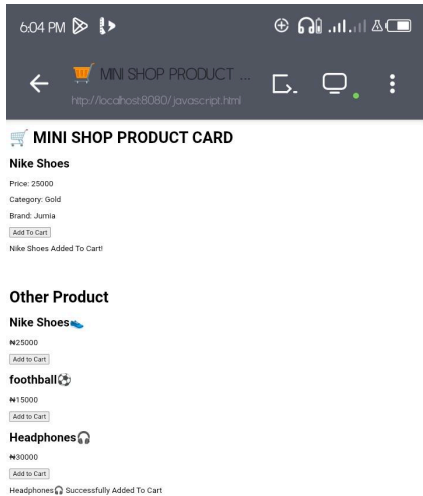
```
button.addEventListener("click", function() {
```

```
  document.querySelector("#cartMessage").textContent = product.name +  
  " Successfully Added To Cart ";
```

```
  })
```

```
});
```

```
6:04 PM < > javascript.html x
< > javascript.
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>🛒 MINI SHOP PRODUCT
  CARD</title>
6 </head>
7 <body>
8
9   <h1>🛒 MINI SHOP PRODUCT
  CARD</h1>
10   <div id="products"></div>
11   <button id="addTo">Add To
  Cart</button>
12   <p id="message"></p><br><br>
13   <h1>Other Product</h1>
14   <div id="product"></div>
15   <p id="cartMessage"></p>
16   <script src="script.js"></
  script>
17 </body>
18 </html>
19
20
21
22
23
<div> <a> <img> [class] [id] h
->| = " " < > </ - <!-- -->
```



Don't worry if the bonus becomes difficult. The main assignment is enough.

DAY 44 SUMMARY

Today you learned:

DESTRUCTURING



Take values out easily

Object:

```
const { name, age } = student;
```

Array:

```
const [first, second] = colors;
```

Real-world use:

API data



Object



Destructuring

↓
Use the information
↓
Display on website

🔥 Your mission now: Do the Practice first, then the Assignment Project. When you're finished, send me your code or screenshot and I'll mark it like your Teacher Coder. 😊👨🏫

DAY 45 — SPREAD OPERATOR ...

This is our new ES6 topic.

And remember our agreement: whenever something is new, I will tell you.

 NEW SYMBOL TODAY:

...

These three dots are called the Spread Operator.

Don't worry about the name yet. Let's understand what it actually does.

❶ What is the Spread Operator?

In very simple "main man" language:

The spread operator ... takes the things inside an array or object and spreads/copies them out.

Think about a bag 🛍️.

You have:

Bag = [Apple, Mango, Orange]

If I say:

"Spread everything out!"

You get:

Apple
Mango
Orange

That's basically what ... does.

② Let's start with an array

Suppose we have:

```
const fruits = ["Apple", "Mango", "Orange"];
```

Normally, fruits is one array:

```
["Apple", "Mango", "Orange"]
```

Now:

```
console.log(...fruits);
```

The spread operator takes the items out:

Apple Mango Orange

So:

```
...fruits
```

means:

"Take the items inside fruits and spread them out here."

③ Why is this useful?

This is where it becomes powerful.

Imagine you have two arrays:

```
const fruits = ["Apple", "Mango"];
```

```
const vegetables = ["Carrot", "Tomato"];
```

You want to combine them.

Older/simple approach

You could do:

```
const food = fruits.concat(vegetables);
```

But with the ES6 spread operator:

```
const food = [...fruits, ...vegetables];
```

Now:

```
console.log(food);
```

gives:

```
["Apple", "Mango", "Carrot", "Tomato"]
```

🔥 That's much cleaner.

④ Understand [...fruits]

This is VERY important.

Look:

```
const fruits = ["Apple", "Mango", "Orange"];
```

```
const newFruits = [...fruits];
```

What happened?

We copied the items from fruits into a new array.

So:

fruits



```
["Apple", "Mango", "Orange"]
```


... spreads

newFruits



["Apple", "Mango", "Orange"]

You now have another array containing the same items.

⑤ Add new items while spreading

This is where you'll start saying:

"Ohhh, I understand this." 😊

```
const fruits = ["Apple", "Mango"];
```

```
const newFruits = [...fruits, "Banana"];
```

Result:

["Apple", "Mango", "Banana"]

We took the old fruits:

...fruits

and added:

"Banana"

⑥ Another example

```
const student1 = ["Ibrahim", "Musa"];
```

```
const student2 = [...student1, "Abdullahi"];
```

Now:

```
console.log(student2);
```

Result:

["Ibrahim", "Musa", "Abdullahi"]

🌐 WHERE IS SPREAD USED IN REAL WEBSITES?

This is important because you always want to know where we actually use the topic.

Spread is used a LOT in real JavaScript applications.

For example, imagine an online shop.

You have:

```
const cart = ["Shoes", "Football"];
```

The customer adds headphones:

```
const newCart = [...cart, "Headphones"];
```

Now:

```
Cart
↓
Shoes
Football
Headphones
```

This is one reason you'll see spread frequently in modern web development.

It's especially common when working with arrays, objects, application state, and frameworks like React later in your journey.

7 SPREAD WITH OBJECTS

Spread isn't only for arrays.

It also works with objects.

Suppose:

```
const user = {
  name: "Ibrahim",
  age: 20
};
```

We can copy it:

```
const newUser = {
  ...user
};
```

Now newUser contains:

```
{
  name: "Ibrahim",
  age: 20
}
```

Again:

...user

means:

"Spread the properties of user into this new object."

⑧ Add something to the object

This is VERY useful.

```
const user = {
  name: "Ibrahim",
  age: 20
};
```

```
const newUser = {
  ...user,
  country: "Nigeria"
};
```

Now:

```
console.log(newUser);
```

Result:

```
{
  name: "Ibrahim",
  age: 20,
  country: "Nigeria"
}
```

🔥 We copied the old object and added something new.

9 Change a value

Look carefully:

```
const user = {  
  name: "Ibrahim",  
  age: 20  
};
```

```
const updatedUser = {  
  ...user,  
  age: 21  
};
```

Now:

name → Ibrahim
age → 21

The age: 21 comes after ...user, so it replaces the previous age in the new object.

This pattern becomes VERY important later.

10 HTML + JavaScript Example

Let's put it into an actual webpage.

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Spread Operator</title>
```

```
</head>
```

```
<body>
```

```
  <h1>My Shopping Cart</h1>
```

```
  <p id="cart"></p>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
const cart = ["Nike Shoes", "Football"];
```

```
const newCart = [...cart, "Headphones"];
```

```
document.querySelector("#cart").textContent =  
  newCart.join(", ");
```

The webpage will show:

My Shopping Cart

Nike Shoes, Football, Headphones

 Let's break that JavaScript down

Step 1

```
const cart = ["Nike Shoes", "Football"];
```

We created our original cart.

Step 2

```
const newCart = [...cart, "Headphones"];
```

This is the important line.

`...cart`

means:

Take everything inside cart.

Then:

`"Headphones"`

adds a new product.

So we get:

```
["Nike Shoes", "Football", "Headphones"]
```

Step 3

```
newCart.join(", ")
```

This turns the array into readable text:

Nike Shoes, Football, Headphones

⚠️ VERY IMPORTANT: Don't confuse Spread with Rest

Later you will see another use of:

...

It is called the Rest Parameter.

For example:

```
function addNumbers(...numbers) {  
  // ...  
}
```

Same three dots.

But today we are learning SPREAD.

I don't want to mix Rest into your brain yet. 😊

When we reach Rest, I'll tell you:

 "This is another use of the ... symbol."

And we'll compare them properly.

 PRACTICE TIME

Don't jump to the assignment yet. Do this practice first.

Practice 1

Create:

```
const fruits = ["Apple", "Mango"];
```

Use spread to create:

Apple
Mango
Banana
Orange

Your code should use:

...

Practice 2

Create:

```
const student = {  
  name: "Ibrahim",  
  age: 20  
};
```

Create a new object containing everything from student plus:

department: "Frontend Development"

Use spread.

Practice 3 🔥

Create two arrays:

```
const shop1 = ["Shoes", "Football"];
```

```
const shop2 = ["Headphones", "Phone"];
```

Use spread to create one array containing all four products.



DAY 45 ASSIGNMENT PROJECT



Mini Shopping Cart — Version 2

Build a small webpage called:

"My Shopping Cart"

Your JavaScript should have:

1. First array

```
const cart = [  
  "Nike Shoes",  
  "Football"  
];
```

2. Use spread to create a new cart

Add:

Headphones

Smartphone

using the spread operator.

3. Display the new cart in HTML.

4. Add a button:

Add New Product

When the button is clicked, display a message such as:

Headphones added to cart!

★ Bonus

Create a user object:

```
const user = {  
  name: "Ibrahim",  
  age: 20  
};
```

Use object spread to create a new object and add:

role: "Frontend Developer"

Display the information on the webpage.

🧠 DAY 45 SUMMARY

Today you learned:

- ✓ What ... means
- ✓ Spread operator
- ✓ Spreading arrays
- ✓ Copying arrays
- ✓ Combining arrays
- ✓ Adding new items to arrays
- ✓ Spreading objects
- ✓ Copying objects
- ✓ Adding properties to objects
- ✓ Updating object properties
- ✓ Where spread is used in real websites

- ✓ HTML + JavaScript example
- ✓ Practice
- ✓ Assignment

The one sentence I want you to remember:

Spread ... takes the contents of an array or object and spreads/copies them into another place.

And don't try to memorize the whole lesson tonight.

Copy it into your docs like you normally do → study it → do the 3 practices → then attempt the assignment yourself.

When you finish, send me the project and I will mark it before we move to Day 46. 🤙🔥💻

DAY 46 — REST PARAMETER ...

1. What is Rest Parameter?

The Rest Parameter allows a function to accept many values and collect them into one array.

The syntax is:

```
function myFunction(...items) {  
  
}
```

The ...items is the Rest Parameter.

Main idea 🧠

Rest collects many values into an array.

For example:

```
function showNames(...names) {  
  console.log(names);  
}
```

```
showNames("Ibrahim", "John", "David");
```

JavaScript collects them like this:

```
["Ibrahim", "John", "David"]
```

So:

```
"Ibrahim"  }  
"John"     } —> ...names —> ["Ibrahim", "John", "David"]  
"David"    }
```

🟡 2. Why do we need Rest?

Imagine you create a shopping cart.

Sometimes a customer buys:

Nike Shoes

Sometimes:

Nike Shoes

Football

Sometimes:

Nike Shoes

Football

Headphones

Phone

Laptop

You don't know how many products the customer will add.

Instead of creating a function with many parameters:

```
function createCart(product1, product2, product3, product4, product5) {  
  
}
```

We can simply use:

```
function createCart(...products) {  
  
}
```

Now the function can accept as many products as we want.

● 3. Rest vs Spread

This is VERY important because both use:

...

But they do opposite jobs.

Spread

Spreads OUT

```
const fruits = ["Apple", "Mango", "Banana"];
```

```
const newFruits = [...fruits];
```

It takes the contents out of the array.

Rest

Collects IN

```
function fruitsList(...fruits) {  
  
}
```

It collects the values into an array.

Think:

SPREAD → 📦 → things come OUT

REST → things → 📦 → things go IN

● 4. Simple Rest Example

```
function addNumbers(...numbers) {  
  
  let total = 0;  
  
  numbers.forEach(function(number) {  
    total += number;  
  });
```

```
    return total;  
}
```

```
console.log(addNumbers(10, 20, 30));
```

Result:

60

Why?

Rest collects:

10, 20, 30

into:

[10, 20, 30]

Then `forEach()` goes through the array.

5. HTML + Rest Parameter

Now let's use it in an actual webpage.

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Rest Parameter</title>
```

```
</head>
```

```
<body>
```

```
  <h1>My Products</h1>
```

```
  <p id="message"></p>
```

```
  <button id="showBtn">Show Products</button>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Notice this:

```
<p id="message"></p>
```

This is where JavaScript will display our products.

And:

```
<button id="showBtn">Show Products</button>
```

This button will start our function.

JavaScript

```
const message = document.getElementById("message");
```

```
const showBtn = document.getElementById("showBtn");
```

```
function showProducts(...products) {
```

```
    message.textContent = products.join(", ");
}
```

```
showBtn.addEventListener("click", function() {
```

```
    showProducts(
        "Nike Shoes",
        "Football",
        "Headphones",
        "Smartphone"
    );
```

```
});
```

When you click the button, you'll see:

Nike Shoes, Football, Headphones, Smartphone

🧠 6. Let's understand the important part

This:

```
function showProducts(...products)
```

means:

"I don't know how many products I'm going to receive, so collect everything into products."

Then:

```
showProducts(  
  "Nike Shoes",  
  "Football",  
  "Headphones",  
  "Smartphone"  
);
```

Rest collects them:

```
products = [  
  "Nike Shoes",  
  "Football",  
  "Headphones",  
  "Smartphone"  
];
```

So we can use normal array methods:

```
products.forEach(...)  
products.join(...)  
products.length
```

because products is an array.

7. Another HTML Example — Shopping Cart

Let's make it more like a real website.

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Shopping Cart</title>
```

```
</head>
```

```
<body>
```

```
  <h1> My Shopping Cart</h1>
```

```
<button id="cartBtn">Add Products</button>
```

```
<h2>Cart:</h2>
```

```
<ul id="cartList"></ul>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

JavaScript

```
const cartBtn = document.getElementById("cartBtn");
```

```
const cartList = document.getElementById("cartList");
```

```
function createCart(...products) {
```

```
    cartList.innerHTML = "";
```

```
    products.forEach(function(product) {
```

```
        const li = document.createElement("li");
```

```
        li.textContent = product;
```

```
        cartList.appendChild(li);
```

```
    });
```

```
}
```

```
cartBtn.addEventListener("click", function() {
```

```
    createCart(
```

```
        "Nike Shoes",
```

```
        "Football",
```

```
        "Headphones",
```

```
        "Smartphone"
```

```
    );
```

```
});
```

Click:

Add Products

You'll get:

Cart:

- Nike Shoes
- Football
- Headphones
- Smartphone

🔥 Now you're using Rest Parameter + functions + arrays + forEach + DOM + events together.

🟡 8. Can we add any number of products?

YES!

That's the beauty of Rest.

We could do:

```
createCart("Nike Shoes");
```

Or:

```
createCart("Nike Shoes", "Football");
```

Or:

```
createCart(  
  "Nike Shoes",  
  "Football",  
  "Headphones",  
  "Smartphone",  
  "Laptop",  
  "Mouse",  
  "Keyboard"  
);
```

We don't need to change the function.

```
function createCart(...products) {  
  
}
```


It accepts them all.

9. Where is Rest used in real websites?

You can use Rest when you don't know how many values a function will receive.

For example:



Shopping websites

```
function addToCart(...products) {
```

```
}
```



Calculators

```
function calculateTotal(...prices) {
```

```
}
```



User systems

```
function createUsers(...users) {
```

```
}
```



Product systems

```
function addProducts(...products) {
```

```
}
```



React/modern JavaScript

You'll also see Rest later when we get deeper into modern JavaScript and React.



10. One more important example

Look at this:

```
function showItems(...items) {
```

```
    console.log(items);
```

```
}
```

```
showItems("Apple", "Mango", "Orange");
```

Inside the function:

items

becomes:

```
["Apple", "Mango", "Orange"]
```

So you can do:

```
console.log(items.length);
```

Result:

3

Or:

```
items.forEach(function(item) {  
  console.log(item);  
});
```

Result:

Apple

Mango

Orange

🧠 REMEMBER THIS

REST

```
function something(...items) {
```

```
}
```

Means:

"Collect all the values into an array."

SPREAD

```
const newArray = [...oldArray];
```

Means:

"Take the contents and spread them out."

So remember:

REST = COLLECTS IN 🍰

SPREAD = SPREADS OUT 🏠

DAY 46 PRACTICE

Create this HTML:

```
<h1>My Products</h1>
```

```
<button id="showBtn">Show Products</button>
```

```
<div id="products"></div>
```

Then JavaScript:

Create:

```
function showProducts(...products) {  
  
}
```

When the button is clicked, send:

Laptop
Phone
Mouse
Keyboard
Headphones

Display all five products on the webpage.

Challenge ★

Try changing it to only:

```
showProducts("Laptop", "Phone");
```

Then try:

```
showProducts(  
  "Laptop",  
  "Phone",  
  "Mouse",  
  "Keyboard",  
  "Headphones",
```

```
"Charger"  
);
```

You should not change the function.

DAY 46 ASSIGNMENT

Project: Flexible Shopping Cart

Build a small webpage containing:

 My Shopping Cart

[Add Products]

Cart:

- Nike Shoes
- Football
- Headphones

Your JavaScript must contain:

```
function createCart(...products) {  
  
}
```

Use Rest Parameter to collect the products.

Your products:

Nike Shoes
Football
Headphones
Smartphone
Laptop

Then display them on the webpage using JavaScript.

Bonus

Create another function:

```
function calculateTotal(...prices) {  
  
}
```

Give it:

```
calculateTotal(25000, 15000, 30000);
```

and make it calculate:

Total: ~~N~~70,000

Don't worry if this looks a little long. The important thing for Day 46 is that you understand one sentence:

Rest ... collects multiple values into an array.

And yes — from now on, when we introduce these JavaScript concepts, I'll keep connecting them to HTML and an actual webpage, not just console examples.